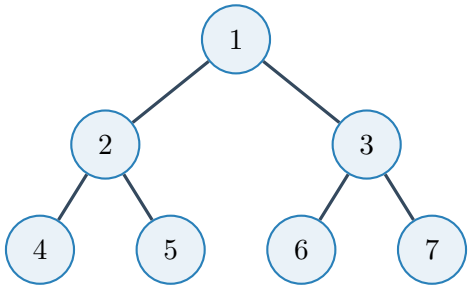
Trees

for Competitive Programming

From Roots and Leaves to Segment Trees and HLD



A rooted binary tree – the first picture of Chapter 1.

31 Chapters • **631 Problems** • **175 Fully Solved**

Written so far: 31 of 31 chapters • 631 problems written • 0 planned in the outline

Comprehensive DSA Guide

October 18, 2026

Contents

How to Read This Book xv

Roadmap xxi

I Foundations 1

1 Tree Basics and Terminology 2
1.1 What Is a Tree? . . . 5
1.2 Depth, Height and Size . . . 15
1.3 Kinds of Trees . . . 28
1.4 Counting Trees . . . 39
Chapter 1 Summary . . . 51
Master Decision Flowchart . . . 52
Common Mistakes Checklist . . . 53
WA Clinic . . . 54
Quick Reference Card . . . 54
Write-From-Memory Drills . . . 55
Complete Problem Checklist . . . 56
Recommended Study Order . . . 58
Difficulty Progression . . . 58
Reduction Gallery . . . 58
Mixed Problem Set . . . 59
The Summit – Scaled: LC 1569 . . . 60
Tool Kit Inventory . . . 60
Related Topics . . . 61

2 Binary Trees: Recursion Patterns 63
2.1 The Node and Structural Recursion . . . 66
2.2 Top-Down: Carrying State Down . . . 80
2.3 Bottom-Up: Returning Values Up . . . 99
2.4 Structural Transformation: Rewiring the Tree . . . 121
Chapter 2 Summary . . . 142
Master Decision Flowchart . . . 143
Common Mistakes Checklist . . . 143
WA Clinic . . . 144
Quick Reference Card . . . 144
Write-From-Memory Drills . . . 145
Complete Problem Checklist . . . 146
Recommended Study Order . . . 147
Difficulty Progression . . . 148
Reduction Gallery . . . 149
Mixed Problem Set . . . 149
The Summit – LC 968 Binary Tree Cameras . . . 150
Tool Kit Inventory . . . 155

Related Topics . . . 156

3 Tree Traversals 158
3.1 The Three Depth-First Orders . . . 162
3.2 Iterative Traversals and Morris . . . 174
3.3 Breadth-First / Level Order . . . 189
3.4 Rebuilding and Serialising . . . 201
Chapter 3 Summary . . . 213
Master Decision Flowchart . . . 214
Common Mistakes Checklist . . . 215
WA Clinic . . . 215
Quick Reference Card . . . 216
Write-From-Memory Drills . . . 217
Complete Problem Checklist . . . 218
Recommended Study Order . . . 220
Difficulty Progression . . . 220
Reduction Gallery . . . 220
Mixed Problem Set . . . 221
The Summit – LC 297 Serialize and Deserialize Binary Tree . . . 222
Tool Kit Inventory . . . 227
Related Topics . . . 229

4 Representing Trees in Memory 230
4.1 Pointers vs Arrays . . . 234
4.2 Rooting an Unrooted Tree . . . 249
4.3 Encodings and Input Formats . . . 268
Chapter 4 Summary . . . 279
Master Decision Flowchart . . . 280
Common Mistakes Checklist . . . 281
WA Clinic . . . 281
Quick Reference Card . . . 282
Write-From-Memory Drills . . . 283
Complete Problem Checklist . . . 284
Recommended Study Order . . . 285
Difficulty Progression . . . 286
Reduction Gallery . . . 286
Mixed Problem Set . . . 287
The Summit – LC 834 Sum of Distances in Tree . . . 288
Tool Kit Inventory . . . 292
Related Topics . . . 293

5 Euler Tour and Subtree Flattening 294
5.1 First-In / Last-Out Times and Bracket Sequences . . . 298
5.2 Subtree as Interval: Flattening to a Line . . . 307
5.3 Path-as-Sequence and the Depth Sequence . . . 316
5.4 Euler Tour for Distance and an RMQ Preview . . . 327
Chapter 5 Summary . . . 339
Master Decision Flowchart . . . 340
Common Mistakes Checklist . . . 341
WA Clinic . . . 341
Quick Reference Card . . . 342
Write-From-Memory Drills . . . 343

Complete Problem Checklist . . . 344
Recommended Study Order . . . 345
Difficulty Progression . . . 345
Reduction Gallery . . . 346
Mixed Problem Set . . . 346
The Summit – LC 2322 Minimum Score After Removals on a Tree . . . 348
Tool Kit Inventory . . . 352
Related Topics . . . 353

6 Constructing Trees from Data 355
6.1 Reconstruct from a Parent Array or an Edge List . . . 360
6.2 Build from Degrees, Sequences and Codes . . . 369
6.3 Is It a Tree? Identification and a Union-Find Pre-Look . . . 379
6.4 Recover the Structure by Queries (Interactive) . . . 388
Chapter 6 Summary . . . 396
Master Decision Flowchart . . . 397
Common Mistakes Checklist . . . 397
WA Clinic . . . 398
Quick Reference Card . . . 398
Write-From-Memory Drills . . . 400
Complete Problem Checklist . . . 401
Recommended Study Order . . . 401
Difficulty Progression . . . 402
Reduction Gallery . . . 402
Mixed Problem Set . . . 402
The Summit – LC 1719 Number of Ways to Reconstruct a Tree . . . 404
Tool Kit Inventory . . . 409
Related Topics . . . 409

II Ordered Trees, Heaps and Tries 411

7 Binary Search Trees 412
7.1 The BST Property and Searching . . . 416
7.2 Insertion and Deletion . . . 426
7.3 BST Queries and Order Statistics . . . 436
7.4 BSTs in Contests: the Ordered Containers . . . 446
Chapter 7 Summary . . . 456
Master Decision Flowchart . . . 458
Common Mistakes Checklist . . . 458
WA Clinic . . . 459
Quick Reference Card . . . 459
Write-From-Memory Drills . . . 460
Complete Problem Checklist . . . 461
Recommended Study Order . . . 462
Difficulty Progression . . . 462
Reduction Gallery . . . 463
Mixed Problem Set . . . 463
The Summit – LC 315 Count of Smaller Numbers After Self . . . 464
Tool Kit Inventory . . . 470
Related Topics . . . 471

8 Balanced Search Trees **473**
8.1 Why Balance? The Rotation . . . 478
8.2 AVL Trees . . . 487
8.3 Treaps, Split/Merge, Implicit Keys . . . 498
8.4 Red-Black Trees (Why `std::set` Is Fast) . . . 509
8.5 Splay Trees: Balance by Locality (Introduction) . . . 514
Chapter 8 Summary . . . 519
Master Decision Flowchart . . . 521
Common Mistakes Checklist . . . 521
WA Clinic . . . 522
Quick Reference Card . . . 523
Write-From-Memory Drills . . . 524
Complete Problem Checklist . . . 525
Recommended Study Order . . . 525
Difficulty Progression . . . 526
Reduction Gallery . . . 526
Mixed Problem Set . . . 527
The Summit – LC 493 Reverse Pairs . . . 528
Tool Kit Inventory . . . 534
Related Topics . . . 534

9 Cartesian Trees and the Monotonic Stack **536**
9.1 The Cartesian Tree in $O(n)$ . . . 541
9.2 Monotonic Stack Applications . . . 550
9.3 Range Minimum Query via the Cartesian Tree . . . 560
9.4 The Cartesian Tree as a Random Treap . . . 568
Chapter 9 Summary . . . 576
Master Decision Flowchart . . . 577
Common Mistakes Checklist . . . 578
WA Clinic . . . 578
Quick Reference Card . . . 579
Write-From-Memory Drills . . . 580
Complete Problem Checklist . . . 581
Recommended Study Order . . . 582
Difficulty Progression . . . 583
Reduction Gallery . . . 583
Mixed Problem Set . . . 583
The Summit – CF 817D Imbalanced Array . . . 585
Tool Kit Inventory . . . 590
Related Topics . . . 591

10 Heaps and Priority Queues **593**
10.1 The Binary Heap . . . 598
10.2 Building a Heap, and Using the Queue . . . 607
10.3 Mergeable Heaps: Heaps You Can Union . . . 616
10.4 Classic Heap Problems . . . 626
Chapter 10 Summary . . . 633
Master Decision Flowchart . . . 635
Common Mistakes Checklist . . . 636
WA Clinic . . . 636
Quick Reference Card . . . 637
Write-From-Memory Drills . . . 638

Complete Problem Checklist . . . 639
Recommended Study Order . . . 640
Difficulty Progression . . . 641
Reduction Gallery . . . 641
Mixed Problem Set . . . 642
The Summit – CF 721D Maxim and Array . . . 643
Tool Kit Inventory . . . 648
Related Topics . . . 649

11 Tries and Binary Tries **651**
11.1 The Trie of Strings . . . 656
11.2 Dictionary Applications . . . 665
11.3 The Binary (XOR) Trie . . . 674
11.4 Compressed and Persistent Tries . . . 683
Chapter 11 Summary . . . 692
Master Decision Flowchart . . . 694
Common Mistakes Checklist . . . 695
WA Clinic . . . 695
Quick Reference Card . . . 696
Write-From-Memory Drills . . . 697
Complete Problem Checklist . . . 698
Recommended Study Order . . . 699
Difficulty Progression . . . 700
Reduction Gallery . . . 700
Mixed Problem Set . . . 701
The Summit – CF 706D Vasiliy's Multiset . . . 702
Tool Kit Inventory . . . 707
Related Topics . . . 708

III Algorithms on Rooted Trees **710**

12 Height, Diameter and Centroid **711**
12.1 Depth, Height and Subtree Size — One DFS for All Three . . . 715
12.2 The Diameter — Two Sweeps, One DP, Same Answer . . . 725
12.3 The Centroid — Cut at the Balance . . . 734
12.4 Eccentricity, Radius, Centre — and the Best Root . . . 743
Chapter 12 Summary . . . 753
Master Decision Flowchart . . . 755
Common Mistakes Checklist . . . 756
WA Clinic . . . 756
Quick Reference Card . . . 757
Write-From-Memory Drills . . . 759
Complete Problem Checklist . . . 760
Recommended Study Order . . . 761
Difficulty Progression . . . 762
Reduction Gallery . . . 762
Mixed Problem Set . . . 763
The Summit – CF 1092F Tree with Maximum Cost . . . 764
Tool Kit Inventory . . . 769
Related Topics . . . 769

13 Tree DP I: Subtree and Path DP **771**
13.1 Subtree DP: One Tuple per Node, Folded from Children . . . 776
13.2 Two-State DP: In / Out at Every Node . . . 785
13.3 Path DP: Harvest at the Peak . . . 795
13.4 Distance-Indexed DP: Histograms over Depth in the Subtree . . . 805
Chapter 13 Summary . . . 817
Master Decision Flowchart . . . 818
Common Mistakes Checklist . . . 819
WA Clinic . . . 819
Quick Reference Card . . . 820
Write-From-Memory Drills . . . 822
Complete Problem Checklist . . . 823
Recommended Study Order . . . 824
Difficulty Progression . . . 825
Reduction Gallery . . . 825
Mixed Problem Set . . . 826
The Summit – CF 1076E Vasya and a Tree . . . 827
Tool Kit Inventory . . . 833
Related Topics . . . 833

14 Tree DP II: Knapsack, States and Budgets **836**
14.1 Tree Knapsack: an Index on the State, and the Merge Bound . . . 841
14.2 States That Grow: the Profile DP . . . 851
14.3 Greedy on Trees: the Exchange Argument . . . 861
14.4 Games on Trees: Grundy Numbers and Win/Lose Flags . . . 870
Chapter 14 Summary . . . 879
Master Decision Flowchart . . . 881
Common Mistakes Checklist . . . 881
WA Clinic . . . 882
Quick Reference Card . . . 883
Write-From-Memory Drills . . . 885
Complete Problem Checklist . . . 887
Recommended Study Order . . . 887
Difficulty Progression . . . 888
Reduction Gallery . . . 888
Mixed Problem Set . . . 889
The Summit – CF 735E Ostap and Tree . . . 890
Tool Kit Inventory . . . 897
Related Topics . . . 898

15 Rerooting: Every Root in One DFS **901**
15.1 The Two-DFS Template: A Down Fold and an Up Fold . . . 906
15.2 The Up/Down Pair: Richer States and the Parent-Side Piece . . . 915
15.3 Prefix/Suffix Merge: When the Combine Has No Inverse . . . 923
15.4 Rerooting a Count: A Verdict for Every Node . . . 931
Chapter 15 Summary . . . 940
Master Decision Flowchart . . . 941
Common Mistakes Checklist . . . 941
WA Clinic . . . 942
Quick Reference Card . . . 943
Write-From-Memory Drills . . . 945
Complete Problem Checklist . . . 946

Recommended Study Order . . . 947
Difficulty Progression . . . 948
Reduction Gallery . . . 948
Mixed Problem Set . . . 948
The Summit – CF 1498F Christmas Game . . . 949
Tool Kit Inventory . . . 954
Related Topics . . . 954

16 Ancestor Queries: LCA and Binary Lifting 956
16.1 Ancestors, the LCA, and the $O(h)$ Walk . . . 962
16.2 LCA by Euler Tour and Range Minimum . . . 967
16.3 LCA by Binary Lifting: The Contest Default . . . 975
16.4 Lifting an Aggregate along a Jump . . . 982
16.5 Distances, Paths and Depth Buckets . . . 989
Chapter 16 Summary . . . 996
Master Decision Flowchart . . . 997
Common Mistakes Checklist . . . 998
WA Clinic . . . 998
Quick Reference Card . . . 998
Write-From-Memory Drills . . . 999
Complete Problem Checklist . . . 1001
Recommended Study Order . . . 1001
Difficulty Progression . . . 1001
Reduction Gallery . . . 1002
Mixed Problem Set . . . 1002
The Summit – CF 1062E Company . . . 1003
Tool Kit Inventory . . . 1007
Related Topics . . . 1008

17 DSU, Small-to-Large and Sack 1010
17.1 Disjoint Set Union: Parent, Rank, Path Compression . . . 1015
17.2 Weighted DSU and Rollback DSU . . . 1023
17.3 Small-to-Large Merging . . . 1034
17.4 DSU on Tree (Sack) . . . 1045
17.5 Offline DSU Tricks: Parallel Binary Search and Dynamic Connectivity . . . 1056
Chapter 17 Summary . . . 1064
Master Decision Flowchart . . . 1065
Common Mistakes Checklist . . . 1066
WA Clinic . . . 1066
Quick Reference Card . . . 1067
Write-From-Memory Drills . . . 1068
Complete Problem Checklist . . . 1070
Recommended Study Order . . . 1070
Difficulty Progression . . . 1071
Reduction Gallery . . . 1071
Mixed Problem Set . . . 1072
The Summit – CF 1009F Dominant Indices . . . 1073
Tool Kit Inventory . . . 1075
Related Topics . . . 1076

IV Query Trees **1078**

18 Fenwick Trees **1079**
18.1 The Binary Indexed Tree . . . 1084
18.2 Range Variants: Adding to Intervals, Summing Intervals . . . 1094
18.3 Order Statistics and Sweeps with a Frequency BIT . . . 1104
18.4 Fenwick on Trees: Driving the Euler Array . . . 1114
18.5 Offline Sweeps: Sort One Axis, BIT the Other . . . 1124
Chapter 18 Summary . . . 1130
Master Decision Flowchart . . . 1132
Common Mistakes Checklist . . . 1133
WA Clinic . . . 1134
Quick Reference Card . . . 1134
Write-From-Memory Drills . . . 1135
Complete Problem Checklist . . . 1136
Recommended Study Order . . . 1137
Difficulty Progression . . . 1138
Reduction Gallery . . . 1138
Mixed Problem Set . . . 1138
The Summit – CF 1181D Irrigation . . . 1140
Tool Kit Inventory . . . 1143
Related Topics . . . 1143

19 Segment Trees **1145**
19.1 The Segment Tree . . . 1150
19.2 Arbitrary Combines and Tuple Nodes . . . 1160
19.3 Searching the Tree . . . 1169
19.4 The Iterative (Bottom-Up) Segment Tree . . . 1178
19.5 Static and Offline Range Problems . . . 1188
Chapter 19 Summary . . . 1197
Master Decision Flowchart . . . 1198
Common Mistakes Checklist . . . 1199
WA Clinic . . . 1200
Quick Reference Card . . . 1200
Write-From-Memory Drills . . . 1202
Complete Problem Checklist . . . 1203
Recommended Study Order . . . 1203
Difficulty Progression . . . 1204
Reduction Gallery . . . 1204
Mixed Problem Set . . . 1205
The Summit – CF 414D Mashmokh and Water Tanks . . . 1206
Tool Kit Inventory . . . 1210
Related Topics . . . 1210

20 Segment Trees II: Lazy **1212**
20.1 The Pending Tag: apply, push, pull . . . 1218
20.2 Composing Lazy Tags . . . 1228
20.3 Range Assign (Paint) . . . 1238
20.4 Lazy on Flattened Subtrees . . . 1246
20.5 When Lazy Propagation Strains . . . 1254
Chapter 20 Summary . . . 1259
Master Decision Flowchart . . . 1260

Common Mistakes Checklist . . . 1261
WA Clinic . . . 1261
Quick Reference Card . . . 1262
Write-From-Memory Drills . . . 1263
Complete Problem Checklist . . . 1265
Recommended Study Order . . . 1265
Difficulty Progression . . . 1266
Reduction Gallery . . . 1266
Mixed Problem Set . . . 1267
The Summit – CF 1146E Hot is Cold . . . 1268
Tool Kit Inventory . . . 1273
Related Topics . . . 1274

21 Merge-Sort and Persistent Trees 1276
21.1 The Merge-Sort Tree: Counting Values in a Range . . . 1283
21.2 Persistence by Path Copying . . . 1292
21.3 Chairman Trees: the k-th Smallest in a Range by Two Root Descents . . . 1303
21.4 Persistent Roots on a Tree: k-th and Count on a Path . . . 1314
21.5 The Sparse Table and Where Each Engine Wins . . . 1325
Chapter 21 Summary . . . 1331
Master Decision Flowchart . . . 1332
Common Mistakes Checklist . . . 1333
WA Clinic . . . 1333
Quick Reference Card . . . 1334
Write-From-Memory Drills . . . 1336
Complete Problem Checklist . . . 1337
Recommended Study Order . . . 1338
Difficulty Progression . . . 1339
Reduction Gallery . . . 1339
Mixed Problem Set . . . 1340
The Summit – CF 840D Destiny . . . 1341
Tool Kit Inventory . . . 1346
Related Topics . . . 1347

22 Heavy-Light Decomposition 1350
22.1 The Chain Decomposition: Cutting a Tree into Heavy Paths . . . 1355
22.2 Path Queries: Jumping Chain to Chain . . . 1363
22.3 Path Updates: A Lazy Tag Down Every Chain . . . 1369
22.4 Subtree and Path in a Single Order . . . 1375
22.5 The Engine Boundary: When Heavy-Light Is the Wrong Tool . . . 1382
Chapter 22 Summary . . . 1385
Master Decision Flowchart . . . 1386
Common Mistakes Checklist . . . 1387
WA Clinic . . . 1387
Quick Reference Card . . . 1388
Write-From-Memory Drills . . . 1390
Complete Problem Checklist . . . 1391
Recommended Study Order . . . 1391
Difficulty Progression . . . 1392
Reduction Gallery . . . 1392
Mixed Problem Set . . . 1393
The Summit – CF 117E Tree or not Tree . . . 1394

Tool Kit Inventory . . . 1400
Related Topics . . . 1401

23 Li Chao Trees, Segment Tree Beats and Function-Tree DP 1404
23.1 The Li Chao Line Container . . . 1408
23.2 The Li Chao Container on a Tree . . . 1415
23.3 Segment Tree Beats and the Chtholly Tree . . . 1421
23.4 Function-Composition Trees . . . 1430
23.5 The Engine Boundary: Which Advanced Tree to Reach For . . . 1437
Chapter 23 Summary . . . 1440
Master Decision Flowchart . . . 1442
Common Mistakes Checklist . . . 1443
WA Clinic . . . 1444
Quick Reference Card . . . 1444
Write-From-Memory Drills . . . 1446
Complete Problem Checklist . . . 1447
Recommended Study Order . . . 1448
Difficulty Progression . . . 1449
Reduction Gallery . . . 1449
Mixed Problem Set . . . 1450
The Summit – CF 932F Escape Through Leaf . . . 1451
Tool Kit Inventory . . . 1457
Related Topics . . . 1458

V Expert Techniques 1461

24 Centroid Decomposition 1462
24.1 Recursion at the Balance . . . 1467
24.2 Counting Paths by Distance . . . 1473
24.3 The Centroid Tree as a Distance Index . . . 1480
24.4 Beyond Path Sums: A Structure in the Bucket . . . 1487
24.5 The Engine Boundary: When Centroid Is the Wrong Tool . . . 1493
Chapter 24 Summary . . . 1496
Master Decision Flowchart . . . 1498
Common Mistakes Checklist . . . 1499
WA Clinic . . . 1499
Quick Reference Card . . . 1500
Write-From-Memory Drills . . . 1502
Complete Problem Checklist . . . 1503
Recommended Study Order . . . 1504
Difficulty Progression . . . 1505
Reduction Gallery . . . 1505
Mixed Problem Set . . . 1506
The Summit – CF 757G Can Bash Save the Day? . . . 1507
Tool Kit Inventory . . . 1514
Related Topics . . . 1515

25 Link-Cut Trees (Dynamic Trees) 1518
25.1 The Splay Engine: Balance by Locality . . . 1523
25.2 Preferred Paths and the Access Operation . . . 1530
25.3 Evert, Link, Cut, and Path Aggregates . . . 1536

25.4 Beyond Paths: Virtual Children and Connectivity . . . 1539
25.5 The Engine Boundary: When Link-Cut Is the Wrong Tool . . . 1542
Chapter 25 Summary . . . 1544
Master Decision Flowchart . . . 1546
Common Mistakes Checklist . . . 1547
WA Clinic . . . 1548
Quick Reference Card . . . 1548
Write-From-Memory Drills . . . 1550
Complete Problem Checklist . . . 1551
Recommended Study Order . . . 1552
Difficulty Progression . . . 1553
Reduction Gallery . . . 1553
Mixed Problem Set . . . 1554
The Summit – CF 487E Tourists . . . 1555
Tool Kit Inventory . . . 1562
Related Topics . . . 1563

26 Virtual Trees (Auxiliary Trees) 1565
26.1 Building a Virtual Tree . . . 1570
26.2 Running a DP on the Compressed Tree . . . 1575
26.3 Two Workhorse Families . . . 1581
26.4 Reading a Statement: Is It a Virtual Tree? . . . 1584
26.5 The Method Boundary: When a Virtual Tree Is the Wrong Tool . . . 1587
Chapter 26 Summary . . . 1590
Master Decision Flowchart . . . 1591
Common Mistakes Checklist . . . 1592
WA Clinic . . . 1592
Quick Reference Card . . . 1593
Write-From-Memory Drills . . . 1594
Complete Problem Checklist . . . 1595
Recommended Study Order . . . 1596
Difficulty Progression . . . 1597
Reduction Gallery . . . 1597
Mixed Problem Set . . . 1597
The Summit – CF 1292D Chaotic V. . . . 1599
Tool Kit Inventory . . . 1605
Related Topics . . . 1606

27 Offline Tree Queries: Mo's on Trees and Tarjan LCA 1608
27.1 Mo's Algorithm: The Block-Order Sweep . . . 1613
27.2 Mo on Subtrees: the Preorder Flatten . . . 1618
27.3 Mo on Paths: the Doubled Tour and the Toggle . . . 1625
27.4 Tarjan's Offline LCA: One DFS and a Disjoint-Set Union . . . 1629
27.5 The Method Boundary: When Mo Is Wrong . . . 1633
Chapter 27 Summary . . . 1637
Master Decision Flowchart . . . 1639
Common Mistakes Checklist . . . 1639
WA Clinic . . . 1640
Quick Reference Card . . . 1640
Write-From-Memory Drills . . . 1641
Complete Problem Checklist . . . 1642
Recommended Study Order . . . 1643

Difficulty Progression . . . 1644
Reduction Gallery . . . 1644
Mixed Problem Set . . . 1644
The Summit – CF 1585E Frequency Queries . . . 1646
Tool Kit Inventory . . . 1652
Related Topics . . . 1653

28 Tree Isomorphism and Hashing **1655**
28.1 Rooted Isomorphism: Sorting Shapes Bottom-Up . . . 1660
28.2 Tree Hashing: $O(1)$ Compares, Priced in Collisions . . . 1664
28.3 Unrooted Isomorphism: Root at the Centre . . . 1671
28.4 Counting Distinct Shapes: Interned Ids Are Isomorphism Classes . . . 1679
28.5 The Method Boundary: When Shape Is Not the Question . . . 1682
Chapter 28 Summary . . . 1687
Master Decision Flowchart . . . 1689
Common Mistakes Checklist . . . 1690
WA Clinic . . . 1690
Quick Reference Card . . . 1691
Write-From-Memory Drills . . . 1691
Complete Problem Checklist . . . 1693
Recommended Study Order . . . 1693
Difficulty Progression . . . 1694
Reduction Gallery . . . 1694
Mixed Problem Set . . . 1695
The Summit – CF 1252F Regular Forestation . . . 1697
Tool Kit Inventory . . . 1704
Related Topics . . . 1705

29 Trees Born from Graphs **1708**
29.1 Bridges and the Bridge Tree . . . 1712
29.2 Articulation Points, Blocks and the Block-Cut Tree . . . 1719
29.3 Functional Graphs: Cycles with Trees on Top . . . 1726
29.4 Cactus Graphs: Cycles That Never Share an Edge . . . 1733
29.5 The Method Boundary: Which Decomposition . . . 1741
Chapter 29 Summary . . . 1743
Master Decision Flowchart . . . 1744
Common Mistakes Checklist . . . 1745
WA Clinic . . . 1746
Quick Reference Card . . . 1746
Write-From-Memory Drills . . . 1746
Complete Problem Checklist . . . 1747
Recommended Study Order . . . 1747
Difficulty Progression . . . 1747
Reduction Gallery . . . 1748
Mixed Problem Set . . . 1748
The Summit – CF 835F Roads in the Kingdom . . . 1749
Tool Kit Inventory . . . 1757
Related Topics . . . 1758

30 Counting Labelled Trees **1762**
30.1 The Prüfer Bijection . . . 1766
30.2 Cayley's Formula and the Forest-Completion Count . . . 1771

30.3 The Matrix-Tree Theorem . . . 1778
30.4 Linearity of Expectation on a Rooted Tree . . . 1782
30.5 The Method Boundary: Labelled vs. Unlabelled . . . 1788
Chapter Summary . . . 1791
Master Decision Flowchart . . . 1791
Common Mistakes Checklist . . . 1791
Wrong-Answer Clinic . . . 1792
Quick Reference Card . . . 1792
Write-From-Memory Drills . . . 1793
Complete Problem Checklist . . . 1793
Recommended Study Order . . . 1793
Difficulty Progression . . . 1793
Reduction Gallery . . . 1794
Mixed Problem Set . . . 1794
The Summit – CF 917D Stranger Trees . . . 1795
Tool Kit Inventory . . . 1800
Related Topics . . . 1800

31 Minimum Spanning Trees and Variants **1802**
31.1 The Cut Property and Kruskal's Algorithm . . . 1806
31.2 Prim's Algorithm and the Frontier View . . . 1813
31.3 The Kruskal-Reconstruction Tree . . . 1817
31.4 Which Edges Can Be in an MST? . . . 1823
31.5 The Method Boundary: Second-Best MST and Hand-Offs . . . 1833
Chapter Summary . . . 1836
Master Decision Flowchart . . . 1837
Common Mistakes Checklist . . . 1837
Wrong-Answer Clinic . . . 1837
Quick Reference Card . . . 1838
Write-From-Memory Drills . . . 1838
Complete Problem Checklist . . . 1839
Recommended Study Order . . . 1839
Difficulty Progression . . . 1839
Reduction Gallery . . . 1839
Mixed Problem Set . . . 1840
The Summit – CF 888G Xor-MST . . . 1840
Tool Kit Inventory . . . 1845
Related Topics . . . 1845

Appendices **1847**

A Formula Reference **1848**
A.1 Tree Basics . . . 1848
A.2 Counting Trees . . . 1848
A.3 Distances and Ancestors . . . 1849

B Code Templates **1850**
B.1 Node Types . . . 1850
B.2 General Trees from an Edge List . . . 1850
B.3 Height and Depth of a Binary Tree . . . 1851
B.4 Level-Order Traversal (BFS) . . . 1851

B.5 Three Binary-Tree Traversals 1852

C Problem-Solving Strategies **1853**
C.1 Recognising a Tree Problem 1853
C.2 Constraints Suggest the Method 1853
C.3 A Five-Step Framework 1854

List of Solved Problems **1855**

Part I

Foundations

The Language of Trees

Beginner – LC Easy/Medium • 6 chapters • 176 problems

Chapters in this part

1 Tree Basics and Terminology 2

2 Binary Trees: Recursion Patterns 63

3 Tree Traversals 158

4 Representing Trees in Memory 230

5 Euler Tour and Subtree Flattening 294

6 Constructing Trees from Data 355

1

Chapter

Tree Basics and Terminology

Level: Beginner – LC Easy/Medium • **Problems:** 37 (8 fully solved) • **Status:** Written

Chapter at a Glance

A **tree** is the simplest structure that captures a hierarchy: one thing at the top, everything else hanging below it, and exactly one way to travel from any object to any other. This chapter builds the whole vocabulary we use for the rest of the book – *root, parent, child, leaf, subtree, ancestor, descendant, depth, height, size* – and proves the few facts that every later algorithm quietly relies on: a tree with n nodes has $n-1$ edges, there is exactly one path between any two nodes, and height can be computed bottom-up with a single recursion.

You will learn to

- say precisely which pictures are trees and which are not, and why; **L1**
- prove and use the $n-1$ edges and unique-path properties; **L1**
- compute depth, height and size of every node in one DFS; **L2**
- name the standard shapes (binary, full, complete, perfect, balanced, skewed) and know the counting rule each one gives; **L2–L3**
- count tree shapes with the Catalan numbers and labelled trees with Cayley's n^{n-2} formula; **L3–L4**
- recognise a *tree-in-disguise* statement and route it to the right tool. **L4**

Prerequisites. None – this is the entry point of the book. We only assume you can read a small C++ function and know what a graph (nodes and edges) is.

In contests and interviews. Every tree problem on LeetCode and most on Codeforces start here: you must know what a root, a leaf and a subtree are, and that a DFS carrying the parent is the workhorse. The counting facts of §1.4 turn up directly in combinatorics and DP problems (Chapter 13).

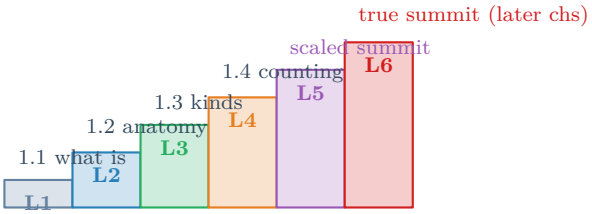
How to Read This Chapter (two paths up the ladder)

Every problem, solution step and warning in this book carries a level badge **L1**–**L6**: **L1** is *"what is a tree"* (rating about 800), **L3** is the solid interview medium (about 1600), **L5**–**L6** is Codeforces 2600–3500+.

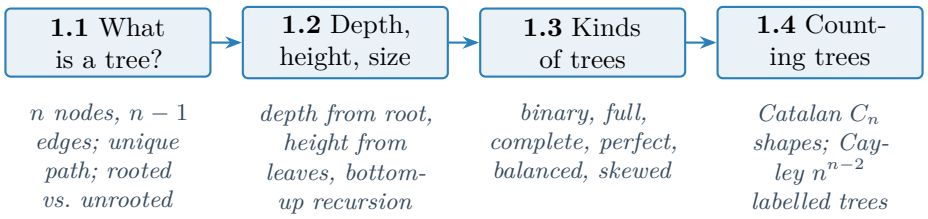
Beginner path (L1–L3). Read §1.1 and §1.2 completely; in §1.3 read the definitions and the first two solved problems; in §1.4 you may *skip on the first pass*: the Catalan recurrence proof, Cayley's proof, the **L4+** variation entries, the hardening notes and the chapter's Summit. Do every recall card and the **L1–L3** practice items. At the end of this path you can solve every **L3** tree-recursion problem in this chapter.

Full path. Everything, in order – it ends with a scaled summit (the hardest problem

solvable with Chapter-1 tools alone) and the reduction gallery that shows how often "it is a tree" is the whole trick. Chapter 1 tops out at **L4**–**L5**; the true **L6** summits begin once Chapters 13–16 hand you DP, lifting and query trees.



Chapter 1 on the six-step ladder: it teaches steps **L1**–**L4** and shows **L5** from the bottom. Beginner path = steps 1–3; the scaled summit sits on step 5 and waits for you after the full path.



The four topics of this chapter, left to right, each with the results you take away from it.

Topic	Key results	Levels	Solved problems
§1.1 What is a tree?	$n-1$ edges; exactly one path between any two nodes	L1–L2	W1.1.1
§1.2 Depth, height and size	height/size recurrences; one-DFS measurement	L1–L3	W1.2.1, W1.2.2
§1.3 Kinds of trees	perfect $2^{h+1}-1$ nodes; full tree leaves = internal +1	L2–L4	W1.3.1, W1.3.2
§1.4 Counting trees	Catalan C_n shapes; Cayley n^{n-2} labelled trees	L3–L5	W1.4.1

Problem Recognition Guide — When You See *this*, Think *that*

The fastest way to get stuck on a tree problem is to reach for the wrong tool. Before you code, match the problem's phrasing against this table; every row is a pattern you will formally build in the section named at the right.

When the problem says...	Think...	Section
"same / identical / duplicate subtree", compare two trees	structural recursion walking both trees in lockstep	§1.1
"for each node, how far from the root / which level"	carry `depth + 1` *down* the recursion (top-down)	§1.2
"height / deepest / how many nodes below / subtree"	*return* a value *up* the recursion (bottom-up)	§1.2
"the tree is complete / perfect / full / balanced"	the shape's counting rule ($2^{h+1}-1$, $L=I+1$) skips work	§1.3
"how many different tree shapes / BSTs from 1..n / labelled trees"	Catalan C_n or Cayley n^{n-2}, usually with a modulus	§1.4
$n \le 10^5$~10^6 and the tree may be a long chain	recursion depth is the danger – iterative DFS or an explicit stack	§1.2

Constraint you see	What it permits	Chapter-1 tool
$n \le 15$	enumerate every shape ($C_{15} = 9{,}694{,}845$ still fits)	§1.4
$n \le 10^3$	$O(n^2)$ is fine – the naive height-at-each-node passes	§1.3
$n \le 10^6$	one visit per node, $O(1)$ combine – single-pass recursion	§1.2
count "modulo 10^9+7"	the answer is a huge formula – Catalan/Cayley DP	§1.4

Not this chapter?

If the tree must *stay ordered* for fast lookup, you need BSTs (Chapter 7); if it must *support updates* while staying shallow, balanced trees (Chapter 8); if queries ask about *paths between two fixed nodes*, that is LCA (Chapter 16). This chapter supplies the vocabulary all of them speak.

Notation Used in This Chapter

Symbol	Meaning	Symbol	Meaning
n	number of nodes	depth(v)	edges from root down to v
T	a tree	height(v)	edges from v to deepest leaf below
parent(v)	the node above v	size(v)	nodes in the subtree rooted at v
children(v)	nodes directly below v	level(v)	same as depth(v) (root = level 0)
deg(v)	number of neighbours of v	leaves(T)	the set of leaves of T
subtree(v)	the tree under v	`null`	the empty / `null` child
h	height of the whole tree	C_n	n-th Catalan number

Throughout the book height and depth are counted in **edges**, so a single lone node has height 0 and height(`null`) $= -1$. The convention note in Appendix A explains why this matters before

you submit.

1.1 What Is a Tree?

In This Section

A tree is a set of nodes joined by edges with no loops and no separate islands – you can get from any node to any other, and there is only one way to do it. File systems, organisation charts, HTML documents and parse trees are all trees, and every data structure in this book is built on this one picture.

Prerequisites. The idea of a graph (nodes and edges) from a first algorithms course.

After this section you can

- tell a tree from a graph that is not a tree, at a glance;
- root a tree and name parents, children, leaves and subtrees correctly;
- prove that a tree on n nodes has $n - 1$ edges and one path per pair;
- read a tree from an edge list into an adjacency list in C++.

Pattern card – lockstep structural recursion

Recognition keywords. "same", "identical", "duplicate / subtree of", "structurally equal", "invert", or any problem that hands you *two* trees (or one tree and a shape) and asks the structures to match.

✓ When to use this

- comparing two trees node-by-node (Same Tree, Subtree check, Symmetric mirror);
- rebuilding a tree in parallel with another traversal (merge, copy);
- deciding tree *identity* – both shape *and* values must agree.

✗ When NOT to use this

- only *values* matter (a multiset comparison – use a hash map, not structure);
- the input is a general graph – verify it is acyclic and connected *first* (Theorem 1.1.4), or your "tree" recursion never terminates;
- one tree but a *global* answer over paths – that is bottom-up aggregation, §1.2 or Chapter 12.

One-line test: *if you can solve it by visiting corresponding nodes of both trees in the same order, this pattern applies.*

1.1.1 Intuition

Folders and family

A tree is a computer's folder structure: one root folder, sub-folders inside folders, files at the bottom, and no folder ever nesting back inside its own descendant. A family tree drawn down from a single ancestor is the same picture. That "nothing is its own ancestor" rule is exactly the no-cycles condition we make precise below.

Look at the picture below. The dots are nodes and the lines are edges. There are no closed loops: starting anywhere, you can never come back to where you began without retracing a step.

That loop-free, fully-connected shape *is* a tree.

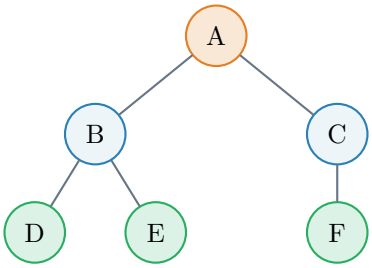


Figure 1.1.1: A tree. The orange node is chosen as the *root*; the green nodes D, E, F are *leaves*. Notice there is exactly one path between any two letters.

Now the three pictures that are **not** trees. A loop (a cycle) is forbidden, and so is a floating piece you cannot reach.

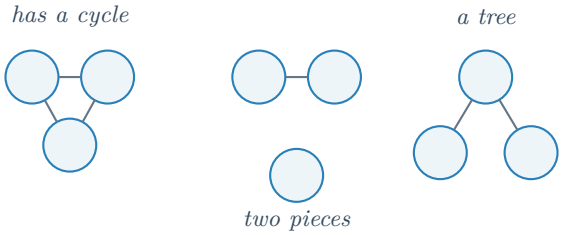


Figure 1.1.2: Left: a cycle, so not a tree. Middle: disconnected (node isolated), so not a tree. Right: connected and loop-free, so a tree.

The same unrooted tree becomes a *rooted* tree the moment we pick a starting node and let gravity decide who is above whom.

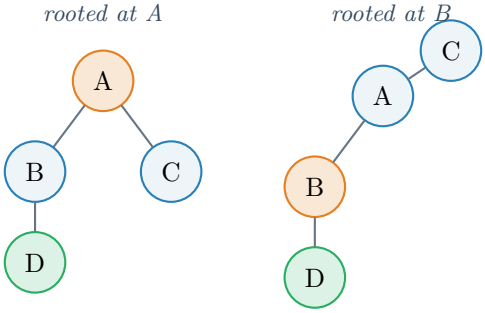


Figure 1.1.3: One tree, two roots. Rooting at B makes A a child of B and D a child of B; rooting at A makes B a child of A. The edges do not change, only our view of parent and child does.

1.1.2 Definitions and Notation

Definition 1.1.1 (Tree and forest)

A **tree** is a connected graph with no cycles. A **forest** is a disjoint union of trees – it drops only the "connected" requirement. Equivalently, a graph on n nodes is a tree if any (and hence all) of these hold: it is connected with $n-1$ edges; it has no cycle but adding any missing edge creates one; every pair of nodes is joined by exactly one path.

In plain words. A tree is a connection with *no loops*: you can travel between any two points, but there is never a roundabout. Four promises that look different – "connected with $n-1$ edges", "no cycle but barely connected", "exactly one path between every pair" – all mean the same thing; grab whichever is easiest to check in the problem in front of you.

Definition 1.1.2 (Rooted tree: parent, child, leaf)

A **rooted tree** is a tree with one distinguished node called the **root**. Directing every edge away from the root gives the relations: if u sits above v and they are adjacent, u is the **parent** of v and v is a **child** of u; a node with no child is a **leaf**; a node that is not a leaf is **internal**. The **subtree** at v, written subtree(v), is v together with all its descendants. An **ancestor** of v is any node on the path from the root to v (including v itself); a **descendant** runs the other way. Siblings share a parent.

In plain words. Choose the one node on top and every relationship becomes family vocabulary measured from it: who sits directly above me (parent), below me (children), who has nobody below (leaf), and the whole line from the top down to me (ancestors). The figure right below puts a face on every one of these words.

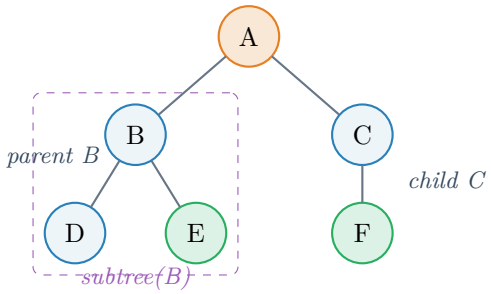


Figure 1.1.4: Anatomy of the rooted tree. B is the parent of D and E (its children); C is a child of A; D, E, F are leaves; the dashed box is the subtree rooted at B. Node A is an ancestor of every other node.

Definition 1.1.3 (Degree, path, distance)

The **degree** $\deg(v)$ of a node is its number of neighbours (in a rooted tree, often split into in-degree = children and out-degree = 1 for the parent). A **path** is a sequence of nodes each adjacent to the next; its **length** is the number of edges. The **distance** $\operatorname{dist}(u, v)$ is the length of the (unique) shortest path between u and v.

In plain words. Degree is just "how many neighbours do I have"; distance is "how many edges do I cross on the only available route" – no tie to break, because Theorem 1.1.5 (two sections down) says there is exactly one route.

1.1.3 Counting Edges, Finding Paths

Two facts about trees come up in almost every proof later in the book. They look obvious, so it is worth seeing exactly why they hold.

Theorem 1.1.4 (Edges of a Tree)

A tree with n nodes has exactly $n-1$ edges.

Proof

Induction on n. For $n = 1$ there are no edges, and $1 - 1 = 0$. For $n \geq 2$, a finite tree always has a leaf: walk a longest simple path, and one of its endpoints must have degree 1 – if it had another neighbour we could either extend the path or close a cycle, both impossible. Call that leaf v; deleting v and its single incident edge leaves a tree on $n - 1$ nodes, which by the induction hypothesis has $n - 2$ edges. Putting v back adds exactly one edge, giving $(n - 2) + 1 = n - 1$.

■

Key Insight

Read it backwards: $n - 1$ edges is the *minimum* needed to connect n nodes. One edge short and you have two islands; one edge too many and you create a cycle. That is why contest statements say "n cities, $n - 1$ roads, all connected" and quietly mean a tree.

Theorem 1.1.5 (Unique Path)

Between any two nodes of a tree there is exactly one simple path.

Proof

Existence: the tree is connected, so some path exists between any two nodes. *Uniqueness:* suppose two different simple paths P_1 and P_2 joined u to v. Walk along P_1 from u until the first node w where P_1 and P_2 split, then continue to where they rejoin at some node z. The two different w-to-z pieces together form a closed loop, i.e. a cycle – contradicting that the graph is a tree. Hence the path is unique.

■

Key Insight

Unique paths are why tree problems rarely need "shortest" in the statement: the one path *is* the shortest. They are also why we can talk about "the" path between u and v without qualification, and why removing a single edge splits a tree into exactly two pieces.

1.1.4 Storing a Tree in Memory

Interview binary trees arrive as `TreeNode*` pointers (see Chapter 2). Contest trees arrive as a list of $n - 1$ edges over nodes numbered $0..n - 1$ or $1..n$; we store them as an adjacency list and remember the parent so the walk never doubles back.

Code 1.1.1: Build an adjacency list from edges and measure sizes with a parent-guard DFS

```
#include <bits/stdc++.h>
using namespace std;

int n;
vector<vector<int>> adj;      // adj[v] = neighbours of v
vector<int> sz;               // sz[v] = size of subtree at v

// Root the tree at 0. p is the node we came from; never walk back to it.
void dfs(int v, int p) {
    sz[v] = 1;                        // count v itself
    for (int to : adj[v]) {
```

```cpp
        if (to == p) continue;       // the parent guard: without this -> infinite
    ↪ loop
        dfs(to, v);
        sz[v] += sz[to];             // fold each child's subtree size upward
    }
}

int main() {
    n = 5;
    adj.assign(n, {});
    vector<pair<int,int>> edges = {{0,1}, {0,2}, {1,3}, {1,4}};
    for (auto [u, v] : edges) { adj[u].push_back(v); adj[v].push_back(u); }
    sz.assign(n, 0);
    dfs(0, -1);
    // edges = n - 1 = 4, and sz[0] == n, confirming the two theorems above.
}
```

Complexity

Building the adjacency list is $O(n)$ (each of the $n-1$ edges pushed twice). The DFS visits every node once and every edge twice, so it is $O(n)$ in time and $O(n)$ memory for the list plus the recursion stack. On a path-shaped tree the stack can be n deep; see §1.2 for the iterative alternative.

1.1.5 Worked Examples

Example 1.1.6 (A six-node tree)

Take the tree of Figure 1.1.4 and add a child G under C. It now has $n = 6$ nodes. Count the edges: AB, AC, BD, BE, CF, CG – that is 6 nodes but only $5 = 6 - 1$ edges, matching Theorem 1.1.4. The path from D to G is $D \to B \to A \to C \to G$, length 4, and it is the only such path (Theorem 1.1.5).

Step-by-Step Trace

Running of Code 1.1.1 on a path $0 - 1 - 2 - 3$ rooted at 0 gives size(3) $= 1$, size(2) $= 2$, size(1) $= 3$, size(0) $= 4$: each node's size is 1 plus its single child's size, so sizes count up from the far leaf.

1.1.6 Variations and Twists

- **Two trees, same order but mirrored.** Lockstep with opposite child pairs (p.left vs q.right) turns "same" into "symmetric" – LC 101 is exactly this twist. **L2**
- **One tree, containment.** "Is t a subtree of s?" (LC 572) runs the lockstep check at *every* node of s as a candidate root. Twists: $O(n+m)$ needs hashing or KMP – Chapter 11 returns with the proof. **L3**
- **Structure from the other side.** "Find the centre of a star graph" (LC 1791, **L1**) hands you $n-1$ edges around one hub: the vocabulary of degrees on a tree, with zero recursion. The star is the height-1 tree of §1.3.
- **Path questions on a tree.** LC 2096 (directions between two nodes) is the unique-path theorem turned into a string: the path $a \to b$ is $a\uparrow\text{lca}\downarrow b$ – preview Chapter 16, solved there. **L3**

- **Disguise.** "Family tree reunion", "file-system merge", "org-chart rewrite" – all lockstep recursion wearing clothes. The reduction is always: identify corresponding nodes, recurse in pairs.

1.1.7 Common Pitfalls

Forgetting the parent guard

A tree stored as an adjacency list has *undirected* edges, so B appears in `adj[A]` and A appears in `adj[B]`. A DFS that does not skip the node it came from bounces $A \to B \to A \to B \ldots$ forever. The line `if (to == p) continue;` is not an optimisation; it is what makes the traversal terminate.

Connected but cyclic, or acyclic but split

Both failures look tree-like. "No cycle" alone allows a forest; "connected" alone allows a graph with a loop. Only together do they define a tree – and only a tree has the $n-1$ edge count. If your edge count is not $n-1$, you do not have a tree.

The two traps as code. Each pitfall above gets a wrong/right pair – compile the WRONG version in your head first, and find the failure yourself.

Code 1.1.2: WRONG – no parent guard: A->B->A->B forever

```
void dfs(int v, int p) { for (int to : adj[v]) dfs(to, v); }
```

Code 1.1.3: RIGHT – skip where you came from

```
void dfs(int v, int p) {
  for (int to : adj[v]) {
    if (to == p) continue;
    dfs(to, v);
  }
}
```

Code 1.1.4: WRONG – edge count alone: a triangle plus an isolated node passes

```
bool isTree = (m == n - 1);
```

Code 1.1.5: RIGHT – count and reach

```
bool isTree = (m == n - 1) && reachedAll(0, n);
```

1.1.8 Problem Lab (fully solved)

Attempt it yourself first

Same Tree (next page) looks trivial – it is still the *most-warrantied* L1 problem in interviews: every wrong submission fails one of the four null-pair cases. Write it before reading; then audit your code against $\varnothing/\varnothing$, $\varnothing/x$, $x/\varnothing$, x/y.

W1.1.1 Same Tree

Source: LeetCode 100 LC Easy L1
Techniques: tree definition, structural equality, recursion

Same Tree

Given the roots `p` and `q` of two binary trees, decide whether they are *the same tree*: same shape *and* same value at every node. Two empty trees are the same; an empty tree and a non-empty one are not.

➤ Understanding the Problem

"Same" here is the precise structural equality the definition gives: the roots match in value, the left subtrees are the same, and the right subtrees are the same. That is already a recursive description, which is why a two-line recursion solves it and why a flat comparison of node values is not enough – shape matters.

➤ Key Observations

Observation 1. Two trees are the same iff their roots carry the same value *and* their left subtrees are the same *and* their right subtrees are the same.

Observation 2. The base case is the `null` pointer: if either node is `null`, the trees match exactly when *both* are `null` at once.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** flatten both trees to a list of values and compare the lists
Killed by: shape is lost – a tree and its mirror image have the same multiset of values and pass, but they are not the same tree.

✗ **Tried:** compare values, and only recurse when the values happen to be equal
Killed by: an unequal value should still be checked against *structure*: if one node is `null` and the other is not, the trees already differ regardless of value.

✓ **Aha!** the definition of "same" is itself recursive – root equal *and* left same *and* right same – so lockstep recursion on the pair (p, q) is not a trick, it is a direct transcription of the definition.

➤ The Approach Ladder

Approach ladder – Same Tree

	Approach	Why it wins / dies	Cost	Verdict
L1	lockstep recursion over the pair (p, q)	the definition of "same" *is* recursive: root equal and left same and right same	$O(\min(n, m))$	✓ Accepted
L2	serialise both trees (pre-order with null markers), compare strings	correct, but $O(n)$ extra memory and a custom format for zero cost gained	$O(n)$ / $O(n)$	✓ Accepted
–	compare multisets of values only	ignores shape entirely – mirror images pass	–	WA

➤ Solution

Recurse in lockstep on the two trees. At each pair of nodes, if both are null return true; if exactly one is null return false; otherwise compare values and recurse on the two children pairs.

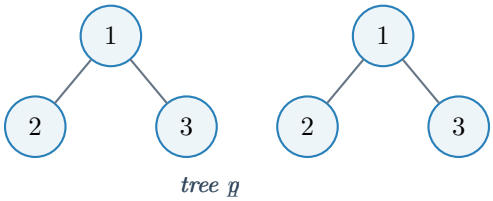


Figure 1.1.5: These two trees are the same: equal values and equal shape, so the answer is `true`. Changing q's right child 3 to a left child would make the shapes differ and the answer `false`.

➤ Proof of Correctness

Induction on tree size

By Theorem 1.1.5 a binary tree's structure is fully described by its root and its two subtrees. Induct on size(p) + size(q). Base: both empty – the code returns `p == q`, correct. Step: if exactly one is empty they differ and the code returns false, correct. Otherwise both are non-empty: by the induction hypothesis the recursive calls correctly decide whether the left subtrees match and whether the right subtrees match; the code returns true exactly when the root values equal *and* both recursive answers are true – precisely the definition of "same tree".

■

➤ Implementation

Code 1.1.6: Same Tree (LeetCode 100)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
```

```
};

class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if (p == nullptr || q == nullptr)   // both null -> equal; one null -> not
            return p == q;
        return p->val == q->val
            && isSameTree(p->left,  q->left)
            && isSameTree(p->right, q->right);
    }
};
```

➤ Trace

Step-by-Step Trace

On the pair in Figure 1.1.5: `isSameTree(1,1)` compares values (equal), then `isSameTree(2,2)` and `isSameTree(3,3)`, each of which recurses into two `nullptr` pairs that return `p == q` = true. Every leaf-of-pair call sees two nulls and reports equal, so the whole call returns true.

➤ Complexity

Complexity

Time $O(\min(n_p, n_q))$: we stop as soon as a mismatch or a null appears, visiting at most the smaller tree. Memory $O(h)$ for the recursion stack, worst case $O(n)$ for a skewed tree. Because the depth can be n, an iterative BFS-pair version (queue of node pairs) avoids stack overflow on pathological inputs.

➤ Verification and Edge Cases

- both empty: `p == q` is true – correct (two nulls match).
- one empty: exactly one is `null`, `p == q` is false – correct.
- equal values, different shape (mirror image): a child pair is $(x, \texttt{null})$ at some point, returning false – shape is enforced. L1

➤ Pitfalls and Variants

Catching only value mismatches

Comparing values but not shape passes a tree against its mirror image – wrong. The `p == q` null test is what enforces equal shape; do not replace it with two separate `return true` branches that can drift apart.

Variants: *Symmetric Tree* (LC 101) compares a tree to its mirror; *Subtree of Another Tree* (LC 572) asks whether one tree appears inside another.

Key Insight

Key takeaways: (1) "same" = root equal AND left same AND right same – the recursion mirrors the definition; (2) the null-pair base case is the entire difficulty, so enumerate all

four cases; (3) structure and values are both checked for free by recursing on the *pair*.

1.1.9 Practice Problems

Stuck? Work through this list

- Did you write the base case *first*? One tree null and the other null is "same"; exactly one null is "different" – three of the four cases below fail on this pair alone.
- Are you comparing *shape* as well as values? `a->val == b->val` alone accepts different structures.
- Draw two 3-node trees that differ only in shape and run your code on them by hand.
- On an adjacency list: did you pass the parent guard (Code 1.1.1)?

Q1.1.1 Invert Binary Tree – LeetCode 226 LC Easy L1

Hint: swap the two children at every node – a one-line post-order recursion.

Q1.1.2 Find Center of Star Graph – LeetCode 1791 LC Easy L1

Hint: no recursion needed: the centre is the node appearing in every edge (degree $n-1$).

Q1.1.3 Univalued Binary Tree – LeetCode 965 LC Easy L1

Hint: check every node equals the root; pass the expected value down in a DFS.

Q1.1.4 Leaf- Similar Trees – LeetCode 865 LC Easy L2

Hint: collect leaf values left-to-right with a DFS, then compare the two lists.

Q1.1.5 Minimum Distance Between BST Nodes – LeetCode 783 LC Easy L2

Hint: Stretch: an in-order walk of a BST is sorted – compare neighbours. See Chapter 7.

10-second refresher

Self-test (answer from memory, tick the box):

- ☐ A tree on n nodes has _____ edges, and between any two nodes there are _____ simple paths.
- ☐ Why does the adjacency-list DFS need `if (to == p) continue;`? What breaks without it?
- ☐ "$m = n - 1$" alone does not prove a graph is a tree – give the counterexample.
- ☐ Same Tree: which three null-pair cases must the base case separate?
- ☐ Write the `isSameTree` skeleton from memory, base case first.

A cycle is a _____; a forest drops the _____ requirement; the lockstep pattern visits corresponding nodes of two trees in the same order.

1.2 Depth, Height and Size

Previously in this chapter . . .

In §1.1 we earned the family vocabulary (root, parent, child, leaf, subtree) and the two theorems every tree algorithm leans on: n nodes $\to n-1$ edges, and one unique path per pair. We also met the parent-guarded DFS of Code 1.1.1. Keep both in hand – this section attaches *numbers* to that vocabulary.

1. A tree on n nodes has ____ edges, and *any* two nodes are joined by _____ simple path(s). *(Ans: $n-1$; exactly one – Theorem 1.1.5.)*
2. In the running example of Figure 1.1.4, is D an ancestor of B, a descendant of B, or a sibling of C? *(Ans: a descendant of B; D and C are not siblings – their parents differ.)*
3. Why does the DFS of Code 1.1.1 pass the parent p to every recursive call? *(Ans: adjacency edges are undirected, so without skipping p the walk bounces back and never terminates.)*

In This Section

Three numbers describe every node of a rooted tree: how far it is from the root (**depth**), how far the deepest leaf below it is (**height**), and how many nodes hang beneath it (**size**). Nearly every tree algorithm is "compute these bottom-up, then combine them," so they are the first tools to make automatic.

Prerequisites. §1.1 (parent, child, subtree) and the DFS of Code 1.1.1.

After this section you can

- state depth, height and size for every node of a small tree by hand;
- compute all three for the whole tree in a single $O(n)$ DFS;
- write the height recursion correctly under both the edges and nodes conventions;
- avoid the stack overflow that a skewed tree causes on recursive solutions.

Pattern card – top-down vs bottom-up measurement

Recognition keywords. “depth / level / distance from root” (carry a parameter *down*); “height / deepest / how many below / subtree total” (return a value *up*). Almost every “compute something for every node” problem is one of these two.

✓ When to use this

- the answer at a node depends only on its *ancestors* ⟶ top-down (depth, path prefixes, root-to-leaf sums);
- the answer at a node depends on its *descendants* ⟶ bottom-up (height, size, subtree sums, any max / min /sum over a subtree);
- both at once – one DFS can carry depth down and return height/size up (Code 1.2.2).

✗ When NOT to use this

- the quantity needs *sibling* subtrees combined at a node and a global best – that is diameter-style logic (Chapter 12) or tree DP (Chapter 13);
- queries mix updates and subtree reads – Euler tour + a data structure (Chapter 4, Chapter 19);
- a plain traversal suffices (visit/print/count all) – BFS/DFS of Chapter 3.

One-line test: *“does the node’s value flow from the root, or bubble up from the leaves?” The*

direction picks the recursion.

1.2.1 Intuition

Root the tree of Figure 1.1.4 at A and imagine two rulers. One hangs from the root downward and reads the *depth* of each node; the other stands on each node and reaches down to the deepest leaf below it, reading the *height*.

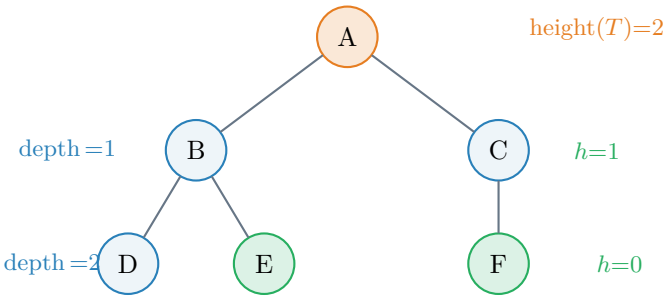


Figure 1.2.1: Blue shows *depth* (measured down from the root, in edges); green shows *height* of a few nodes (edges to the deepest leaf below them). The whole tree has height 2, the height of its root A. Leaves have height 0.

Depth vs Height

Depth depth(v)
edges from the *root* down to v; shared by the walk from the top

Height height(v)
edges from v down to its deepest *leaf*; computed from the bottom up

1.2.2 Definitions and Notation

Definition 1.2.1 (Depth and level)

The **depth** depth(v) of a node is the number of edges on the path from the root to v; the root has depth $= 0$. The **level** of a node is its depth (some books call the root level 1; we always start levels at 0). Siblings share a depth.

In plain words. Depth answers "how many steps down from the boss am I?" – the boss herself counts as 0. Think generations in a family photo: same generation, same depth, always.

Definition 1.2.2 (Height)

The **height** height(v) of a node is the number of edges on the longest path from v down to a leaf in its subtree; a leaf has height $= 0$ and the empty child has height(`null`) $= -1$. The height of the tree is height(T) $=$ height(root).

In plain words. Height answers the opposite question: "looking *down*, how long is the deepest chain I manage?" A leaf manages nothing, so 0. The odd-looking height(`null`) $= -1$ is not philosophy – it is the one convention that makes the max formula of Theorem 1.2.4 compute leaves correctly all by itself, with no special case in your code.

Definition 1.2.3 (Size)

The **size** size(v) is the number of nodes in subtree(v), counting v itself; so size(`null`) $= 0$ and $\sum$ of nothing is 0.

In plain words. Size is a headcount inside your own department – and you count yourself. An empty department has nobody to count: size(`null`) $= 0$.

1.2.3 The Bottom-Up Recurrences

Height and size both "bubble up" from the leaves: a node reads the answers its children computed and folds them together. This is the pattern behind half the book.

Theorem 1.2.4 (Height and Size Recursion)

With height(`null`) $= -1$ and size(`null`) $= 0$, for any node v

$$\text{height}(v) = 1 + \max_{c \in \text{children}(v)} \text{height}(c), \qquad \text{size}(v) = 1 + \sum_{c \in \text{children}(v)} \text{size}(c).$$

A leaf (children(v) $= \emptyset$) gets height $= 0$ and size $= 1$.

Proof

The longest downward path from v must leave v through exactly one child, then follow the longest path from that child; that is $1 + \max_c \text{height}(c)$, and if there is no child the max over the empty set is height(`null`) $= -1$, giving height(v) $= 0$. Every node of subtree(v) is either v itself or a node of exactly one child's subtree, so the counts add to $1 + \sum_c \text{size}(c)$. Both are well-founded because a tree has no cycles, so recursion reaches a leaf in finitely many steps.

■

Key Insight

Depth is a *top-down* quantity (carry it into the recursion as a parameter), while height and size are *bottom-up* (returned from it). One DFS can do all three: push depth in, return height and size out.

Where this is used. Chapter 8 (AVL balance factors), Chapter 12 (diameter) and Chapter 24 (centroid decomposition) – this same "compute a value per node bottom-up, combine children" move returns again and again with a different local formula.

1.2.4 One DFS that Measures Everything

Code 1.2.1: Height of a binary tree (height in edges, height(null) = -1)

```
// height in EDGES: null -> -1, leaf -> 0.  (See the note below for the nodes version .)
int height(TreeNode* r) {
    if (r == nullptr) return -1;
    return 1 + max(height(r->left), height(r->right));
}
```

Code 1.2.2: One DFS that fills depth, height and size for a general tree

```cpp
vector<vector<int>> adj;
vector<int> depth_, height_, size_;

void measure(int v, int p, int d) {          // p = parent, d = depth of v
    depth_[v]  = d;
    size_[v]   = 1;                           // count v itself
    height_[v] = -1;                          // a leaf ends at -1 + 1 = 0
    for (int to : adj[v]) {
        if (to == p) continue;                // parent guard (see 1.1)
        measure(to, v, d + 1);                // push depth down
        size_[v]   += size_[to];              // pull size up
        height_[v]  = max(height_[v], height_[to]);
    }
    height_[v] += 1;                          // add the edge to the deepest child
}
```

Complexity

Each node is visited once and each edge twice, so the DFS is $O(n)$ time and $O(n)$ memory (arrays plus a recursion stack up to the height of the tree). On a path of n nodes the stack is $O(n)$ deep – see the pitfall below.

Edges vs nodes: an off-by-one you must choose

This book counts height/depth in **edges**: height(`null`) $= -1$, a lone node has height 0. LeetCode's *Maximum Depth* (Code 1.2.5 below) counts in **nodes**: height(`null`) $= 0$, a lone node has depth 1. The two answers differ by exactly one. Read the statement, pick the base case, and never mix them in one program.

1.2.5 Worked Examples

Example 1.2.5 (Reading all three numbers)

In Figure 1.2.1: D has depth 2, height 0, size 1; B has depth 1, height 1, size 3; A has depth 0, height 2, size 6. Note size$(A) = 6$ equals the node count, and height(A) is the tallest root-to-leaf run of edges ($A \to B \to D$ or $A \to B \to E$).

Step-by-Step Trace

Running of Code 1.2.2 on a path $0 - 1 - 2$ rooted at 0 fills depth $\{0, 1, 2\}$, height $\{2, 1, 0\}$ and size $\{3, 2, 1\}$. Height grows from the far leaf back to the root, size collapses onto the root – the two recurrences running in opposite directions along the same walk.

1.2.6 Variations and Twists

- **Direction flip.** Every measurement here has a top-down twin: depth is carried *in*, height is returned *out*. LC 1123 (LCA of deepest leaves) is the bottom-up height recursion with an LCA twist – Chapter 16. **L3**
- **Count the level, not the edge.** Replace max by "which level has the most nodes" and the same DFS becomes level statistics – width questions (*Maximum Width of Binary Tree*, LC 662) index nodes by position and run on the same rails. **L3**
- **Two quantities, one DFS.** The stretch problem below (LC 2265) needs *size and sum*

returned together – the exact shape of Code 1.2.2, one field richer. This is the whole gateway into tree DP (Chapter 13): a state per node, folded bottom-up. L4

- **The iterative demand.** On judges with $n \leq 10^5$ chain-shaped trees every entry here needs a stack machine version; see the hardening note under the stretch solution. L3–L4
- **Disguise.** "Time needed to inform all employees" (LC 1376) is a general tree with depths; "deepest leaves sum" is depth-bucketed accumulation. Spotting *it is just depth* is the entire solve.

1.2.7 Common Pitfalls

Recursion on a skewed tree

A tree that is really a path of 10^5 nodes makes the DFS recurse 10^5 deep and can overflow the call stack. Prefer an explicit stack (iterative post-order) or raise the stack limit when the judge allows. Every recursive tree routine in this book gets an iterative twin for exactly this reason.

Off-by-one on the base case

Writing height(`null`) $= 0$ when the problem wants edges (or vice-versa) shifts every answer by one. Decide the convention *first*, set the null base case to match, and test on a single-node tree – the fastest way to catch it.

The traps as code.

Code 1.2.3: WRONG – mixed conventions: LC-104 statement (nodes) with the edges base case

```
int maxDepth(TreeNode* r) {
  if (!r) return -1;   // edges base ...
  return 1 + max(maxDepth(r->left),
                 maxDepth(r->right)); // ... +1 per node!
}
```

Code 1.2.4: RIGHT – pick one convention and keep it (nodes here)

```
int maxDepth(TreeNode* r) {
  if (!r) return 0;                      // nodes: null = 0
  return 1 + max(maxDepth(r->left), maxDepth(r->right));
}
```

1.2.8 Problem Lab (fully solved)

Attempt it yourself first

Maximum Depth is L1 – write it cold in under 3 minutes, both conventions from memory. **Minimum Depth** is the one that eats interviews: get a sheet of paper, draw the three trees in its trap figure, and predict the answer *before* reading a line of code.

W1.2.1 Maximum Depth of Binary Tree

Source: LeetCode 104 LC Easy L1
Techniques: recursion, height of a binary tree

Maximum Depth

Given the root of a binary tree, return the number of *nodes* on the longest path from the root down to the farthest leaf. A single node has depth 1; an empty tree has depth 0.

➤ Understanding the Problem

This is the height of the tree counted in *nodes*, not edges – LeetCode's convention. So the answer is the edge-height of Theorem 1.2.4 plus one. The natural formulation is recursive: the deepest path through a node is one more than the deeper of its two children.

➤ Key Observations

Observation 1. Counting depth in nodes: $\text{depth}(v) = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$, with the base case $\text{depth}(\texttt{null}) = 0$.

Observation 2. This mirrors the height recursion exactly; only the base case changes (0, not -1).

➤ Solution

Recurse, returning 0 for a null child and $1 + \max(\text{left}, \text{right})$ otherwise.

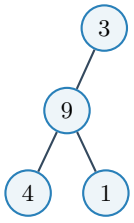


Figure 1.2.2: The longest root-to-leaf path is $3 \to 9 \to 4$ (or $3 \to 9 \to 1$), three *nodes*, so the answer is 3. Counting edges instead would give 2.

➤ Proof of Correctness

Induction on $\text{size}(v)$

Base: $v = \texttt{null}$, the code returns 0, correct. Step: assume the calls on the children return their correct node-depths. The deepest path from v goes to the deeper child and then adds the single node v, so $1 + \max$ is exactly the longest root-to-leaf node count at v – the same recurrence the theorem states, shifted by the base case.

■

➤ Implementation

Code 1.2.5: Maximum Depth (LeetCode 104)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (root == nullptr) return 0;                    // nodes convention
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

➤ Complexity

Complexity

Time $O(n)$ (every node visited once). Memory $O(h)$ recursion stack, worst-case $O(n)$ on a skewed tree; a level-order (BFS) count of how many levels there are gives the same answer iteratively with $O(w)$ queue space (w = widest level).

➤ Pitfalls and Variants

Edges vs nodes

Subtracting one here (to answer in edges) or adding one to an edge-height solution is the classic wrong answer. Re-read whether the problem counts nodes or edges before submitting.

Variant: *Minimum Depth* (W1.2.2) below; *Diameter of Binary Tree* (LC 543) uses the same bottom-up idea on two paths at once (Chapter 12).

Key Insight

Key takeaway: max-depth is Theorem 1.2.4 with the base case set to the convention the problem asks for. Master the single recursion, then change only the null-return between 0 (nodes) and -1 (edges).

W1.2.2 Minimum Depth of Binary Tree

Source: LeetCode 111 LC Easy L2
Techniques: recursion, the one-child trap

Minimum Depth

Return the number of nodes on the shortest path from the root to the nearest *leaf* (a node with no children). The path must end at a real leaf.

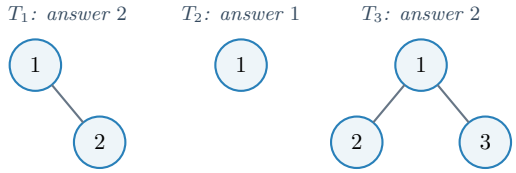


Figure 1.2.3: The three test trees. T_1 is the trap: a naive min over children (with the null child scoring 0) answers 1 – but the lone node 1 is *not* a leaf, the real answer is 2. Every wrong submission dies on T_1.

➤ Understanding the Problem

The trap: a naive $\min(\text{left}, \text{right})$ with $\text{depth}(\texttt{null}) = 0$ reports depth 1 for a root that has only a right child, because the missing *left* (a null, not a leaf) looks shortest. We must only compare branches that actually reach a leaf.

➤ Key Observations

Observation 1. A null child is *not* a leaf and must never win the min.

Observation 2. If a node has exactly one child, its minimum depth is 1 plus that child's minimum depth – the min has only one candidate.

➤ Solution

Three cases: null returns 0; a node missing one side recurses on the side it has; a node with both children takes $1 + \min$ of the two.

➤ Proof of Correctness

Induction on $\text{size}(v)$

For $v = \texttt{null}$ the answer is 0. If v has exactly one child c, the only root-to-leaf paths go through c, so the minimum is $1 + \text{depth}_{\min}(c)$, which the code returns. If v has two children, every leaf lies under one of them, and the shortest path picks the smaller of the two correct recursive answers plus v. In the remaining case (v is a leaf) both children are null, the two-child formula gives $1 + \min(0, 0)$ only if we let nulls score 0 – which is why we special-case missing children above instead.

■

➤ Implementation

Code 1.2.6: Minimum Depth (LeetCode 111)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int minDepth(TreeNode* root) {
        if (root == nullptr) return 0;
        if (root->left  == nullptr) return 1 + minDepth(root->right);  // only right
        if (root->right == nullptr) return 1 + minDepth(root->left);   // only left
        return 1 + min(minDepth(root->left), minDepth(root->right));   // both
    }
};
```

➤ Complexity

Complexity

Time $O(n)$, memory $O(h)$ recursion stack ($O(n)$ worst case). A BFS that returns as soon as it pops a leaf is often faster in practice (it stops at the first leaf) and is naturally iterative.

➤ Pitfalls and Variants

The one-child case is the whole problem

Collapsing to min(left, right) with nulls scoring 0 fails the moment a node has a single child – a huge fraction of the tests. Keep the two guard lines.

Variant: minimum depth to *any* node with a property (e.g. the nearest leaf matching a predicate) is the same BFS-with-early-exit pattern.

Attempt it yourself first

LC 2265 (next) asks for something you have never computed: "nodes whose value equals the average of their subtree." Try it – the wall you hit is the *reason* bottom-up pairs exist. Give yourself 15 minutes.

Peek (1 min lost): You are already computing size(v) in one DFS. What is the second quantity the average needs?

Nudge (5 min lost): Return a *pair* (sum, count) from each subtree, like Code 1.2.2 returns height and size. Integer division truncates toward zero – does that ever mis-round a true average?

Full hint (you will not solve this one): Floor-division kills you: $7/2 = 3$ but the true mean is 3.5. Check sum = count × val exactly – the node counts iff its value times the size equals the subtree sum.

W1.2.3 Count Nodes Equal to Average of Subtree

Source: LeetCode 2265 LC Medium L3
Techniques: bottom-up pair return, one DFS, truncation trap

Count Nodes Equal to Average of Subtree

Given the root of a binary tree, count nodes whose value equals the *floor of the average* of the values in their subtree (the node itself included). $1 \le n \le 1000$, $1 \le \text{val} \le 10^5$.

➤ Understanding the Problem

For every node we need its whole subtree's sum and size, then compare $\lfloor \text{sum}/\text{size} \rfloor$ with the node's value. "For every node, a subtree statistic" is the signature of the bottom-up return – exactly the machinery of Code 1.2.2 with richer payloads.

➤ Key Observations

Observation 1. depth-style top-down passes cannot answer this: a node's subtree average is built from its *descendants*, so information must flow *up*.

Observation 2. Sum and size are both foldable in one DFS: $\text{sum}(v) = \text{val}(v) + \sum_c \text{sum}(c)$, $\text{size}(v) = 1 + \sum_c \text{size}(c)$ – the pair returns together.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** one DFS to fill arrays, then a second loop averaging each subtree by re-walking it **Killed by:** that is $O(n^2)$ on a chain – passes for $n \le 1000$, but it teaches nothing and dies at 10^5.

✗ **Tried:** computing sum bottom-up but doing the division at the end (sum/size with ints) **Killed by:** floor division: a subtree $\{3, 4\}$ gives $7/2 = 3$, and if the root value is 3 you count it – but $\lfloor \text{mean} \rfloor$ *is* 3, so here it is right; the real killer is thinking val = sum/size is equivalent to val · size = sum. It is *not*: $\{3, 4, 4\}$ has mean 3.67, floor 3, yet $3 \cdot 3 \neq 11$.

✓ **Aha!** return pair(sum, size) up the recursion and test sum/size == val using the *same* integer division the problem defines. The division is part of the *specification*, not our shortcut – so the floor question vanishes.

➤ The Approach Ladder

Approach ladder – Count Nodes Equal to Average of Subtree

	Approach	Why it wins / dies	Cost	Verdict
L2	recompute each subtree's average by walking it	correct but quadratic on chains ($n \geq 10^5$ dies); no reusable skill	$O(n^2)$	TLE
L3	one DFS returning pair(sum, size); test at each node	both stats fold bottom-up for free	$O(n)$ / $O(h)$	3 Accepted
L4	same DFS, but test val · size = sum "to avoid division"	silently changes the spec (floor vs exact mean) – WA on $\{3, 4, 4\}$	$O(n)$	WA

➤ Solution

One post-order DFS. Each call returns pair(sum, size) of its subtree; on the way back up, after both children have returned, the node computes its own pair, and increments the global answer when sum/size (integer division, exactly as the judge does it) equals its value.

➤ Proof of Correctness

Induction on size(v)

For $v =$ `null` the returned pair is $(0, 0)$, correct. For v non-nil, by the induction hypothesis the two children return the correct (sum, size) of their subtrees; the node adds its own value and one node, giving the correct pair for subtree(v). The node is counted exactly when the spec's condition $\lfloor \text{sum}/\text{size} \rfloor = \text{val}$ holds, which the code tests with the same integer division the judge uses. Summing these per-node decisions yields the required count.

■

➤ Implementation

Code 1.2.7: Count Nodes Equal to Average (LeetCode 2265)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
    int ans = 0;
    // returns {subtree_sum, subtree_size}
    pair<long long,int> dfs(TreeNode* v) {
        if (!v) return {0, 0};
        auto [sl, cl] = dfs(v->left);
        auto [sr, cr] = dfs(v->right);
        long long sum = sl + sr + v->val;
        int   cnt = cl + cr + 1;
```

```
19          if (sum / cnt == v->val) ++ans;   // floor div == the spec
20          return {sum, cnt};
21      }
22  public:
23      int averageOfSubtree(TreeNode* root) {
24          ans = 0;
25          dfs(root);
26          return ans;
27      }
28  };
```

➤ Trace

Step-by-Step Trace

Tree $\{3,4,4\}$ (root 3, two leaf children $4,4$): each leaf returns $(4,1)$ and is counted $(4/1=4)$. The root sums to $3+4+4=11$, count 3, $11/3=3$ (integer) = root value, so it is counted too – answer 3. Note the WRONG val $\cdot$ size = sum test would reject the root ($3\cdot 3\neq 11$).

➤ Complexity

Complexity

Time $O(n)$ – one visit per node, $O(1)$ combine. Memory $O(h)$ stack. Sum in `long long`: at $n=1000$, val $\leq 10^5$ the bound is 10^8 – safe even in `int`, but the habit is what saves you at $n=10^5$ in the next problem.

➤ Verification and Edge Cases

- single node $\{5\}$: sum/size = 5 – counted. **L3**
- chain of equal values: every subtree averages to that value – answer n.
- floor trap from the thinkbox: $\{3,4,4\}$ root 3, mean $11/3$ floor 3 – counted (integer div is the *spec*); the WRONG val $\cdot$ size = sum test rejects it – that is the **L4** WA rung.

➤ Pitfalls and Variants

Truncation is the specification

Do not "improve" the floor division into an exact-mean test. The problem defines the average as integer floor; matching the judge's rounding is part of being correct.

Key Insight

Key takeaways: (1) "for every node, a subtree statistic" $\Rightarrow$ return a *pair* bottom-up in a single DFS; (2) the naive re-walk is $O(n^2)$ – the pair return makes it $O(n)$; (3) reproduce the spec's rounding, do not substitute a "nicer" test. This pair-return is the doorway to tree DP (Chapter 13).

AC $\longrightarrow$ red: hardening this code

L4 stress: n up to 10^5 as a chain – the $O(h)$ stack overflows on many judges. Convert to the explicit-stack post-order twin (pairs stored in a parallel array, exactly the Code 1.2.2 bookkeeping). **L5 variant:** same pair return but the average is over the *path to root*

(top-down accumulator, no pairs needed) – practise flipping direction on purpose. **L5 stretch:** count nodes equal to the *median* of their subtree – pairs are no longer enough; that is DSU-on-tree territory (Chapter 11).

1.2.9 Practice Problems

Stuck? Work through this list

- Test your base case on a *single node*: is the answer 0 (edges) or 1 (nodes)? Convert conventions explicitly (1.2 convention note).
- "Minimum depth" on a one-child node: a null child is *not* a leaf – ignoring this gives the classic wrong answer.
- Skewed tree of 10^5 nodes: will your recursion survive? If not, convert to an explicit stack before debugging anything else.
- For diameter-style problems, separate the two quantities: what you *return* (best downward path) vs what you *update globally* (best path through a node).

Q1.2.1 Sum of Left Leaves – LeetCode 404 LC Easy L2

Hint: recurse, passing a flag for "I am a left child"; add the value when a leaf is left.

Q1.2.2 Tree Diameter (Longest Path) – CSES 1131 CSES L3

Hint: two BFS/DFS sweeps: farthest node from an arbitrary start, then farthest from that. See Chapter 12.

Q1.2.3 Diameter of Binary Tree – LeetCode 543 LC Easy L2

Hint: bottom-up: return height, update a global best with left-height + right-height.

Q1.2.4 Binary Tree Longest Consecutive Sequence – LeetCode 298 LC Medium L3

Hint: Stretch: carry the current run length down in a DFS; compare both children, do not branch-max naively.

10-second refresher

Self-test (answer from memory, tick the box):

☐ With the edges convention, height(`null`) = _____ and a lone node has height _____.
☐ Which quantities flow *down* the recursion (carried in) and which bubble *up* (returned)?
☐ Min-depth dies on which specific tree shape? Draw it.
☐ In LC 2265 why is testing val · size = sum a WA against the floor-average spec?
☐ Write the one-DFS ⟨height, size⟩ measurement skeleton from memory, base case first.

One DFS fills depth, height and size by pushing depth in and returning ⟨height, size⟩ out.

1.3 Kinds of Trees

Previously in this chapter . . .

So far: the family vocabulary and the two theorems ($n-1$ edges, unique path), and one DFS that pushes *depth* in and returns *height* and *size* out (Code 1.2.2). This section asks the next question – what happens when the shape itself is *promised* to you – and every promise turns into a formula.

1. With the edges convention, height(`null`) = ____ and a lone node has height ____. *(Ans: -1; 0.)*
2. In the running example of Figure 1.2.1, what are height and size of the root A? *(Ans: height$(A) = 2$, size$(A) = 6$.)*
3. Which quantity is carried *down* the recursion and which are returned *up*? *(Ans: depth down; height and size up.)*

In This Section

"Tree" hides a family of shapes, and each extra restriction buys a counting rule or a faster algorithm. Knowing whether your tree is *full*, *complete*, *perfect* or *balanced* tells you its height from its size, whether an array can store it, and which problems become one line instead of a recursion.

Prerequisites. §1.2 (height, size) and Theorem 1.1.4.

After this section you can

- recognise and name binary, full, complete, perfect, balanced and skewed trees;
- never confuse "full", "complete" and "perfect" again;
- derive the node and leaf counts of perfect and full trees;
- bound the height of any binary tree with n nodes by $\lfloor \log_2 n \rfloor$ and $n-1$.

Pattern card – exploit the shape promise

Recognition keywords. “complete”, “perfect”, “full”, “balanced”, “height-balanced”, “every level filled except possibly the last” – any word that *promises* a shape.

✓ When to use this

- the problem guarantees a shape: use its counting rule instead of traversing (perfect subtree $= 2^h - 1$ nodes, Theorem 1.3.7; the $O(\log^2 n)$ node count below is this idea at work);
- the problem *asks* whether the shape holds (balanced, symmetric): a bottom-up check that returns per-node data and aborts early;
- a height bound lets you size arrays/stacks: $\lfloor \log_2 n \rfloor \le h \le n-1$ (Corollary 1.3.9).

✗ When NOT to use this

- no promise is stated – never *assume* completeness; a hidden skewed test case will break $O(\log n)$ logic;
- the task is to *make* the tree balanced under updates – that is a real self-balancing tree (Chapter 8);
- the shape is arbitrary but you need subtree aggregates – tree DP (Chapter 13).

One-line test: *find the sentence that promises the shape, then ask what it makes free: a formula, a bound, or a halving argument.*

1.3.1 Intuition

A **binary tree** restricts every node to at most two children, conventionally called `left` and `right`. That single rule is what makes tree algorithms branch in two and lets us store a tree in an array. The other shapes in this chapter are binary trees with one more promise each.

Single-elimination tournament

A *perfect* binary tree is a knockout bracket: two players per match (the children), the winners fill the next round (level) completely, and the number of matches is fixed by the number of players. Cut the last round short and you get a *complete* tree; let someone get a bye and it is no longer complete. The bracket picture is why the counts come out as powers of two.

1.3.2 Definitions and Notation

Definition 1.3.1 (Binary and m-ary tree)

A **binary tree** is a rooted tree in which each node has at most two children, named `left` and `right` (the two slots are distinguished, so a node with one child keeps track of whether it is the left or the right one). An **m-ary tree** allows at most m children; a general tree is an m-ary tree with unbounded m.

In plain words. Two seats per node, and the seats are *named*: "one left child" and "one right child" are *different* binary trees. This one nuisance decides inorder's meaning and half the reconstruction puzzles of Chapter 3.

Definition 1.3.2 (Skewed (degenerate) tree)

A binary tree is **skewed** (or **degenerate**) when every internal node has exactly one child, so the tree is really a linked list: all-left or all-right. A skewed tree on n nodes has height $n - 1$ – the worst case for any recursive tree code.

In plain words. A tree that forgot to branch – it is a linked list in disguise. Whenever your solution is "$O(n)$ time but recursion depth n", this shape is the judge's test case that kills it.

Definition 1.3.3 (Full (proper) binary tree)

A binary tree is **full** (or **proper**, or strict) when every internal node has *exactly two* children – a node has 0 or 2 children, never 1.

In plain words. No loners: every node you meet has either no children at all or exactly two. One is not on the menu.

Definition 1.3.4 (Perfect binary tree)

A binary tree is **perfect** when every internal node has two children *and* all leaves are at the same depth. It is a completely filled bracket of height h.

In plain words. Fully packed, all the way down – level ℓ holds exactly 2^ℓ nodes with zero gaps. The pyramid is perfect.

Definition 1.3.5 (Complete binary tree)

A binary tree is **complete** when every level except possibly the last is full, and the last level is filled left to right with no gaps. Every perfect tree is complete; the converse needs the last level to be full.

In plain words. Cinema filling: every row packed left-to-right with no empty seat in front of an occupied one; only the last row may be half full. This is exactly the shape an *array* can store without wasting slots – which is why heaps (Chapter 10) and segment trees promise it.

Definition 1.3.6 (Height-balanced tree)

A binary tree is **(height-)balanced** when for every node the heights of its left and right subtrees differ by at most 1. Balancedness is what keeps the height logarithmic so searches stay fast; it is the property Chapters 5–6 fight to maintain.

In plain words. No node may lean too hard: at every fork, one side may deepen past the other by at most *one* level. That tiny local promise keeps the global height near $\log_2 n$ – the difference between instant searches and walking a list.

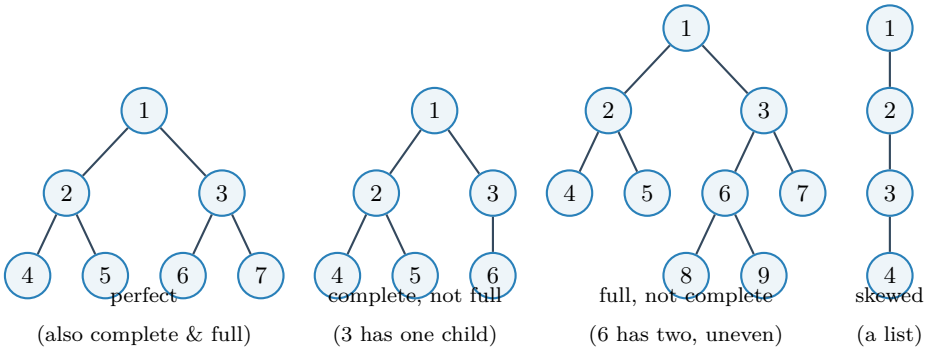


Figure 1.3.1: Four shapes, four promises. Read the boxes under each to see which definition the tree satisfies and which it breaks.

Full vs Complete vs Perfect

Full / Perfect
full: every internal node has two children; perfect adds "all leaves level"

Complete
every level filled left-to-right, last level may be short but has no gaps

1.3.3 Counting Nodes and Leaves

Each extra restriction pins down the size. These identities are the ones interviewers expect cold.

Theorem 1.3.7 (Perfect Tree Counts)

A perfect binary tree of height h (counted in edges) has $2^{h+1} - 1$ nodes, 2^h leaves, and 2^ℓ nodes on level ℓ (levels numbered from 0).

Proof

Level ℓ doubles each step (each node has two children), so it holds 2^ℓ nodes; levels run $0..h$, giving $\sum_{\ell=0}^{h} 2^\ell = 2^{h+1} - 1$ nodes by the geometric series. The only nodes with no children are those on the last level h, so leaves $= 2^h$. ■

Key Insight

Invert it: a perfect tree with n nodes has $h = \log_2(n+1) - 1$, so $n+1$ is a power of two. This is why "is it perfect?" can be answered by counting levels, never by pointer chasing.

Theorem 1.3.8 (Leaves of a Full Binary Tree)

In a full binary tree with I internal nodes and L leaves, $L = I + 1$.

Proof

Count edges two ways. Every internal node has exactly two children, contributing two parent-to-child edges, so the tree has $2I$ edges. But any tree on $n = I + L$ nodes has $n - 1$ edges (Theorem 1.1.4). Therefore $2I = (I + L) - 1$, i.e. $L = I + 1$. ■

Quick feel. Start with one node: one leaf, zero internal – the leaves lead by exactly one. Every split converts a leaf into an internal node that sprouts *two* new leaves: leaves $-1 + 2$, internal $+1$ – the gap of one never changes. The proof above counts the same phenomenon through edges.

Corollary 1.3.9 (Height Bounds of a Binary Tree)

A binary tree with n nodes has height (in edges) $\lfloor \log_2 n \rfloor \le$ height $\le n - 1$. The lower bound is attained by a perfect (or complete) tree, the upper bound by a skewed tree.

Proof

A tree of height h holds at most $2^{h+1} - 1$ nodes (a perfect tree), so $n \le 2^{h+1} - 1 < 2^{h+1}$, giving $h > \log_2 n - 1$, i.e. $h \ge \lfloor \log_2 n \rfloor$. On the other hand each edge can increase the longest root-to-leaf path by at most one, and there are only $n - 1$ edges, so $h \le n - 1$. Both extremes are the perfect and skewed trees above. ■

Key Insight

"Balanced means height $O(\log n)$." Keeping h near its lower bound is the entire point of AVL and red–black trees (Chapter 8): it turns $O(n)$ lists into $O(\log n)$ searches.

Where this is used. Chapter 8 (self-balancing BSTs), Chapter 10 (heaps in an array) and Chapter 19 (segment trees) – complete and perfect trees are exactly the ones an array stores with no wasted slots, and balanced trees are the ones whose height stays $O(\log n)$.

1.3.4 Worked Examples

Example 1.3.10 (Reading the gallery)

In Figure 1.3.1 the third tree has internal nodes $\{1, 2, 3, 6\}$ ($I = 4$) and leaves $\{4, 5, 8, 9, 7\}$ ($L = 5$): indeed $L = I + 1$, matching Theorem 1.3.8. The first is perfect with $h = 2$, so $2^3 - 1 = 7$ nodes and $2^2 = 4$ leaves – count them to confirm.

Step-by-Step Trace

For $n = 6$ nodes, Corollary 1.3.9 says $\lfloor \log_2 6 \rfloor = 2 \le h \le 5$. A complete tree of six nodes has height 2 (near the best), the skewed tree has height 5 (the worst) – both inside the band.

1.3.5 Variations and Twists

- **Verify the promise.** Instead of using completeness, *test* it: LC 958 (Check Completeness) is level-order with a "once you see a gap, only leaves allowed" flag. **L2**
- **Array-shaped trees.** A complete tree lives in an array at indices $1..n$ with children at $2i, 2i+1$ – the address arithmetic behind heaps (Chapter 10) and segment trees (Chapter 19). Every "pointer" problem here has an array twin; find it as practice. **L3**
- **Balanced is a spectrum.** Height-balanced (AVL, **L3** preview Chapter 8) versus weight-balanced versus merely "the depth difference of any two leaves is at most 1" – read which definition the statement uses; they are not equivalent. **L4**
- **m-ary generalisation.** Perfect m-ary trees count as $\sum_{\ell} m^{\ell} = (m^{h+1} - 1)/(m - 1)$ – the same geometric series with a different ratio; K-ary heaps (the same array arithmetic as Chapter 10) and K-ary tries reuse it. **L3**
- **Disguise.** "Tournament bracket", "file-system block", "heap snapshot", "memory segment layout" – all complete/perfect vocabulary. The recognition sentence to hunt for: “every level except possibly the last is completely filled”.

1.3.6 Common Pitfalls

Confusing full / complete / perfect

They are not synonyms and not even comparable in one direction: a complete tree need not be full (a last-level parent can have one child), and a full tree need not be complete (uneven depth). "Perfect" is the only one that is both. Sketch a two-level and a three-level example of each before you assert a property.

Balanced is per-node, not just root

Checking the height difference at the root only is not enough – W1.3.1 has test cases where the root looks balanced but a deep subtree is not. The condition must hold for *every* node.

The trap as code.

Code 1.3.1: WRONG – root-only balance test

```
bool isBalanced(TreeNode* r) {
  if (!r) return true;
  return abs(height(r->left) - height(r->right)) <= 1;
```

```
}
```

Code 1.3.2: RIGHT – per-node check via the −1 sentinel (first solved problem)

```
bool isBalanced(TreeNode* r) { return check(r) != -1; }
// check() tests |lh-rh|<=1 at EVERY node while
// returning the height it already computed
```

1.3.7 Problem Lab (fully solved)

Attempt it yourself first

Both problems below are top-**L3** interview staples. Do **Balanced** with the single-pass sentinel from memory (if you wrote two recursive functions, you built the **L1** rung, not the **L2** one), then attempt **LC 222** cold – the question "what does *complete* let me skip?" is the entire solve.

W1.3.1 Balanced Binary Tree

Source: LeetCode 110 LC Easy L2
Techniques: bottom-up height, early exit, sentinel

Balanced Binary Tree

Given a binary tree, decide whether it is height-balanced: for every node, the heights of its two subtrees differ by at most one.

➤ Understanding the Problem

The naive plan – a full height computation at every node – is $O(n^2)$ (height is $O(n)$ and we call it n times). We can do one bottom-up pass that returns each node's height and simultaneously checks its balance, reusing every child's height instead of recomputing it.

➤ Key Observations

Observation 1. If a subtree is unbalanced the whole tree is unbalanced, so we can abort early by propagating a special "unbalanced" signal upward.

Observation 2. Returning −1 as that signal works because a real height is never negative (with the null = 0 nodes convention here).

➤ The Approach Ladder

Level	Approach	Idea	Time	Verdict
L1	Recompute height at every node	definition straight from the book	$O(n^2)$	✗ TLE
L2	Single-pass bottom-up with −1 sentinel	reuse each child's height	$O(n)$	✓ Accepted

Rung 1 is correct and worth saying out loud in an interview – then collapse it to rung 2 by noticing that `height` is recomputed on subtrees that were already measured.

➤ Solution

A recursive `check` returns the height of a balanced subtree, or -1 if that subtree is unbalanced. A node is balanced iff both children returned non-negative and their heights differ by at most one.

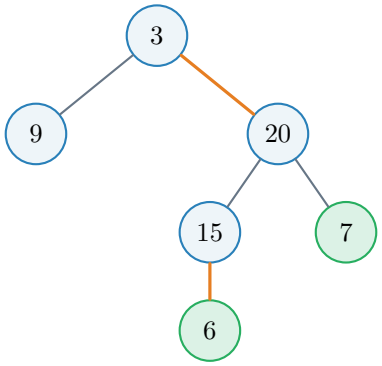


Figure 1.3.2: Not balanced: at node 20 the left subtree (height 2 through the orange edges down to 6) and the right subtree (height 0) differ by more than one, even though the root itself looks fine.

➤ Proof of Correctness

Induction on size(v)

By Theorem 1.2.4 the returned height is correct whenever the subtree is balanced. If a child returns -1, its subtree violates the condition at some node (induction hypothesis) and v correctly propagates -1. Otherwise v's subtree is balanced exactly when both children's subtrees are (induction) *and* v itself satisfies $|\,\text{height(left)} - \text{height(right)}| \le 1$, which the code tests. The root's return is negative iff the whole tree is unbalanced.

■

➤ Implementation

Code 1.3.3: Balanced Binary Tree (LeetCode 110)

```cpp
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int check(TreeNode* r) {                // height if balanced, -1 if not
        if (r == nullptr) return 0;
        int lh = check(r->left);   if (lh == -1) return -1;
        int rh = check(r->right);  if (rh == -1) return -1;
        if (abs(lh - rh) > 1) return -1;
        return 1 + max(lh, rh);
    }
    bool isBalanced(TreeNode* root) { return check(root) != -1; }
```

```
20 };
```

➤ Trace

Step-by-Step Trace

On Figure 1.3.2: `check(6)` returns 0, so `check(15)`$= \max(0, \text{null } 0) + 1 = 1$; `check(7)`$= 0$; at 20, $|1 - 0| = 1$ ok, returns 2; at 9, both null, returns 0; at root 3, $|0 - 2| = 2 > 1$ returns -1. The root return is negative, so `isBalanced` is `false`.

➤ Complexity

Complexity

Time $O(n)$: each node is visited once and its children's heights are reused, not recomputed (versus $O(n^2)$ for the root-only-height approach). Memory $O(h)$ stack, worst case $O(n)$.

➤ Pitfalls and Variants

Two traversals double the cost

Writing `isBalanced` to call a separate `height` on every node is correct but $O(n^2)$. The single-pass sentinel version above is what interviewers want to see.

Variant: *Minimum Absolute Difference in BST* or any per-node property uses the same "return a value, abort early" bottom-up shape.

Key Insight

Key takeaway: fold two questions into one DFS – *return* the height and use an impossible value (-1) as the "already broken" signal. The sentinel early-abort is the reusable pattern for every per-node promise.

W1.3.2 Count Complete Tree Nodes

Source: LeetCode 222 LC Medium L3
Techniques: complete-tree structure, perfect shortcut, divide and conquer

Count Complete Tree Nodes

Given the root of a *complete* binary tree, return the number of nodes. Aim for better than $O(n)$: the structure should let you skip whole perfect subtrees.

➤ Understanding the Problem

A plain DFS counting nodes is $O(n)$ and ignores the promise. In a complete tree, compare the length of the left spine to the right spine of a subtree: if they are equal, that subtree is *perfect* and holds $2^{\text{height}} - 1$ nodes (Theorem 1.3.7) – count it instantly. Otherwise recurse, but only down the side that actually changes.

➤ Key Observations

Observation 1. If the left-most and right-most paths from a node have the same length, the subtree is perfect and its size is a power of two minus one.

Observation 2. If they differ, exactly one child subtree is perfect, so each step halves the problem.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** plain DFS over all n nodes
Killed by: accepted – but the statement *promised* completeness and you paid nothing for it; the judge's follow-up question is "why is your answer $O(n)$?"

✗ **Tried:** counting the last level by BFS-ing to depth h
Killed by: the full tree above the last level has $2^h - 1 = O(n)$ nodes – visiting them to "find" the last level is the whole $O(n)$ you promised to avoid.

✓ **Aha!** some *subtrees* inside a complete tree are *perfect* – and a perfect subtree's size is a closed formula. Detect one by comparing the left and right spines, then never walk inside it again.

➤ The Approach Ladder

Level	Approach	Idea	Time	Verdict
L2	DFS counting every node	works, but ignores the shape promise	$O(n)$	✓ Accepted
L3	Perfect-subtree shortcut	spine test, $2^h - 1$ count, halve each step	$O(\log^2 n)$	✓ Accepted

Rung 1 is accepted by the judge yet defeats the purpose of the problem – rung 2 is the intended solution and is the one to present.

➤ Solution

Compare the left and right spine heights of the current subtree. Equal ⇒ perfect ⇒ return $2^{\text{spine}} - 1$; otherwise return 1 plus the counts of the two (still-complete) children.

Code 1.3.4: Count Complete Tree Nodes (LeetCode 222)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x = 0) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
```

```
    int leftHeight(TreeNode* n)  { int h = 0; while (n) { ++h; n = n->left;  } return
    ↪ h; }
    int rightHeight(TreeNode* n) { int h = 0; while (n) { ++h; n = n->right; } return
    ↪ h; }

    int countNodes(TreeNode* root) {
        if (root == nullptr) return 0;
        int lh = leftHeight(root), rh = rightHeight(root);
        if (lh == rh) return (1 << lh) - 1;              // perfect subtree: 2^lh - 1
        return 1 + countNodes(root->left) + countNodes(root->right);
    }
};
```

➤ Proof of Correctness

Proof

When the two spine lengths agree ($= h$) the complete tree of height h must have its last level full, hence perfect, so the count is $2^h - 1$ by Theorem 1.3.7. When they differ, the root contributes 1 and we recurse on the (still complete) children; correctness follows by induction on height, since every subtree of a complete tree rooted at a node is again complete.

■

➤ Trace

Step-by-Step Trace

A perfect tree of height 2 (7 nodes): at the root $lh = rh = 3$, so we return $2^3 - 1 = 7$ without recursing. A complete tree of 6 nodes: at the root $lh = 3 \neq rh$ (right spine shorter), so we return $1 + \text{count(left)} + \text{count(right)}$; the left child is perfect ($2^2 - 1 = 3$) and the right has 2, giving $1 + 3 + 2 = 6$.

➤ Complexity

Complexity

Spine lengths cost $O(\log n)$ each; we do this on one root-to-leaf path, so $O(\log^2 n)$ time – a large win over $O(n)$ when the tree is deep and full. Memory $O(\log n)$ stack.

➤ Verification and Edge Cases

- empty tree: returns 0. **L3**
- perfect tree: shortcut fires at the root – $O(\log n)$ total.
- $1 \ll lh$ with $lh = 31$: overflows `int`; use `1LL << lh` and check the node-count constraint first (see the hardeningbox).

➤ Pitfalls and Variants

$1 \ll lh$ overflow

For very tall trees 2^{lh} can exceed `int`; if n may be large use `long long` and $1 \ll lh$ only up to $lh < 31$. Contest node counts keep lh small, but check the constraint.

Variant: the same "measure both spines, shortcut perfect subtrees" idea finds the last (level-order) node or the insertion point in a heap-backed array.

Key Insight

Key takeaways: (1) a promise is a coupon – *complete* buys the perfect-subtree formula $2^h - 1$; (2) detect a perfect subtree in $O(\log n)$ by comparing the two spines; (3) each non-shortcut step halves the tree, giving $O(\log^2 n)$. Never assume completeness the statement did not give you.

AC ⟶ red: hardening this code

L4 sharpen: the $O(\log^2 n)$ above re-measures spines at every level. Walk down a *single* root-to-leaf path instead, deciding left-vs-right at each node by one spine comparison – $O(\log n)$, the textbook optimum for this problem. The move ("spend $O(\log n)$ tests to choose a side, then commit") is the same binary-search spirit you will meet in Chapter 19. **L4 stress:** $1 \ll lh$ with lh = 31: switch to `1LL « lh` and check the constraint table first.

1.3.8 Practice Problems

Stuck? Work through this list

- Write the promise as a formula first: perfect ⇒ $2^{h+1} - 1$ nodes; full ⇒ $L = I + 1$; complete ⇒ array storage (§1.3).
- Balanced checks fail most often because they test only the root – the condition is *per node*; return height and a sentinel together.
- If your solution never uses the promised shape, ask: which subtree can I count, skip, or halve in one step instead?
- $1 \ll h$ overflows for $h \geq 31$ – check the node-count constraint before using the perfect-subtree shortcut.

Q1.3.1 Symmetric Tree – LeetCode 101 LC Easy L2

Hint: compare a tree with its mirror: recurse on (left.left, right.right) and (left.right, right.left).

Q1.3.2 Check Completeness of a Binary Tree – LeetCode 958 LC Medium L2

Hint: BFS with a flag: after the first node with a missing child, every remaining node must be a leaf.

Q1.3.3 Binary Tree Level Order Traversal – LeetCode 102 LC Medium L2

Hint: BFS with a queue; process one whole level per iteration to fill the levels.

Q1.3.4 Convert Sorted Array to BST – LeetCode 108 LC Easy L2

Hint: pick the middle element as root, recurse on halves – builds a balanced tree. See Chapter 7.

Q1.3.5 Binary Tree Right Side View – LeetCode 199 LC Medium L3

Hint: Stretch: the last node of each BFS level; or a DFS that visits right before left.

Q1.3.6 Complete Binary Tree Inserter – LeetCode 919 LC Medium L3

Hint: Stretch: keep a queue of parents that still have a free slot to insert left-to-right.

10-second refresher

Self-test (answer from memory, tick the box):

- ☐ Perfect tree of height h (edges): nodes = _____, leaves = _____.
- ☐ Full tree with I internal nodes has _____ leaves – and the proof counts _____ two ways.
- ☐ Which shape promise lets you count nodes in $O(\log^2 n)$, and what exactly do you compare to spot a shortcut subtree?
- ☐ Give a tree that is full but not complete, and one that is complete but not full.
- ☐ Write the balanced-check sentinel `check` from memory, base case first.

Balanced means a *per-node* condition; the sentinel return -1 collapses "height" and "is-it-valid" into one $O(n)$ DFS – the same trick you will re-use in every early-exit bottom-up problem of the book.

1.4 Counting Trees

Previously in this chapter . . .

Shapes have now been promised, named and measured (§1.3); the perfect tree gave us $2^{h+1}-1$ and the full tree $L = I + 1$. This final section turns the question around: not *what* shapes exist, but *how many* – and the two answers (Catalan, Cayley) are the only counting facts in this book you must own cold.

1. A perfect tree of height h (edges) has _____ nodes and _____ leaves. *(Ans: $2^{h+1}-1$; 2^h.)*
2. A full tree with I internal nodes has _____ leaves. *(Ans: $I+1$ – Theorem 1.3.8.)*
3. In §1.3, the $O(\log^2 n)$ node count relied on detecting a _____ subtree by comparing _____. *(Ans: perfect; the left and right spine lengths.)*

In This Section

"How many trees are there?" has two very different answers depending on what you are counting: the number of *shapes* (Catalan numbers) and the number of ways to put *distinct labels* on those shapes (Cayley's n^{n-2}). Both show up as DP counts and as combinatorics problems, so it pays to know the two formulas and which question each answers.

Prerequisites. §1.3 (binary shapes) and basic binomial coefficients.

After this section you can

- count binary-tree shapes with the Catalan recurrence and closed form;
- state Cayley's formula and say what a labelled tree is;
- compute both modulo a prime without overflow;
- recognise a Catalan count hiding inside a DP problem (Chapter 13).

Pattern card – count, do not build

Recognition keywords. "how many different / structurally unique trees (BSTs)...", "number of shapes / labelled trees", often with "modulo 10^9+7".

✓ When to use this

- a count of *configurations*, never the configurations themselves: look for a root-split

recurrence $\sum_r G(r-1)G(n-r)$ – that shape *is* Catalan;
- "keys $1..n$ in a BST" – the labels are forced by in-order sorting, so BST count = binary-shape count $= C_n$;
- free (unrooted, unbinary) labelled trees on n vertices – Cayley n^{n-2}.

✗ When NOT to use this

- the problem asks you to *output* every tree (LC 95) – recursion building node lists, output-size-bound, not a DP over counts;
- the tree is given as input and you merely traverse it – no counting formula applies;
- the answer is not tree-shaped: split the "root first" question – if no choice of a root decomposes the rest, it is a different DP (Chapter 13).

One-line test: *"can I pick the root, multiply two independent halves, and sum over root choices?" If yes, you are counting Catalan objects.*

1.4.1 Intuition

An **ordered** binary tree distinguishes a left child from a right child, so a three-node tree can be arranged in five different shapes – one per way the single "extra" node can hang off a two-node skeleton.

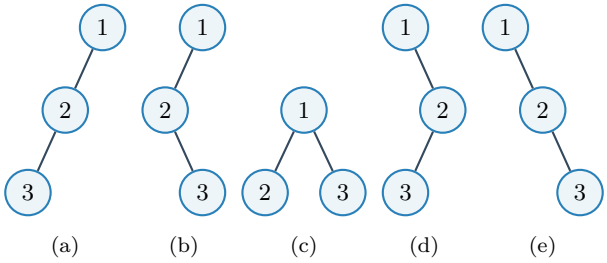


Figure 1.4.1: All five shapes of a three-node binary tree, so $C_3 = 5$. They differ only in which of the four slots the nodes occupy – empty slots are drawn with `[,phantom]`.

1.4.2 Definitions and Notation

Definition 1.4.1 (Shape vs labelled tree)

The **shape** of a rooted (binary) tree is the tree with its values ignored – only the parent–child connections matter. A **labelled tree** on n vertices keeps the shape *and* assigns the distinct names $1, 2, \ldots, n$ to the nodes. Two trees are equal as shapes iff an isomorphism maps one to the other preserving children; they are equal as labelled trees only if the labels match too.

In plain words. Rub out every number and look at the skeleton – that skeleton is the *shape*. Two trees with the same skeleton are the same shape no matter how different their values look. The separate question is *who wears which number*.

Definition 1.4.2 (Counted objects)

We count three related quantities for n nodes: the number of *binary-tree shapes* (ordered, values ignored); the number of *binary trees labelled* $1..n$ (a shape times the ways to fill it); and the number of *labelled trees* whose edges are chosen freely (not restricted to be binary, and undirected). Each has its own formula below.

In plain words. Three questions people constantly mix up: how do the skeletons arrange themselves (Catalan C_n), a fixed skeleton filled with numbers (shapes $\times n!$, or forced to 1 if it must stay a BST), and edges chosen freely (n^{n-2}). Re-read the question being asked before grabbing a formula.

1.4.3 Shapes: the Catalan Numbers

Theorem 1.4.3 (Binary Tree Shapes are Catalan)

The number of distinct shapes of a binary tree with n nodes is the Catalan number

$$C_n = \frac{1}{n+1}\binom{2n}{n} = 1, 1, 2, 5, 14, 42, 132, \ldots \quad (n = 0, 1, 2, \ldots)$$

equivalently $C_0 = 1$ and $C_n = \sum_{i=0}^{n-1} C_i\, C_{n-1-i}$.

Proof

Root-splitting counts the recurrence. A tree with $n \geq 1$ nodes spends one node on the root; the remaining $n-1$ split into i nodes in the left subtree and $n-1-i$ in the right, for some $i = 0, \ldots, n-1$. Left and right shapes are chosen independently, giving $C_i \cdot C_{n-1-i}$ trees for that i, and summing over i yields $C_n = \sum_i C_i C_{n-1-i}$ with $C_0 = 1$ (the empty shape). This is exactly the Catalan recurrence, whose closed form is $\frac{1}{n+1}\binom{2n}{n}$ (obtainable by generating functions or by a bijection with balanced parenthesis strings).

■

In plain words. The root burns one node and hands the other $n-1$ away as two independent piles; fix the left pile's size i and the count is "C_i ways left $\times$ C_{n-1-i} ways right". Sum over every split. That same one habit – split by where the first item's partner sits – also counts balanced parentheses and stack pop-sequences; this is why the number keeps reappearing.

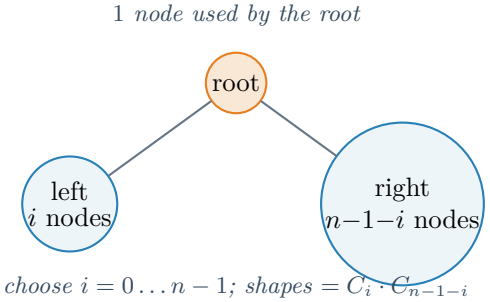


Figure 1.4.2: The Catalan recurrence: the root splits the other $n-1$ nodes between two independent subtrees, and we sum the products over every split.

The number of labelings for a fixed shape

Given a binary shape of n nodes, filling it with labels $1..n$ so that it stays a *BST* is forced (in-order must be sorted), which is why counting BSTs = counting shapes = C_n. Filling it with labels with no BST rule gives $n!$ labelled trees on that shape.

1.4.4 Labelled Trees: Cayley's Formula

Theorem 1.4.4 (Cayley's Formula)

The number of distinct trees on n *labelled* vertices $1, \dots, n$ (edges arbitrary, not restricted to binary or rooted) is n^{n-2}. Rooted at any vertex, this becomes n^{n-1} labelled rooted trees.

Proof idea (Prüfer sequences)

There is a bijection between labelled trees on n vertices and sequences ("Prüfer codes") of length $n-2$ whose entries are any of the n labels. Since each of the $n-2$ positions holds one of n choices freely, there are n^{n-2} such sequences, hence that many trees. The bijection deletes the smallest leaf repeatedly and records its neighbour; reversing the process rebuilds the tree from the code. The construction and its inverse are built in full in Chapter 30.

■

In plain words. Name five cities and let them choose four roads so all are connected and loop-free: exactly $5^{5-2} = 125$ distinct choices. The Prüfer code is the beautiful bijection that proves it – met again in Chapter 30; for now, own the formula, and never answer $n!$ by reflex.

Key Insight

$n = 1$: one tree (1^{-1} read as 1). $n = 2$: $2^0 = 1$ edge. $n = 3$: $3^1 = 3$. $n = 4$: $4^2 = 16$. Growing far slower than $n!$ – most permutations of a shape collapse to the same unrooted tree.

Where this is used. Chapter 13 and Chapter 15 (DP that counts trees), and Chapter 30 (Prüfer codes) – Catalan-style root-splitting recurrences reappear directly in tree-counting DPs, and the Prüfer bijection behind Cayley's formula – including building a tree from its code – is constructed in Chapter 30.

1.4.5 Computing the Counts

Code 1.4.1: Catalan numbers by DP, $C_0 \dots C_n$ (grows fast – use big integers past $n \approx 30$)

```cpp
#include <bits/stdc++.h>
using namespace std;

// C[0..n], the number of binary-tree shapes with k nodes is C[k].
vector<long long> catalan(int n) {
    vector<long long> C(n + 1, 0);
    C[0] = 1;
    for (int k = 1; k <= n; ++k)
        for (int i = 0; i < k; ++i)
            C[k] += C[i] * C[k - 1 - i];     // root split: i left, k-1-i right
    return C;
}
```

Code 1.4.2: Cayley modulo a prime: $n^{n-2} \bmod m$ by fast exponentiation

```cpp
long long modpow(long long b, long long e, long long m) {
    long long r = 1 % m;
    b %= m;
    for (; e > 0; e >>= 1, b = b * b % m)
        if (e & 1) r = r * b % m;
```

```
    return r;
}
// number of labelled trees on n vertices modulo m
long long labelledTrees(long long n, long long m) {
    return n <= 1 ? 1 % m : modpow(n, n - 2, m);   // n^(n-2) mod m
}
```

Complexity

The Catalan DP is $O(n^2)$ time, $O(n)$ memory; use $O(n)$ only if you switch to the closed form with modular inverses. Cayley with binary exponentiation is $O(\log n)$ after the $n-2$ exponent is set. Both counts overflow `long long` almost immediately (C_{35} exceeds 2^{63}), so contests ask for the answer modulo a prime – then reduce every multiply.

1.4.6 Worked Examples

Example 1.4.5 (Small values)

Figure 1.4.1 shows $C_3 = 5$. Extending the root-split, $C_4 = C_0C_3 + C_1C_2 + C_2C_1 + C_3C_0 = 5 + 2 + 2 + 5 = 14$, and $C_5 = 42$. For labelled trees, Cayley gives $n = 4 \Rightarrow 4^2 = 16$ and $n = 5 \Rightarrow 5^3 = 125$.

Step-by-Step Trace

of Code 1.4.1 builds the array $C = [1, 1, 2, 5, 14, 42, 132]$ for $N = 6$: each new entry is the running sum of products from Figure 1.4.2, matching Theorem 1.4.3 line for line.

1.4.7 Variations and Twists

- **Same recurrence, new clothes.** Balanced parenthesis strings, stack pop-sequences, non-crossing handshakes, triangulations of a polygon – all C_n. When a DP splits into two independent halves at a chosen point, hum $1, 1, 2, 5, 14$ and check. **L3**
- **Counting → generating.** LC 95 builds every tree instead of counting: the same root split, but the entries are *lists of nodes* – and the cost is the output size $\Theta(C_n)$. **L3**
- **Insertion order instead of shape.** LC 1569 (the stretch solved problem below): reorder $1..n$ so the insertion-built BST equals a target – a Catalan count wearing binomial clothing. **L5**
- **Rooted vs free vs binary.** Binary shapes C_n; rooted labelled free trees n^{n-1}; unrooted labelled n^{n-2}; *unlabelled* shapes of free trees need Pólya – far outside this chapter (and mostly outside CP). Which pair of questions (labels? binary?) the statement answers decides the formula. **L4**
- **Disguise.** "structurally unique", "how many brackets", "how many merge orders" – the recognition sentence is always the root-split: pick the first element, two independent remainder problems.

n	0	1	2	3	4	5	6
C_n (shapes)	1	1	2	5	14	42	132
labelled trees n^{n-2}	–	1	1	3	16	125	1296

1.4.8 Common Pitfalls

Shapes vs labelled trees

C_n counts *shapes* (labels ignored); n^{n-2} counts *labelled* trees (edges free, labels matter, not necessarily binary). Quoting the wrong one is a whole- problem zero. Ask first: do node identities matter, and is binary structure required?

Overflow and modulo

Catalan and n^{n-2} explode; in contests the modulus is part of the task. Never compute $\binom{2n}{n}$ as raw integers – use modular inverses or the DP, all under the modulus.

The trap as code.

Code 1.4.3: WRONG – raw binomial, then reduce: explodes at $n \approx 30$

```
long long C = 1;
for (int i = 0; i < n; i++)
  C = C * (2*n - i) / (i + 1);   // overflows ll;
                                 // '/' is illegal mod p
```

Code 1.4.4: RIGHT – numerator/denominator as modular inverses

```
// C_n = binom(2n,n)/(n+1) mod p, p prime:
long long num = 1, den = 1;
for (int i = 0; i < n; i++) {
  num = num * (2*n - i) % p;
  den = den * (i + 1) % p;
}
long long C = num % p * modpow(den, p - 2, p) % p;
```

1.4.9 Problem Lab (fully solved)

Attempt it yourself first

LC 96 is the chapter's core **L3**: derive the root-split recurrence on paper before coding. The stretch (LC 1569, **L5**) is one of the book's first true contest-Hards – struggle with it for a full session; the ladder there reuses *only* tools taught above.

W1.4.1 Unique Binary Search Trees

Source: LeetCode 96 LC Medium L3
Techniques: Catalan numbers, root-splitting DP

Unique BSTs

Given n, count how many *structurally unique* BSTs store the keys $1 \dots n$. For $n = 3$ the answer is 5.

➤ Understanding the Problem

A BST's in-order traversal is the sorted keys, so once we pick which key is the root, the smaller keys are forced left and the larger are forced right. Different shapes give different trees, and two trees with the same shape but different keys can't both be valid BSTs on $1..n$. So counting BSTs is exactly counting binary shapes on n nodes – the Catalan number C_n (Theorem 1.4.3).

➤ Key Observations

Observation 1. Choosing root key r splits keys into $r-1$ smaller and $n-r$ larger; the two sides form independent subtrees.

Observation 2. So $G(n) = \sum_{r=1}^{n} G(r-1)\,G(n-r)$ with $G(0) = G(1) = 1$ – the Catalan recurrence.

➤ The Approach Ladder

Level	Approach	Idea	Time	Verdict
L2	Build every BST, count them	LC 95 machinery used just to count	$\Theta(C_n)$	✗ TLE/✗ MLE
L3	Root-split DP over sizes	$G(n) = \sum_r G(r-1)G(n-r)$	$O(n^2)$	✓ Accepted
L4	Closed form $\frac{1}{n+1}\binom{2n}{n}$	factorials and modular inverse	$O(n)$	✓ Accepted

➤ Solution

Bottom-up DP over subtree size, summing the products of left and right counts, exactly Code 1.4.1.

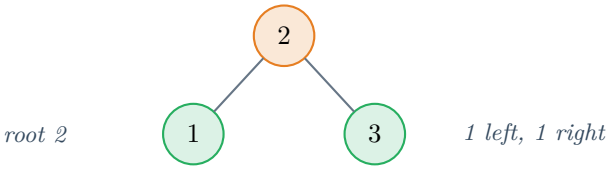


Figure 1.4.3: One of the five BSTs on $\{1,2,3\}$: root 2 forces 1 left and 3 right. Pick root 1 and both $2,3$ go right (two shapes), root 3 mirrors it – 5 in total.

➤ Proof of Correctness

Proof

By the BST in-order property, key r as root uses exactly keys $< r$ on the left and $> r$ on the right, and the subtrees are independent valid BSTs, contributing $G(r-1)G(n-r)$ trees. The cases partition all BSTs by their root, so $G(n) = \sum_r G(r-1)G(n-r)$. The DP computes this with base $G(0) = G(1) = 1$ and returns $G(n) = C_n$.

■

➤ Implementation

Code 1.4.5: Unique Binary Search Trees (LeetCode 96)

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int numTrees(int n) {
        vector<int> dp(n + 1, 0);
        dp[0] = 1; dp[1] = 1;
        for (int nodes = 2; nodes <= n; ++nodes)
            for (int root = 1; root <= nodes; ++root)
                dp[nodes] += dp[root - 1] * dp[nodes - root];
        return dp[n];
    }
};
```

➤ Trace

Step-by-Step Trace

$dp = [1, 1, \dots]$. $dp[2] = dp[0]dp[1] + dp[1]dp[0] = 2$. $dp[3] = dp[0]dp[2] + dp[1]dp[1] + dp[2]dp[0] = 2 + 1 + 2 = 5$. Matches $C_3 = 5$ and the answer for the sample.

➤ Complexity

Complexity

Time $O(n^2)$, memory $O(n)$. For LC $n \le 19$ the value fits in `int`; on larger inputs or with a modulus, do the same DP with `long long` and reduce. A closed-form C_n with modular inverses is $O(n)$.

➤ Verification and Edge Cases

- $n = 0$: $dp[0] = 1$ (the empty tree) – the recurrence base, correct. **L3**
- $n = 1$: answer 1; $n = 2$: answer 2; $n = 3$: answer 5 – check against C_n.
- the **L2** build-every-tree rung is the LC 95 problem: same math, but you store node lists – $\Theta(C_n)$ output, so it MLEs when you only needed a count.

➤ Pitfalls and Variants

Counting vs generating

W1.4.1 only *counts*; *Unique BST II* (LC 95) builds every tree, so the same recurrence is applied to node objects instead of integers – $O(C_n)$ output, and recursion, not a flat DP.

Key Insight

Key takeaways: (1) a BST on keys $1..n$ is fully determined by its *shape*, so counting BSTs = counting shapes = C_n; (2) the recurrence is root-split × two independent halves summed over the split; (3) recognise $1, 1, 2, 5, 14$ the moment a DP splits at a chosen point.

Attempt it yourself first

LC 1569 (next) is the chapter's summit at **L5** – a genuine LeetCode Hard that uses *only* tools built above: the root-split view of a BST, binomial coefficients, and modular arithmetic. Give yourself a full 45-minute session with paper first. Do not read the ladder until you have written *something*.

Peek (1 min lost): Which element of the array is forced to be the root of the built BST? Once the root is fixed, what decides which later elements end up left vs right?

Nudge (5 min lost): Relative order is all that matters: elements that land in the left subtree must keep their *relative* order among themselves (and likewise the right), but the two streams can be interleaved arbitrarily. How many interleavings of a k-slot and $(n-1-k)$-slot stream are there?

Full hint (you will not solve this one): Answer $= \binom{n-1}{k} \cdot f(\text{left}) \cdot f(\text{right})$ where k is the count of elements below the root – the root is locked to position 0 of any reorder, so choose which of the remaining $n-1$ insertion slots the smaller values occupy. Recurse, precompute Pascal's triangle mod 10^9+7, and subtract 1 at the end because the original order must not be counted.

W1.4.2 Number of Ways to Reorder Array to Get Same BST

Source: LeetCode 1569 LC Hard L5
Techniques: root-split decomposition, binomial interleaving, Pascal DP mod p

Reorder Array to Get Same BST

Given a permutation-like array a of n distinct positive integers, a BST is built by inserting the elements *in array order* (standard BST insertion: smaller goes left, larger goes right, first element is the root). Count the number of *reorderings* of a that build the *exact same* BST, excluding the original order. Return the count modulo 10^9+7. $1 \le n \le 1000$.

➤ Understanding the Problem

Insertion always makes a_0 the root, then each following element walks down left/right by comparison. Two facts unlock the problem:

1. which elements land in the left subtree is *forced* – exactly the values below the root – so only their *relative arrival order* and the *interleaving* with the right-subtree arrivals are free;
2. within one side, the same logic applies recursively.

So the count is a product over nodes of *interleaving choices*, each a binomial coefficient.

➤ Key Observations

Observation 1. Root is fixed to position 0 in every valid reorder. With k smaller and $n-1-k$ larger elements, the $n-1$ remaining slots are split between the two streams: $\binom{n-1}{k}$ ways to seat the left stream (the right stream takes the rest).

Observation 2. The two streams interleave *arbitrarily* without changing either subtree: a left element's descent only compares against elements already on the left path, and relative order within a stream is what the recursive structure constrains. Hence $f(S) = \binom{|S|-1}{|L|} f(L)\, f(R)$, $f(|S| \le 1) = 1$.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** "this is Catalan – count shapes and multiply by $n!$"
Killed by: the *shape* is not what we vary; the BST is fixed by the original array, and most permutations build a *different* tree. We reorder so the tree stays identical, which pins each element's side – a multinomial-of-streams count, not C_n.

✗ **Tried:** enumerate all $n!$ orders, insert each, compare trees
Killed by: $O(n! \cdot n)$ – dies at $n = 20$, let alone $n = 1000$. Enumerating to *count* is always the signal to find the independent sub-choices instead.

✓ **Aha!** the only freedom is *how the two forced streams weave together*. Pick the root's slot pattern with $\binom{n-1}{|L|}$, recurse on each side, multiply – the same root-split anatomy as Catalan, with binomials replacing a flat $G(r-1)G(n-r)$ because here the *positions*, not just sizes, are being chosen.

➤ The Approach Ladder

Level	Approach	Idea	Time	Verdict
L2	enumerate permutations	insert each, compare trees	$O(n! \cdot n)$	TLE/CE
L3	Catalan reflex	quote C_n (or $C_n \cdot n!$)	$O(n)$	WA
L4	stream-interleaving recursion	$\binom{n-1}{\|L\|} \cdot f(L) \cdot f(R)$, Pascal mod p	$O(n^2)$	✓ Accepted

➤ Solution

Split the array around its first element into L (smaller) and R (larger), recursively count each side, and multiply by $\binom{n-1}{|L|}$ for the interleavings. All arithmetic mod $10^9 + 7$; $\binom{\cdot}{\cdot}$ comes from a Pascal table built once (adds only – no division, so no modular inverses needed).

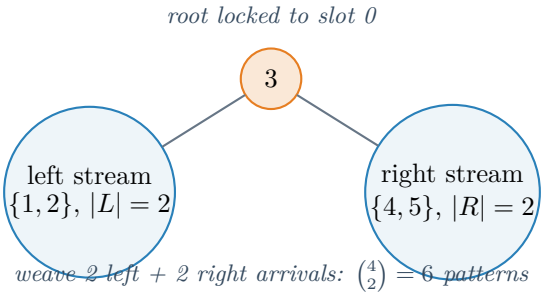


Figure 1.4.4: LC 1569 on $[3, 4, 5, 1, 2]$: the root 3 is fixed; whichever slots the smaller stream $\{1, 2\}$ takes (in relative order), the larger stream fills the rest – and each stream's internal count recurses. $\binom{4}{2} \cdot f(\{1, 2\}) \cdot f(\{4, 5\}) = 6$.

➤ Proof of Correctness

Proof

Induction on $|S|$. *Base*: $|S| \le 1$, one order, $f = 1$. *Step*: every valid reorder has S_0 first (it is the root, else the tree's root value differs), sends exactly L to the left subtree and R to the right (insertion comparisons force this), and, once the slot set for L is chosen, builds BST(L) and BST(R) independently – interleaving cannot move an element between streams because side placement depends only on value-vs-root, not timing. Choosing the L-slots ($\binom{|S|-1}{|L|}$ ways), then valid orders within each stream ($f(L)$, $f(R)$ by IH) is a bijection with valid reorders of S. Multiplying and subtracting the original order at the top gives the answer.

■

➤ Implementation

Code 1.4.6: Ways to Reorder Array to Get Same BST (LeetCode 1569)

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
    static const int P = 1e9 + 7;
    vector<vector<long long>> C;          // C[n][k] = binom(n,k) mod P

    long long solve(const vector<int>& a) {
        int n = a.size();
        if (n <= 1) return 1;
        vector<int> L, R;                 // streams forced by the root a[0]
        for (int i = 1; i < n; ++i)
            (a[i] < a[0] ? L : R).push_back(a[i]);
        long long ways = C[n - 1][L.size()];          // interleavings
        ways = ways * solve(L) % P;
        ways = ways * solve(R) % P;
        return ways;
    }

public:
    int numOfWays(vector<int>& nums) {
        int n = nums.size();
        C.assign(n + 1, vector<long long>(n + 1, 0)); // Pascal triangle, adds only
        for (int i = 0; i <= n; ++i) {
            C[i][0] = 1;
            for (int j = 1; j <= i; ++j)
                C[i][j] = (C[i - 1][j - 1] + C[i - 1][j]) % P;
        }
        return (solve(nums) - 1 + P) % P;             // exclude the original order
    }
};
```

➤ Trace

Step-by-Step Trace

$[3,4,5,1,2]$: root 3, $L = \{1,2\}$ (in array-relative order $[1,2]$), $R = \{4,5\}$. $f(\{1,2\})$: root 1, one element right, $\binom{1}{0} = 1$. $f(\{4,5\}) = 1$ likewise. Top: $\binom{4}{2} \cdot 1 \cdot 1 = 6$ reorders including the original, answer $6 - 1 = 5$ – matches LeetCode's example.

➤ Complexity

Complexity

Pascal table $O(n^2)$ time and memory. The recursion: each solve frame costs $O(m)$ to split, and elements re-split down their root-to-node path – $O(n^2)$ worst case (skewed BST), $O(n \log n)$ balanced. Overall $O(n^2)$, within $n \leq 1000$. A factorial/inverse-factorial table replaces Pascal at the same asymptotics.

➤ Verification and Edge Cases

- $n = 1$ or $n = 2$: answer 0 (only the original order) – the recurrence gives $f = 1$, then $-1 = 0$. **L5**
- sorted input (chain BST): every split has $|L| = 0$, so $\binom{n-1}{0} = 1$ at each node – $f = 1$, answer 0 (a chain has a unique insertion order).
- forgetting the -1: returns 6 instead of 5 on the sample – the classic WA.

➤ Pitfalls and Variants

Off-by-species

Three silent killers: forgetting the final -1 (the original order is counted by the recurrence); $\binom{n-1}{k}$ written as $\binom{n}{k}$ (the root does *not* join the weaving); and computing / on reduced values if you switch to the factorial form – use modular inverses (§1.4 RIGHT listing) or stay with Pascal.

Key Insight

Key takeaways: (1) the tree being *fixed* turns a "how many orders" question into a count of *weavings of value-forced streams*; (2) the recurrence is Catalan's root-split anatomy with a binomial factor replacing the flat product; (3) under a prime, prefer Pascal (division-free) or modular inverses – never integer division on reduced values; (4) subtract the original order last. This is the chapter's scaled summit: three Chapter-1 tools (root-split, binomials, modulo) composed.

AC ⟶ red: hardening this code

L5 stress: $n = 1000$ *sorted* input – the BST degenerates to a chain, recursion depth ≈ 1000: fine on LeetCode, but on judges with small stacks convert Code 1.4.6's `solve` to an explicit stack, or note the chain answer is $\binom{n-1}{1}^{n-1}$-style and test overflow of the intermediate product against P^2 (use `long long`, never `int` for products). **L5 variant:** same count *including* the original order – one line different, favourite judge twist. **L5 stretch:** Generalize to building the same *shape* (values free to swap sides if depths match) – the streams are no longer value-forced; think before coding, then see Chapter 13.

1.4.10 Practice Problems

Stuck? Work through this list

- Decide what is being counted: shapes (C_n), labelled free trees (n^{n-2}), or labelled rooted trees (n^{n-1})? One wrong choice and every number misses.
- If your recurrence has a sum over a split point with a product of two independent halves,

compute the first values by hand – $1, 1, 2, 5, 14$ means Catalan.
- Modulus given? Reduce after *every* addition and multiplication; division needs a modular inverse, never "/" on the reduced value.
- Check $C_3 = 5$ and n^{n-2} at $n = 4$ (16) against your code before submitting.

Q1.4.1 Unique Binary Search Trees II – LeetCode 95 LC Medium L3

Hint: same root split, but build and combine actual left/right subtree lists.

Q1.4.2 Different Ways to Add Parentheses – LeetCode 241 LC Medium L3

Hint: each operator as a root splits the expression – a Catalan-shaped interval DP. See Chapter 13.

Q1.4.3 Count Binary Trees With Factors – LeetCode 852 LC Medium L4

Hint 1: sort values; for each root count factor pairs that multiply to it – Catalan's cousin: the split is by *values*, not by position.

Hint 2: DP: answer[v] = 2 * sum over proper factor pairs (a,b) of answer[a]*answer[b]; memoise with a hash map of value to count.

10-second refresher

Self-test (answer from memory, tick the box):

- ☐ Binary shapes on n nodes: $C_n =$ _____ (closed form), and the recurrence is $C_n =$ _____.
- ☐ Labelled free trees on n vertices: _____; rooted: _____.
- ☐ BSTs on keys $1..n$ count C_n because _____.
- ☐ In LC 1569 the only freedom after the root is _____, which is why a _____ coefficient appears.
- ☐ Write the LC 96 root-split DP from memory, base case first.

The two questions that pick the formula: do labels matter? is binary structure forced? (Both no: n^{n-2}. No/yes: $C_n \cdot n!$ shape-x-label. Yes/yes BST: C_n.)

Chapter 1 Summary

Everything in this chapter is a tool the rest of the book assumes. The tables collect the results with the exact conditions they need, and the algorithms with the deciding phrase that selects them.

Result	Statement (with conditions)	Where
Edges of a tree	A tree on n nodes has exactly $n-1$ edges.	Theorem 1.1.4
Unique path	Exactly one simple path joins any two nodes (connected, acyclic).	Theorem 1.1.5
Height recursion	height(`null`) $= -1$, height$(v) = 1 +$ $\max_c$ height(c).	Theorem 1.2.4
Size recursion	size(`null`) $= 0$, size$(v) = 1 + \sum_c$ size(c).	Theorem 1.2.4
Perfect counts	Height h (edges) $\Rightarrow 2^{h+1} - 1$ nodes, 2^h leaves.	Theorem 1.3.7
Full tree leaves	Every internal node has 2 children $\Rightarrow L = I+1$.	Theorem 1.3.8
Height bounds	$\lfloor \log_2 n \rfloor \le$ height $\le n-1$ (binary, n nodes).	Corollary 1.3.9
Catalan shapes	Binary shapes on n nodes $= C_n = \frac{1}{n+1}\binom{2n}{n}$.	Theorem 1.4.3
Cayley trees	Labelled (free) trees on n vertices $= n^{n-2}$; rooted n^{n-1}.	Theorem 1.4.4

Algorithms from this chapter – the decision column is the point.

Algorithm	Time	Memory	When to use (deciding phrase)	Code
Adjacency + parent-guard DFS	$O(n)$	$O(n)$	input is an *edge list*, not pointers	Code 1.1.1, Code 1.2.2
Top-down parameter DFS	$O(n)$	$O(h)$	the node's value flows *from the root* (depth, prefix)	Code 1.2.2
Bottom-up return / pair	$O(n)$	$O(h)$	the value *aggregates a subtree* (height, size, sum, avg)	Code 1.2.7, W1.2.3
Lockstep double recursion	$O(\min(n,m))$	$O(h)$	*two* trees compared node-for-node	Code 1.1.6
Sentinel early-abort	$O(n)$, $\Omega(h)$	$O(h)$	per-node promise check that should fail fast	Code 1.3.3
Spine shortcut (complete)	$O(\log^2 n)$	$O(\log n)$	node count when *completeness is promised*	Code 1.3.4
BFS by level size	$O(n)$	$O(n)$	anything about *levels* (view, width, completeness)	W1.3.1 usage in 1.3
Catalan root-split DP	$O(n^2)$	$O(n)$	*count* objects that decompose at a chosen root	Code 1.4.1
Pascal-mod + interleaving	$O(n^2)$	$O(n^2)$	count reorderings/interleavings modulo a prime	Code 1.4.6

Master Decision Flowchart

Post-study revision ladder: walk it top to bottom and stop at the first YES. It must *decide*, not decorate – a reader who walks it never opens the wrong section.

```
Read the problem. Is the input a tree (n nodes, n-1 edges, connected & acyclic)?
|
NO  --> a graph problem; prove acyclicity with the n-1 edge test    [1.1]
|       or leave the chapter (graph series).
|
YES
|
```

```
Does it compare TWO trees / shapes ("same", "subtree of", "symmetric")?
+-- YES --> lockstep structural recursion; base case handles null pairs [1.1]
|           ...and is the answer known only at leaves of the *pair*
|           recursion (both sides seen)?  --> post-order + combine     [1.2]
|
Does every node need a value that flows FROM the root (depth, level, prefix)?
+-- YES --> top-down DFS, carry the parameter down                     [1.2]
|
Does the answer AGGREGATE a subtree (height, size, sum, max below)?
+-- YES --> bottom-up recursion, return the value (or a PAIR) up       [1.2]
|           ...and is the best path THROUGH a node also needed?
|           +-- YES --> return down-gain, update a global best --
|                       the two-channel split (dia/tilt)          [1.2, c09, c10]
|           ...and is the answer a floor-average / exact division?
|           +-- YES --> test sum == val * count; keep the division
|                       the SPEC asks for, not your own shortcut       [1.2]
|
Does the statement PROMISE a shape (complete / perfect / full / balanced)?
+-- YES --> spend the promise: 2^(h+1)-1 shortcut, L=I+1, height
|           bounds, O(log^2 n) node count                             [1.3]
|           ...promise to VERIFY, not given? --> per-node check with
|           a sentinel that aborts early (LC 110, LC 958)             [1.3]
|
Does it ask HOW MANY trees / shapes / BSTs / valid ORDERS exist?
+-- YES --> root-split: Catalan C_n (shapes), n^(n-2) (labelled free),
|           C(n-1,|L|) * f(L) * f(R) (insertion orders) mod p          [1.4]
|           ...asks to OUTPUT every tree (LC 95)? --> generate lists,
|           cost = output size, not a DP over counts                  [1.4]
|
None of these?
--> ordered lookups [c05]; updates with shallow guarantee [c06]; priority of
    extremes [c07]; prefix/shared strings [c08]; traversals as tools [c03];
    path queries between nodes [c11]; everything else: DP [c10].
```

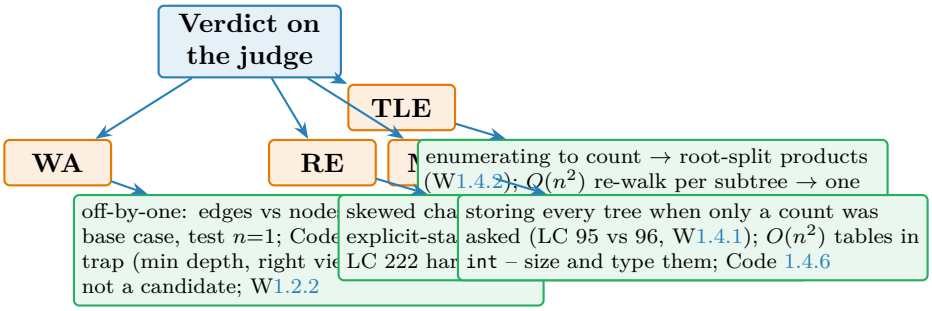
Common Mistakes Checklist

- **Missing the `null`/leaf base case.** Every tree recursion must say what a null (or a leaf) returns *before* recursing. Fix: write the base case first.
- **Depth vs height, edges vs nodes.** They differ by one and run in opposite directions. Fix: pick the convention, set the base case to match, test a one-node tree.
- **Forgetting the parent guard.** A DFS on an adjacency list without `if (to == p) continue;` loops forever. Fix: always pass the parent.
- **Recursion on a skewed tree.** A path of 10^5 nodes overflows the stack. Fix: iterative post-order with an explicit stack.
- **Treating a null child as depth 0 in min-depth.** The one-child trap: Code 1.2.6 skips null sides instead of minimising them.
- **Full / complete / perfect mixed up.** They give different counting rules. Fix: sketch a two- and three-level example of each before asserting one.
- **Balanced checked only at the root.** The condition is per-node. Fix: return height bottom-up and a sentinel the moment any node breaks it.
- **Shapes vs labelled trees.** C_n ignores labels; n^{n-2} is unrooted and unbinary. Fix: ask whether identities matter and whether binary structure is required.
- **Overflow on counts.** Catalan and n^{n-2} explode past tiny n. Fix: reduce modulo the given prime at every multiply; `long long` for products.

- **Dividing under a modulus.** `/` on reduced values is wrong. Fix: Pascal table (adds only) or modular inverses – the 1.4 RIGHT listing.
- **Forgetting -1 in reorder counts (LC 1569).** The recurrence counts the original order too. Fix: subtract it at the top, W1.4.2.

WA Clinic

Judge verdicts are *symptoms*; the causes are a short list this chapter trained you in. Diagnose top-down; each leaf points at the drill ground.



Chapter 1 verdict tree – only the failures this chapter's problems can actually produce.

Backup table (same content, terse):

Verdict	Usual cause	One-line fix	Trained in
WA	convention mix (edges/nodes)	fix base case, test $n = 1$	W1.2.1
WA	null child fed to min	skip one-child sides explicitly	W1.2.2
WA	missing final -1	subtract the original order	W1.4.2
TLE	counting by constructing	count with products, not lists	W1.4.1
TLE	per-subtree re-walk	single DFS, return pairs	W1.2.3
RE	recursion on a chain	explicit stack / iterative post-order	1.2 pitfalls
MLE	generating all trees	keep only counts	§1.4

Quick Reference Card

Chapter 1 at a glance

Definitions. Tree = connected + acyclic. Rooted: parent/child/leaf/subtree/ancestor. depth down from root, height down to deepest leaf, size nodes in subtree.

Core identities. n nodes $\Rightarrow$ $n-1$ edges. Unique path between any two nodes. height(`null`)$=-1$, height(v)=1+ max$_c$ height(c); size(v)=1+$\sum_c$ size(c). Perfect: $2^{h+1}-1$ nodes, 2^h leaves. Full binary: $L{=}I{+}1$. Height: $\lfloor \log_2 n \rfloor \le h \le n{-}1$.

Counting. Binary shapes $C_n = \frac{1}{n+1}\binom{2n}{n} = 1, 1, 2, 5, 14, 42, \ldots$; labelled trees n^{n-2}; labelled rooted trees n^{n-1}; insertion orders of a fixed BST: $\binom{n-1}{|L|} f(L) f(R) - 1$.

Conventions. Heights/depths in *edges* unless a problem says nodes (LeetCode *Max Depth* counts nodes: subtract 1 to convert).

Code 1.4.7: Chapter 1 code kit: the two shapes you reuse everywhere

```cpp
#include <bits/stdc++.h>
using namespace std;

// (1) General tree from an edge list: DFS with a parent guard, all three measures.
int n; vector<vector<int>> adj; vector<int> dep, hei, sz;
void measure(int v, int p, int d) {
    dep[v] = d; sz[v] = 1; hei[v] = -1;                // edges convention
    for (int to : adj[v]) if (to != p) {
        measure(to, v, d + 1);
        sz[v]   += sz[to];
        hei[v]   = max(hei[v], hei[to]);
    }
    ++hei[v];
}

// (2) Binary tree height (edges) and node-count height (nodes) in two lines.
int heightEdges(TreeNode* r) { return !r ? -1 : 1 + max(heightEdges(r->left),
    ↪ heightEdges(r->right)); }
int heightNodes(TreeNode* r) { return !r ?  0 : 1 + max(heightNodes(r->left),
    ↪ heightNodes(r->right)); }
```

Write-From-Memory Drills

Four signature skeletons; the marked lines are yours. Each drill blank comes from a listing that exists in the chapter – close the book, fill the blanks, then diff against the original.

Code 1.4.8: Drill 1 – parent-guard DFS (from Code 1.2.2)

```cpp
void measure(int v, int p, int d) {
    dep[v] = d; sz[v] = 1; hei[v] = -1;
    for (int to : adj[v]) {
        // FILL 1: skip the parent _____________________________
        // FILL 2: recurse with _______________________________
        // FILL 3: pull size and height up: ___________________
    }
    ++hei[v];
}
```

Key: `if (to == p) continue;  measure(to, v, d + 1);  sz[v] += sz[to]; hei[v] = max(hei[v], hei[to]);`

Code 1.4.9: Drill 2 – bottom-up pair return (from Code 1.2.7)

```cpp
pair<long,long> dfs(TreeNode* r) {          // {sum, count}
    if (!r) return {0, 0};
    auto [sl, cl] = dfs(r->left);
    auto [sr, cr] = dfs(r->right);
    long s = sl + sr + r->val, c = cl + cr + 1;
    // FILL: count r exactly when _____________________________
    return {s, c};
}
```

Key: `s / c == r->val` – the *spec's* floor division, or exactly `s == r->val * c` (identical for positive values; 1.2 hardening-box).

Code 1.4.10: Drill 3 – sentinel early-abort (from Code 1.3.3)

```
int h(TreeNode* r) {                 // -1 = "some node already broke"
    if (!r) return 0;
    int lh = h(r->left), rh = h(r->right);
    // FILL 1: propagate failure: _____________________________
    // FILL 2: local check and return: ___________________________
}
```

Key: `if (lh < 0 || rh < 0 || abs(lh-rh) > 1) return -1;  return 1 + max(lh, rh);`

Code 1.4.11: Drill 4 – mod-safe counting (from Code 1.4.6)

```
long long solve(vector<int>& a) {             // |a| = n, root = a[0]
    if (n <= 1) return 1;
    split a[1..] into L (smaller), R (larger);
    long long w = C[n - 1][L.size()];         // interleavings
    // FILL 1: combine the streams: ______________________________
    // FILL 2: top-level adjustment: ____________________________
}
```

Key: `w = w * solve(L) % P * solve(R) % P;  (solve(nums) - 1 + P) % P` – never a bare -1 under a modulus.

Complete Problem Checklist

Every problem this chapter offers – 8 fully solved, 17 practice, 6 mixed. Tick the box only after you solved it *unaided*; a solved-with-hints attempt counts as half.

#	Problem	Source	Key technique	What to look for	Done?
Section 1.1 – what a tree is, lockstep recursion					
W1.1.1	Same Tree	LC 100	L1 lockstep structural recursion	all four null/value cases	☐
Q1.1.1	Invert Binary Tree	LC 226	L1 swap children, recurse	post-order vs pre-order both work	☐
Q1.1.2	Find Center of Star Graph	LC 1791	L1 degree/edge voting	a star is a tree with one hub	☐
Q1.1.3	Univalued Binary Tree	LC 965	L1 one-value propagation	compare child to parent	☐
Q1.1.4	Leaf-Similar Trees	LC 865	L2 collect leaf sequence	pre-order gives leaf order	☐
Q1.1.5	Min Distance Between BST Nodes	LC 783	L2 inorder + running prev	a BST hides sorted data	☐
Section 1.2 – depth, height, size					
W1.2.1	Maximum Depth	LC 104	L1 bottom-up max	edges vs nodes convention	☐
W1.2.2	Minimum Depth	LC 111	L2 bottom-up min + trap	null child is not a leaf	☐
W1.2.3	Count Nodes = Subtree Avg	LC 2265	L3 two-channel pair return	floor division is the spec	☐
Q1.2.1	Sum of Left Leaves	LC 404	L2 flag direction when descending	"left" is a parent property	☐
Q1.2.2	Tree Diameter	CSES 1131	L3 two BFS/DFS or bottom-up	general tree, not binary	☐
Q1.2.3	Diameter of Binary Tree	LC 543	L2 return down, update global	the two-quantity split	☐
Q1.2.4	Longest Consecutive Sequence	LC 298	L3 chain length up from node	reset on broken run	☐
Section 1.3 – shape promises					
W1.3.1	Balanced Binary Tree	LC 110	L2 -1 sentinel bottom-up	per-node, not root-only	☐
W1.3.2	Count Complete Tree Nodes	LC 222	L3 spine test + perfect shortcut	$O(\log^2 n)$ beats $O(n)$	☐
Q1.3.1	Symmetric Tree	LC 101	L2 mirror lockstep (1.1 + 1.3)	compare left-outer to right-outer	☐
Q1.3.2	Check Completeness	LC 958	L2 BFS order + gap flag	no node after a hole	☐
Q1.3.3	Level Order Traversal	LC 102	L2 BFS by levels	queue size = level width	☐
Q1.3.4	Convert Sorted Array to BST	LC 108	L2 halve the array	builds a height-balanced tree	☐
Q1.3.5	Right Side View	LC 199	L3 last node per BFS level	or DFS right-first with depth	☐
Q1.3.6	Complete Binary Tree Inserter	LC 919	L3 BFS queue as insertion frontier	reuse the queue for insert	☐
Section 1.4 – counting trees					
W1.4.1	Unique Binary Search Trees	LC 96	L3 Catalan root-split DP	$G(0) = 1$ base	☐
W1.4.2	Ways to Reorder → Same BST	LC 1569	L5 interleaving × recursion	root locked to slot 0	☐
Q1.4.1	Unique BSTs II	LC 95	L3 build every subtree list	counting vs generating	☐
Q1.4.2	Different Ways to Add Parentheses	LC 241	L3 Catalan-shaped interval DP	operator as root split	☐
Q1.4.3	Count Binary Trees With Factors	LC 852	L4 value-DP over factor pairs	ordered pairs count ×2	☐
Mixed set (chapter end)					
M1.1	Merge Two Binary Trees	LC 617	L2 lockstep + construction	null side returns the other	☐

Recommended Study Order

Two focused weeks at one hour a day; days 1–7 are the spine, 8–14 consolidation and stretch. Do not skip the hand-drawn traces – this chapter *is* the tracing skill.

Day	Focus	Tasks
1	Read 1.1	prove $n-1$ edges by hand on 4-node trees; W1.1.1
2	Practise 1.1	Q1.1.1–Q1.1.5, all without looking back at the code
3	Read 1.2	trace depth/height/size on a 5-node tree in a table
4	Practise 1.2	W1.2.1–W1.2.2 re-implemented, Q1.2.1, Q1.2.3
5	Hardest of 1.2	W1.2.3 cold, then Q1.2.2 (CSES 1131) – two-DFS version first
6	Read 1.3	memorise perfect/full/complete definitions with 2-level sketches
7	Practise 1.3	W1.3.1 both rungs of the ladder; W1.3.2; Q1.3.1, Q1.3.2
8	Finish 1.3	Q1.3.3–Q1.3.6 (queue as frontier)
9	Read 1.4	derive $C_3 = 5$ from the root-split by hand
10	Practise 1.4	W1.4.1, Q1.4.1; postpone Q1.4.2–Q1.4.3 if mod arithmetic is shaky
11	Mixed M1.1–M1.3	decide the tool from the phrasing before coding
12	Summit prep	re-run the 1.4 ladder on paper: why C_n fails LC 1569
13	Summit day	W1.4.2 (LC 1569) fully from memory, incl. Pascal table
14	Stretch	M1.4–M1.6 with Chapter 16 beside you (postpone M1.6 to day 15 if tired)

Difficulty Progression

Tiers are competency gates, not rating buckets – name the skill each unlocks.

Tier	Problems	What passing this tier proves
L1 ★	LC 100, 104, 226, 1791, 965	base case + lockstep are reflexes; conventions never mixed
L2 ★★	LC 110, 111, 112, 572, 617, 101, 108, 102, 958, 865, 783, 404, 543	direction choice (carry down vs return up) is deliberate, not lucky
L3 ★★★	LC 199, 919, 298, 96, 95, 241, 2265, CSES 1131	two-channel returns; Catalan root-split derived, not quoted
L4 ★★★★	LC 235, 236, 852, 124	global-best vs returned-gain split; value-DP with factor pairs
L5 ★★★★★	LC 1569	counting composes: binomial interleaving × recursion × mod discipline

Reduction Gallery

The tree is a disguise; each line collapses a family onto one chapter tool. No new theory – just where the skeleton hides.

- "minimum absolute difference in a BST" (LC 783) ⟶ inorder is sorted, so the answer is adjacent-pair min – one running predecessor, §1.1.
- "count nodes equal to subtree average" (LC 2265) ⟶ two bottom-up numbers that ride one return value: the pair channel, W1.2.3.

- "deepest leftmost value", "longest consecutive run" (LC 298) ⟶ depth carried down × chain length returned up – both channels at once, §1.2.
- "right side view" (LC 199) ⟶ the level promise: queue size is the width, the last pop is the view – §1.3.
- "insert into a complete tree" (LC 919) ⟶ the queue *is* the left-to-right frontier of 1.3's completeness definition – §1.3.
- "is it complete?" (LC 958) ⟶ gap flag over BFS order: an index promise, not a recursion – §1.3.
- "$\leq 10^5$ node count on a complete tree" (LC 222) ⟶ the perfect-subtree coupon $2^{h+1} - 1$ applied log times – Theorem 1.3.7.
- "add parentheses to an expression" (LC 241) and stack pop-sequences ⟶ any choose-the-split-point-and-multiply structure *is* Catalan – Theorem 1.4.3.
- BST insertion logs ⟶ the tree is a *function of the order*; count weavings of forced streams – W1.4.2.
- heap/array parent formulas $2i, 2i{+}1$ ⟶ shape arithmetic on a complete tree; no pointers needed at all – §1.3 (Chapter 10).

Mixed Problem Set

These blend the chapter's ideas and do not name the topic; decide the tool yourself.

M1.1 Merge Two Binary Trees – LeetCode 617 LC Easy L2
Hint: recurse in lockstep, but build a new node from the sum at each pair.

M1.2 Path Sum – LeetCode 112 LC Easy L2
Hint: carry a running total; a root-to-leaf walk must hit the target exactly at a leaf.

M1.3 Subtree of Another Tree – LeetCode 572 LC Easy L3
Hint: at each node try the two-tree equality; otherwise recurse into both children.

M1.4 Lowest Common Ancestor of a BST – LeetCode 235 LC Medium L3
Hint: the order does the work: descend until the two targets split sides. See Chapter 16.

M1.5 Lowest Common Ancestor of a Binary Tree – LeetCode 236 LC Medium L4
Hint: Stretch: post-order returning what was found; the node where both appear is the answer (Chapter 16).

M1.6 Binary Tree Maximum Path Sum – LeetCode 124 LC Hard L4
Hint 1: Stretch: like the diameter – a global best uses the best path THROUGH each node.
Hint 2: DFS returns the best downward gain from a node $(\max(0, \text{child}))$; update the answer with node + left + right.

M1.7 Employee Importance – LeetCode 690 LC Medium L3
Hint: The hierarchy of bosses is the tree: sum an employee plus every subordinate underneath, so a plain downward DFS with a running total answers it.

M1.8 Find the Town Judge – LeetCode 997 LC Easy L2
Hint: The judge trusts no one and is trusted by everyone – an out-degree of 0 and an in-degree of $n - 1$; track the two counters like parent and child links.

M1.9 Find if Path Exists in Graph – LeetCode 1971 LC Easy L2
Hint: Store the edge list as an adjacency list, then walk from the source and stop the moment you touch the target – reachability is the whole question once the edges are stored the right way.

M1.10 Number of Islands – LeetCode 200 LC Medium L3

Hint: The grid is a forest under 4-neighbour adjacency; one DFS flood that erases visited land costs exactly one island – count the floods, not the cells.

M1.11 Keys and Rooms – LeetCode 841 LC Medium L3

Hint: Rooms are nodes and keys are out-edges: DFS from room 0 until no unvisited door is reachable, and success means every room got stamped.

The Summit – Scaled: LC 1569

Why scaled

The full Summit slot (a CF 3000+ solved red-coder style) needs composition tools this book builds later. Chapter 1's ceiling is the **L5 Ways to Reorder Array to Get Same BST** (W1.4.2, §1.4) – solved there in full pipeline. It is the scaled summit: real contest Hard, two chapter tools composed (root-split counting + modular binomials), one earlier-chapter tool (subtree recursion from §1.2).

Revision-only ladder – say *why* each rung dies before you re-code the winner:

Level	Rung	Killing constraint
L1–L2	simulate & compare trees	only *verifies* one order; counting needs the decomposition
L2	enumerate all $n!$ orders	$20! \gg 10^{18}$; dead by $n = 20$
L3	quote C_n (or $C_n \cdot n!$)	wrong object – the BST (hence streams) is value-forced
L4	root-split recursion with *sizes only*	forgets *which* slots the left stream takes: the $\binom{n-1}{\lvert L\rvert}$ factor
L5	$\binom{n-1}{\lvert L\rvert} f(L) f(R)$, Pascal mod p, final -1	survives – the solved version in 1.4

Re-derive cold: the bijection (why interleaving never moves an element between streams) in one paragraph; the Pascal table without lookups; the three WA-clinic entries this problem trains (W1.4.2).

Tool Kit Inventory

The machine assembled so far – cumulative through Chapter 1; every later chapter appends rows and this table is where they are registered.

Tool	Since	Level	Trigger phrase	Where
Adjacency list + parent-guard DFS	1.1	L1	edge list given, not pointers	Code 1.1.1
Lockstep double recursion	1.1	L1	two trees compared	W1.1.1
$n-1$ edges / unique path tests	1.1	L1	"is this really a tree?"	Theorem 1.1.4
Top-down carried parameter	1.2	L1	value flows from the root	Code 1.2.2
Bottom-up return (single)	1.2	L1	subtree aggregate	Theorem 1.2.4
Two-channel pair return	1.2	L3	two numbers per node, one DFS	W1.2.3
Return-down / update-global split	1.2	L2	best path through a node	W1.2.2 run-ups, LC 543
Sentinel early-abort	1.3	L2	per-node promise, fail fast	Code 1.3.3
Shape-coupon counting ($2^{h+1}-1$, $L{=}I{+}1$)	1.3	L2	a shape is promised	Theorem 1.3.7
Spine test + perfect shortcut	1.3	L3	huge complete tree, count nodes	Code 1.3.4
BFS level sizing	1.3	L2	anything about levels	Q1.3.3 (LC 102)
Catalan root-split DP	1.4	L3	count root-decomposable objects	Theorem 1.4.3
Cayley n^{n-2} + Prüfer preview	1.4	L4	labelled free trees	Theorem 1.4.4
Pascal-mod table (division-free binomials)	1.4	L4	binomials under a prime	Code 1.4.6
Stream interleaving count	1.4	L5	reorderings that preserve structure	W1.4.2

Related Topics

- Chapter 2 – Binary trees in depth: the `TreeNode` model, construction, more traversals.
- Chapter 3 – Preorder, inorder, postorder and level order as reusable walks.
- Chapter 4 – Storing trees: arrays, adjacency, parent/child pointers, Euler tour.
- Chapter 8 – Self-balancing trees, where the height-bounds corollary becomes a design goal.
- Chapter 10 – Heaps: the complete-tree shape promise exploited for storage.
- Chapter 12 – Diameter and centroid, built on the same bottom-up height idea.
- Chapter 13 – Tree DP: "combine children, return a value", formalised; Catalan DPs live there.
- Chapter 16 – Lowest common ancestor, the path tool this chapter only hinted at.
- Chapter 30 – Prüfer codes: the full bijection behind Cayley's n^{n-2}.

Last Word – Chapter 1

"In a tree there is exactly one way to get anywhere; every problem is deciding which way to walk it."

What this chapter actually installed is not formulas but **two reflexes**:

1. *direction*: values that flow down from the root are carried as parameters; values that bubble up from leaves are return values. Guess wrong and no amount of cleverness saves you.
2. *promise*: every shape adjective in a statement (complete, perfect, balanced) is a discount coupon – a formula, a bound, or a halving. Spotting it is the whole optimization.

Carry three habits forward: write the null base case first, test the one-node tree, and keep a stack-overflow reflex on chains longer than 10^4. Everything later in the book – BSTs that keep themselves ordered, heaps that keep themselves shallow, tries that keep themselves sparse – is this skeleton wearing a promise.

Next: Chapter 2 takes the same recursion and asks what a *node* really holds, splitting the chapter's two reflexes into a named technique each.

6

Chapter

Constructing Trees from Data

Level: LC Easy–Hard • **Problems:** 19 (5 fully solved) • **Status:** Written

Chapter at a Glance

Chapters 4 and 5 pushed *forward*: here is a tree, store it, then lay it on a line. This chapter pulls *backward*. A contest hands you **raw data** – a list of edges, a **parent array**, a **degree sequence**, an **encoding**, even *answers to questions you ask* – and the real task is to **rebuild the tree** and then *prove the rebuild is legitimate*: connected, acyclic, exactly $n-1$ edges, a single root, degrees summing to $2(n-1)$. That reverse move – *identify the structure hiding inside the data* – is the skill every later chapter silently assumes the moment it reads `edges[i]=[u,v]`. The three decisions this chapter forces on you:

- **Given pairs, is there a tree, and which one?** An edge list or parent array determines a tree *only* if it passes the **tree certificate**: n nodes, $n-1$ edges, connected *and* acyclic (any two of the three force the third). Root it by finding the node with no parent. **L2**–**L4**
- **Given degrees or a code, can a tree realise them?** A list of positive integers is some tree's degree sequence *iff* it sums to $2(n-1)$; the **Prüfer code** turns every labelled tree into a length-$(n-2)$ sequence, so a degree array even tells you *how many* trees exist. **L3**–**L5**
- **Given only yes/no and numbers from an oracle, can you still find it?** Ask a *distance* or *ancestor* query, get one integer or bit back, and adapt. Each honest answer halves the candidates – reconstructing a hidden tree is binary search on a shape. **L2**–**L4**

You will learn to

- rebuild an adjacency structure from an edge list or a parent array and locate the root in one pass; **L2**–**L3**
- certify that a graph *is* a tree using the $n-1$-edges / connected / acyclic triangle, with a union-find pre-look; **L3**–**L4**
- decide whether a degree sequence is realisable and construct a tree that realises it; **L3**–**L4**
- read the Prüfer code as the bridge between trees, degree arrays and counting (full bijection in Chapter 30); **L4**–**L5**
- repair an over-full edge set to a tree (delete the redundant edge) and recover a hidden structure from adaptive queries. **L2**–**L5**

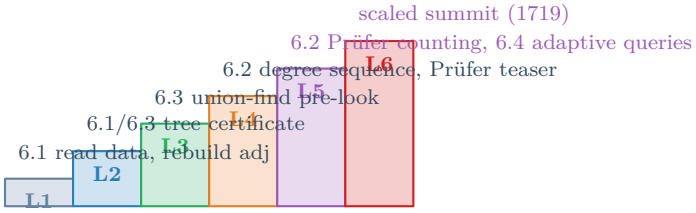
Prerequisites. Representation and rooting from Chapter 4 (§4.2, Code 4.2.1) – you already know how to pour edges into an `adj` list and DFS with a parent guard; this chapter adds the *validity* and *reconstruction* decisions on top. The timestamp/distance machinery of Chapter 5 (Chapter 5) is *used* as an oracle in §6.4, never re-derived.

How to Read This Chapter (two paths up the ladder)

Level badges L1–L6 as before: L2 is "read the data, rebuild the array", L3 is the solid interview medium, L5 is an LC Hard reconstruction with a case split.

Beginner path (L2–L3). Read §6.1 fully – it turns an edge list or parent array into a rooted tree and states the tree certificate. Then §6.3 up through the union-find "is it a tree?" test. Take every L2–L3 solved problem. You may *skip on a first pass*: the degree/Prüfer material of §6.2, the interactive protocol of §6.4, every L4+ variation, the hardening notes and the Summit. At the end of this path you can rebuild any tree from an edge list and certify it in $O(n)$.

Full path. Everything, in order – it ends with the scaled Summit (LC Hard 1719 *Number of Ways to Reconstruct a Tree*: read an ancestor-pair table, sort nodes by degree, greedily attach each node to the smallest feasible parent, and decide *unique / many / impossible*) and the reduction gallery showing that "rebuild from a parent array", "realise a degree sequence", "delete the redundant edge" and "find the node by distance queries" are the *same* certificate checked against four different data sources. The heavier theory – the Disjoint-Set-Union structure itself and the Prüfer/Cayley counting proofs – lives in Chapter 17 and Chapter 30.



Chapter 6 on the six-step ladder: rebuilding and certifying carry you from raw pairs to "this is a tree and here is its root" (L3); the degree/Prüfer and interactive topics and the scaled Summit (1719) reach L5. Beginner path = steps 2–3.

The Running Example: One Hidden Tree, Four Data Shadows

This chapter has *one* running example: the general tree T below, drawn once and then *never shown again* – instead every topic receives it as a different pile of *data*. The point is that four unrelated-looking inputs (an edge list, a parent array, a degree sequence, a bag of distance answers) all describe *this exact tree*, and reconstructing it is the whole game. Rooted at node 1, children visited in ascending order:

$$\text{edges } \{1\text{–}2,\ 1\text{–}3,\ 2\text{–}4,\ 2\text{–}5,\ 3\text{–}6,\ 3\text{–}7,\ 7\text{–}8\}.$$

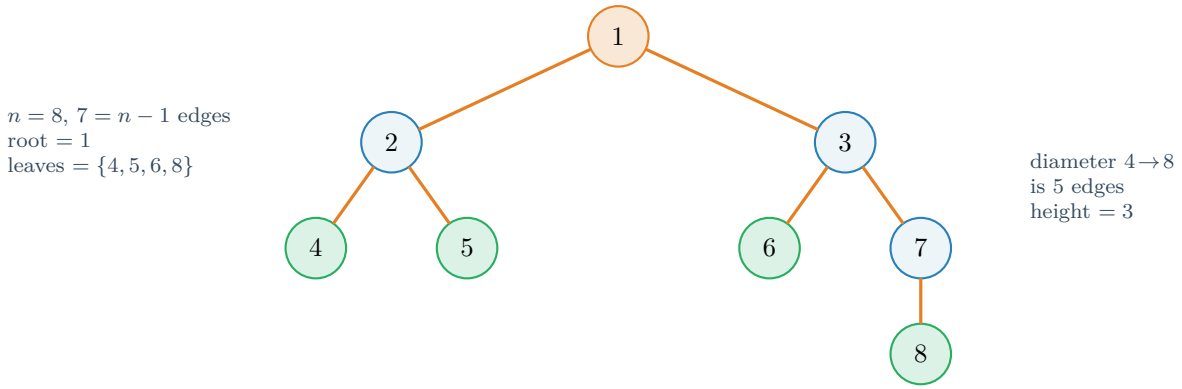
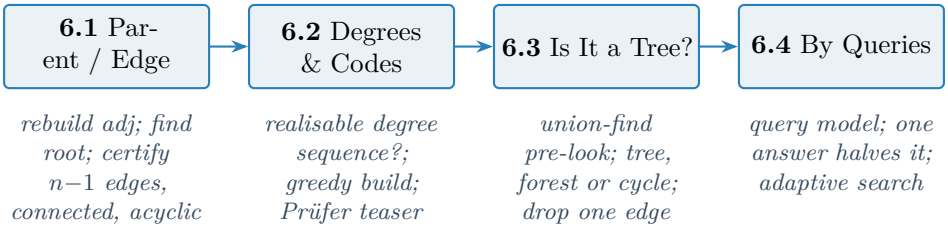


Figure 6.0.1: Running example T (rooted at 1). Four different data sources – the edge list above, the parent array, the degree sequence and a set of distance answers – each alone is enough to rebuild this tree, and no two of them look alike. That is the chapter in one picture.

Keep these four "shadows" of T in mind – they are re-derived, never merely asserted, in §6.1 and §6.2:

Data shadow	**Value for T (rooted at 1, nodes 1..8)**
edge list (undirected)	$[1,2], [1,3], [2,4], [2,5], [3,6], [3,7], [7,8]$ – $n-1 = 7$ pairs
parent array parent[·]	$[\,0, 1, 1, 2, 2, 3, 3, 7\,]$ (index i is node i; parent[1] $= 0$ marks the root)
degree sequence deg[·]	$[\,2, 3, 3, 1, 1, 1, 2, 1\,]$, sum $= 14 = 2(n-1)$
Prüfer code (teaser)	$(2, 2, 1, 3, 3, 7)$ – length $n - 2 = 6$; node v appears $\deg(v) - 1$ times
distance answers (6.4)	$\mathrm{dist}(4, 8) = 5$, $\mathrm{dist}(5, 6) = 4$, $\mathrm{dist}(1, 8) = 3$, …



The four topics, left to right, are the *same certificate* tested against four data sources. 6.1 rebuilds from *pairs*; 6.2 from *degrees/codes*; 6.3 *certifies* (and repairs) a suspicious edge set; 6.4 recovers the shape from *answers*. Each is construction from data seen one way.

Topic	Key results	Levels	Solved problems
§6.1 Parent Array / Edge List	rebuild adjacency; root-finding; the tree certificate; repair an over-edge set	L2–L4	W6.1.1
§6.2 Degrees, Sequences & Codes	degree sequence realisable $\iff$ sum $2(n-1)$; greedy build; Prüfer teaser	L3–L5	W6.2.1
§6.3 Is It a Tree? (Union-Find)	connected + acyclic + $n-1$ edges triangle; union-find test; tree/forest/cyclic	L2–L3	W6.3.1
§6.4 Recover by Queries	interactive model; what one query reveals; adaptive halving for a hidden node	L2–L4	W6.4.1

Problem Recognition Guide — When You See *this*, Think *that*

Construction problems announce themselves by *withholding the drawing*: you get pairs, counts, an array or an oracle instead of pointers. Match the phrasing; every row points at the section that builds the tool.

When the problem says...	Think...	Section
"`edges[i]=[u,v]`", "`parent[i]` gives the parent of i", a forest given as pairs	rebuild `adj`, find the root, verify the certificate	§6.1
"is it a valid tree?", "one extra edge added – remove it", "which edge is redundant"	the $n-1$/connected/acyclic test; union-find pre-look	§6.3
"given the degrees of every node, build a tree", "how many trees match this degree array"	degree sequence sums to $2(n-1)$; Prüfer code counts them	§6.2
"an encoding / grid / nested string – construct the tree"	recursive build-by-rule (quad tree); a stack parse	§6.2
"you may ask *queries*; find the hidden node/edge in k questions"	each answer is a bit – adapt, halve, descend	§6.4
"reconstruct so that every listed ancestor-pair holds; is it unique?"	sort by degree, greedy parent, transitivity check (Summit)	W6.4.2

Constraint you see	What it forces	Chapter-6 tool
$n-1$ edges promised, graph may be corrupt	do not trust it – run the certificate/union-find	§6.3
a tree *reconstructed* then *queried* many times	rebuild once from the array, answer later	§6.1
"how many / which structure fits this degree list"	Prüfer bijection: degrees pin the count	§6.2 (Chapter 30)
interactive judge, tight query budget	information-per-query drives the strategy	§6.4
N up to $2 \cdot 10^5$ edge list	rebuild *iteratively*; $O(n\alpha)$ union-find	§6.3 (Chapter 17)

Not this chapter?

If the tree is *already given* as pointers or an `adj` list and you only need to walk or store it, that is Chapter 4 (Chapter 4). If you must rebuild a *binary* tree from two of its *traversal orders* (inorder+preorder, inorder+postorder) or serialise/parse it, that is Chapter 3 (Chapter 3) – construction from *traversals*, a different data source than the *arbitrary* data here. If the real question is once the tree stands – subtree ranges, distances, LCA – that is Chapter 5 (Chapter 5) and Chapter 16 (Chapter 16). If you need the Disjoint-Set-Union structure itself (path compression, rank), that is Chapter 17 (Chapter 17); we borrow only its *pre-look* as a tree test. If you want the full Prüfer bijection and Cayley's n^{n-2} *proof*, that is Chapter 30 (Chapter 30). This chapter owns *reading a tree out of whatever data you were handed, and proving you read it right.*

Notation Used in This Chapter

Symbol	Meaning	Symbol	Meaning
T	the running general tree (Figure 6.0.1)	parent(v)	v's parent; $0/-1$ marks the root
$\deg(v)$	number of neighbours of v (degree)	$\mathcal{E}$	edge set, $\lvert\mathcal{E}\rvert = n-1$ for a tree
$\mathcal{P}$	the Prüfer code, length $n-2$	$\mathcal{C}$	number of connected components
$\mathrm{dist}(u,v)$	edges on the u–v path (from Chapter 5)	$\mathcal{R}$	a query/answer oracle for a hidden tree
find, uni	union-find ops (Chapter 17)	**L3**	difficulty badge (runs **L1**–**L6**)

As in earlier chapters, dist and depth count **edges**, general trees are stored as an adjacency list `adj` (`vector` of `vector<int>`) with a parent guard, and every listing is complete *C++* (standard header included). Node numbering follows the problem: many tree-input problems are 1**-indexed** (LeetCode 684/685 label nodes $1..n$), which is why the running example T is 1-indexed here; `adj` arrays are 0- or 1-sized to match.

6.1 Reconstruct from a Parent Array or an Edge List

Previously in this chapter . . .

1. How do you pour an undirected edge list into an `adj` list, and why does the DFS need a *parent guard*? (§4.2, Code 4.2.1)
2. For a rooted tree, what single number per node is 0 for the root and ≥ 1 for every other node, and how do you read it from an array? (§4.2)
3. How many edges does any tree on n nodes have, and why? (Chapter 1, Chapter 5)
4. What is $\sum_v \deg(v)$ for the running tree T of Figure 6.0.1?

In This Section

Almost every tree problem *begs the question in code first*: `vector<vector<int>> edges` or `vector<int> parent`. Before any algorithm runs, you must turn that data back into the object – an adjacency structure with a distinguished root – and then *prove the data really describes a tree.* Chapter 4 showed you how to *store* a tree you already trust; here the trust must be *earned.* Two objects do all the work. The **tree certificate** is the trio "$n-1$ edges, connected, acyclic": any two of the three *force* the third, so checking two is enough. The **degree sum** $\sum_v \deg(v) = 2(n-1)$ is a cheap arithmetic canary that instantly rejects corrupt input. Read the parent array's zeros, or an edge list's in-degrees, and the root falls out. Everything in this section is bookkeeping with one theorem behind it – and one Hard problem (LC 685) where the bookkeeping *is* the difficulty. **Prerequisites.** §4.2 (adjacency build, parent-guarded rooting, parent / depth / size); Chapter 1 ($n-1$ edges). This section adds *verification and root detection from raw data*; it does not re-teach traversal.

After this section you can

- rebuild `adj` from an edge list *or* a parent array in one $O(n)$ pass; **L2**
- find the unique root from in-degrees or the parent array without any DFS; **L2–L3**
- certify "is this data a tree?" by checking any two of the certificate's three legs; **L3**
- repair an *over*-full edge set into a tree by deleting the one redundant edge. **L4**

Pattern card – rebuild the tree from its data

Recognition keywords. the input is *not* pointers or a drawing but a pile of *pairs*: `edges[i]=[u,v]`, a `parent[i]` array, an `ancestor[i]` list, or a table of node attributes you must link into a tree before solving anything else.

✓ When to use this

- the statement hands you `edges` / `parent` and the real question (rooting, subtree sizes, a repair) needs the `adj` built first;
- "is the input a valid tree?" / "which edge is extra?" – certificate + degree sum;
- "rebuild so a stated rule holds" (every ancestor pair, every degree) – §6.2 and the Summit continue this idea;
- the data is *nearly* a tree (one edge too many) – W6.1.1.

✗ When NOT to use this

- you are given `TreeNode*` / `root` already – skip straight to Chapter 3;
- you must rebuild from two *traversal orders* – that is Chapter 3, not arbitrary data;
- the tree is trusted and the work is answering subtree ranges – Chapter 5.

One-line test: *is there a tree* ***hiding inside an array of pairs****? Build it, then certify it –*

this section.

6.1.1 Intuition

An edge list is a tree *in disguise*: n nodes, and a pair $[u, v]$ is a single connection. To see the object, put every pair into both endpoints' neighbour lists – exactly the `adj` build of §4.2 – and, if the tree is *rooted*, orient it with one DFS. The only *new* questions are: **which node is the root, and is this even a tree?**

Root detection needs no DFS at all. In a rooted tree exactly one node has no parent. So: from a *parent array* the root is the index v with $\text{parent}[v] = 0$ (or -1, or $\text{parent}[v] = v$); from a directed *edge list* $[u, v]$ meaning "u is v's parent," the root is the node that never appears as a v (in-degree 0); from an *undirected* list there is no root until you *choose* one and root it (§4.2). Validity is a counting game: n nodes demand $n - 1$ edges, and every node except the root must have exactly one parent.

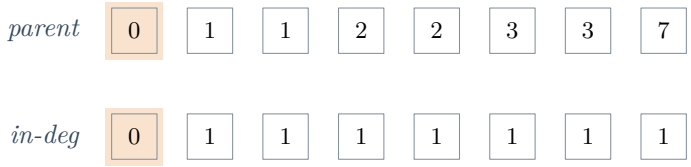


Figure 6.1.1: Root detection is *arithmetic*, not traversal. Read the parent array: the single 0 marks node 1. Or read the directed edge list's in-degrees: the unique 0 (node 1) is the root and every other node is exactly 1 – if any node shows in-degree 2, the data is *not* a rooted tree (that is exactly LC 685's alarm, W6.1.1).

6.1.2 Definitions and Notation

Definition 6.1.1 (Reconstruction from pairs)

Given a set $\mathcal{E}$ of $n - 1$ undirected pairs on nodes $\{1, \ldots, n\}$, the **reconstruction** is the adjacency structure adj with $v \in adj[u]$ and $u \in adj[v]$ for every $\{u, v\} \in \mathcal{E}$. A **parent array** $\text{parent}[\cdot]$ with $\text{parent}[root] = 0$ is reconstructed by adding the directed edge $\text{parent}[v] \to v$ for every $v \neq root$. The reconstruction is *valid* if the result is a tree (rooted, if directed).

In plain words. Reconstruction just means "fill the neighbour lists from the pairs, and read the root from the array." The data can be a lie – two parents for one node, a cycle, a disconnected piece – and validity is what we test next.

Theorem 6.1.2 (The tree certificate)

For a graph G on n nodes, any **two** of the following three conditions imply the third, and all three are equivalent to "G is a tree":

1. G has exactly $n - 1$ edges;
2. G is connected;
3. G is acyclic.

Proof

A tree (connected + acyclic) has $n-1$ edges by the classical count (each of the $n-1$ merges in a spanning process adds one edge). If (1) $n-1$ edges and (2) connected hold but a cycle existed, breaking one cycle edge keeps connectivity and drops the count to $n-2$, contradicting that a connected graph needs $\geq n-1$ edges – so (3). If (1) and (3) hold but G is disconnected with $k \geq 2$ components, each acyclic component on n_i nodes has $n_i - 1$ edges, totalling $n - k < n - 1$ edges, contradicting (1) – so (2). If (2) and (3) hold, a DFS tree uses all n nodes and every non-tree edge would close a cycle, so exactly $n-1$ edges: (1). Hence any two legs force the third.

■

Key Insight

You never check all three – checking two is the whole certificate, and each check is one cheap $O(n)$ pass: count the edges ($= n-1$?), and run one DFS or union-find to test connectedness *or* acyclicity. Pick the two legs your representation makes easiest: an edge list gives (i) for free; a union-find gives (iii) directly (§6.3).

Theorem 6.1.3 (Handshaking / degree sum for trees)

For any tree on n nodes, $\sum_v \deg(v) = 2(n-1)$. Consequently the number of *leaves* (degree 1) equals $2 + \sum_{v:\deg(v)\geq 2}(\deg(v) - 2)$, and a positive-integer list is a candidate tree degree sequence only if it sums to $2(n-1)$.

Proof

Each edge touches exactly two nodes, so counting "+1 per endpoint" over all edges gives 2 times the edge count $= 2(n-1)$, which is the sum of degrees. Writing $d-2$ for each node's *excess* over 2 and separating the $\sum(1) = n$ baseline: $\sum_v \deg(v) = n + \sum_v(\deg(v) - 1)$, so $\sum_v(\deg(v) - 1) = 2(n-1) - n = n - 2$; splitting into leaves (deg $= 1$, term 0) and the rest gives the leaf identity.

■

Proposition 6.1.4 (Reading the root)

In a rooted tree on n nodes given by a directed edge list $[u, v]$ ("u is v's parent") or a parent array: the root is *unique* and is the sole node of in-degree 0 (equivalently the sole index with parent$[v] = 0$), and every other node has in-degree 1. A data set with zero or two or more in-degree-0 nodes, or any node of in-degree ≥ 2, is **not** a rooted tree.

Proof

The root has no parent (in-degree 0); every other node has exactly one parent (in-degree 1), so in-degrees are a permutation of $\{0, 1, \ldots, 1\}$ summing to $n-1$. If the root were not unique, two nodes with no parent would have no common ancestor to connect them (contradicts connectivity), and if some node had two parents the in-degree sum would exceed $n-1$, breaking Theorem 6.1.2's edge count. Hence exactly one root, and the in-degree histogram certifies the orientation.

■

6.1.3 The algorithm: build *adj*, find the root, certify

Code 6.1.1: Rebuild adjacency from an edge list or parent array, find the root, certify validity

```cpp
#include <bits/stdc++.h>
using namespace std;

// (A) undirected edge list -> adjacency; root chosen by caller.
vector<vector<int>> buildFromEdges(int n, const vector<vector<int>>& edges) {
    vector<vector<int>> adj(n + 1);
    for (auto& e : edges) { adj[e[0]].push_back(e[1]); adj[e[1]].push_back(e[0]); }
    return adj;
}

// (B) parent array (1-indexed, parent[root]=0) -> children adj + root + validity.
vector<vector<int>> buildFromParent(const vector<int>& par, int& root, bool& ok) {
    int n = (int)par.size() - 1;               // par[0] unused
    vector<vector<int>> adj(n + 1);
    vector<int> indeg(n + 1, 0);
    root = 0; int zeros = 0; ok = true;
    for (int v = 1; v <= n; ++v) {
        int p = par[v];
        if (p == 0) { root = v; ++zeros; }     // candidate root(s)
        else if (p >= 1 && p <= n) { adj[p].push_back(v); ++indeg[v]; }
        else ok = false;                        // parent out of range
    }
    if (zeros != 1) ok = false;                 // need exactly one root
    for (int v = 1; v <= n; ++v)
        if (v != root && indeg[v] != 1) ok = false;   // every other node: one parent
    // optional full check: DFS from root must visit all n nodes (connectivity)
    return adj;
}
```

Complexity

Time $O(n)$ – one pass per edge / array slot, plus (optionally) one DFS for connectivity. Memory $O(n)$ for `adj` and the in-degree histogram. This is the cheapest possible reconstruction: you never rebuild what a single scan already gives. On a corrupt input the `ok` flag or an in-degree ≥ 2 is the early warning; §6.3 turns that warning into a full cycle test.

6.1.4 Worked Example

Example 6.1.5 (Rebuilding T from its two data shadows)

Take the running tree T (Figure 6.0.1), $n = 8$. *From the parent array* parent $= [0, 1, 1, 2, 2, 3, 3, 7]$ (index 0 unused): the only 0 is at slot 1, so *root* $= 1$ and *zeros* $= 1$ (✓). Every other slot has one parent, so each $v \neq 1$ is appended to $adj[\text{parent}[v]]$ and gets in-degree 1 (✓). *From the edge list* $[1, 2], [1, 3], [2, 4], [2, 5], [3, 6], [3, 7], [7, 8]$: seven edges $= n - 1$ (✓), each added to both endpoints. Degree check: deg $= [2, 3, 3, 1, 1, 1, 2, 1]$, sum $= 14 = 2(n - 1)$ (✓). One DFS from 1 visits all 8 nodes (connected). Two legs of Theorem 6.1.2 hold $\Rightarrow$ it *is* a tree, and 1 is its root – reconstructed, not assumed.

The degree-sum canary rejects two corrupt inputs instantly

Corrupt A: add edge $[4, 5]$ to T's edge list. Now 8 edges for 8 nodes: count leg (1) fails ($8 \neq n - 1$), $\sum_v \deg(v) = 16 \neq 14$ – reject before any traversal. *Corrupt B:* set parent$[7] = 2$ (so node 7 has parents 2 *and*, via a stray edge, another). In-degree of 7 becomes 2: Proposition 6.1.4 fires – no valid root assignment – reject. This is exactly the shape LC 685 must repair (W6.1.1).

6.1.5 Variations and Twists

- **Root from a directed edge list**: mark every v that appears as a child; the root is the node never marked. $O(n)$ with a boolean array, no histogram needed. **L2**
- **Undirected, root free**: no in-degree tells you the root – root at *any* node (§4.2) or, if the problem wants the "true" root, look for extra clues (an `isRoot` flag, or node 0/1 by convention). **L2–L3**
- **Reconstruct-then-query**: build once, store children in *given* order, and answer many descendants/subtree questions on the rebuilt tree (Chapter 5 takes over). **L3**
- **Weighted pairs**: the edge $[u, v]$ carries a value; build `adj` of `(to, w)` pairs – the reconstruction logic is identical (the LC 399 practice below is exactly this). **L3–L4**
- **Near-trees**: n edges (one too many) or a lone in-degree-2 node – do not reject, *repair* by deleting the offending edge (W6.1.1). **L4**

6.1.6 Common Pitfalls

Trap 1 – trusting that the input is a tree

Statements say "it is a tree" but corrupt or off-by-one input is a classic WA source. At minimum check $|\mathcal{E}| = n - 1$ and one root (Proposition 6.1.4); §6.3 adds the full cycle test. Never run your elegant subtree DP on a graph with a cycle – the DFS loops forever.

Code 6.1.2: WRONG: assume the array is a valid tree, root hard-coded

```
int root = 1;                       // what if node 1 isn't the root?
for (int v = 2; v <= n; ++v) adj[par[v]].push_back(v);
```

Code 6.1.3: RIGHT: detect the root and validate

```
int root = 0, zeros = 0;
for (int v = 1; v <= n; ++v) if (par[v] == 0) { root = v; ++zeros; }
if (zeros != 1) { /* not a valid rooted tree: reject / handle */ }
```

Trap 2 – mixing 1-indexed data with 0-indexed arrays

LeetCode tree-input problems label nodes $1..n$ (684, 685) while your `adj`/`vector` indices start at 0. Size arrays $n+1$ and keep the intended index, or translate *consistently* everywhere – a half-translation drops node n or reads slot 0 as a real node.

Code 6.1.4: WRONG: n-sized array on $1..n$ labels – index n out of range

```
vector<int> indeg(n);       // labels run 1..n but slots are 0..n-1
```

Code 6.1.5: RIGHT: size $n + 1$, ignore slot 0

```
vector<int> indeg(n + 1, 0);        // one slot per real label
```

Trap 3 – forgetting the parent guard when treating edges as a tree

If you rebuild *undirected* `adj` and then DFS without a parent guard, the first step back to the parent re-visits the node you came from and, in a graph that *isn't* yet certified, may loop. Always pass `parent` down the DFS (Code 4.2.1) *and* run the certificate.

6.1.7 Problem Lab

The section's anchor is LC 685 – reconstruct/repair a *rooted* tree given n directed edges (one too many). It forces every idea above at once: the in-degree histogram, the unique-root proposition, and the union-find cycle test you meet properly in §6.3.

W6.1.1 Redundant Connection II

Source: LeetCode 685 LC Hard L4
Techniques: directed rooted tree, in-degree (parent) analysis, union-find cycle test, case split

Redundant Connection II

A **rooted tree** on n nodes (labels $1 \dots n$) had exactly **one** extra directed edge added, so the input `edges` has n edges (each `[u,v]` means "u is v's parent"). The extra edge makes either (a) some node have **two parents**, or (b) a **cycle**, or both. Return *one* edge to remove so the rest is a valid rooted tree; if several answers exist, return the one that appears **last** in `edges`. $3 \le n \le 1000$.

Attempt it yourself first – L4, give yourself 35 minutes

A fair first attempt: a rooted tree has $n-1$ edges and every non-root node one parent, so *one* edge is wrong. Enumerate which edge to delete and test validity each time.

Peek (1 min lost): Look at **in-degrees**. If some node has in-degree 2, one of its two incoming edges is the culprit – which one? If *no* node has in-degree 2, what does n edges guarantee?

Nudge (5 min lost): In-degree 2 gives two candidate edges. *Temporarily drop the later one* and run a union-find over the rest: if the rest is still cyclic, the culprit is the *earlier* edge; otherwise the dropped later edge is the answer. No in-degree 2? Just find the edge that closes a cycle.

Full hint (you will not solve this one): Compute in-degrees. Find a node with in-degree 2; label its two incoming edges `first` (earlier index) and `banned` (later index). Union-find over all edges, *skipping* `banned`: the first edge whose endpoints are already connected is a cycle – if a two-parent node existed return `first`, else return that cycle edge. If no cycle remains, return `banned`. Full build and proof below.

➤ Understanding the Problem

A valid rooted tree has $n-1$ edges with exactly one node of in-degree 0 (the root) and $n-1$ nodes of in-degree 1. One extra edge pushes the count to n edges and breaks the picture in one of three *overlapping* ways: a node gains a **second parent** (in-degree 2); or the new edge points *into* an ancestor, creating a **cycle**; or it does both at once (the second parent is also on a cycle). The removal must leave a legal rooted tree, and ties go to the **last** edge in the input – a tie-break you honour by scanning in order and only *proposing* later edges.

➤ Key Observations

Observation 1. In-degree 2 is *local* evidence: if node c has parents p_1, p_2 (edges at indices $i_1 < i_2$), the redundant edge is one of those two – no other edge can create the second parent. So the search space collapses from n edges to **two**.

Observation 2. To choose between the two, *assume* the later edge i_2 is the intruder and test whether the remaining $n-1$ edges (including i_1) are *acyclic*. If they are, i_2 was indeed safe to drop. If a cycle survives, the cycle must use i_1, so i_1 is the true intruder.

Observation 3. With *no* in-degree 2 node, n edges on n nodes force exactly one cycle (Theorem 6.1.2: edges $> n-1 \Rightarrow$ cyclic). A union-find that processes edges in input order reports the edge that *closes* the cycle – automatically the last such edge.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** Try deleting every edge and re-checking "is it a rooted tree" with a fresh DFS.
Killed by: $O(n)$ candidates $\times$ $O(n)$ validity $= O(n^2)$ *tests*, each with fiddly reachability – correct, but you reinvent the cycle check n times and still need the "last edge" tie-break. A single union-find pass subsumes all of it.

✗ **Tried:** Treat a two-parent node and a cycle with one rule.
Killed by: They interact: the extra edge can *both* give a second parent *and* sit on a cycle. Ignoring the second parent (just chasing any cycle) returns the wrong edge when the fix is to cut the *earlier* of the two parent edges.

✓ **Aha!** Two facts, one scan. In-degree 2 names the *only* two candidate edges; a union-find that *skips the later* candidate tells you which of the two survives the cycle test. No two-parent node means the cycle edge itself is the answer.

➤ Solution

Compute in-degrees. If a node c has two incoming edges at input indices $i_1 < i_2$, set `banned`$= i_2$ and `first`$= i_1$. Run union-find over $1..n$ processing every edge *except* `banned` in input order: on the first edge whose endpoints are already in one set, a cycle exists – return edges[`first`] if a two-parent node was found (dropping the later edge did *not* kill the cycle), otherwise return that closing edge. If the scan finishes with no cycle, return edges[`banned`] (the later parent edge was the redundant one) – or an empty answer in the degenerate no-cycle-no-two-parent case, which cannot occur for valid input.

➤ Proof of Correctness

Proof

Case 1: some node has in-degree 2. Let its parent edges be e_1 (earlier) and e_2 (later). Any valid tree must drop at least one of them, since no node may have two parents. If removing e_2 yields an acyclic edge set, those $n-1$ edges with the surviving single-parent structure are a rooted tree, so e_2 is a correct removal; e_2 is later than e_1, matching the tie-break, so we return e_2. If removing e_2 still leaves a cycle, then e_1 lies on a cycle (the union-find reports a cycle among the edges excluding e_2), so removing e_2 alone cannot work – the redundant edge must be e_1, which also resolves the double parent. Hence e_1 is returned. *Case 2: no*

node has in-degree 2 and n edges exist: exactly one cycle (*Theorem* 6.1.2 contrapositive). The union-find returns the last edge closing that cycle. Removing it breaks the only cycle and keeps every in-degree ≤ 1, so the rest is a rooted tree; it is the latest valid answer because we scan in order and any earlier cycle edge is not chosen. In both cases the returned edge is exactly the one whose removal restores a valid rooted tree, chosen to appear last among ties.

■

➤ Implementation

Code 6.1.6: Redundant Connection II – in-degree scan + union-find with one edge banned (LeetCode 685)

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
    vector<int> p;
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool uni(int a, int b) {                    // false == already connected (cycle)
        a = find(a); b = find(b);
        if (a == b) return false;
        p[b] = a; return true;
    }
public:
    vector<int> findRedundantDirectedConnection(vector<vector<int>>& edges) {
        int n = (int)edges.size();              // n nodes, n edges
        p.assign(n + 1, 0);
        for (int i = 0; i <= n; ++i) p[i] = i;
        vector<int> indeg(n + 1, 0);
        for (auto& e : edges) ++indeg[e[1]];

        int banned = -1, first = -1;            // the two edges into an in-degree-2
    ↪ node
        for (int i = 0; i < n; ++i) if (indeg[edges[i][1]] == 2) {
            if (first == -1) first = i;         // earlier incoming edge
            else             banned = i;        // later incoming edge -> skip it
    ↪ first
        }
        for (int i = 0; i < n; ++i) {
            if (i == banned) continue;          // pretend e_later is already removed
            if (!uni(edges[i][0], edges[i][1])) // cycle closes here
                return (banned != -1) ? edges[first] : edges[i];
        }
        return (banned != -1) ? edges[banned] : vector<int>{};
    }
};
```

➤ Trace

Two official inputs, one per case

Ex. 1 `edges=[[1,2],[1,3],[2,3]]`. In-degrees: node 3 has 2 (from 1 and 2). So `first=1` ($[1,3]$), `banned=2` ($[2,3]$). Union-find skipping index 2: union $1,2$; union $1,3$; no cycle. Loop

ends clean ⇒ return edges[`banned`]= $[2, 3]$. ✓
Ex. 2 `edges=[[1,2],[2,3],[3,4],[4,1],[1,5]]`. All in-degrees ≤ 1 (node 1 from 4), so `banned=first=-1`. Union in order: 1–2, 2–3, 3–4 merge $\{1, 2, 3, 4\}$; edge $[4, 1]$: 4, 1 already connected ⇒ cycle, `banned=-1` so return $[4, 1]$. ✓(matches the expected $[4, 1]$).

➤ Complexity

Complexity

Time $O(n\,\alpha(n))$ – one in-degree pass and one union-find pass; α is the inverse Ackermann (≤ 4 for all realistic n). Space $O(n)$. At $n \leq 1000$ the $O(n^2)$ brute force also passes, but this is the clean linear solution. Verdict: ✓ **Accepted**.

➤ Pitfalls and Variants

Common Mistake

Three ways to break it: (i) skipping the *earlier* parent edge instead of the later one – the tie-break and the cycle logic both want the *later* edge banned first. (ii) Running the union-find *including* the banned edge – then the two-parent node always looks cyclic and you return the wrong parent. (iii) Forgetting 1-indexing: nodes are $1..n$, so `p/indeg` are sized $n + 1$. *Variant:* LC 684 is the undirected, single-cycle special case (just Case 2) – W6.3.1.

Key takeaways

1. **In-degree is evidence.** A second parent pins the culprit to two edges – the whole search space – because the degree sum Theorem 6.1.3 and Proposition 6.1.4 leave no room for a third possibility.
2. **One union-find pass replaces n validity tests** and hands you the "appears last" edge for free, because edges are consumed in input order.
3. **Reconstruction and repair are the same skill:** read structure out of degrees and pairs, then *test* it – exactly the certificate of Theorem 6.1.2.

AC ⟶ red: hardening this code

- **Both defects at once**: if a two-parent node is *also* on the cycle, banning the later edge still leaves a cycle, and the code correctly returns the *earlier* parent edge – worth an explicit test input like `[[2,1],[3,1],[4,2],[1,4]]`.
- **Larger n**: the path-compressed union-find keeps it near-linear; avoid a per-candidate DFS re-check (the $O(n^2)$ trap). Full DSU theory (rank, compression) is Chapter 17.
- **Generalisation**: "delete k edges to make a forest" needs the component count of §6.3; this problem is the $k = 1$, rooted, tie-broken special case.

6.1.8 Practice Problems

Stuck? Work through this list

- Did you detect the root by *counting* (one in-degree-0 node), not by hard-coding? (Trap 1, Proposition 6.1.4.)
- Are your arrays sized for 1-indexed labels? (Trap 2.)
- When choosing between two parent edges, did you *ban the later* one first and test for a

surviving cycle? (W6.1.1.)
- Did you use the $n-1$ / degree-sum canary before trusting the data? (Theorem 6.1.3.)

Q6.1.1 Evaluate Division – LeetCode 399 LC Medium L3

Hint: You are handed raw *pairs* `equations[i]=[a,b]` with a value a/b, and must answer queries x/y. There is no tree given – you must *reconstruct* a structure from the pairs: build an adjacency list where $a \to b$ carries weight w and $b \to a$ carries $1/w$, then each query is a weighted path product through that rebuilt graph (DFS/BFS, or a weighted union-find). Recognise it as "build `adj` from pairs, then answer" – §6.1 in weighted clothing, with connectivity deciding "answerable vs -1".

10-second refresher – 6.1 Reconstruct from Parent Array / Edge List

Turn pairs into `adj` in one pass; find the root as the sole in-degree-0 node (Proposition 6.1.4); certify with any *two* of "$n-1$ edges, connected, acyclic" (Theorem 6.1.2); reject bad input with $\sum_v \deg(v) = 2(n-1)$ (Theorem 6.1.3). To *repair* one extra edge (LC 685), let the in-degree-2 node pin two candidates and a union-find (skipping the later) choose (W6.1.1).

Self-test (answer from memory, tick the box):

- ☐ Rebuild T from its parent array $[0,1,1,2,2,3,3,7]$: name the root and the leaves.
- ☐ State the tree certificate and prove that "$n-1$ edges + acyclic" forces connectedness.
- ☐ Why must a rooted tree's in-degrees be a permutation of $\{0,1,\ldots,1\}$?
- ☐ In LC 685, why do you ban the *later* parent edge, not the earlier, before the cycle test?
- ☐ Give the degree-sum of any tree on n nodes and explain the leaf formula.

6.2 Build from Degrees, Sequences and Codes

Previously in this chapter ...

1. State the handshaking identity for a tree's degrees and the exact value of $\sum_v \deg(v)$ for the running tree T. (Theorem 6.1.3)
2. Why can a graph with a node of in-degree 2 never be a rooted tree? (Proposition 6.1.4)
3. How many edges must a realisation of a degree sequence have, and how do you read that straight off the sequence? (Theorem 6.1.2)
4. In the leaf-peel of a tree, why is there *always* a node of degree 1 to remove? (Chapter 1)

In This Section

Section 6.1 rebuilt a tree from *pairs*. Here the input is even coarser: a list of *numbers*. You are handed a **degree sequence** $d_1, \ldots, d_n$ and asked (i) does *any* tree have exactly these degrees, and (ii) if so, *build one* – or even *count* them. The answer is shockingly clean: a list of positive integers is a tree degree sequence **iff** it sums to $2(n-1)$ (Theorem 6.2.2), and the constructive proof is a leaf-peel that is the exact inverse of the **Prüfer code** – the length-$(n-2)$ sequence that bijects labelled trees with multisets, and the reason a degree array also tells you *how many* trees realise it (Proposition 6.2.4; the full bijection lives in Chapter 30). The same "rebuild a branching object from aggregate data" instinct reappears in a completely different guise: **constructing a quad tree** by recursively partitioning a grid (LC 427), where the data is a matrix and the tree is a divide-and-conquer shape. And a *topological order* (LC 210) is a linear "tree" peeled from in-degrees – the identical degree-peel

mechanic. One idea – *read structure back out of degrees* – three faces. **Prerequisites.** Theorem 6.1.3, Proposition 6.1.4 from §6.1. The Prüfer decoding is *used* as a construction; its bijective proof and Cayley's count are Chapter 30. This section adds *realisability, greedy construction and counting from a degree array*, plus recursive-shape construction.

After this section you can

- decide in $O(n)$ whether a degree list is realisable by a tree, and say why; **L3**
- construct *an* explicit tree realising a given degree sequence by a leaf-peel; **L3–L4**
- read the Prüfer code as $\deg(v) - 1$ occurrences and count realising trees; **L4–L5**
- build a quad tree (or any recursive partition tree) from raw grid data by divide and conquer. **L3**

Pattern card – realise a tree from degrees / codes / aggregate data

Recognition keywords. the input is a list of *numbers*, not pairs: a `degree[]` array, an `ancestor-count`, a Prüfer-like *code*, a grid you must turn into a partition tree, or an in-degree table you peel into an order.

✓ When to use this

- "does a tree exist with these degrees? / build one" – Theorem 6.2.2 plus the leaf-peel;
- "how many labelled trees have this degree sequence?" – Prüfer multinomial Proposition 6.2.4;
- "compress this $n \times n$ grid into a recursive tree" – quad tree, W6.2.1;
- "order the tasks so every dependency comes first" – in-degree peel (LC 210, the practice).

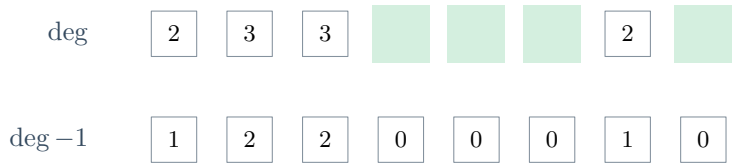
✗ When NOT to use this

- you already have the edges and only need the root – §6.1;
- the sequence is an *arbitrary graph*'s degrees (not necessarily a tree): that is the Erdős–Gallai / Havel–Hakimi question for general graphs, out of scope here;
- you must answer subtree/path queries on a trusted tree – Chapter 5.

One-line test: *is the input a pile of **numbers** that must become a **shape**? Check the sum, then peel leaves – this section.*

6.2.1 Intuition

Forget the labels for a moment. Every tree on n nodes has $n - 1$ edges, and each edge donates 1 to two endpoints, so the degrees must sum to $2(n - 1)$ – there is simply no other total (Theorem 6.1.3). The surprising direction is the *converse*: as long as every $d_i \geq 1$ and the sum is $2(n - 1)$, you can *always* wire up a tree with exactly those degrees. The recipe is a peeling game. Because the average degree is $2(n - 1)/n < 2$, some node has degree 1 – a forced *leaf*. Pin that leaf onto any node that still has degree to spare (> 1), spend one unit from each, and recurse on the smaller problem. Each pin removes exactly one leaf and never creates an illegal degree, so after $n - 1$ pins you have a genuine tree. This is the same bookkeeping that underlies Prüfer encoding, seen from the decode side.



top row sums to $14 = 2(n-1)$ ✓ *realisable;*
green cells are the forced leaves the peel starts with;
bottom row sums to $6 = n-2$ *and is the Prüfer multiset of* T

Figure 6.2.1: Two arithmetic views of the same degree array. The *top* row is T's degrees; the $\sum = 2(n-1)$ test is the whole realisability certificate (Theorem 6.2.2). The *bottom* row subtracts 1 from each entry – its sum is $n-2$, and reading node v exactly $\deg(v)-1$ times gives the Prüfer *multiset* $(2,2,1,3,3,7)$ of §6.1's teaser.

6.2.2 Definitions and Notation

Definition 6.2.1 (Tree degree sequence; realisability)

A list $d = (d_1, \ldots, d_n)$ of positive integers is **tree-realisable** if there exists a tree G on nodes $\{1, \ldots, n\}$ with $\deg(i) = d_i$ for every i. The construction problem asks for one such G; the counting problem asks how many labelled G realise d.

In plain words. "Can these numbers be the degrees of *some* tree, and if so show me the tree (or count them)." Nothing about positions or edges is given – only the per-node totals.

Theorem 6.2.2 (Degree-sequence realisability for trees)

A list $d_1, \ldots, d_n$ of positive integers is tree-realisable **if and only if** $\sum_{i=1}^{n} d_i = 2(n-1)$. When $n \geq 2$, the leaf-peel of Code 6.2.1 constructs a realising tree.

Proof

(Only if) Every tree has $n-1$ edges and each edge contributes 2 to the degree sum (Theorem 6.1.3), so any realisation forces $\sum d_i = 2(n-1)$, and clearly each $d_i \geq 1$ (a node in a connected graph on $n \geq 2$ vertices has a neighbour). *(If)* We run the peel. Since every $d_i \geq 1$ and $\sum d_i = 2(n-1) < 2n$, the average is below 2, so some node has $d_i = 1$ – a leaf exists at every stage. Pin a current leaf ℓ to any node p of current degree > 1 and append edge (ℓ, p), decrementing both. The invariant "all remaining degrees positive and their sum equals twice the edges still to add" is preserved: the sum drops by 2 (one from ℓ, one from p) and we remove one leaf, so there are $n-1$ iterations before all degrees reach 0. The $n-1$ produced edges connect all n nodes (each new edge ties a not-yet-used leaf into the growing component, and the final edge joins the last two leaves), giving $n-1$ edges and, by Theorem 6.1.2, a tree whose degrees are exactly d.

■

Key Insight

This is a *sharp* contrast with general graphs: for an arbitrary graph the realizability test is the Erdős–Gallai inequalities or a Havel–Gakimi peel. For *trees* the single equation $\sum d_i = 2(n-1)$ (plus $d_i \geq 1$) is necessary *and* sufficient – trees are rigid enough that one number decides everything. The degree array is "missing information" only about *which*

tree, never about *whether* one exists.

Definition 6.2.3 (Prüfer code (construction view))

The **Prüfer code** of a labelled tree on $\{1,\dots,n\}$ is a sequence of length $n-2$ over the alphabet $\{1,\dots,n\}$ obtained by repeatedly deleting the smallest leaf and recording its neighbour. Crucially, $\deg(v)-1$ equals the number of times v appears in the code, so *any* ordering with those multiplicities decodes to *a* tree realising d. (The deletion–recording bijection and its inverse are proven in full in Chapter 30; here we only *use* the decode.)

Proposition 6.2.4 (Counting realising trees from a degree sequence)

If $d_1,\dots,d_n$ is tree-realisable ($n \geq 2$), the number of *labelled* trees whose degree sequence is exactly d is

$$\frac{(n-2)!}{\prod_{i=1}^{n}(d_i-1)!}.$$

Summing this multinomial over all realisable d recovers Cayley's n^{n-2} labelled trees.

Proof

By Definition 6.2.3, a labelled tree is uniquely coded by a length-$(n-2)$ sequence in which symbol i appears exactly d_i-1 times; the sum of these multiplicities is $\sum(d_i-1) = 2(n-1)-n = n-2$, so the count is the number of distinct permutations of that multiset – the multinomial $(n-2)!/\prod(d_i-1)!$. Since the Prüfer map is a bijection (proven in Chapter 30), this multinomial *is* the number of realising labelled trees.

■

Example 6.2.5 (How many trees look like T's degrees?)

$d = [2,3,3,1,1,1,2,1]$, so $d_i - 1 = [1,2,2,0,0,0,1,0]$ and the multinomial is $6!/(1!\,2!\,2!\,0!\,0!\,0!\,1!\,0!) = 720/4 = \mathbf{180}$ distinct labelled trees share T's degree sequence. Cayley gives $8^6 = 262144$ trees in total on 8 labels; the 180 above are just those with this particular degree profile – the array narrows the world by three orders of magnitude, yet still cannot name *which* tree.

6.2.3 The algorithm: peel leaves to realise a degree sequence

Code 6.2.1: Construct a tree realising a degree sequence, or report it impossible

```cpp
#include <bits/stdc++.h>
using namespace std;

// vertices are 0..n-1; deg[v] = required degree of v.
// Returns n-1 edges realising deg, or an empty list if deg is not tree-realisable.
vector<pair<int,int>> treeFromDegrees(const vector<int>& deg) {
    int n = (int)deg.size();
    long long sum = 0;
    for (int d : deg) { if (d < 1) return {}; sum += d; }     // positive required
    if (n == 1) return {};                                    // single node, no edges
    if (sum != 2LL * (n - 1)) return {};                      // the WHOLE test (Thm 6.2)

    vector<int> cur = deg;                                    // live degrees during the peel
```

```cpp
    // Pruefer-style leaf-min-heap: always peel the smallest current leaf.
    priority_queue<int, vector<int>, greater<int>> leaf;
    for (int i = 0; i < n; ++i) if (cur[i] == 1) leaf.push(i);

    // Order vertices by (deg-1) copies -> a Pruefer sequence of length n-2.
    vector<int> prufer;
    for (int i = 0; i < n; ++i) for (int k = 1; k < deg[i]; ++k) prufer.push_back(i);

    vector<pair<int,int>> E;
    for (int x : prufer) {                                  // decode: attach a leaf
    ↪ to x
        int l = leaf.top(); leaf.pop();
        E.emplace_back(l, x);
        if (--cur[x] == 1) leaf.push(x);                    // x became a leaf
    }
    int a = leaf.top(); leaf.pop();                         // last two leaves join
    int b = leaf.top(); leaf.pop();
    E.emplace_back(a, b);
    return E;                                               // exactly n-1 edges,
    ↪ degrees = deg
}
```

Complexity

Time $O(n \log n)$ – the Prüfer multiset holds $n-2$ entries and every heap operation is $\log n$; building the multiset is itself $O(n)$ since $\sum(\deg(v)-1) = n-2$. Memory $O(n)$. The realisability *decision* alone is $O(n)$ (one sum). The construction is exactly the Prüfer *decoder* run on a conveniently chosen code, which is why it always succeeds when Theorem 6.2.2's equation holds.

6.2.4 Worked Example

Example 6.2.6 (Realising T's degree sequence by the peel)

Feed $d = [2,3,3,1,1,1,2,1]$ (nodes 0..7 relabel 1..8) to Code 6.2.1. The sum is $14 = 2(n-1)$ ✓, so we proceed. Initial leaves (degree 1): $\{4,5,6,8\}$ (1-indexed). The Prüfer multiset, node v listed $\deg(v)-1$ times, is $\{1, 2, 2, 3, 3, 7\}$ (length 6). Decoding in ascending order: attach leaf $4 \to 1$ (node 1 drops to 1, joins the leaf pool); attach leaf $5 \to 2$; attach leaf $6 \to 2$ (node 2 now degree 1, a leaf); attach leaf $1 \to 3$; attach leaf $2 \to 3$ (node 3 drops to 1); then node 7; the final two free leaves join. The result is *some* tree with degrees exactly $[2,3,3,1,1,1,2,1]$ – not necessarily T itself, but a legal realisation, which is all the theorem promises. Re-running with a different Prüfer *order* would yield a different one of the 180 trees counted in Example 6.2.5.

The peel never gets stuck

At every step the "positive degrees, sum = twice-remaining-edges" invariant holds, so the average degree of the live graph is below 2 and a degree-1 node is guaranteed to be in the heap. If you *do* hit an empty leaf heap, the input failed Theorem 6.2.2 – e.g. $d = [2,2,2]$ sums to $6 \neq 4$, or $d = [3,1]$ has a node wanting 3 with only one partner: both return empty.

6.2.5 From grids to recursive trees: the quad tree

The degree array is aggregate data of type "counts". A **quad tree** is aggregate data of type "a square matrix", and the reconstruction is *recursive partition*: if a square is uniform, it collapses to a leaf; otherwise split it into four quadrants and rebuild each. The output tree's shape is forced by the grid – no edges are ever given, yet a genuine tree pops out. This is the construction soul of LC 427.

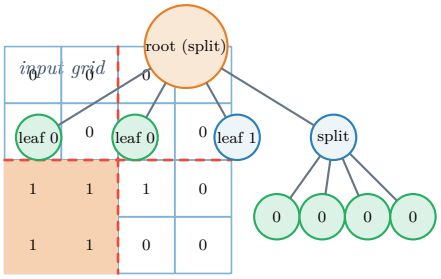


Figure 6.2.2: The same grid, two representations. The 4×4 grid on the left is split (dashed) into four 2×2 quadrants; three are uniform (leaf 0, leaf 0, leaf 1) and the bottom-right still mixes 0/1, so it recurses one more level. The quad tree on the right is the *reconstruction*: a node is a leaf exactly when its square is uniform, internal otherwise – LC 427.

Definition 6.2.7 (Region quad tree)

A **quad tree** over an $n \times n$ grid is a tree whose root covers the whole square. A node covering a square S is a **leaf** if every cell of S holds the same value (that value is stored on the leaf); otherwise it is **internal** and has exactly four children covering the top-left, top-right, bottom-left and bottom-right halves of S.

6.2.6 Variations and Twists

- **Sorted vs labelled degrees**: Theorem 6.2.2 is about the labelled model (node i has degree d_i). If only the *multiset* of degrees matters, divide the count by automorphisms – still a Prüfer argument (Chapter 30). L4–L5
- **Bounded-degree trees**: constrain which d_i are allowed (e.g. binary: all $d_i \le 3$ for a rooted binary tree); realisability then needs a matching argument, not just the sum. L5
- **Prefer a specific tree**: to *rebuild* T *itself*, the degree array is not enough – you need a full Prüfer sequence (the order matters), Chapter 30. L4
- **Quad tree, faster uniformity test**: replace the $O(\text{area})$ scan with a 2-D prefix sum of the values so "is S uniform?" is $O(1)$; total build $O(n^2)$. L4
- **k-d tree / octree**: same partition-tree idea with k dimensions or splits on a data point instead of uniformity – construction-by-recursion, LC 427's template. L3–L4

6.2.7 Common Pitfalls

Trap 1 – accepting a sequence with a zero or a bad sum

$d_i = 0$ with $n \ge 2$ is impossible in a connected graph (an isolated node), and $\sum d_i \ne 2(n-1)$ is fatal – yet both sneak past code that only "starts peeling". Test positivity *and* the sum *before* constructing (see the two early `return {}` lines of Code 6.2.1).

Code 6.2.2: WRONG: peel without validating

```
for (int i=0;i<n;i++) if (deg[i]==1) leaf.push(i);   // sum may be != 2(n-1): heap can
    ↪ empty mid-run
```

Code 6.2.3: RIGHT: check the certificate first

```
long long s=0; for(int d:deg){ if(d<1) return {}; s+=d; }
if (s != 2LL*(n-1)) return {};                          // reject before any peel
```

Trap 2 – assuming the constructed tree is *the* tree

A degree sequence fixes only the *degrees*, not the wiring. Unless a Prüfer sequence's *order* is given, you recover *one of* the $\prod$-many realisations – fine for "build any", wrong for "recover exactly T". Do not claim the answer is unique when Proposition 6.2.4 returns a count > 1.

Trap 3 – quad tree: splitting uniform squares

The base case is *uniform value*, not *size* 1. A 2×2 block of all 1s is a single leaf; recursing past a uniform square spawns empty children and corrupts `isLeaf`. Test uniformity *first*, and only split when it fails (Code 6.2.4).

6.2.8 Problem Lab

The anchor, LC 427, is the purest "data → tree" reconstruction in the chapter: the edges are never mentioned – they are *implied* by the recursive partition of a grid.

W6.2.1 Construct Quad Tree

Source: LeetCode 427 LC Medium L3
Techniques: divide and conquer, recursive construction, uniformity test, tree building from grid

Construct Quad Tree

Given an $n \times n$ grid `grid` of 0s and 1s, build its **quad tree** and return the root. A **leaf** has `isLeaf=true` and `val` equal to the (single) value in its region. An **internal** node has `isLeaf=false`, arbitrary `val`, and four children `topLeft`, `topRight`, `bottomLeft`, `bottomRight` covering the four quadrants. n is a power of 2, $1 \le n \le 6$.

Attempt it yourself first – L3, give yourself 25 minutes

Fair first attempt: recursion on a square. Base case when the whole square is one value.

Peek (1 min lost): How do you decide "the square $(r_0, c_0) \to (r_1, c_1)$ is uniform"? And when it *isn't*, into how many sub-squares does it break, and what are their corners?

Nudge (5 min lost): Scan the square; if all cells equal *val*, return `new Node(val, true)`. Otherwise split at the mid row/col into four rectangles – top-left $[r_0, rm) \times [c_0, cm)$, etc. – and recurse; return `new Node(val, false, tl, tr, bl, br)`.

Full hint (you will not solve this one): `build(r0,c0,r1,c1)`: let $v = grid[r_0][c_0]$; if the region is all v return a leaf; else $rm = (r_0+r_1)/2$, $cm = (c_0+c_1)/2$ and recurse on the four half-open quadrants, threading them as the four children. Top call `build(0,0,n,n)`. Full build and proof below.

➤ Understanding the Problem

The tree's shape is completely determined by the grid: a node is a leaf iff its region is uniform. The four children are ordered, so this is a *fixed* 4-ary recursion – the only real work is (a) testing uniformity and (b) getting the quadrant boundaries right (half-open rectangles so the four pieces

tile the parent exactly). Unlike §6.1 there are no edges to read – the edges are *created* by the partition.

➤ Key Observations

Observation 1. n is a power of two, so halving always lands on integer boundaries and bottoms out at 1×1 (a trivially uniform square), guaranteeing the recursion terminates.

Observation 2. "Uniform" is a property of a region, checked independently at every node; the naive scan is $O(\text{area})$. Because LC 427 caps $n \le 6$, the naive total $O(n^2 \log n)$ is trivially fine – but a 2-D prefix sum makes each test $O(1)$ and the whole build $O(n^2)$ for large grids.

Observation 3. For an internal node the `val` is unspecified by the problem, so any consistent choice works; the standard is the top-left cell's value – it must not be mistaken for a leaf decision.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** Build the tree by first converting the grid into an explicit edge list.
Killed by: There *are* no edges in the input – you would invent a graph, then re-derive the very partition you were trying to encode. Quad trees are built *top-down by recursion*, not from pairs.

✗ **Tried:** Split before testing uniformity.
Killed by: Then a solid 4×4 block of 1s becomes 16 tiny leaves and four levels of pointless children – you produce a non-minimal quad tree and fail the `isLeaf` checks. Uniformity is the *base case*; test it first.

✓ **Aha!** A node is a leaf *iff* its square is uniform. So: test, then either return a leaf or split into the four half-open quadrants and recurse – the recursion *is* the construction.

➤ Solution

Define $build(r_0, c_0, r_1, c_1)$ over the half-open square $[r_0, r_1) \times [c_0, c_1)$. Read $v = grid[r_0][c_0]$; if every cell in the square equals v, return a leaf `Node(v,true)`. Otherwise compute midpoints rm, cm and return an internal `Node(v,false,...)` built from the four recursive calls on the top-left $[r_0, rm) \times [c_0, cm)$, top-right $[r_0, rm) \times [cm, c_1)$, bottom-left $[rm, r_1) \times [c_0, cm)$ and bottom-right $[rm, r_1) \times [cm, c_1)$ quadrants. The root is $build(0, 0, n, n)$.

➤ Proof of Correctness

Proof

Termination: each recursive call halves the side length, reaching 1×1 (uniform, hence a leaf), so the depth is $\log_2 n$ and every branch terminates. *Leaf correctness*: *build* returns `isLeaf=true` only after scanning the whole region and confirming all cells equal v, matching the definition. *Internal correctness*: when the uniformity scan fails, the four half-open quadrants are disjoint and their union is exactly the parent square, so the four children cover the parent's region with no gaps or overlaps, each built recursively to represent its sub-region. By induction on region size, every returned node faithfully represents its square, and the root represents the full grid – so the constructed quad tree is the required one.

■

➤ Implementation

Code 6.2.4: Construct Quad Tree – recursive partition of a uniform test (LeetCode 427)

```cpp
#include <bits/stdc++.h>
using namespace std;

// Definition for a quad-tree node.
class Node {
public:
    bool val;
    bool isLeaf;
    Node* topLeft;
    Node* topRight;
    Node* bottomLeft;
    Node* bottomRight;
    Node() : val(false), isLeaf(false),
             topLeft(nullptr), topRight(nullptr),
             bottomLeft(nullptr), bottomRight(nullptr) {}
    Node(bool _val, bool _isLeaf) : val(_val), isLeaf(_isLeaf),
             topLeft(nullptr), topRight(nullptr),
             bottomLeft(nullptr), bottomRight(nullptr) {}
    Node(bool _val, bool _isLeaf, Node* a, Node* b, Node* c, Node* d)
        : val(_val), isLeaf(_isLeaf), topLeft(a), topRight(b), bottomLeft(c),
    ↪ bottomRight(d) {}
};

class Solution {
    vector<vector<int>> g;
    bool uniform(int r0, int c0, int r1, int c1, int v) {
        for (int i = r0; i < r1; ++i)
            for (int j = c0; j < c1; ++j)
                if (g[i][j] != v) return false;
        return true;
    }
    Node* build(int r0, int c0, int r1, int c1) {
        int v = g[r0][c0];
        if (uniform(r0, c0, r1, c1, v)) return new Node(v == 1, true);   // leaf (base
    ↪ case)
        int rm = (r0 + r1) / 2, cm = (c0 + c1) / 2;                      // split into
    ↪ 4
        return new Node(v == 1, false,
            build(r0, c0, rm, cm), build(r0, cm, rm, c1),               // top-left,
    ↪ top-right
            build(rm, c0, r1, cm), build(rm, cm, r1, c1));              // bottom-left,
    ↪ bottom-right
    }
public:
    Node* construct(vector<vector<int>>& grid) {
        g = grid;
        int n = (int)grid.size();
        return build(0, 0, n, n);
    }
};
```

➤ Trace

The Figure 6.2.2 grid

Figure 6.2.2: bottom-left 2×2 is all 1; top-left and top-right 2×2 are all 0; bottom-right is mixed. `build(0,0,4,4)` scans the whole 4×4 – mixed $\Rightarrow$ internal. Four calls: `build(0,0,2,2)` uniform 0 $\Rightarrow$ leaf; `build(0,2,2,4)` uniform 0 $\Rightarrow$ leaf; `build(2,0,4,2)` uniform 1 $\Rightarrow$ leaf; `build(2,2,4,4)` mixed $\Rightarrow$ split into four 1×1 leaves $(1, 0, 0, 0)$. The returned tree has root with 3 leaf children and one internal child with 4 leaves – exactly Figure 6.2.2. ✓

➤ Complexity

Complexity

Time $O(n^2 \log n)$ with the naive scan: level 0 scans n^2, the 4 children together scan $4 \cdot (n/2)^2 = n^2$, and so on for $\log n$ levels – $O(n^2)$ per level. Space $O(n^2)$ nodes in the worst checkerboard (every 1×1 is its own leaf, $O(n^2)$ nodes). With $n \leq 6$ this is free; for large grids, a 2-D prefix sum makes `uniform` $O(1)$ and the build $O(n^2)$. Verdict: **✓ Accepted**.

➤ Pitfalls and Variants

Common Mistake

Three breakers: (i) splitting a uniform region (Trap 3) – test *first*. (ii) Off-by-one quadrants: use half-open $[r_0, r_1)$ and midpoints so the four children tile exactly; closed boundaries overlap and one cell gets two parents. (iii) Forgetting `isLeaf` on internal nodes (`new Node(v,false,...)`) – the judge compares structure, so a mis-flagged leaf fails. *Variant:* LC 427's sibling is the "intersection of two quad trees" and "quad-tree compression" – same recursion, extra combine step.

Key takeaways

1. **Degrees are a certificate.** One equation, $\sum d_i = 2(n-1)$, decides existence outright (Theorem 6.2.2) – far cleaner than the general-graph case.
2. **The peel *is* Prüfer decoding.** Leaf-peel construction and Prüfer counting are the same mechanism seen constructively vs. enumeratively (Chapter 30).
3. **Recursive partition is reconstruction too.** A quad tree turns grid data into a tree with no edges ever named – construction from aggregate data, one shape per data value.

AC ⟶ red: hardening this code

- **O(1) uniformity**: build a 2-D prefix sum `pre` of the grid; a region is uniform iff its cell-sum equals 0 or equals its area – turns W6.2.1 into $O(n^2)$.
- **Recover *exactly* T**: replace Code 6.2.1's chosen Prüfer order with the given code, then decode – the standard Prüfer→tree decoder (Chapter 30).
- **Existential edge count**: among the 180 realisations of T's degrees, ask how many have node 2 adjacent to node 7; a Prüfer-window counting exercise, Chapter 30.

6.2.9 Practice Problems

Stuck? Work through this list

- Did you validate positivity *and* the $\sum = 2(n-1)$ equation before peeling? (Trap 1, Theorem 6.2.2.)
- Did you resist claiming uniqueness when Proposition 6.2.4 gives a count > 1? (Trap 2.)
- In the quad tree, did you test uniformity *before* splitting? (Trap 3, W6.2.1.)
- Are your quadrant rectangles half-open so the four children tile the parent? (W6.2.1.)

Q6.2.1 Course Schedule II – LeetCode 210 LC Medium L3

Hint: Given prerequisite *pairs*, return a valid ordering of all courses, or `[]`. This is the **in-degree peel** of Theorem 6.2.2 on a *directed acyclic* graph: repeatedly emit every node of current in-degree 0 (a "source", the DAG analogue of a degree-1 leaf), delete its outgoing edges, and repeat (Kahn's algorithm). Recognise the shared mechanic – peel by degree, reconstruct an order/structure from aggregate counts – and note the *cycle detector*: if the peel finishes with nodes still having positive in-degree, the input is not a DAG (the "realisability fails" moment). Rebuild `adj` from the pairs first, exactly §6.1.

10-second refresher – 6.2 Degrees, Sequences & Codes

A positive list is a tree degree sequence *iff* $\sum d_i = 2(n-1)$ (Theorem 6.2.2); build a realisation by peeling the smallest leaf onto a degree-> 1 node (Code 6.2.1) – the same move as Prüfer decoding, where node v appears $\deg(v) - 1$ times (Definition 6.2.3). The count of realising labelled trees is $(n-2)!/\prod(d_i - 1)!$ (Proposition 6.2.4); full bijection and Cayley are Chapter 30. A quad tree reconstructs a grid by uniform-then-split recursion (W6.2.1); a topological order reconstructs a DAG by the in-degree peel (LC 210).

Self-test (answer from memory, tick the box):

- ☐ Decide realisability of $[3, 2, 2, 1, 1, 1]$ for $n = 6$: yes or no, and by which test?
- ☐ Run one peel step on T's degrees – which node do you pin first and to what?
- ☐ How many labelled trees share a degree array with deg -1 multiset $\{1, 2, 2, 1\}$?
- ☐ In LC 427, why must uniformity be tested *before* you split a square?
- ☐ Explain how Kahn's algorithm is the directed in-degree sibling of the leaf-peel.

6.3 Is It a Tree? Identification and a Union-Find Pre-Look

Previously in this chapter ...

1. State the tree certificate and explain why checking only *two* of its three legs suffices. (Theorem 6.1.2)
2. In W6.1.1, what did a union-find *rejection* (an edge whose endpoints were already connected) tell you? (W6.1.1)
3. A forest with n nodes and k components has how many edges? Guess, then prove it in this section. (Theorem 6.1.3)
4. Why does banning the *later* of two candidate edges and testing for a surviving cycle pick the right redundant edge? (W6.1.1)

In This Section

Sections 6.1 and 6.2 *built* a tree from trustworthy-ish data. This section is the decision problem *before* you build: **given an edge list, is it a tree at all – or is it a forest, or does it hide a cycle, and if there is a cycle, which edge is redundant?** The union-find you touched in W6.1.1 becomes the star: process edges left to right, merging components, and the very first edge whose endpoints are *already* in one set is a **cycle-closing** edge. That single observation *proves* the tree certificate operationally, *counts components* for free (the "number of provinces" problem), and *identifies and deletes* the one surplus edge (LC 684 – the undirected little sibling of LC 685). This is a **pre-look**: enough union-find to certify trees. The *full* data-structure – union by rank and path compression analysis – lives in Chapter 17 (its use in Kruskal is Ch. 31, Chapter 31); we borrow only the "are these two already connected?" query here. **Prerequisites.** Theorem 6.1.2 (the certificate), W6.1.1 (the union-find cycle alarm). This section formalises *why* that alarm is exactly acyclicity and turns it into a decision procedure.

After this section you can

- decide "is this a tree?" in $O(n)$ with the $n-1$-edge count plus one union-find pass; **L2–L3**
- classify any edge set as tree / forest / unicyclic / multi-cyclic by counting rejections; **L3**
- count connected components (provinces) with the same machinery; **L2–L3**
- find and delete the last redundant edge that keeps a graph a tree (LC 684). **L3**

Pattern card – certify, classify, or repair a suspicious edge set

Recognition keywords. you are given `edges` on n nodes and the question is about *validity or shape*, not a subtree computation: "is it a tree?", "how many connected pieces?", "which single edge makes it cyclic?", "delete one edge to get a tree".

✓ When to use this

- "return the redundant connection" – the edge closing a cycle, last-in-input wins (W6.3.1);
- "number of provinces / friend circles / islands" – count components (§6.3);
- "can all pairs be connected / is this network a tree?" – $n-1$ edges + connectivity;
- the directed / two-parent version – W6.1.1 already used this pre-look.

✗ When NOT to use this

- you only need to build `adj` for a trusted tree – §6.1;
- you need the *minimum spanning tree* weight, not the yes/no shape – Chapter 31;
- connectivity under edge *insertions and deletions* online – that needs more than this pre-look.

One-line test: *is the question "does this edge set form one acyclic whole?" Merge with union-find and watch for a rejection – this section.*

6.3.1 Intuition

A tree is *maximally acyclic* and *minimally connected*: add any edge and you create a cycle; delete any edge and you disconnect. That dual tightness is the certificate of Theorem 6.1.2, and union-find is the machine that *enforces* it edge by edge. Feed edges in order, each time asking "are these two endpoints already reachable from each other?" – if yes, this edge is *redundant* (it

closes a cycle); if no, merge the two pieces. After all edges: if nothing was rejected and you used exactly $n-1$ edges, you watched n separate nodes coalesce into one – a tree. A single rejection is a cycle; a leftover set of pieces is a forest.

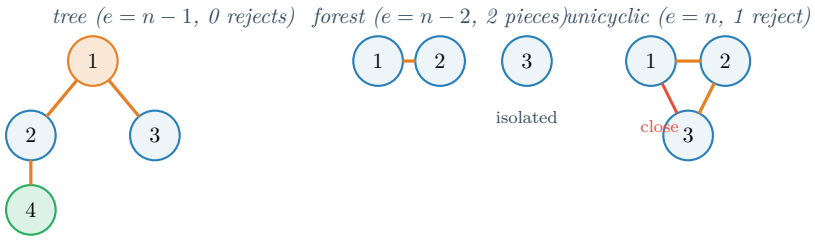


Figure 6.3.1: The three shapes union-find separates. A **tree**: $n-1$ edges, every merge accepted. A **forest**: acyclic but $< n-1$ edges, several surviving sets. A **unicyclic** graph: n edges, the red edge finds its endpoints already joined and is *rejected* – the redundant connection (W6.3.1). The number of rejects is the cyclomatic number $e-(n-k)$.

6.3.2 Definitions and Notation

Definition 6.3.1 (Forest, component, spanning tree)

A **forest** is an acyclic graph. Its maximal connected subgraphs are **components**; a connected forest (one component) is a tree. A graph on n nodes with k components that is a forest has exactly $n-k$ edges. A **spanning tree** of a connected graph uses all n nodes and exactly $n-1$ of its edges.

Proposition 6.3.2 (Union-find pre-look at acyclicity)

Process a graph's m edges one at a time, initially n singleton sets, merging (unioning) the two endpoint sets for each *accepted* edge and *rejecting* any edge whose endpoints already share a set. Then:

1. an edge is rejected **iff** it closes a cycle among the edges seen so far;
2. the number of accepted edges is exactly $n-k$, where k is the number of connected components of the whole graph;
3. the number of rejections is $m-(n-k)$ – the graph is a forest **iff** this is 0.

Proof

(1) At the moment $[u,v]$ is examined, the accepted edges define reachability; u, v already share a set precisely when a path of accepted edges connects them, so adding $[u,v]$ closes a cycle. Conversely any cycle's last-processed edge sees its two neighbours already joined. (2) Each accepted union lowers the set count by exactly one, from n down; the surviving sets are the connected components of the graph (accepted edges and rejected edges connect the same pairs, since a rejected edge joined already-connected nodes), so exactly k sets remain and $n-k$ unions were accepted. (3) Rejections $= m -$ accepted $= m-(n-k)$. A forest has no cycles, hence no rejections; a graph with no rejections never closed a cycle, hence is acyclic – a forest.

■

Corollary 6.3.3 (The operational tree test)

A graph on n nodes is a tree **iff** it has $n-1$ edges and union-find rejects none, and **iff** it has $n-1$ edges and union-find merges everything into a single set. This is Theorem 6.1.2 turned into a one-pass algorithm: the count is leg (1), the rejection-free scan is leg (3), and either plus the count forces leg (2).

Key Insight

You get *three* answers from one scan for free: reject-count > 0 means "not a tree, has a cycle"; reject-count $= 0$ but $\text{sets}_{\text{final}} > 1$ means "a forest"; reject-count $= 0$ and exactly one final set with $n-1$ edges means "a tree". LC 684 reads off the *last* rejected edge; LC 547 reads off the *final set count*.

6.3.3 The algorithm: one union-find pass to certify and classify

Code 6.3.1: Union-find pre-look: decide tree / forest / cyclic, and count components

```cpp
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> p;
    DSU(int n) : p(n) { iota(p.begin(), p.end(), 0); }        // p[i]=i: n singleton sets
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }   // path compression
    bool uni(int a, int b) {                                   // false => already connected
        a = find(a); b = find(b);
        if (a == b) return false;                              // rejected edge: closes a cycle
        p[b] = a; return true;                                 // accepted: merges two sets
    }
};

// Is the graph on nodes 0..n-1 exactly ONE tree?
bool isTree(int n, const vector<pair<int,int>>& edges) {
    if ((int)edges.size() != n - 1) return false;              // leg (1): the edge count
    DSU d(n);
    for (auto& e : edges)
        if (!d.uni(e.first, e.second)) return false;           // leg (3): a reject = a cycle
    return true;                                               // n-1 edges + acyclic => connected
}

// Number of connected components (provinces).
int countComponents(int n, const vector<pair<int,int>>& edges) {
    DSU d(n);
    int comps = n;
    for (auto& e : edges) if (d.uni(e.first, e.second)) --comps;  // each merge removes one piece
    return comps;
}
```

Complexity

Time $O((n+m)\,\alpha(n))$ – every `find/uni` is inverse-Ackermann with path compression ($\alpha \le 4$ in practice), so essentially linear. Memory $O(n)$. This one-pass approach is far cheaper than the alternative DFS colouring for cycle detection, and *both* answers (tree? how many components?) come from one pass. Full DSU complexity analysis (why path compression alone is already near-linear) is Chapter 17.

6.3.4 Worked Example

Example 6.3.4 (Classifying T and T plus one bad edge)

T on $n = 8$ has $7 = n - 1$ edges. `isTree`: the count passes, and processing the seven edges in {1-2,1-3,2-4,2-5,3-6,3-7,7-8} merges $8 \to 1$ set with no rejection – **a tree** ✓, `countComponents`$= 1$. Now append the bad edge $[4, 5]$: 8 edges for 8 nodes fails the count leg instantly, and if you ran the scan anyway, edges up to 7 build one set, then $[4, 5]$ finds 4, 5 already joined $\Rightarrow$ **reject** (one cycle, the 2–4–5 triangle). The *last* rejected edge is exactly $[4, 5]$ – the edge LC 684 would return.

Edge-by-edge on LC 684's Example 2

`edges=[[1,2],[2,3],[3,4],[1,4],[1,5]]`, nodes 1..5. Sets start $\{1\}\{2\}\{3\}\{4\}\{5\}$. $[1, 2]$: merge $\{1, 2\}$. $[2, 3]$: merge $\{1, 2, 3\}$. $[3, 4]$: merge $\{1, 2, 3, 4\}$. $[1, 4]$: $find(1) = find(4)$ – **already one set** $\Rightarrow$ reject, return $[1, 4]$. The scan stops at the first rejection; $[1, 4]$ is the redundant connection, matching the official answer. ✓

6.3.5 Variations and Twists

- **DFS cycle test**: root the graph and watch for a back-edge to a non-parent ancestor – equivalent to union-find on undirected graphs, $O(n + m)$; union-find is less code. **L3**
- **Directed version**: acyclicity now means a *topological order* exists (LC 207), and the peel is in-degree based (§6.2's practice) – union-find does NOT apply to directed cycles. **L3–L4**
- **Multiple surplus edges**: to delete k edges and leave a forest, *count* rejections; a graph is a k-cyclomatic forest iff rejections $= k$. **L4**
- **Dynamic connectivity**: edges arriving over time, ask "connected yet?" – union-find answers monotone insertions directly (this pre-look already does); deletions need more (Chapter 17). **L4**
- **Bipartite / 2-colour check**: union-find with parity answers "can nodes be split into two sets across every edge" – the same merge mechanic with an extra flag (Chapter 17). **L4**

6.3.6 Common Pitfalls

Trap 1 – trusting the edge count alone

"$n - 1$ edges" does *not* mean "tree": n nodes with $n - 1$ edges could be a disconnected piece plus a small cycle elsewhere (the two defects cancel in the count). Union-find rejects none *and* the count is $n - 1$ – both legs (Corollary 6.3.3); or check connectivity instead of acyclicity.

Code 6.3.2: WRONG: count only – a unicyclic disconnected graph slips past

```
return (edges.size() == n-1);        // 4 nodes, edges {1-2,2-3,3-1}: count 3? no. See
    ↪ the trap.
```

Code 6.3.3: RIGHT: count + rejection-free union-find

```
if ((int)edges.size() != n-1) return false;
DSU d(n);
for (auto& e : edges) if (!d.uni(e.first, e.second)) return false;   // a reject =
    ↪ cycle
return true;
```

Trap 2 – union-find on a *directed* cycle

The rejection test detects *undirected* cycles. On the directed graph of LC 207 the two prerequisite edges $a \to b$ and $b \to a$ are a genuine directed cycle but union-find would merge them silently the same as any pair. Detecting directed cycles needs Kahn's in-degree peel or DFS colour – not this tool (§6.2).

Trap 3 – forgetting path compression / recursion depth

Without `p[x]=find(p[x])` the trees get tall; at $n \approx 2 \cdot 10^5$ a *recursive* `find` on a degenerate chain can even overflow the stack. Keep compression (as in Code 6.3.1) or make `find` iterative; the union-by-rank upgrade is Chapter 17.

6.3.7 Problem Lab

LC 684 is the section distilled: one undirected extra edge, return the last redundant one – the *first* rejection of a union-find run in input order *is* the answer.

W6.3.1 Redundant Connection

Source: LeetCode 684 LC Medium L3
Techniques: undirected graph, union-find cycle detection, spanning tree, edge deletion

Redundant Connection

A **tree** on n nodes (labels $1 \dots n$) gained exactly **one** extra *undirected* edge, so `edges` now has n edges (each `[u,v]` with $u \neq v$). Find the edge that can be removed to make the graph a tree again; if several answers exist, return the one appearing **last** in `edges`. $2 \le n \le 1000$.

Attempt it yourself first – L3, give yourself 20 minutes

Fair first attempt: a tree has $n-1$ edges; here there are n, so exactly one is on a cycle. Find a cycle, return its latest edge.

Peek (1 min lost): Process edges left to right. When you are about to add $[u, v]$, what simple question tells you this edge is the one closing the cycle?

Nudge (5 min lost): Maintain connected components with union-find. The first edge whose two endpoints are *already connected* closes the only cycle. Because you scan in input order, the *last* such edge you could point at is naturally the one you return – there is exactly one cycle here.

Full hint (you will not solve this one): Run union-find over $1..n$ (arrays sized $n+1$) in input order; return the first edge $[u, v]$ with $find(u) = find(v)$. That edge is redundant and, since the input has a single cycle and you scanned in order, it is the latest edge of that cycle. Full build and proof below.

➤ Understanding the Problem

The graph has n nodes and n edges: one more edge than a tree needs, so it holds *exactly one* cycle (cyclomatic number $m - (n - k) = n - (n - 1) = 1$ with $k = 1$). Removing any cycle edge yields a tree; the tie-break wants the *last* cycle edge in input order. A rooted/two-parent complication is absent – edges are undirected, so this is W6.1.1 with Case 2 only.

➤ Key Observations

Observation 1. Scanning edges in order and merging with union-find, the *first* edge whose endpoints are already connected closes the cycle – that is the unique redundant edge of a unicyclic graph, and it appears *last* among that cycle's edges by construction of the scan.

Observation 2. You never need to find the cycle's *members* – only its closing edge. Union-find identifies the closing edge directly, in $O(n\,\alpha(n))$, with no DFS and no parent bookkeeping.

Observation 3. This is Corollary 6.3.3 run just past the point of failure: a tree would reject nothing; the first rejection here names the culprit.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** DFS for the cycle, collect its edges, take the max-index one.
Killed by: Workable but you must track a back-edge, reconstruct the cycle from parent pointers, then scan for the largest input index – three fiddly steps. The union-find scan *is* the cycle-closing test, in-order, so the "last edge" tie-break is automatic.

✗ **Tried:** Check every edge by deleting it and re-testing *isTree*.
Killed by: $O(n)$ tests $\times$ $O(n)$ each $= O(n^2)$; fine for $n \le 1000$ but it re-derives connectivity n times and still needs careful tie-breaking. One pass beats it.

✓ **Aha!** Process edges in input order with union-find; return the *first* rejected edge. Because there is exactly one cycle, the closing edge is the last-in-order member of it – the exact answer.

➤ Solution

Initialise union-find on $1..n$. Walk `edges` in order: for $[u, v]$, if $\mathit{find}(u) = \mathit{find}(v)$ return $[u, v]$ (the cycle-closing redundant edge); otherwise union them. Because a valid input guarantees one extra edge, the loop always returns before finishing.

➤ Proof of Correctness

Proof

The graph has n edges on n nodes, so it is connected with cyclomatic number 1 – a single cycle C. Claim: the first edge e^* the scan rejects is the member of C appearing last in `edges`. When $e^* = [u, v]$ is rejected, u, v are already joined by accepted edges, forming a u–v path P that avoids e^*, so $P \cup \{e^*\}$ is a cycle; the input has only one cycle C, hence $P \cup \{e^*\} = C$ and $e^* \in C$. All earlier members of C were accepted (they did not reject, else the scan had stopped), and every edge processed after e^* is not examined, so e^* is the last member of C in input order. Removing e^* breaks C and leaves $n - 1$ edges with no cycle – a tree (Corollary 6.3.3) – and it is the latest valid deletion, matching the tie-break.

■

➤ Implementation

Code 6.3.4: Redundant Connection – return the first edge union-find rejects (LeetCode 684)

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
    vector<int> p;
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
public:
    vector<int> findRedundantConnection(vector<vector<int>>& edges) {
        int n = (int)edges.size();                  // n nodes, n edges (labels 1..n)
        p.resize(n + 1);
        iota(p.begin(), p.end(), 0);
        for (auto& e : edges) {
            int a = find(e[0]), b = find(e[1]);
            if (a == b) return e;                   // already connected => closes the
    ↪ cycle
            p[a] = b;                               // merge
        }
        return {};                                  // unreachable for valid input
    }
};
```

➤ Trace

Both official examples

Ex. 1 `[[1,2],[1,3],[2,3]]`: merge $1, 2$; merge $1, 3$ (set $\{1, 2, 3\}$); $[2, 3]$: $find(2) = find(3)$ $\Rightarrow$ return $[2, 3]$. ✓
Ex. 2 `[[1,2],[2,3],[3,4],[1,4],[1,5]]`: merge $1, 2, 3, 4$ into one set across the first three edges; $[1, 4]$ already connected $\Rightarrow$ return $[1, 4]$ (matches expected). ✓

➤ Complexity

Complexity

Time $O(n\,\alpha(n))$ – one in-order union-find pass (α inverse-Ackermann, ≤ 4). Space $O(n)$ for the parent array. Beats the $O(n^2)$ delete-and-retest brute force, and at $n \leq 1000$ either passes. Verdict: **✓ Accepted**.

➤ Pitfalls and Variants

Common Mistake

(i) Size arrays $n + 1$ for 1-indexed labels (Trap 2 of §6.1). (ii) Return the *first* rejected edge, not the last rejected – with a single cycle they coincide, but if you kept scanning past the reject you might overwrite the answer; `return e` immediately. (iii) Do not use this on the directed LC 685 without the in-degree pre-pass (W6.1.1). *Variant:* LC 685 is this with a possible two-parent node; LC 687-style problems add a second edge to delete – re-run the

scan after removing the found edge.

Key takeaways

1. **A rejection *is* a cycle.** Union-find turns "is it a tree?" into "did I reject an edge?" – acyclicity for free, and count components on the same pass (Proposition 6.3.2).
2. **Scan order = tie-break.** The first rejected edge is the last-in-order cycle member; the "return latest" rule needs no extra comparison.
3. **684 ⊂ 685.** This is W6.1.1's Case 2 – undirected, one cycle, no two-parent node – the certificate's legs checked one at a time.

AC ⟶ red: hardening this code

- **Two edges to delete**: run W6.3.1, remove the found edge, re-run to find the next reject – reject-count equals the cyclomatic number $m - (n - k)$.
- **Bigger inputs**: add union-by-rank to keep *find* shallow (or make it iterative, Trap 3); full analysis Chapter 17.
- **From tree to MSW**: if edges are weighted and you want the *max*-weight edge to delete, sort by weight and run the same reject logic – that is Kruskal in reverse (Chapter 31).

6.3.8 Practice Problems

Stuck? Work through this list

- Did you check the $n-1$ count *and* a rejection-free scan (not just one leg)? (Corollary 6.3.3, Trap 1.)
- Did you return on the *first* rejection instead of scanning past it? (W6.3.1.)
- Are your arrays sized for 1-indexed nodes? (Trap 2 of §6.1.)
- For counting components, did you start the tally at n and decrement on each accepted merge? (Code 6.3.1, W6.3.1.)

Q6.3.1 Number of Provinces – LeetCode 547 LC Medium L2

Hint: A $n \times n$ adjacency matrix `isConnected` of friend relationships; count the **provinces** (connected components). Rebuild the edge list from the matrix (read only $i < j$ to skip duplicates) and call `countComponents` from Code 6.3.1 – start at n and decrement on each accepted union – or DFS the matrix counting roots. This is exactly the "reject-count = 0 but sets > 1" branch of §6.3: the answer is the number of surviving union-find sets (or DFS trees), and it needs *no* edge-count leg since cycles are allowed. Recognise it as the component-counting half of this section.

10-second refresher – 6.3 Is It a Tree? (Union-Find pre-look)

Certify with *two* certificate legs: exactly $n-1$ edges and a union-find pass that rejects nothing (or merges to one set) (Corollary 6.3.3). A rejection is a cycle; rejections $= m - (n - k)$ and k is the final set count (Proposition 6.3.2). LC 684 returns the *first* rejected edge in input order – the last cycle member (W6.3.1); the directed sibling is LC 685 (W6.1.1); components-province count is LC 547. Full DSU (rank, compression) is Chapter 17, and its use in Kruskal is Ch. 31 (Chapter 31).

Self-test (answer from memory, tick the box):

☐ Give an n-node edge set with $n - 1$ edges that is *not* a tree; which certificate leg fails?

- ☐ State the reject-count formula and use it to prove a graph with 0 rejections is a forest.
- ☐ In LC 684, why is the *first* rejected edge also the "last appearing" valid answer?
- ☐ Why must you NOT use union-find to detect a *directed* cycle? (Name the right method.)
- ☐ Compute *countComponents* for T (8 nodes, 7 edges) and for T with a self-loop added.

6.4 Recover the Structure by Queries (Interactive)

Previously in this chapter . . .

1. What does a single comparison in binary search do to the size of the candidate set? Why $\log_2 n$ comparisons in total? (Chapter 1, Chapter 5 preview)
2. In §6.1, what $O(1)$ fact about the root made it readable from an array? (Proposition 6.1.4)
3. How could a *distance* or *ancestor* answer about a tree be produced by the timestamp machinery? (Chapter 5, Theorem 5.1.2)
4. If a yes/no answer is truthful and splits the world in half, how many do you need to pin one of N possibilities?

In This Section

Every topic so far *handed* you the data. Here the data is hidden behind an **oracle**: you may only *ask questions*, and each answer is a small, costly bit of information you must *use adaptively* – your next query depends on the previous answers. Reconstructing a hidden object by queries is the same skill as the parent/degree/identification problems of §6.1–§6.3, turned inside out: instead of reading a full array you *elicit* it, one strategic question at a time. The purest instance – and our worked anchor LC 374 – is **guess the number**: pick from $[1, n]$, an oracle replies "higher / lower / correct", and the optimal strategy is textbook binary search, which provably uses the minimum possible number of questions (Theorem 6.4.3). Two ideas carry the whole section: a truthful answer should *halve* the candidate space, and the number of candidates you can distinguish is $2^{\#\text{answers}}$, so you cannot beat $\lceil \log_2 \rceil$. Where the *cost* of a wrong guess matters (LC 375) the halving point shifts from the middle to a min-max choice – an interval DP – and where the hidden object is a *tree* rather than a number, the same adaptive idea becomes group-testing on **distance/ancestor** queries answered by the Chapter 5 oracle Chapter 5. This is construction from data where you get to *choose* the data. **Prerequisites.** Binary-search intuition; the Chapter 5 distance/ancestor oracle (Chapter 5) used as a black box. This section adds the *query model, the information-theoretic lower bound, and adaptive vs. cost-aware strategies.*

After this section you can

- model a problem as an adaptive query protocol and identify the candidate space; **L2**
- apply the halving principle and prove $\lceil \log_2 N \rceil$ questions are necessary; **L2–L3**
- implement optimal search against a three-way (higher/lower/equal) oracle (LC 374); **L2**
- recognise when the optimal split is *not* the middle (worst-case cost, LC 375). **L3–L4**

Pattern card – reconstruct by asking the oracle

Recognition keywords. you do not see the hidden value/object; instead you are given a *feedback function* (a `guess()`, an API that answers "higher/lower", a distance/ancestor query, a yes/no test) and must find the answer with as few *queries* as possible, adapting each to the last.

✓ When to use this

- "guess the number / word / root" with a comparison oracle – binary search (W6.4.1);
- "find x using $f(x)$ queries, minimise the *number* of queries" – halve the space;
- "guarantee a win, but a wrong guess *costs* you" – min-max, not midpoint (LC 375);
- "rebuild a hidden tree using distance / ancestor queries" – adaptive group testing on the Chapter 5 oracle.

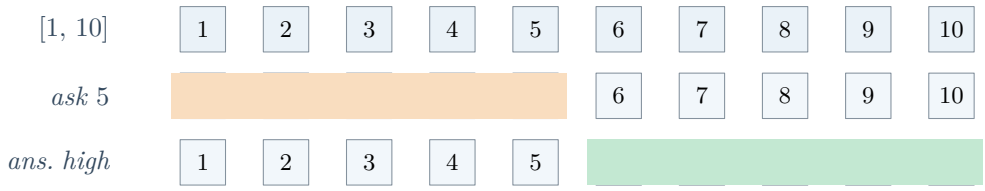
✗ When NOT to use this

- the full array/matrix is visible up front – §6.1/§6.2;
- you just need to search a *sorted* array once, not drive an interactive protocol – ordinary binary search, Chapter 1;
- answers may be *lies* (Ulam's game) – that needs error-correcting splits, out of scope.

One-line test: *can you only learn the answer by **asking**, and does each reply shrink the possibilities? Choose the question that halves – this section.*

6.4.1 Intuition

You are on a number line $[1, n]$; the secret x is somewhere on it and a referee only says "higher", "lower" or "got it" after each of your guesses. The clever move is always to guess the *middle* of the still-possible interval: whatever the referee says, you discard half the remaining candidates. The interval of possibilities shrinks $n \to n/2 \to n/4 \to \cdots$, so about $\log_2 n$ guesses finish it – and you cannot do better, because each three-way answer can at best divide the world into three and the count of distinguishable secrets grows only exponentially in the number of questions. This is the whole game: *a good query is one whose answers leave you with equally small sub-cases.*



guess the midpoint 5; if told "secret is higher" the survivors $[6, 10]$ are half of what remains – each truthful answer throws away $\approx$ half the candidates

Figure 6.4.1: Adaptive halving on a number line. One well-chosen question (the midpoint) turns a size-10 candidate set into a size-≤ 5 one no matter which branch the oracle confirms. Repeating gives a decision *tree* of depth $\lceil \log_2 n \rceil$ – exactly the shape the tree chapters draw, but here its nodes are *questions*, not data.

6.4.2 Definitions and Notation

Definition 6.4.1 (Query (oracle) model)

A **query model** is a tuple $(\mathcal{X}, \mathcal{Q}, ans)$: a hidden target $x \in \mathcal{X}$ (the candidate space), a set of allowed **queries** $q \in \mathcal{Q}$, and an answer function $ans(q, x)$ returning a feedback symbol. An **adaptive strategy** chooses each next query as a function of the answers so far, and must halt with a unique x. Its **cost** is the number (or weighted total) of queries in the worst case over x.

In plain words. You can't see x; you may only poke questions and get terse truthful replies; a

strategy is good if it always names x using few questions, and *adaptive* means your next question depends on the last answer.

Proposition 6.4.2 (Halving and the decision-tree bound)

If every query has at most b possible answers, then k queries can distinguish at most b^k targets, so any correct adaptive strategy needs $k \geq \lceil \log_b |\mathcal{X}| \rceil$ queries in the worst case. For a three-way comparison oracle ($b = 3$: higher/lower/equal) on N ordered secrets this is $\lceil \log_3 N \rceil$ *in principle*, and binary search attains the (slightly stronger, because answers must respect order) bound of $\lceil \log_2(N{+}1) \rceil$ guesses.

Proof

Each answer can at best multiply the number of distinguishable outcomes by b: after k queries the transcript of answers has $\leq b^k$ distinct values, and a correct strategy must map different targets to different transcripts, so $|\mathcal{X}| \leq b^k$, i.e. $k \geq \log_b |\mathcal{X}|$. The decision process is literally a b-ary *tree* (nodes = queries, edges = answers, leaves = conclusions) of depth k and hence $\leq b^k$ leaves. For an ordered search each comparison splits the remaining interval into a left block, a point, and a right block; always splitting so the larger side is as small as possible yields the recurrence $T(N) = 1 + T(\lfloor N/2 \rfloor)$ whose solution is $\lceil \log_2(N{+}1) \rceil$, attained by guessing the midpoint.

■

Theorem 6.4.3 (Binary search is query-optimal)

Against a truthful higher/lower/equal oracle, guessing the midpoint of the current candidate interval minimises the worst-case number of queries; it finds any secret in $[1, n]$ within $\lceil \log_2(n{+}1) \rceil$ guesses, matching Proposition 6.4.2's lower bound up to the base. No strategy can guarantee fewer in the worst case.

Key Insight

Optimality is *about the split*, not the midpoint per se: you want the answer that leaves the *largest* remaining branch as small as possible. For a pure *count* of questions on a uniformly ordered line, that is the midpoint – binary search. *When answers cost something* (LC 375: you pay the number you guessed) the split is no longer the middle – you minimise the worst case *sum*, giving $k + \max(\text{left}, \text{right})$ rather than max over sizes. Same decision tree, different objective.

6.4.3 The algorithm: optimal play against a three-way oracle

Code 6.4.1: Guess Number Higher or Lower – overflow-safe binary search on a comparison oracle

```cpp
#include <bits/stdc++.h>
using namespace std;

// Forward-declared oracle (provided by the judge):
//   returns -1 if num is HIGHER than the secret ("too high"),
//            1 if num is LOWER than the secret ("too low"),
//            0 if num equals the secret.
int guess(int num);

class Solution {
```

```
public:
    int guessNumber(int n) {
        int lo = 1, hi = n;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;      // NOT (lo+hi)/2 -- lo+hi overflows int
    ↪ (n up to 2^31-1)
            int g = guess(mid);
            if (g == 0)  return mid;           // hit
            if (g == -1) hi = mid - 1;         // mid too high -> secret is in [lo,
    ↪ mid-1]
            else         lo = mid + 1;         // mid too low  -> secret is in [mid+1,
    ↪ hi]
        }
        return -1;                             // unreachable for a valid secret in
    ↪ [1,n]
    }
};
```

Complexity

Time $O(\log n)$ guesses – the interval halves every iteration (Theorem 6.4.3); at $n = 2^{31}-1$ that is ≤ 31 oracle calls. Memory $O(1)$. The *overflow guard* `lo + (hi-lo)/2` is the real content of an "easy" problem: with n up to $2^{31}-1$, the naive `(lo+hi)/2` exceeds `INT_MAX` and mis-splits.

6.4.4 Worked Example

Example 6.4.4 (Finding $x = 2$ in $[1, 10]$ by adaptive halving)

Secret $x = 2$. Start $[lo, hi] = [1, 10]$. Guess $mid = 1+(10-1)/2 = 5$; $guess(5) = -1$ (too high) $\Rightarrow hi = 4$, interval $[1, 4]$. Guess $mid = 2$; $guess(2) = -1$ (too high) $\Rightarrow hi = 1$, interval $[1, 1]$. Guess $mid = 1$; $guess(1) = 1$ (too low) $\Rightarrow lo = 2$, and the loop's $mid = 2$ gives 0 after at most $\lceil \log_2 11 \rceil = 4$ guesses. Every step threw away about half the remaining candidates – the Figure 6.4.1 shrink in action, finishing within the Theorem 6.4.3 bound.

Why the midpoint is the right question

From a size-m interval a midpoint guess leaves $\lfloor m/2 \rfloor$ candidates on the "too high" branch and $\lceil m/2 \rceil - 1$ on the "too low" branch, so the larger survivor is $\leq \lceil m/2 \rceil$. Guessing *any other* k makes one branch $k-1$ or $m-k$ candidates – both can be $> m/2$ – so no single guess has a smaller worst case. That is the Proposition 6.4.2 lower bound met exactly.

6.4.5 Variations and Twists

- **Cost-aware split (LC 375)**: you *pay* the guessed number and want to minimise worst-case total cash; optimal first guess is not the middle – interval DP over $[i, j]$. **L3–L4**
- **Reconstruct a hidden *tree* by distance queries**: query dist(u, v) against an oracle (answerable via Chapter 5); adaptively locate a node or edge by which split shrinks the candidate-tree set – group testing on a tree. **L4–L5**
- **Ancestor/yes/no queries**: "is u an ancestor of v?" answered in $O(1)$ by tin / tout (Theorem 5.1.2) is a binary query; n of them rebuild parent pointers by bisecting each node's ancestor chain. **L4**
- **Non-adaptive (batch) queries**: if all questions must be fixed upfront (no adaptation), you need combinatorial group designs – more queries than the adaptive log. **L5**

- **Lies allowed**: Ulam's game – each answer might be wrong e times; the candidate count is spread over $\sum_{i \le e} \binom{k}{i}$ transcripts, inflating queries. L5

6.4.6 Common Pitfalls

Trap 1 – `(lo+hi)/2` overflowing

With n up to $2^{31}-1$, `lo+hi` exceeds `INT_MAX` and wraps negative, corrupting the midpoint (wrong answer or an infinite loop). Always write `lo + (hi-lo)/2` – mathematically equal, overflow-free.

Code 6.4.2: WRONG: overflows when `lo+hi` exceeds `INT_MAX`

```
int mid = (lo + hi) / 2;          // lo,hi up to 2^31-1  ->  signed overflow (UB)
```

Code 6.4.3: RIGHT: overflow-safe midpoint

```
int mid = lo + (hi - lo) / 2;     // hi-lo can't overflow; result identical
```

Trap 2 – flipping the oracle's sign

`guess(mid)` $= -1$ means *your guess is too high*, so the secret is *smaller*: shrink *hi*, not *lo*. Swapping the two branches makes the interval grow and the loop never ends (or returns wrong). Read the oracle's contract literally.

Trap 3 – non-shrinking bounds ⇒ infinite loop

Updating to $hi = mid$ (instead of $mid - 1$) while *lo* stays can keep *mid* constant forever. Because you already know *mid* is *not* the answer on that branch, exclude it: $hi = mid - 1$ / $lo = mid + 1$.

6.4.7 Problem Lab

LC 374 is the section's cleanest specimen: a hidden integer, a three-way oracle, and the provably optimal halving strategy – with one real trap (overflow) hiding behind an "easy" label.

W6.4.1 Guess Number Higher or Lower

Source: LeetCode 374 LC Easy L2
Techniques: binary search, interactive/oracle model, three-way comparison, overflow-safe midpoint

Guess Number Higher or Lower

We pick a number x in $[1, n]$. You guess a number and a pre-defined function `guess(num)` returns -1 if your guess is *higher* than x ("too high"), 1 if your guess is *lower* than x ("too low"), and 0 if correct. Return the number x. $1 \le n \le 2^{31} - 1$.

Attempt it yourself first – L2, give yourself 12 minutes

Fair first attempt: obviously binary search. But n is enormous – make sure your loop terminates and your midpoint never overflows.

Peek (1 min lost): What is the largest value `lo+hi` can take when $n = 2^{31} - 1$, and what does `int` do with it?

Nudge (5 min lost): Keep a $[lo, hi]$ window. Compute `mid` as `lo+(hi-lo)/2`. On `guess(mid)`$= -1$ move `hi` below `mid`; on 1 move `lo` above `mid`; on 0 return.

Full hint (you will not solve this one): Standard three-way binary search with an overflow-safe midpoint and correct branch direction ($-1 \Rightarrow$ `secret smaller`). Loop while `lo<=hi`. Full build and proof below.

➤ Understanding the Problem

A monotone predicate: define $P(k) \equiv (guess(k) \leq 0)$, i.e. $k \geq x$. Truth grows with k, so x is the smallest k with $P(k)$ true – exactly binary search on a sorted yes/no boundary. The oracle gives three-way feedback $(<, =, >)$, so you can finish the moment $guess(mid) = 0$. The only genuine difficulty is numeric: n reaches `INT_MAX`, so careless midpoint arithmetic overflows.

➤ Key Observations

Observation 1. Each answer collapses the candidate interval to at most half if you guess the midpoint (Proposition 6.4.2), so $\leq \lceil \log_2(n{+}1) \rceil$ guesses (about 31 for the largest n) always suffice – and no strategy guarantees fewer (Theorem 6.4.3).

Observation 2. `guess(mid)`$= -1$ means the secret is *strictly below* `mid`; $= 1$ means *strictly above*. Because `mid` is known wrong on that branch, exclude it (`mid`± 1) – this both prevents infinite loops and keeps the count at log.

Observation 3. `lo+hi` can overflow `int`; `lo+(hi-lo)/2` cannot, since $0 \leq hi - lo \leq n - 1$ fits. This one line is the difference between AC and a Wrong Answer on large n.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** Linear scan $1, 2, \ldots$ until `guess`$= 0$.
Killed by: $O(n)$ oracle calls – at $n = 2^{31}$ it is hopeless (TLE) even though each call is cheap. Queries are the currency here.

✗ **Tried:** Guess `(lo+hi)/2`.
Killed by: On the biggest inputs `lo+hi` overflows to a negative `int`, `mid` lands outside $[lo, hi]$, and you loop or return wrong. The idea is right; the *arithmetic* betrays it.

✓ **Aha!** Binary search with `mid=lo+(hi-lo)/2`: halve the window each answer, move to `mid`± 1 (mid is known-wrong on that branch), stop when `guess=0`. Optimal and overflow-free.

➤ Solution

Maintain a closed candidate interval $[lo, hi] = [1, n]$. While non-empty, compute the overflow-safe midpoint, ask `guess`: on 0 return it; on -1 set $hi = mid - 1$ (secret lower); on 1 set $lo = mid + 1$ (secret higher). Monotonicity guarantees the secret stays inside the window; halving guarantees termination in log steps.

➤ Proof of Correctness

Proof

Invariant: the secret $x \in [lo, hi]$ at every loop start (true initially). If $guess(mid) = -1$ then $x < mid$, so setting $hi = mid - 1$ keeps x in the window; the 1 case is symmetric; the 0 case returns $mid = x$. Thus the invariant is preserved and the returned value equals x. *Termination*: each iteration shrinks $hi - lo$ to at most $\lceil (hi - lo)/2 \rceil - 1 < hi - lo$, so after $\leq \lceil \log_2(n{+}1) \rceil$ steps the window empties (and must have hit 0 first, since x is always inside it). *Optimality*: by Proposition 6.4.2, any strategy needs $\Omega(\log n)$ three-way answers to distinguish n ordered secrets, and Theorem 6.4.3 shows this attains the bound, so it is query-optimal.

■

➤ Implementation

See Code 6.4.1 above – the listing in this section *is* the accepted solution: three-way binary search with `lo+(hi-lo)/2` and correct branch directions.

➤ Trace

Official $n = 10, x = 6$

$[lo, hi] = [1, 10]$: $mid = 5$, $guess = 1$ (low) $\Rightarrow lo = 6$. $[6, 10]$: $mid = 8$, $guess = -1$ (high) $\Rightarrow hi = 7$. $[6, 7]$: $mid = 6$, $guess = 0 \Rightarrow$ return 6. ✓ Three guesses for $x = 6$ – within $\lceil \log_2 11 \rceil = 4$.

➤ Complexity

Complexity

Time $O(\log n)$ oracle calls (≤ 31 at the maximum n); each call is treated as $O(1)$. Space $O(1)$. Optimal by Theorem 6.4.3. Verdict: ✓ **Accepted**.

➤ Pitfalls and Variants

Common Mistake

(i) `(lo+hi)/2` overflow (Trap 1) is the single most common WA/TLE here. (ii) Sign flip of the oracle (Trap 2). (iii) Using $hi = mid$ / $lo = mid$ instead of ± 1 (Trap 3) – infinite loop on the boundary. *Variant:* LC 374's "guess the word" (1095) and "find peak element" (162) are the same adaptive halving on a different oracle – binary search without an explicit = answer.

Key takeaways

1. **Choose questions that halve.** With an oracle, your power is picking the query whose answers split the candidate space evenly (Proposition 6.4.2).
2. **Counting bound = decision tree.** log questions is a tree of depth log – the same object as every other chapter, but nodes are *queries*.
3. **Easy label, real trap.** "Easy" hid an integer-overflow bug; correctness of the *arithmetic* matters as much as the algorithm.

AC ⟶ red: hardening this code

- **Prove optimality**: show no decision tree with fewer than $\lceil \log_2(n+1) \rceil$ leaves covers n ordered secrets – a counting argument (Proposition 6.4.2).
- **Cost-aware version**: replace the midpoint with the min-max choice $\arg\min_k (k + \max(T[lo, k-1], T[k+1, hi]))$ – LC 375.
- **Tree reconstruction by distance**: to find a hidden node x, repeatedly query $\text{dist}(centroid, x)$ (answerable in $O(1)$ after Chapter 5) to discard whole subtrees – centroid-halving instead of line-halving (Chapter 12 if centroid known).

6.4.8 Practice Problems

Stuck? Work through this list

- Did you write `lo+(hi-lo)/2`, not `(lo+hi)/2`? (Trap 1.)
- Did you map `guess`$= -1$ to "search *lower*" (shrink `hi`)? (Trap 2.)
- Did you exclude the known-wrong `mid` with ± 1? (Trap 3.)
- Can you state why fewer than $\log_2 n$ guesses cannot always work? (Proposition 6.4.2.)

Q6.4.1 Guess Number Higher or Lower II – LeetCode 375 LC Medium L3

Hint: Same three-way oracle as W6.4.1, but a wrong guess of k *costs you* $\$k$ and you must return the minimum cash needed to **guarantee** a win regardless of the secret. The halving midpoint is no longer optimal – you now minimise the worst-case *total cost*, a different objective on the same decision tree. Model it with interval DP: let $T[i][j]$ be the guarantee for secrets $i..j$; then $T[i][j] = \min_{k \in [i,j]} (k + \max(T[i][k-1], T[k+1][j]))$ with $T[i][j] = 0$ when $i \geq j$ (the adversary picks the pricier branch, so you prepare for the max). This is the "cost-aware split" variation from §6.4; it also generalises the optimal-BST / matrix-chain family – an interval-DP technique, not a tree-representation one.

10-second refresher – 6.4 Recover by Queries (Interactive)

You only see the answer by *asking*: a query model is (space, allowed questions, feedback). A truthful answer is worth a branch, so k b-way answers distinguish b^k targets – you cannot beat $\lceil \log_b \rceil$ (Proposition 6.4.2). Guess the midpoint: three-way binary search finds a secret in $[1, n]$ in $\lceil \log_2(n+1) \rceil$ guesses, provably optimal (Theorem 6.4.3), with `lo+(hi-lo)/2` to dodge `INT_MAX` overflow and $mid \pm 1$ to keep shrinking. When wrong guesses *cost*, minimise $k + \max(\text{left}, \text{right})$ over the interval (LC 375); when the hidden object is a tree, halve with distance/ancestor queries off the Chapter 5 oracle.

Self-test (answer from memory, tick the box):

- ☐ State the query model triple and give the oracle for LC 374.
- ☐ Prove $\lceil \log_2(n+1) \rceil$ is a lower bound on worst-case guesses for a three-way oracle.
- ☐ Why does `(lo+hi)/2` fail at $n = 2^{31} - 1$, and what replaces it?
- ☐ In LC 375, why do we take max (not sum/average) of the two sub-intervals plus the cost k?
- ☐ Give one distance/ancestor query that would help reconstruct a hidden tree, and why it halves.

Chapter 6 Summary

Four sections, one inverse question: *given raw data a tree left behind, rebuild the tree and prove the rebuild is the tree.* The parent/edge view (6.1) turns pairs into adjacency and reads the root off an in-degree histogram; the degree/code view (6.2) decides whether a degree list can be a tree at all and counts the ones that are; the identification view (6.3) uses a union-find pre-look to say *tree, forest, or cycle* in one pass; the query view (6.4) rebuilds a hidden object one adaptive question at a time. Every entry below is a pointer, not a re-derivation.

Result	Statement (with conditions)	Where
Tree certificate	For a graph on n nodes, any **two** of $\{n-1$ edges, connected, acyclic$\}$ force the third; check two, never all three.	Theorem 6.1.2
Degree sum	In every tree $\sum_v \deg(v) = 2(n-1)$; the count leg of the certificate in disguise.	Theorem 6.1.3
Read the root	A rooted structure has exactly one node of in-degree 0 (the root) and every other node of in-degree exactly 1; two parents or two roots means *not a tree.*	Proposition 6.1.4
Degree-sequence realisability	Positive $d_1, \ldots, d_n$ is tree-realisable **iff** $\sum d_i = 2(n-1)$; the leaf-peel builds one.	Theorem 6.2.2
Prüfer / count	Node v appears $\deg(v)-1$ times in the length-$(n-2)$ code; the number of labelled trees with degrees d_i is $(n-2)! \,/\, \prod_i (\deg(v_i)-1)!$.	Proposition 6.2.4
Operational tree test	n nodes form a tree **iff** there are $n-1$ edges *and* union-find rejects none; k rejections means k surplus (cyclomatic) edges.	Corollary 6.3.3
Query-optimal search	Against a truthful higher/lower/equal oracle the midpoint split attains $\lceil \log_2(n+1) \rceil$ guesses, and no strategy is better in the worst case.	Theorem 6.4.3

Algorithms from this chapter – rebuild, then certify.

Algorithm	Time	Memory	When to use (deciding phrase)	Code
Rebuild adj, find root, certify	$O(n)$	$O(n)$	"edge list / parent array given, is it a valid rooted tree?"	Code 6.1.1
Degree-realise by leaf-peel	$O(n \log n)$	$O(n)$	"build *a* tree with these degrees / is this degree list a tree?"	Code 6.2.1
Union-find pre-look	$O((n+m)\,\alpha)$	$O(n)$	"is it a tree? / how many components? / delete the redundant edge"	Code 6.3.1
Optimal guess (binary search)	$O(\log n)$	$O(1)$	"find a hidden value through a higher/lower oracle, fewest questions"	Code 6.4.1
Summit: degree-greedy parent hunt	$O(n^3)$	$O(n^2)$	"given all ancestor pairs, how many rooted trees match?"	Code 6.4.10

Master Decision Flowchart

Post-study ladder: walk top to bottom, stop at the first branch that matches. It decides *which pile of data you were handed* and hence which rebuild-and-certify tool to run – a follower never trusts a count that was never a certificate.

```
You are handed DATA about a tree. What shape is the data?
|
|-- an EDGE LIST or PARENT ARRAY (the pairs are explicit)
|     +-- undirected, exactly n-1 edges -> count leg + one union-find pass   [6.3]
|     +-- rooted / directed -> read in-degree: one 0 (root), rest exactly 1  [6.1,
  ↪ LC 685]
|     +-- n edges, one on a cycle -> delete the LAST rejected edge           [6.3,
  ↪ LC 684]
|     +-- raw equation / relationship pairs -> rebuild adj, DFS the ratio    [6.1,
  ↪ LC 399]
|
|-- a DEGREE SEQUENCE or CODE (only statistics survive)
|     +-- all d_i >= 1 and sum(d) == 2(n-1)?  else NOT realisable           [6.2]
|     +-- yes -> leaf-peel (Pruefer decode) builds one tree                  [6.2]
|     +-- "how many trees?" -> (n-2)! / prod (d_i - 1)!  (Cayley = all d free)[6.2]
|     +-- grid / region uniformity -> recurse into 4 quadrants (quad tree)   [6.2,
  ↪ LC 427]
|
|-- "IS IT a tree / how many components?" (cycle & connectivity ask)
|     +-- union-find: a rejected edge = a cycle; components = final set count [6.3]
|     +-- directed acyclicity (order exists) -> topological sort, NOT UF     [6.3,
  ↪ LC 210]
|
|-- an ORACLE (you only get to ASK)
|     +-- higher/lower/equal -> binary search, ceil(log2(n+1)) is optimal     [6.4,
  ↪ LC 374]
|     +-- a wrong guess COSTS you -> min-max k + max(left,right), interval DP  [6.4,
  ↪  LC 375]
|
+-- recursion / matrix sized by n -> a chain input is O(n) deep: explicit stack,
    or a vector<vector<bool>> sized n^2 when an adj list would do. Full DSU &
    MST -> [c17]; Pruefer/Cayley proofs -> [c32]; distance/LCA oracle -> [c24].
```

Common Mistakes Checklist

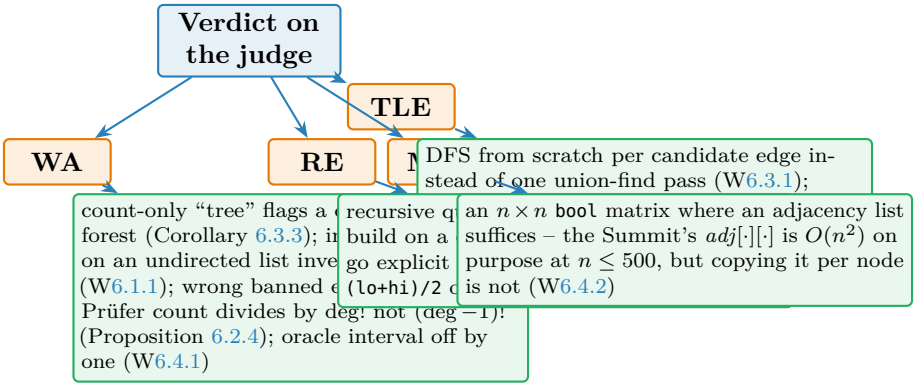
- **Trusting $n-1$ edges alone.** A disconnected forest can carry $n-1$ edges. The certificate needs *two* of the three legs – pair the count with connectivity or with a cycle-free union-find pass, never the count by itself (Theorem 6.1.2, Trap 1).
- **Reading in-degree on an undirected list.** Feeding both directions makes the root look like it has a parent and every leaf look like a non-root – a rooted instance (LC 685) must scan *directed* edges (Proposition 6.1.4, Trap 2).
- **Banning the wrong edge in the two-parent repair.** When a node has two parents, *remove the later edge first*, then run the cycle scan on the rest; returning the second parent without testing the cycle mis-picks the answer (W6.1.1).
- **Confusing the degree *sum* with a *realisation*.** $\sum d = 2(n-1)$ with all $d_i \geq 1$ is necessary *and* sufficient – but a 0 or a negative sneaking in breaks the peel; screen for it first (Theorem 6.2.2, Trap 1).
- **Inverting the Prüfer count.** Node v appears $\deg(v)-1$ times *in the code*, and the count divides by $\prod(\deg(v)-1)!$ – not $\deg(v)!$ (Proposition 6.2.4, Trap 2).
- **Running union-find on a directed acyclicity ask.** "is a topological order possible?"

(LC 210) is *not* the undirected cycle test; union-find ignores edge direction and gives the wrong verdict (§6.3, Trap 3).

- **Off-by-one and overflow in the oracle.** The interval is [lo, hi] closed – move to `mid-1`/`mid+1`; and `(lo+hi)/2` overflows for n up to $2^{31}-1$, so use `lo+(hi-lo)/2` (W6.4.1, Trap 1).

WA Clinic

Verdicts are symptoms; the causes are the short list this chapter drilled. Diagnose top-down – every leaf names the training problem.



Chapter 6 verdict tree – only the failures this chapter's problems can produce.

Backup table (same content, terse):

Verdict	Usual cause	One-line fix	Trained in
WA	count-only "is a tree"	pair $n-1$ edges with a UF pass	Corollary 6.3.3
WA	two-parent false alarm	scan *directed* edges only	W6.1.1
WA	Prüfer count deg!	divide by $(\deg(v)-1)!$	Proposition 6.2.4
WA	oracle off-by-one	closed [lo, hi], move ± 1	W6.4.1
TLE	re-DFS per candidate edge	one union-find sweep	W6.3.1
RE	dense *adj* matrix	size once at $n \le 500$; explicit stack	W6.4.2

Quick Reference Card

Chapter 6 at a glance

One question. The rest of the book *walks* a tree someone drew for you; this chapter answers the inverse – *what data pins a tree down, and how do I rebuild and certify it* from that data alone? Rebuild the object, then prove it is a tree with the certificate (Theorem 6.1.2).

Two invariants, always. $m = n - 1$ edges (equivalently $\sum_v \deg(v) = 2(n-1)$, Theorem 6.1.3) and *connected* (or *acyclic*) – any two of the three certificate legs force the third (Theorem 6.1.2). A degree list is realisable iff every $d_i \ge 1$ and the sum is exactly $2(n-1)$ (Theorem 6.2.2).

Four shadows of the same tree T. Edge list / parent array $\rightarrow$ rebuild and read the root

(§6.1, Example 6.1.5). Degree sequence → peel and count (§6.2, Example 6.2.6). Component count → union-find rejections (§6.3, Example 6.3.4). Oracle answers → adaptive halving (§6.4, Example 6.4.4). All four describe the *identical* T of Figure 6.0.1.

Prüfer one-liner. Node v shows up $\deg(v) - 1$ times in the length-$(n-2)$ code, so a degree sequence is realised by $(n-2)! \,/\, \prod_v (\deg(v)-1)!$ labelled trees (Proposition 6.2.4) – Cayley's n^{n-2} is that sum with degrees free (Chapter 30).

Code 6.4.4: Chapter 6 code kit: rebuild, realise, union-find, guess

```cpp
#include <bits/stdc++.h>
using namespace std;

// (1) Edge list / parent array -> adjacency, read the root, certify.   [6.1]
vector<vector<int>> buildFromParent(const vector<int>& par, int& root, bool& ok) {
    int n = (int)par.size() - 1; vector<vector<int>> adj(n + 1); vector<int> indeg(n + 1);
    root = 0; int zeros = 0; ok = true;
    for (int v = 1; v <= n; ++v) { int p = par[v];
        if (p == 0) { root = v; ++zeros; }                  // candidate root(s)
        else if (p >= 1 && p <= n) { adj[p].push_back(v); ++indeg[v]; }
        else ok = false; }                                  // parent out of range
    if (zeros != 1) ok = false;                             // exactly one root
    for (int v = 1; v <= n; ++v) if (v != root && indeg[v] != 1) ok = false;
    return adj;                                             // + DFS from root for connectivity
}

// (2) Degree sequence -> a realising tree by the leaf-peel (Pruefer decode). [6.2]
bool realisable(const vector<int>& deg) {                   // the WHOLE test (Thm 6.2)
    long long sum = 0;
    for (int d : deg) if (d < 1) return false; else sum += d;
    return sum == 2LL * ((int)deg.size() - 1);
}

// (3) Union-find pre-look: a rejected edge closes a cycle; count components. [6.3]
struct DSU { vector<int> p;
    DSU(int n) : p(n) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool uni(int a, int b) { a = find(a); b = find(b);
        if (a == b) return false; p[b] = a; return true; } };   // false => cycle

// (4) Optimal guess against a higher/lower/equal oracle -- overflow-safe.    [6.4]
int guessNumber(int n) {
    int lo = 1, hi = n;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;                       // NOT (lo+hi)/2 -- overflow
        int g = guess(mid);                                 // -1 too high, 1 too low, 0 hit
        if (g == 0) return mid;
        if (g == -1) hi = mid - 1; else lo = mid + 1;
    }
    return -1;
}
```

Write-From-Memory Drills

Five signature skeletons, each lifted from a listing already in the chapter. Cover the key line, reproduce it from memory, then check the answer one line below.

Code 6.4.5: Drill 1 – rebuild a rooted tree from its parent array (from Code 6.1.1)

```
for (int v = 1; v <= n; ++v) { int p = par[v];
    if (p == 0) { root = v; ++zeros; }
    else if (1 <= p && p <= n) { /* FILL: ______ */; ++indeg[v]; }
    else ok = false;
}
if (/* FILL: zeros != 1 */) ok = false;                    // exactly one root
```

Key: append v to `adj[p]` for the child list; validity is *one* zero in-degree and in-degree 1 for every other node (Proposition 6.1.4).

Code 6.4.6: Drill 2 – the degree-sum gate (from Code 6.2.1)

```
long long sum = 0;
for (int d : deg) { if (/* FILL: d < 1 */) return {}; sum += d; }
if (/* FILL: sum != 2LL*(n-1) */) return {};               // Thm 6.2 is the WHOLE
    ↪ test
```

Key: positivity first, then the single equation $\sum d = 2(n-1)$ (Theorem 6.2.2); if it passes, the leaf-peel never gets stuck.

Code 6.4.7: Drill 3 – the Prüfer multiset (from Code 6.2.1)

```
for (int i = 0; i < n; ++i)
    for (int k = 1; k < /* FILL: deg[i] */; ++k) prufer.push_back(i);   // deg(v)-1
    ↪ copies
```

Key: node v appears $\deg(v)-1$ times, so the code has length $\sum(\deg(v)-1) = n-2$ (Proposition 6.2.4).

Code 6.4.8: Drill 4 – is-it-a-tree by one union-find pass (from Code 6.3.1)

```
bool isTree(int n, const vector<pair<int,int>>& edges) {
    if ((int)edges.size() != /* FILL: n - 1 */) return false;         // leg (1)
    DSU d(n);
    for (auto& e : edges) if (!d.uni(e.first, e.second)) return false; // a reject = a
    ↪  cycle
    return true;
}
```

Key: $n-1$ edges *and* no rejected edge $\Rightarrow$ connected and acyclic at once (Corollary 6.3.3).

Code 6.4.9: Drill 5 – optimal play against the oracle (from Code 6.4.1)

```
int lo = 1, hi = n;
while (lo <= hi) {
    int mid = /* FILL: lo + (hi - lo)/2 */;
    int g = guess(mid);
    if (g == 0) return mid;
    if (g == -1) /* FILL: hi = mid - 1 */; else /* FILL: lo = mid + 1 */;
}
```

Key: closed interval, so shrink to `mid-1`/`mid+1`; the midpoint guard avoids overflow at $n = 2^{31}-1$ (Theorem 6.4.3, Trap 1).

Complete Problem Checklist

Every problem this chapter offers – 4 fully solved in-section, 4 practice, one Summit, five mixed. Tick *only* after an unaided solve; a hinted attempt counts as half. Cite by name and number – URLs appear once, in the solved and practice entries.

#	Problem	Source	Key technique	What to look for	Done?
Section 6.1 – parent array / edge list					
W6.1.1	Redundant Connection II	LC 685	**L4** in-degree + union-find repair	two-parent case, ban later edge, then cycle scan	☐
P6.1.2	Evaluate Division	LC 399	**L3** rebuild graph, ratio DFS	pour equation pairs into weighted adj, walk a path	☐
Section 6.2 – degrees, sequences, codes					
W6.2.1	Construct Quad Tree	LC 427	**L3** recursive uniform partition	all-equal test then split into four quadrants	☐
P6.2.2	Course Schedule II	LC 210	**L3** topological order (directed)	prerequisite pairs → an ordering, or the empty list on a cycle	☐
Section 6.3 – is-it-a-tree / union-find					
W6.3.1	Redundant Connection	LC 684	**L3** union-find first reject	n edges, return the last edge that closed a cycle	☐
P6.3.2	Number of Provinces	LC 547	**L2** count components	matrix → union-find, answer is the final set count	☐
Section 6.4 – queries / interactive					
W6.4.1	Guess Number Higher or Lower	LC 374	**L2** binary search on oracle	overflow-safe midpoint, closed-interval shrink	☐
P6.4.2	Guess Number Higher or Lower II	LC 375	**L4** min-max interval DP	wrong guess costs k: $k + \max(\text{lo}, \text{hi})$	☐
Summit (chapter end)					
S6	Number of Ways to Reconstruct a Tree	LC 1719	**L5** degree-greedy parent hunt + transitivity	equal-degree tie → 2, missing containment → 0	☐

Recommended Study Order

Five focused days. Day 1 fixes the certificate and the rebuild; day 2 the degree/count machinery; day 3 the union-find identification; day 4 the query model; day 5 the Summit. Solve in this order even where it differs from section order – each day's tool is already in hand the day before.

Day	Focus	Tasks
1	Read + do 6.1	rebuild T from parent array and edge list (Example 6.1.5); LC 685; LC 399; write the root-read cold
2	Read + do 6.2	realise T's degree sequence by the peel (Example 6.2.6); LC 427; LC 210; count the 180 realisations
3	Read + do 6.3	classify T and T+bad edge (Example 6.3.4); LC 684; LC 547
4	Read + do 6.4	halve $[1, 10]$ to find x (Example 6.4.4); LC 374; LC 375
5	Summit + revise	LC 1719 degree-greedy hunt; walk the decision flowchart on five questions

Difficulty Progression

Tiers are competency gates – each row names the *skill* the tier unlocks, not just a rating bucket.

Tier	Problems	What passing this tier proves
L2 ★★	LC 374, 547	you can drive an oracle optimally and count components with one union-find pass
L3 ★★★	LC 684, 427, 399, 210	rebuild from raw pairs, spot the redundant edge, and partition a grid recursively
L4 ★★★★	LC 685, 375	split on in-degree *and* cycle in a directed repair; shift the split when a guess costs
L5 ★★★★★	LC 1719 (Summit)	reconstruct from partial (ancestor) data and decide existence and multiplicity

Reduction Gallery

Construction is a disguise; each line collapses a family onto one rebuild-and-certify move. No new theory – just where the certificate is already hiding.

- "is this a tree?" $\longrightarrow$ $n-1$ edges plus one union-find pass with no rejection (Corollary 6.3.3).
- "delete one edge to make it a tree" (LC 684) $\longrightarrow$ the last edge union-find rejects is on the unique cycle (W6.3.1).
- "one node has two parents / a cycle too" (LC 685) $\longrightarrow$ in-degree scan splits the two repair cases before the cycle test (W6.1.1).
- "build a tree with these degrees" (LC-ish) $\longrightarrow$ check $\sum d = 2(n-1)$, then peel leaves off a Prüfer multiset (Theorem 6.2.2, Code 6.2.1).
- "how many trees share this degree sequence?" $\longrightarrow$ decode count $(n-2)!/\prod(\deg(v)-1)!$ (Proposition 6.2.4).
- "is this $n \times n$ grid uniform?" (LC 427) $\longrightarrow$ a tree built from the data by four-way recursion, not from edges (W6.2.1).
- "can I order these prerequisites?" (LC 210) $\longrightarrow$ a *directed* acyclicity test (topological sort), the cousin union-find cannot answer (§6.3).
- "group accounts sharing an email" (LC 721) $\longrightarrow$ union-find where the keys are strings – the same merge mechanic as Code 6.3.1.
- "are these equality/inequality constraints satisfiable?" (LC 990) $\longrightarrow$ union the equals, then any inequality whose two ends are already joined is a contradiction – a cycle in disguise (Corollary 6.3.3).
- "rebuild an N-ary tree inside a binary one" (LC 431) $\longrightarrow$ first-child / next-sibling encoding, the structural mirror of §6.1's parent-array rebuild.
- "how many rooted trees match these ancestor pairs?" (LC 1719) $\longrightarrow$ degree-based parent hunt plus transitivity containment (W6.4.2).

Mixed Problem Set

These refuse to name their pattern; decide *which rebuild-and-certify move* you need before opening the code. A hint tells you the shape of the answer, never the tool.

M6.1 Course Schedule – LeetCode 207 LC Medium L3

Hint: "can you finish all courses" is "is the prerequisite digraph acyclic" – but note the direction matters, so this is the *topological* twin of §6.3, not the undirected union-find test of Corollary 6.3.3: a Kahn/DFS in-degree peel decides it.

M6.2 Regions Cut By Slashes – LeetCode 959 LC Hard L4

Hint: scale the grid to a cell complex and union the four neighbours of every open cell; the answer is the *component count* of Code 6.3.1 after you stop merging – the exact "components = final set count" reading of §6.3.

M6.3 Accounts Merge – LeetCode 721 LC Medium L3

Hint: emails that co-occur on one account belong to one person: union-find where the universe is email *strings*, not indices. Group by the **find** representative – the same merge mechanic as Code 6.3.1, keyed differently.

M6.4 Satisfiability of Equality Equations – LeetCode 990 LC Medium L3

Hint: union every "$a == b$" pair, then every "$a! = b$" whose ends are *already* joined is a contradiction – an inequality edge closing a cycle over the equalities, so it is Corollary 6.3.3's rejection test in a fresh costume.

M6.5 Encode N-ary Tree to Binary Tree – LeetCode 431 LC Hard L4

Hint: a lossless encoding *is* a reconstruction: map each node to *first child* and *next sibling* pointers so the N-ary tree survives as a binary one, then rebuild it exactly as §6.1 rebuilds T from a parent array – the arrow reversed.

M6.6 Remove Max Number of Edges to Keep Graph Fully Traversable – LeetCode 1579 LC Hard L4

Hint: Type-3 edges serve both players, so claim them last: union the type-1 owners, then the type-2, then spend Code 6.3.1's cycle test on the shared edges – the maximal-removal question is §6.3's "which edges are forced" run backwards.

M6.7 Lexicographically Smallest Equivalent String – LeetCode 1061 LC Medium L3

Hint: The equalities partition 26 letters into classes; rewrite every character as its class's *smallest* member – Code 6.3.1's canonical-representative habit, one find per letter.

M6.8 Most Stones Removed with Same Row or Column – LeetCode 947 LC Medium L3

Hint: Every removal leaves the component connected, so what survives is one stone per component over shared row/column coordinates: union, then count the final representatives – §6.3's component ledger with stones as vertices.

M6.9 Possible Bipartition – LeetCode 886 LC Medium L3

Hint: Disliked pairs must land on opposite sides: BFS two-colouring answers directly, or union each person with all of a rival's rivals – a failed merge is Corollary 6.3.3's rejection test wearing an odd cycle.

M6.10 Count the Number of Complete Components – LeetCode 2685 LC Medium L3

Hint: Group vertices with Code 6.3.1, then tally vertices and edges per component: complete exactly when the edges hit $v(v-1)/2$. The certify-after-building step is §6.2's degree audit counted on edges.

The Summit – LC 1719 Number of Ways to Reconstruct a Tree

Why this is the chapter's top rope

Sections 6.1–6.4 each rebuild or verify a tree from *one clean shadow*: a full edge list, a degree sequence, a bag of union-find answers, an oracle. The Summit hands you the faintest shadow imaginable – only the set of *ancestor–descendant pairs*, and not even the tree – then asks two questions at once: does *any* rooted tree realise exactly these pairs, and if so are there *several*? The whole chapter converges here: you must *reconstruct* a structure from partial data (§6.1), reason about *degrees* to guess who can be a parent (§6.2), and *certify* validity – here by the transitivity of ancestry rather than a union-find cycle count (§6.3). A node's candidate parent is the *lowest-degree node still connected to it*: ancestry is transitive, so every neighbour of a node must also be a neighbour of its parent. When two nodes can occupy the same parent slot (equal degree) the answer is "more than one"; when a required containment fails, "none."

W6.4.2 Number of Ways to Reconstruct a Tree

Source: LeetCode 1719 LC Hard L5
Techniques: degree-sorted parent hunt + ancestor-transitivity containment test; equal-degree tie → multiple

Number of Ways to Reconstruct a Tree

You are given `pairs`, a list of pairs$[i] = [x_i, y_i]$ with $x_i < y_i$ and no duplicates, describing an unknown rooted tree on the nodes that appear. A pair $[x, y]$ is present **iff** one of x, y is an *ancestor* of the other (nodes on the root-to-node path, the node itself excluded). The tree is *not* binary – a node may have any number of children. Return **0** if no rooted tree matches, **1** if exactly one does, and **2** if more than one does. Two trees differ when some node has a different parent. ($1 \le |\text{pairs}| \le 2 \cdot 10^5$, $1 \le x_i < y_i \le 500$.)

Attempt it yourself first – L5, give yourself 50 minutes

A fair first attempt: notice that a node connected to *every* other node must be the root (its ancestor set is everyone), and that if you fix a root, a node's parent is *some* ancestor. But enumerating trees is exponential – the challenge is to decide existence and multiplicity *greedily* from the degrees alone.

Peek (1 min lost): sort the nodes by how many pairs they appear in. Who is the most likely parent of a node x – a node of larger or smaller degree?

Nudge (5 min lost): a parent must be connected to all of x's neighbours too (transitivity), so look for x's parent among nodes *connected to* x with degree at least $\deg(x)$. The lowest such degree is the tightest fit – the nearest ancestor.

Full hint (you will not solve this one): Build the adjacency and degrees. Process nodes in *increasing* degree; for node x scan the higher-degree nodes for the first one y that is *adjacent* to x – that is x's parent candidate. If $\deg(x) = \deg(y)$ the two could swap parent slots, so mark *multiple*. Verify y is adjacent to *every* neighbour of x (containment); if any fails, no tree exists (0). If more than one node ends up with no parent, more than one root (0). Else return 2 if a tie was seen, otherwise 1. Full build and proof below.

➤ Understanding the Problem

The input is not edges of the tree – it is the *ancestor relation* compressed into pairs. So $[x, y] \in$ pairs does not mean x is y's parent, only that one is somewhere above the other. Two structural facts drive everything. First, ancestry is *transitive*: if p is x's parent, then every node related to x (its ancestors, and, going down, its descendants that reach through x) must also be related to p – in the pair graph, p's neighbour set must *contain* x's neighbour set. Second, a parent has *at least* as many relations as its child, so $\deg(p) \geq \deg(x)$. The reconstruction is therefore a greedy hunt: attach each node to the *minimum-degree* neighbour that is still at least as connected as it is.

➤ Key Observations

Observation 1. Sorting nodes by degree means when you look at x, every possible parent is *later* in the list. The first later node adjacent to x (smallest such degree) is the closest ancestor – the tightest fit, exactly as §6.2 reads structure off degrees.

Observation 2. If x and its chosen parent y have the *same* degree, either could be the other's parent without changing any pair count – so the tree is not unique: return 2 if a valid tree exists. This is the multiplicity signal (Proposition 6.2.4 counts a kindred "swap" freedom).

Observation 3. Containment is the validity test: for every neighbour z of x, z must be adjacent to y (or be y/x itself). A failure means no rooted tree can make y the parent of x – return 0. It is the transitivity the certificate of Theorem 6.1.2 silently assumes.

➤ Think It Through With Me

Think it through with me

✗ **Tried:** Treat `pairs` as tree edges and run a union-find.
Killed by: They are *not* edges – a pair can join a node to a grandparent. Union-find would happily accept a triangle $\{1, 2, 3\}$ as a 2-edge tree and miss that two of those are the same ancestor chain. The relation is ancestry, not adjacency.

✗ **Tried:** Try every root and every parent assignment.
Killed by: Exponential. Degrees already order the search: a parent has degree $\geq$ its child, so a single increasing-degree sweep fixes the candidate for each node in one pass.

✓ **Aha!** Sort by degree; for each node find the first adjacent higher-degree node as its parent, flag an equal-degree tie as *multiple*, and verify the parent contains all the node's neighbours. Count roots; more than one root $\to$ 0, a tie $\to$ 2, else 1.

➤ The Approach Ladder

#	Approach	Idea	Time	Verdict
1	**L3** pairs-as-edges union-find	treat each pair as a tree edge	$O(m\,\alpha)$	**WA**
2	**L4** root + parent enumeration	fix root, recurse on ancestor sets	exponential	**✗ TLE**
3	**L5** degree-sorted hunt + containment	nearest higher-degree adjacent parent	$O(n^3)$	**✓ Accepted**

Rung 1 is the tempting trap – the pairs are ancestry, not edges, so a cycle-free count means nothing (Rung 1's failure is precisely why §6.3's test needs the *right* edges). Rung 2 is correct but explodes. Rung 3 uses the degree order to pin one parent per node and the containment check to certify it.

➤ Solution

Build an adjacency *matrix* $adj[\cdot][\cdot]$ and neighbour lists from `pairs`; self-loops $adj[v][v]$ are set so the containment test reads uniformly. Collect the nodes that actually appear, sort them by increasing degree. For each node x (at position i), scan $j > i$ for the first node y adjacent to x – that is x's parent candidate. If none is found, x is a root (increment the root count). Otherwise, if $\deg(x) = \deg(y)$ flag *multiple*, and verify every neighbour z of x is adjacent to y; a miss returns 0. At the end, more than one root returns 0; otherwise a *multiple* flag returns 2, else 1 (Code 6.4.10).

➤ Proof of Correctness

Nearest higher-degree parent, containment, and the tie count

Lemma 1 (a parent has degree at least its child). If p is x's parent, every node related to x is related to p (transitivity of ancestry), and p is additionally related to x, so $\deg(p) \geq \deg(x)+1 > \deg(x)$ unless p and x have identical relation sets, in which case $\deg(p) = \deg(x)$. Hence scanning higher-or-equal degree positions only can find x's parent. *Lemma 2 (the first adjacent one is the closest ancestor).* Among the nodes adjacent to x with degree $\geq \deg(x)$, the minimum-degree one is the deepest possible ancestor – any other adjacent node of larger degree is an ancestor of it, not of x directly. Choosing it loses no valid tree. *Lemma 3 (containment is necessary and sufficient).* y can be x's parent iff y is adjacent to every neighbour of x: *necessary* by transitivity; *sufficient* because attaching x under such a y preserves exactly the given pairs (no pair is created – y already relates to all of x's relations – and none is lost). *Multiplicity and existence.* If some x shares a degree with its parent y, the two are interchangeable (equal deg, equal relation sets equal), so > 1 trees exist, flagged by *multiple*; if two nodes have no parent they are two roots, impossible in one tree (0); if a containment fails, no parent works (0). Otherwise the greedy parents are forced, giving exactly one tree. Returning 0/2/1 by these tests is therefore correct. ■

➤ Implementation

Code 6.4.10: Number of Ways to Reconstruct a Tree (LeetCode 1719) – degree-sorted parent hunt + containment

```cpp
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int checkWays(vector<vector<int>>& pairs) {
        int N = 501;                                        // node labels are <= 500
        vector<vector<char>> adj(N, vector<char>(N, 0));    // adjacency as a bit
    ↪ matrix
        vector<vector<int>> nbr(N);
        for (auto& pr : pairs) {
            int A = pr[0], B = pr[1];
            adj[A][B] = adj[B][A] = 1;
```

```
            nbr[A].push_back(B); nbr[B].push_back(A);
        }
        vector<int> active;
        for (int id = 1; id < N; ++id)
            if (!nbr[id].empty()) { active.push_back(id); adj[id][id] = 1; }  // self-
    ↪ loop: uniform test
        sort(active.begin(), active.end(),
             [&](int a, int b) { return nbr[a].size() < nbr[b].size(); });    //
    ↪ increasing degree

        bool multiple = false;
        int roots = 0;
        for (int i = 0; i < (int)active.size(); ++i) {
            int x = active[i], j = i + 1;
            while (j < (int)active.size() && !adj[x][active[j]]) ++j;         // first
    ↪  adjacent, >= degree
            if (j < (int)active.size()) {
                int y = active[j];
                if (nbr[x].size() == nbr[y].size()) multiple = true;          // swap
    ↪ -> >1 tree
                for (int z : nbr[x]) if (!adj[y][z]) return 0;                //
    ↪ transitivity fail -> none
            } else {
                ++roots;                                                     // x has
    ↪ no parent -> root
            }
        }
        if (roots > 1) return 0;                                             // two
    ↪ roots impossible
        return multiple ? 2 : 1;
    }
};
```

➤ Trace

Worked example – `pairs = [[1,2],[2,3],[1,3]]`

All three nodes appear; degrees $\deg(1) = \deg(2) = \deg(3) = 2$. After self-loops, *adj* is complete on $\{1, 2, 3\}$. Sorted by degree (tie, order $1, 2, 3$).

- $x = 1$: first adjacent later node is 2; $\deg(1) = \deg(2)$ so *multiple*=true; neighbours of 1 are $\{2, 3\}$, both adjacent to 2 ✓.
- $x = 2$: first adjacent later node is 3; tie keeps *multiple* true; neighbours $\{1, 3\}$ both adjacent to 3 ✓.
- $x = 3$: no later node, so 3 is the **root**; *roots* = 1.

One root and a tie ⇒ return **2**. Indeed the three pairs are the ancestor relation of a *chain* of length 3, and either of the two non-root orders ($1 \to 2 \to 3$ or $3 \to 2 \to 1$, plus rotations) is valid – more than one tree. ✓

A quick "none" case – `pairs = [[1,2],[2,3]]`

deg $= [0, 1, 2, 1]$ for nodes $1, 2, 3$ (node $1, 3$ leaves, node 2 centre). Sorted: $1, 3, 2$. $x = 1$ attaches to 2 (adjacent), no tie, neighbours of $1 = \{2\}$ contained in 2 ✓. $x = 3$ attaches to 2 ✓. $x = 2$ is last → root, *roots* = 1. No tie, no containment break ⇒ return **1**: the unique tree is 2 as root with children $1, 3$.

➤ Complexity

Complexity

Time $O(n^3)$ in the worst case, where $n \leq 500$ is the number of distinct nodes: the outer sweep is n, the parent scan is $O(n)$, and the containment loop inspects up to n neighbours – about $500^3 = 1.25 \cdot 10^8$ elementary operations at the extreme, and $O(m)$ to build the matrix ($m \leq 2 \cdot 10^5$). Space $O(n^2)$ for the adjacency matrix (501^2 bytes). The $n \leq 500$ label bound is what makes a dense $adj[\cdot][\cdot]$ affordable here – contrast §6.1, where an edge list would blow up at n large.

➤ Verification and Edge Cases

Case	What the code does
Star (one node in every pair)	that node has max degree, becomes the single root; leaves tie-free → 1
Perfect chain	every consecutive degree differs by one; no tie, one root → 1
Two nodes same relation set	equal-degree tie fires → *multiple* → 2
Contradictory pairs	a containment check finds a neighbour z with $adj[y][z] = 0$ → 0
Two components (two roots)	`roots`> 1 at the end → 0

➤ Pitfalls and Variants

Common Mistake

Three classic bugs: (i) reading `pairs` as tree *edges* and running Corollary 6.3.3's $n-1$-count test – they are *ancestry* pairs, so the count is meaningless (Rung 1's WA). (ii) Forgetting the equal-degree tie: a valid tree with a swappable parent must report 2, not 1. (iii) Skipping the `roots`> 1 guard – a forest of two "trees" has two parentless nodes and must be 0. Variant – if the problem asked to *build* one tree rather than count, the same greedy parents give it directly; and the "distinct labels ≤ 500" bound is what lets a dense matrix stand in for the adjacency lists of §6.1.

Key takeaways – three things to remember

1. *Partial data still pins structure.* Ancestor pairs, though not edges, order a unique parent per node once you sort by degree – the degree-reading skill of §6.2.
2. *Transitivity is the certificate here.* Containment (y relates to all of x's relations) plays the role the union-find rejection plays in §6.3.
3. *Multiplicity* $\iff$ *an interchangeable slot.* Equal-degree adjacent nodes can swap, exactly as Proposition 6.2.4's count is > 1 whenever two degrees repeat.

AC ⟶ red: hardening this code

- **Larger label range**: replace the $O(n^2)$ `char` matrix with a hash set of neighbours and test containment by set-subtraction – same logic, $O(m)$ memory, so the $n \leq 500$ ceiling lifts.
- **Return a real tree**: keep the chosen y as `parent[x]` during the sweep; the greedy is already a reconstruction, closing the loop with §6.1.

- **This is counting-in-disguise**: the "how many" $\{0, 1, 2\}$ answer mirrors Proposition 6.2.4's exact count $(n-2)!/\prod(\deg-1)!$; the full Prüfer bijection that turns such counts into closed forms is Chapter 30.

Tool Kit Inventory

The machine assembled so far – cumulative through Chapter 6. Chapters 1–5's rows are carried forward; Chapter 6's rows are appended. Pointers only, never prose.

Tool	Since	Level	Trigger phrase	Where
Chapters 1–5 – carried forward				
Adjacency list + parent-guard DFS	1.1	L1	edge list given, not pointers	Code 1.1.1
Three-slot DFS (pre/in/post)	3.1	L1	a node sequence in a named order	Code 3.1.1
Root: read parent, depth, size, height	4.2	L3	undirected edges, need above/below	Code 4.2.1
Stamp tin / tout; $O(1)$ ancestor test	5.1	L2	"is u above v?" / DFS-order listing	Code 5.1.1
Flatten: subtree = range $[\text{tin}, \text{tin}+\text{size})$	5.2	L3	many whole-subtree queries	Code 5.2.1
Chapter 6 – new this chapter				
Rebuild adj, read root, certify	6.1	L3	edge list / parent array, valid rooted tree?	Code 6.1.1
Degree-sum gate + leaf-peel realisation	6.2	L3	"build/count a tree from degrees"	Code 6.2.1
Union-find pre-look (tree? components?)	6.3	L2	"is it a tree / how many provinces"	Code 6.3.1
Optimal guess on a 3-way oracle	6.4	L2	"find a hidden value, fewest questions"	Code 6.4.1
Degree-greedy parent hunt + containment	Summit	L5	"reconstruct from ancestor pairs"	Code 6.4.10

Related Topics

- Chapter 3, Chapter 4 – backward: rebuilding from a *pair of traversals* (in+pre) lives in Chapter 3; this chapter rebuilds from *arbitrary* data, and the mechanical adjacency build / parent-guarded rooting is §4.2 – we add only the validity and root-detection decisions.
- Chapter 5 – forward/backward: 6.4 *uses* the distance/ancestor oracle (tin / tout, dist, LCA) as a black box to answer queries about a hidden tree; it never re-derives it.
- Chapter 17 – forward: the full Disjoint-Set-Union structure, union by rank and path compression live there (its greedy use in MST is Ch. 31, Chapter 31); §6.3 is only the one-pass pre-look.
- Chapter 30 – forward: the Prüfer-code bijection and Cayley's n^{n-2} proof; here the code is a *teaser* that explains why a degree sequence pins a counting problem (Proposition 6.2.4).
- Chapter 16, Chapter 27 – LCA and Mo-on-trees both *assume* a rebuilt, rooted tree – the object this chapter produces and certifies.

Last Word – Chapter 6

*"Every earlier chapter walked a tree you were **given**; this chapter asked whether a tree can be summoned back from the **marks** it left – and proved the summoning was honest."*

Chapters 3–5 taught you to traverse, store, and flatten a tree that was already drawn for you. This chapter inverted the arrow. You were handed a pile of pairs and asked to *rebuild* the shape (§6.1), handed a list of degrees and asked whether *any* tree wears those numbers (§6.2), handed an edge stream and asked *is this even a tree* (§6.3), and finally handed nothing but an oracle and asked to *elicit* the answer one question at a time (§6.4). Four shadows – edge list, parent array, degree sequence, distance answers – all of the *same* T in Figure 6.0.1, and each enough to rebuild it.

Carry three reflexes forward:

1. *Certify with two legs, never one.* $n-1$ edges *and* connected, or $n-1$ edges *and* acyclic – any two of the certificate force the third (Theorem 6.1.2); the count alone is a trap a forest happily passes.
2. *Degrees are data.* $\sum \deg = 2(n-1)$ decides realisability, $(n-2)!/\prod(\deg -1)!$ counts the trees, and the Summit's parent hunt runs entirely on degree order (Theorem 6.2.2, Proposition 6.2.4).
3. *A good question halves the world.* Against an oracle, choose the query whose answers leave equally small sub-cases; $\lceil \log_2 \rceil$ is not a convenience, it is a lower bound (Theorem 6.4.3).

And three habits, drilled until boring: write T's parent array, degree sequence, and Prüfer code by hand and check $\sum \deg = 2(n-1)$ before you trust a rebuild; keep straight the *undirected* cycle test (union-find) from the *directed* one (topological sort) – they answer different questions and each fails the other; and never answer an oracle by a linear scan when a midpoint halves the candidate set.

Next: Chapter 7 keeps building on the rebuild instinct but installs an *order* inside the tree – a Binary Search Tree is a tree that knows its own inorder, so search, insert, delete and rank queries all collapse into a single root-to-leaf descent, and the degeneration of that descent is the cliffhanger that opens the balancing chapter.

Part II

Ordered Trees, Heaps and Tries

Search Trees, Heaps, Tries

LC Medium – CF 1600 • 5 chapters • 102 problems

Chapters in this part

7 Binary Search Trees 412

8 Balanced Search Trees 473

9 Cartesian Trees and the Monotonic Stack 536

10 Heaps and Priority Queues 593

11 Tries and Binary Tries 651

Part III

Algorithms on Rooted Trees

DFS, Tree DP, Ancestors

CF 1400–2000 • 6 chapters • 123 problems

Chapters in this part

12 Height, Diameter and Centroid 711

13 Tree DP I: Subtree and Path DP 771

14 Tree DP II: Knapsack, States and Budgets 836

15 Rerooting: Every Root in One DFS 901

16 Ancestor Queries: LCA and Binary Lifting 956

17 DSU, Small-to-Large and Sack 1010

16

Chapter

Ancestor Queries: LCA and Binary Lifting

Level: CF 1700–2500, CSES, LC Hard • **Problems:** 21 (5 fully solved) • **Status:** Written

Chapter at a Glance

Chapters 12–15 all rooted the tree and walked away with one number (or one number per root). This chapter asks the question they kept deferring: a *query* now names *two arbitrary nodes* – find their join point, measure the route between them, fetch the k-th ancestor – and there are Q up to $2 \cdot 10^5$ such queries on a static tree. The single object that answers all of them is the **lowest common ancestor**: the deepest node that lies on both root-chains.

Everything here is *one question, three machines.* Machine 1 is the honest walk: equalise the depths, then step both pointers up together – $O(h)$ per query, zero setup, and the baseline every proof argues against (§16.1). Machine 2 converts LCA into a range-minimum query on the $2n-1$ **tour of the DFS walk**: record every visit, keep the first occurrence `first[]` of each node, and the LCA is the shallowest node between the two firsts – a **sparse table** answers that in $O(1)$ (§16.2). Machine 3 precomputes `up`$[v][j]$, the 2^j-th ancestor of every node, and climbs any distance in $O(\log n)$ bit-steps (§16.3); the same table, carrying a second payload `agg`$[v][j]$, lifts *any associative aggregate* – max, sum, xor, a count – along a jump, which is the merged old-Chapter-12 material of §16.4. Machine 3 is what contests actually use: it upgrades *every* earlier tool (the k-th ancestor, distance-through-the-hinge, depth buckets for CF 208E's cousins) for the price of $O(n \log n)$ memory.

You will learn to

- state the ancestor chain property (Proposition 16.1.3) and the equalise-then-step walk, and read a tree's LCA table by hand; **L2**
- build the $2n-1$ tour, `first[]`, and prove that LCA is a range-minimum of depths over the tour span (Theorem 16.2.2); **L3**
- answer that RMQ with a sparse table in $O(1)$ – including the two-overlapping-blocks trick and the idempotence caveat that makes it legal for min but never for sums; **L4**
- build `up`$[v][j]$, climb k in $O(\log n)$, and write the descending-pairs LCA that *returns* `par`$[u]$ – not u; **L3–L4**
- carry an aggregate on each jump (Theorem 16.4.1), split a path query at the hinge, descend to the first ancestor past a threshold, and walk to the k-th node *on a route*; **L4–L5**
- turn the hinge into measurements – distances, on-path tests, per-depth buckets – and combine everything on the Summit CF 1062E. **L5**

Prerequisites. Chapter 12 (Chapter 12) for the rooted-view DFS and the depth/parent measures – the setup DFS here is Code 12.1.1 verbatim. Chapter 5 (Chapter 5) for the

entry-only tin / tout flattening we reuse as an ancestry test (but *not* for the tour: the $2n-1$ LCA tour is a different object, contrasted in 16.2). Chapter 13 (Chapter 13) for the subtree fold whose pair-harvest this chapter makes queryable, and for the distance identity $\text{dist}(s,u)+\text{dist}(u,v)=\text{dist}(s,v)$ iff u is on the route. Chapter 19 (Chapter 19) is *not* needed: the sparse table is built from scratch here.

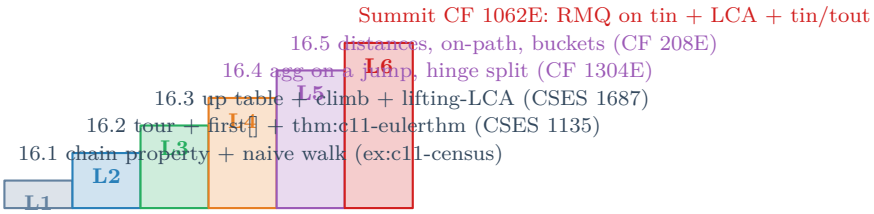
How to Read This Chapter (three paths)

Level badges L1–L6: L2 is computing an LCA by the naive walk on a drawn tree; L3 is either machine built correctly under time pressure (a tour table, or an `up` table with the sentinel discipline); L4 needs *two* ideas composed (sparse table over the tour, or a hinge-split aggregate query); L5 is a query that reduces to LCA through a translation layer you must invent yourself (route parity for CF 1304E, depth buckets for CF 208E, the removal analysis of the Summit); L6 is knowing which machine – or which decomposition from Ch.19–24 – the constraint table is actually asking for.

Walk-first path (L2–L3, "I have never seen an LCA"). Read §16.1 end to end: ancestors, the chain property, the hand-computed census *Example* 16.1.4, and the equalise-then-step walk. Then §16.3: the `up` table and CSES 1687's k-th ancestor are the smallest complete win in the chapter. Only then return to §16.2 and see the tour as a *reduction*.

Lifting-first path (L3–L4, "I need the workhorse"). §16.1 for notation, then §16.3 twice: once for the table, once for the descending-pairs LCA and its $u=v$ shortcut. Then §16.4 – the second table $\texttt{agg}[v][j]$, the two accumulators, the hinge split – and CF 1304E. This path covers what the majority of contest problems actually use.

RMQ-first path (L4, "I already know sparse tables"). Go straight to §16.2: the $2n-1$ tour, `first[]`, Theorem 16.2.2, and the two-overlapping-blocks query; CSES 1135 is the case study. Read the caveat in §16.4 before reusing the trick anywhere non-idempotent. Backward ropes: Chapter 9's RMQ → LCA-on-Cartesian-tree direction (Chapter 9) is the mirror image of what you just did.



Chapter 16 on the six-step ladder: the census and the honest walk sit at L2; each machine used standalone is L3; composing a sparse table over the tour, or an aggregate over a jump, is L4; the four solved problems climb L3–L5; the Summit CF 1062E is step 5 read from step 6. Walk-first path = steps 2–3; full contest path runs to 6.

The Running Example: One Ten-Node Tree, Three Machines

One tree does all the work – the same T Chapters 12–15 used, now wearing ancestor-query luggage. Nodes $1,\dots,10$, rooted at 1; edges

$$1{-}2,\ 1{-}3,\ 2{-}4,\ 2{-}5,\ 3{-}6,\ 3{-}7,\ 5{-}8,\ 7{-}9,\ 9{-}10.$$

Child sets in that view: $1\to\{2,3\}$, $2\to\{4,5\}$, $3\to\{6,7\}$, $5\to\{8\}$, $7\to\{9\}$, $9\to\{10\}$; leaves $\{4,6,8,10\}$; height 4, reached on the spine $1,2,5,8$ and the spine $1,3,7,9,10$. The tree is

deliberately *thin*: depth 4 on $n = 10$ means the naive walk is already slow enough to feel, and a chain-shaped worst case is only one re-root away.

Figure 16.0.1 stamps T with everything the machines consume. Below each node: `d` = depth (root at 0), `p` = parent, with the sentinel `par`$[1] = 0$ made visible; above-left/above-right: tin / tout under the Ch. 5 entry-only convention with tout = last stamp inside the subtree. These exact numbers are re-used, unchanged, in every section: the $2n - 1$ tour of 16.2 is the DFS *walk* over this drawing, the `up` table of 16.3 is built from the `p` row, and every distance in 16.5 is read off the `d` row.

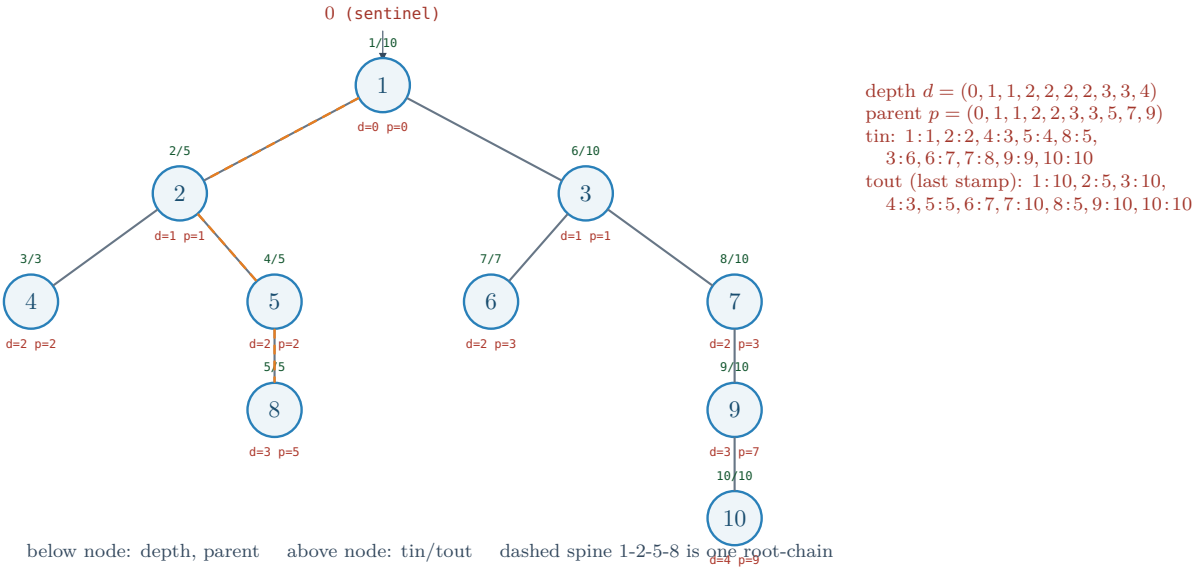


Figure 16.0.1: The running tree T with the ancestor-query overlay. Below each node: depth d (root at 0) and parent p; above each node: tin / tout from the entry-only DFS of Chapter 5 (Chapter 5), children visited in ascending label order, tout = the last stamp inside the subtree. The orange dashed spine $1-2-5-8$ is one root-chain, so ancestors$(8) = \{8, 5, 2, 1\}$. Sanity checks used across the chapter: node 10 is the unique depth-4 node; $u \preceq v$ iff $\text{tin}(u) \le \text{tin}(v) \le \text{tout}(u)$ — e.g. $7 \preceq 10$ as $8 \le 10 \le 10$, while $9 \preceq 6$ fails as $9 > 7$. Every tour, table and climb in 16.2–16.5 re-uses exactly these stamps.

Four angles, four hand-computed overlays – each lives in the section named on its row and never changes:

Angle on T	What comes out	Traced in
ancestor census	$\text{lca}(8, 10) = 1$, $\text{lca}(4, 5) = 2$, $\text{lca}(6, 7) = 3$, degenerate $\text{lca}(9, 10) = 9$	§16.1
$2n - 1$ tour + RMQ	tour $1, 2, 4, 2, 5, 8, 5, 2, 1, 3, 6, 3, 7, 9, 10, 9, 7, 3, 1$ (length 19), `first`$[8] = 5$, `first`$[10] = 14$, span min depth $0 \to$ LCA 1	§16.2
`up`$[v][j]$ table	`up`$[10][2] = 1$, `up`$[8][1] = 2$; climb for $\text{lca}(8, 10)$ equalises $10 \to 9$, jumps $2 \to 1$ region, returns `par`$[2] = 1$	§16.3
distances + buckets	$\text{dist}(8, 10) = 7$, $\text{dist}(4, 8) = 3$; depth-2 bucket $\{3, 4, 7, 8\}$ answers "cousins at distance 2" counts	§16.5

Three Machines, One Question (the cost chart)

All three answer the *same* question – $\text{lca}(u, v)$ on a fixed rooted tree – and the right one is chosen by the shape of the input, not by taste. Write h for the height and n for the node count.

Machine	Setup	Per query	Memory	When it wins
naive walk (16.1)	$O(1)$ (just `par`, `depth`)	$O(h)$	$O(n)$	one-off questions, shallow trees, proofs-by-baseline
Euler + RMQ (16.2)	$O(n \log n)$	$O(1)$	$O(n \log n)$	pure LCA/distance queries only, nothing else needed from the table
binary lifting (16.3)	$O(n \log n)$	$O(\log n)$	$O(n \log n)$	the default: LCA + k-th ancestor + aggregates + threshold descents

One question is worth three machines

The RMQ machine is *faster* per query ($O(1)$) but the lifting machine is *more useful*: the same `up` table also answers the k-th ancestor, lifts max/sum/xor payloads, and descends to the first ancestor past a threshold – queries that have no RMQ translation at all. When the constraints fit both, contests pick lifting; the tour is kept because the reduction LCA→RMQ is one of the two or three great problem-transformations in this book (Chapter 9 owns the reverse one).

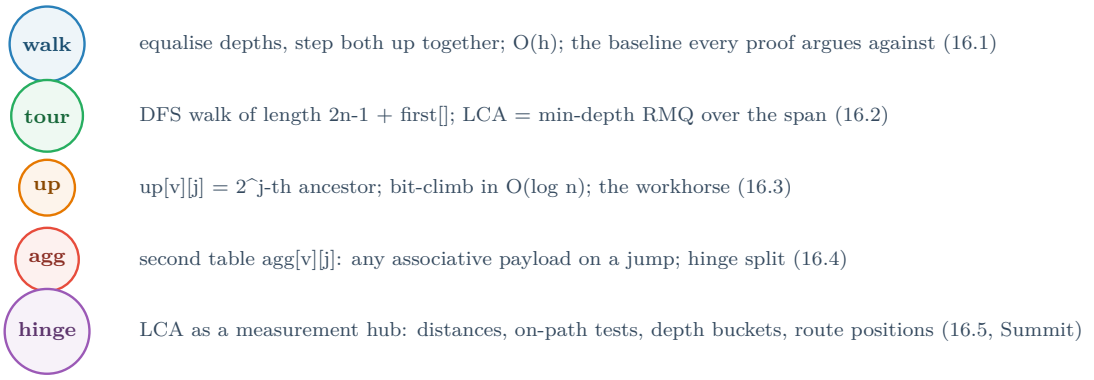


Figure 16.0.2: The chapter's five-state taxonomy: rows 1–3 are the three machines for the question "lca(u, v) =?", row 4 is the same lifting machine carrying a payload (the merged old-Chapter-12 topic), row 5 is what the hinge *measures*. Every solved problem in the chapter sits on exactly one row: CSES 1135 on row 2, CSES 1687 on row 3, CF 1304E on row 4, CF 208E on row 5, the Summit CF 1062E across rows 2 and 5.

Who Owns What (so nothing is learned twice)

Idea	Owner	This chapter's share
entry-only tin / tout flattening, subtree-as-interval	Ch. 5 (Chapter 5)	reused as the ancestry test *Proposition* 16.5.4; never re-derived
rooted DFS, depth/height/size measures	Ch. 12 (Chapter 12)	Code 12.1.1 reused verbatim as the setup pass
subtree fold, pair-at-LCA harvest	Ch. 13 (Chapter 13)	the harvest becomes *queryable*; Example 16.1.4 is its worked basis
RMQ via Cartesian tree	Ch. 9 (Chapter 9)	the reverse reduction; this chapter owns static sparse tables
LCA + lifting + lifted aggregates	**Ch. 16 (this)**	three machines, the hinge, and the merged old-Ch. 12 payload material
dynamic path updates / subtree updates	Ch. 20 (Chapter 20), Ch. 22 (Chapter 22)	*not* here – the moment values change between queries, hand off (HLD)
distance *distributions* over all pairs	Ch. 24 (Chapter 24)	counting over every pair is centroid territory; per-pair queries stay here

Pattern card – "the answer is a distance, route or k-th ancestor between named pairs"

Recognition keywords. a *static* tree plus Q questions of the form: how far apart are u and v; is w on the path u–v; who is the lowest common boss of employees x, y; the k-th ancestor / k-th node on the route; the deepest ancestor whose value still passes a test; a count inside the subtree at depth d from v.

✓ When to use this

- per-pair distance or LCA queries on a fixed tree – any machine from this chapter; §16.2 or §16.3;
- "k-th ancestor of v", or "first node above v with property P" – §16.3 and the bit-descent of §16.4;
- an extremum / sum / xor *along a path*, values frozen – `agg`$[v][j]$, §16.4;
- "cousins at distance j from v" and similar depth-indexed subtree counts – hinge + buckets, §16.5 (CF 208E);
- walk $a \to b$, detour through a fresh edge, parity/wiggle room – route length via distances, §16.4 (CF 1304E).

✗ When NOT to use this

- edge or vertex values *change* between queries – that is segment trees + heavy-light, Chapters 20/22 (Chapter 20, Chapter 22); nothing here survives an update;
- the answer must be reported for every *root* – rerooting, Chapter 15 (Chapter 15);
- counting pairs over the whole tree (a distance *distribution*) – centroid decomposition, Chapter 24 (Chapter 24);
- a single BFS-solvable distance on an unweighted graph that happens to be a tree *forest* with per-component roots – check the input is really rooted first (the super-root trick,

16.3).

One-line test: *do the queries name two nodes and ask what lies between them (or above one of them) on the unique route, while the tree itself never changes? Then this chapter's machine is the answer structure.*

Reading the Constraints (which machine the limits want)

Constraint signature	What it rules in/out	Tool
$n, Q \le 10^5$, tree is a chain in the worst case	$O(h)$ walk is $O(n)$ per query – TLE; any $O(\log)$ machine fits	§16.3, default
$n \le 10^5$, $Q \le 5 \cdot 10^4$, pure LCA/distance, nothing else	$O(1)$ RMQ query pays off; one sparse table, no second table	§16.2, CSES 1135
queries also ask k-th ancestor / path extrema / threshold climbs	RMQ cannot express them; `up` + `agg` can	§16.3, §16.4
"depth j subtree count" per query	buckets keyed by depth + tin-range inside the subtree	§16.5, CF 208E
$n, Q \le 10^5$ but h small in test data	the walk *passes* – fragile; contests have anti-h tests	walk only as a checker
values updated between queries	everything here is static; the table goes stale	Ch. 20/22 hand-off

Notation Used Only Here

Root the tree at 1 unless stated (every contest input in this chapter either says so or is a forest given a super-root 0; see 16.3's sentinel discipline). `par`$[v]$ and `depth`(v) come from one measuring DFS (Code 12.1.1); `par`[root] $= 0$ and vertex 0 is a **sentinel**, never a real answer – every table row for 0 is 0 and every loop must stop before returning it. ancestors(v) is the root-chain $v, \texttt{par}[v], \ldots, 1$, and $u \preceq v$ ("u is an ancestor of v", including $u = v$) is the **ancestor order** whose chains Proposition 16.1.3 exploits. The $2n-1$ tour is `tour[]` with `first`$[v]$ = index of v's first appearance and `dep`$[\,]$ the depth alongside it (16.2's convention: indices are 0-based). The sparse table is `st`$[j][i]$ = index of the minimum-depth entry among `tour`$[i..i+2^j-1]$. `up`$[v][j]$ = the 2^j-th ancestor of v, LOG is the smallest integer with $2^{LOG} > n$ ($LOG = 4$ suffices for T); `agg`$[v][j]$ is the aggregate over the 2^j vertices *starting at v and going up, excluding the landing vertex* – the convention pinned in 16.4 and never varied. dist(u, v) is the number of *edges* on the route.

Every section of this chapter is a machine run on the *same* tree on the *same* stamps: if a table here and a trace there disagree, this opener is wrong and the section is right – fix it here, never the other way.

16.1 Ancestors, the LCA, and the $O(h)$ Walk

What you already know – 4 questions

1. From Chapter 12 (Ch. 12): the rooted-view DFS Code 12.1.1 stamps `depth`(v), `par`$[v]$ and `size`(v) in one pass. *(Which array do you need before any LCA question can even be posed?)*
2. From Chapter 13 (Ch. 13): the subtree fold harvests pair-values at the LCA (Proposition 13.3.2). *(What was the "top node" ℓ in that construction?)*
3. From Chapter 5 (Ch. 5): the entry-only tin / tout convention identifies subtrees as intervals. *(Can you test "u is an ancestor of v" with just* tin*?)*
4. From Figure 16.0.1: the shared running tree T rooted at 1, with the depth/parent overlay already printed. *(Which two root-chains meet at node 3?)*

In This Section

Three lines of definition and one picture carry the whole chapter: root the tree, and every node drags a chain upward to the root; two chains climb together after they meet and never separate again. "Where do they meet?" is the LCA question. This section proves the meeting is well defined (common ancestors form a chain – Proposition 16.1.3), computes a small census of LCAs on the running tree T *by hand, from the definition* (Example 16.1.4) – because every fast machine in §16.2–§16.3 is just this picture compressed – and then writes the naive walk that is already correct and sometimes already fast enough. We also plant the root-dependence warning here, where it is cheapest to understand.

After this section you can

- state lca from the chain-intersection definition and compute any pair's LCA on paper in $O(h)$, including the ancestor-of-each-other shortcut case; **L2**
- prove common ancestors form a chain (so "lowest" is unique) and use the characterisation $\mathrm{lca}(u,v) = u \iff u \preceq v$; **L2–L3**
- write the equalise-then-step-together walk and explain why its structure is *the* template both fast machines will keep; **L3**
- give the worst-case cost honestly ($O(h)$ per query: fine on a star, fatal on a path) and name when it matters (Q large). **L2**

16.1.1 Chains Upward

Definition 16.1.1 (Ancestor, descendant, parent chain)

Fix a root r. The **ancestor chain** of v is $\mathrm{ancestors}(v) = (v = a_0,\ a_1,\ \ldots,\ a_{\mathrm{depth}(v)} = r)$, the nodes of the unique route $v \to r$ in order; a_j is the **j-th ancestor** of v. We write $u \preceq v$ when $u \in \mathrm{ancestors}(v)$ (u is an **ancestor** of v, v a **descendant** of u), counting $v \preceq v$ itself; $\mathtt{par}(v) = a_1$.

Read it as: the route, not a relation to prove. $\preceq$ is exactly "appears on the way up", and the subtree-as-interval language of Ch. 5 translates: $u \preceq v$ iff $\mathrm{tin}(u) \le \mathrm{tin}(v) \le \mathrm{tout}(u)$. The self-convention ($v \preceq v$) is what makes the next definition not lie about pairs like $(9, 10)$.

Definition 16.1.2 (Lowest common ancestor)

For nodes u, v: $\mathrm{lca}(u,v)$ is the deepest node of $\mathrm{ancestors}(u) \cap \mathrm{ancestors}(v)$ – the common ancestor with maximum depth.

Proposition 16.1.3 (Common ancestors form a chain)

ancestors$(u) \cap$ ancestors$(v) \neq \emptyset$ (both contain r), and it equals ancestors(ℓ) where ℓ is its deepest element. In particular the LCA is unique, w is a common ancestor of u, v iff $w \preceq \mathrm{lca}(u, v)$, and $\mathrm{lca}(u, v) = u \iff u \preceq v$.

Proof

ancestors(u) is a chain by definition (one route, no branches). Take any two common ancestors w_1, w_2 of u and v: both lie on the single route $u \to r$, and nodes of one chain are linearly ordered by depth – so $w_1 \preceq w_2$ or $w_2 \preceq w_1$. A deepest element ℓ exists (finite chain), and every common ancestor is an ancestor of ℓ: *common* = ancestors(ℓ). The final equivalence: if $u \preceq v$, then ancestors$(u) \subseteq$ ancestors(v), so the deepest common node is u; conversely lca $= u$ forces u into ancestors(v).

■

Three vocabularies for one node

ℓ is the *deepest common ancestor*; the *parting point* where the chains from u and v merge; the *top node / harvest point* of Ch. 13's pair decomposition (Proposition 13.3.2) *if* $u \neq v$ and neither is an ancestor of the other – the degenerate rows of *Example* 16.1.4's census ride on ℓ itself, one arm of length 0.

16.1.2 The Parting-Point Picture on T

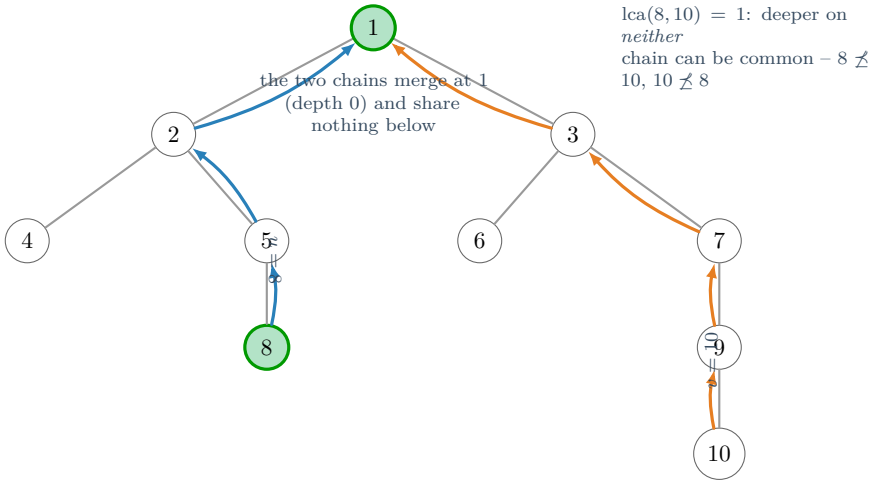


Figure 16.1.1: The parting-point picture on T: $\ell = 1$ is where the chains $8 \to 5 \to 2 \to 1$ and $10 \to 9 \to 7 \to 3 \to 1$ first touch; above ℓ they coincide (both empty), below it they never meet again (else two routes would join twice – no cycles in a tree). The hinge 1 is the root.

16.1.3 Computing by Hand First (the census)

Example 16.1.4 (An LCA census straight from the definition)

Intersect chains, take the deepest – no machine yet, just Figure 16.0.1. On T rooted at 1:

u	v	ancestors$(u) \cap$ ancestors(v)	lca(u, v)
8	10	$\{1\}$	1
4	5	$\{2, 1\}$	2
6	7	$\{3, 1\}$	3
8	9	$\{1\}$	1
9	10	$\{9, 7, 3, 1\}$ and $9 \preceq 10$	**9**
5	5	ancestors$(5) = \{5, 2, 1\}$	**5**

The last two rows are the shortcut cases Proposition 16.1.3 licenses: when one node is an ancestor of the other the LCA *is* that node (the chains did not part – one contains the other), and $\text{lca}(u, u) = u$. A machine that special-cases these two lines separately is a machine that did not read the proposition.

Reading *Example* 16.1.4 in Ch. 13

Chapter 13's distance-DP section (`04_distance_dp.tex`) references this exact example by label (*Example* 16.1.4) when proving that $\text{dist}(s, u) + \text{dist}(u, v) = \text{dist}(s, v)$ iff u is on the route s-v. The census rows $(4, 5)$ and $(6, 7)$ serve as witnesses there.

The LCA lives on the *rooted* tree – root changes answers

Re-root T at 6: the chains change, and $\text{lca}_6(3, 6) = 6$ (since $6 \preceq_6 3$ now), while $\text{lca}_1(3, 6) = 3$. Distances, by contrast, never moved – the $O(1)$-query machinery of §16.5 routes around this on purpose. Fix one root for a problem, use it everywhere, and when a statement is phrased unrooted ("the junction of the three paths", CSES 1136-style), any fixed root is *legal*: the answers that matter come out root-independent.

16.1.4 The Naive Walk: Equalise, Then Step Together

Proposition 16.1.5 (The $O(h)$ walk)

The following returns $\text{lca}(u, v)$: (i) while $\text{depth}(u) > \text{depth}(v)$: $u \leftarrow \texttt{par}[u]$; (ii) symmetrically for v; (iii) while $u \neq v$: step *both* up. Cost is at most $\text{depth}(u) + \text{depth}(v) + 1$ parent moves.

Proof

Phase (i)–(ii) cannot change the answer: climbing past the current LCA is impossible while depths exceed the other's (the shallower node's chain *contains* the LCA of the pair – Proposition 16.1.3 applied to the shortened chain), and after equalisation both nodes sit at the same depth on their chains. In phase (iii) equal depths mean the two walk in lockstep and can only *land* on a common node simultaneously – the first such landing is the deepest common ancestor: any deeper one would have been hit one step earlier. Termination: both reach r at latest.

■

Code 16.1.1: The walk in five lines – the template 16.2 and 16.3 only change how far one "step" may jump

```
1 int lca_naive(int u, int v) {
```

```
    while (dep[u] > dep[v]) u = par[u];          // phase i:  raise the deeper side
    while (dep[v] > dep[u]) v = par[v];          // phase ii: until depths match
    while (u != v) { u = par[u]; v = par[v]; }   // phase iii: lockstep until first
    ↪ meet
    return u;
}
```

Believe it because: phase (iii) is *Example* 16.1.4 in motion – the two chains *merge at ℓ and coincide above it*, so walking equal depths forward, the first agreement *must* be ℓ; they can never cross without agreeing. Every fast method below keeps this two-phase shape and merely buys a faster "step".

Walk on T: lca(8, 10)

depth(8) = 3, depth(10) = 4. Phase (ii): $v \leftarrow$ par[10] = 9, now depth(9) = 3 = depth(8). Phase (iii): step both:

step	u	v
0	8	9
1	5	7
2	2	3
3	1	1 ✓

First agreement at depth 0: $\text{lca}(8, 10) = 1$. Matches the census. Cost: $1 + 3 = 4$ parent moves $= \text{depth}(8) + \text{depth}(10) - 2 \cdot \text{depth}(1) + 1 = 3 + 4 - 0 + 1 = 8$. Wait: the +1 is the final loop-iteration check, so at most $3 + 4 + 1 = 8$ pointer assignments. For a pair on the same side $(4, 5)$: phase (i) moves 4 → nothing (same depth), phase (iii) steps $4 \to 2$, $5 \to 2$: one move. $h = 4$, so even the worst case is small on T.

Tree shape	**Worst-case walk**	**Verdict for Q queries**
star / bushy ($h = O(1)$)	$O(1)$ per query	naive *wins* – no preprocessing needed
balanced binary ($h = O(\log n)$)	$O(\log n)$	naive is already contest-fast
path / chain ($h = n$)	$O(n)$	$Q \geq 10^3 \Rightarrow$ TLE: bring §16.2 or §16.3

So the honest first move in an LCA problem: *bound the height.* If the statement hands you a shallow tree (or a random one, where Ch. 8's BST analysis says $h = O(\log n)$ in expectation), five lines of walk beat forty lines of table. The rest of the chapter is for adversarial paths with large Q.

16.1.5 Variations and Twists

- **Multiple LCAs at once.** Given k nodes, $\text{lca}(v_1, v_2, \ldots, v_k)$ is the LCA of the min-depth and max-depth nodes in any tin-sorted order of the set. Proof sketch: the intersection of k chains is a chain (each intersection of two chains is a chain), so the deepest common element exists and equals the pairwise LCA of the two extremes. The Summit (Chapter 16 99 station 12) uses this. **L4**
- **Lowest common ancestor with weights.** If edges carry a length, the *hinge* is the same (graph structure unchanged), but the *distance* is $\text{dist}(r, u) + \text{dist}(r, v) - 2\,\text{dist}(r, \ell)$. §16.5 makes this the definition. **L3**
- **Ancestor test in $O(1)$.** $u \preceq v$ iff $\text{tin}(u) \leq \text{tin}(v) \leq \text{tout}(u)$ – this is Ch. 5's interval test, restated for clarity in Proposition 16.5.4 (§16.5). **L2**
- **Binary-tree specialisation.** On a *binary* tree with parent pointers, the walk is the classic LC 236 post-order solution (used in Ch. 1). **L2**
- **Forest → super-root.** If the input is a forest with Q queries spanning components, add

node 0 and connect all component roots to it; the walk and every table below extend verbatim. Queries within different components return 0. L3

16.1.6 Common Pitfalls

Trap 1: Forgetting the self-convention

The LCA of a node with itself is that node ($\mathrm{lca}(v, v) = v$), not its parent. If your walk starts by checking $u \neq v$ and skips the loop, you never produce this row. Test on *Example* 16.1.4's row 6.

Trap 2: Treating $\preceq$ as strict

Some code writes 'if (isAnc(u,v)) return u;' using a *strict* ancestor test that excludes $u = v$. Then $\mathrm{lca}(v, v)$ falls through to the loop and the loop never fires because $u = v$ already: either you return the wrong node or loop forever. Pin the convention at the first line: $u \preceq v$ includes $u = v$.

Trap 3: Assuming the root is fixed

On the tree T, $\mathrm{lca}_1(8, 10) = 1$ but re-rooting at node 3 changes it: the chain $8 \to 5 \to 2 \to 1 \to 3$ and $10 \to 9 \to 7 \to 3$ now meet at 3 (depth 0 in the new view). Contest problems always say "rooted at 1" or give a parent array. If they don't, *read the sample* – the hinge answers decode the implicit root.

16.1.7 Practice Corner

Q16.1.1 Lowest Common Ancestor of a Binary Tree II – LeetCode 1644 LC Medium L2

Hint: premium statement; solve from this hint: it is LC 236 (Ch. 1's mixed set – the pure *definition* exercise: no tables, just the recursion) with one twist: p and q are *not guaranteed to exist*. Keep Ch. 1's post-order shape – return the node if it is p or q, else the side that found something, else the merge of both – but count arrivals: report an LCA only if the traversal found *both* targets; a lone sighting is not an ancestor of a missing node.

Back-cites. LC 236 (Ch. 1 mixed set M1.1, Chapter 1) – the binary-tree LCA is the base case of *Example* 16.1.4's structure. Ch. 5 (*Chapter* 5) – the tin / tout interval convention re-used in Proposition 16.5.4. Ch. 13 (Proposition 13.3.2) – the "harvest at the hinge" identity that *Example* 16.1.4 makes concrete.

Section 16.1 in 10 seconds

Root the tree. Every node drags a *chain* up to the root. Two chains meet and never part again: the meeting point is $\mathrm{lca}(u, v)$. Common ancestors form a chain (Proposition 16.1.3) so the LCA is unique. Shortcut: $\mathrm{lca}(u, v) = u$ iff $u \preceq v$ (including $u = v$). The naive walk equalises depths and steps in lockstep; cost $O(h)$, fine if $h = O(\log n)$ or Q small, fatal on a path with $Q = 10^5$.

Self-test (answer from memory, tick the box):

- ☐ State lca in one sentence.
- ☐ What does $\preceq$ include that a strict "below" does not?
- ☐ Give the census row $(8, 10)$ and explain why lca $= 1$.
- ☐ The walk's three phases? Why does lockstep find the *first* agreement?
- ☐ When is the naive walk fast enough? Give h and Q bounds.

16.2 LCA by Euler Tour and Range Minimum

Three questions from 16.1 and earlier chapters

1. From *Example* 16.1.4: $\mathrm{lca}(8, 10) = 1$ because the chains $8 \to 5 \to 2 \to 1$ and $10 \to 9 \to 7 \to 3 \to 1$ share only the root. *(If you had a sequence recording every DFS visit, where would the shallowest entry between the first sightings of 8 and 10 sit?)*
2. From Ch. 12 (*Code* 12.1.1): the measuring DFS visits each edge twice (down and up) and produces $2n - 1$ node recordings in that walk. *(That is exactly what the tour below records.)*
3. From Ch. 9 (*Chapter* 9): the *Cartesian tree* solves RMQ-on-array by building a tree; here we run the reduction in the *opposite* direction – turn LCA-on-tree into RMQ-on-array. *(Same algorithm, different port. Which array do we RMQ?)*

In This Section

The walk of §16.1 is correct but $O(h)$. This section converts LCA into a *range-minimum query* on an array derived from the DFS walk: build the $2n-1$ tour, mark the first occurrence of each node, and the LCA is the shallowest tour-entry between the two firsts – one sentence theorem (Theorem 16.2.2), then a $O(n \log n)$ sparse table answers *any* such query in $O(1)$ per lookup. The result: Q distance/LCA queries in $O(n \log n + Q)$ total. Static trees only – the moment a value changes between queries, hand off to segment trees (Ch. 19/20).

Prerequisites. §16.1 (chains, $\preceq$, the census), Ch. 12's DFS (*Code* 12.1.1) for the walk discipline and the depth stamp. Ch. 9's RMQ section is *optional* – we rebuild the sparse table here from scratch, but knowing the Cartesian direction makes the contrast vivid.

After this section you can

- build the $2n-1$ tour, `first[]`, and `dep[]` by hand on T; **L3**
- state and prove the Euler-RMQ reduction (Theorem 16.2.2), and read an LCA off a depth span; **L3**
- construct a sparse table in $O(n \log n)$ and query it in $O(1)$ via two *overlapping* blocks; **L4**
- explain why overlapping blocks are legal for idempotent operations (min, max, gcd) but *never* for sum, xor, or count; **L4**
- solve a multi-query distance problem (CSES 1135) using this machine end-to-end. **L3–L4**

16.2.1 The $2n - 1$ Tour – What It Is and Is Not

Definition 16.2.1 (Euler tour of a rooted tree)

Run the root's DFS; append the current node to `tour` *when you enter it and every time a child's call returns to it.* For n nodes this records n entries + $n - 1$ returns = $2n - 1$ slots. Alongside each slot store the *depth* of the node at that slot. Define `first`$[v]$ = the *earliest* index of v in `tour` (0-based).

This tour is NOT Ch. 5's flattening tour

Chapter 5 (*Chapter* 5) records n entries *only* (enter a node; never re-log on return) and produces the tin / tout interval view that makes subtree queries array-range queries. The $2n - 1$ LCA tour here is a *different object*: it logs *every* visit (enter *and* each return), and its whole purpose is to create an array whose range-minimum of depths answers LCA. The two

tours coexist freely in the same problem – tin / tout for ancestry tests and subtree ranges, the $2n-1$ tour for LCA – but do not confuse them.

16.2.2 Hand-Computing the Tour on T

The DFS visits children in ascending label order. Walk T (root 1, edges as in Figure 16.0.1) and record:

Tour construction on T (0-based index)

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
`tour`$[i]$	1	2	4	2	5	8	5	2	1	3	6	3	7	9	10	9	7	3	1
`dep`$[i]$	0	1	2	1	2	3	2	1	0	1	2	1	2	3	4	3	2	1	0

`first`$[\,]$: $1\to0$, $2\to1$, $3\to9$, $4\to2$, $5\to4$, $6\to10$, $7\to12$, $8\to5$, $9\to13$, $10\to14$.

Verify: $\mathrm{lca}(8,10)$: span is $[$`first`$[8],$ `first`$[10]] = [5,14]$. The minimum depth in `dep`$[5..14] = (3,2,1,0,1,2,1,2,3,4)$ is 0 at $i=8$, and `tour`$[8]=1$. Census says $\mathrm{lca}(8,10)=1$. ✓

$\mathrm{lca}(4,8)$: span $[2,5]$, `dep` $=(2,1,2,3)$, min 1 at $i=3$, `tour`$[3]=2$. Census: $\mathrm{lca}(4,5)=2$; since $\mathrm{lca}(4,8)=\mathrm{lca}(4,5)=2$ (8 is below 5). ✓

$\mathrm{lca}(9,10)$: span $[13,14]$, `dep` $=(3,4)$, min 3 at $i=13$, `tour`$[13]=9$. Census: $9 \preceq 10 \Rightarrow$ lca $=9$. ✓

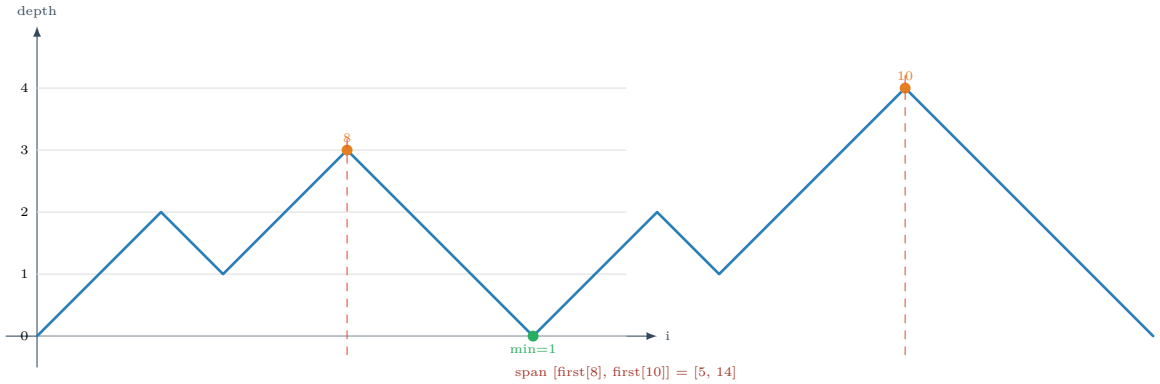


Figure 16.2.1: The $2n-1$ depth profile of T. The LCA of 8 and 10 is the node at the shallowest point between their first occurrences: here the global minimum 0 at $i=8$, which is node 1.

16.2.3 The Reduction Theorem

Theorem 16.2.2 (Euler-tour LCA = RMQ on depth)

For any two nodes u, v in a rooted tree with the $2n-1$ tour `tour`, depth array `dep`, and first-occurrence `first`:

$$\mathrm{lca}(u,v) \;=\; \texttt{tour}\Big[\arg\min_{\texttt{first}[u]\le i\le \texttt{first}[v]} \texttt{dep}[i]\Big]$$

(assuming `first`$[u] \le$ `first`$[v]$; swap otherwise).

Proof

WLOG $f_u = \texttt{first}[u] \le f_v = \texttt{first}[v]$. The tour records the DFS walk. Between f_u and f_v the walk must leave u's subtree (to reach v) and can only do so by climbing to some ancestor of u. Let w be the shallowest tour entry in $[f_u, f_v]$. Since the walk starts at u (of depth $\text{depth}(u)$) and ends at v, the path $\texttt{tour}[f_u..f_v]$ descends into the subtree(s) and always returns through the same ancestors. w is on the walk from u up toward the root (so $w \preceq u$) and also on the walk from the root down toward v (so $w \preceq v$); hence w is a common ancestor. If some common ancestor a were strictly deeper than w, both u and v would be in a's subtree, so the DFS would visit both without ever surfacing above a – contradicting that w is the minimum. Hence $w = \ell$ (the deepest common ancestor).

■

16.2.4 The Sparse Table – Building and Querying

The theorem reduces LCA to an *RMQ*: given an array $A[0..N-1]$ (here $N = 2n-1$), answer $\arg\min$ over any $[L, R]$ in $O(1)$. The standard tool is the **sparse table**.

Definition 16.2.3 (Sparse table)

Let $\texttt{st}[j][i]$ store the *index of the minimum* A-value among $A[i \,..\, i+2^j-1]$ (a **sparse-table block** of length 2^j starting at i). Write $a = \texttt{st}[j-1][i]$ and $b = \texttt{st}[j-1][i+2^{j-1}]$ (the two halves). Recurrence:

$$\texttt{st}[0][i] = i; \qquad \texttt{st}[j][i] = \begin{cases} a & \text{if } A[a] \le A[b] \\ b & \text{otherwise.} \end{cases}$$

Build: $O(N \log N)$, space $O(N \log N)$.

Query for $[L, R]$: let $j = \lfloor \log_2(R - L + 1) \rfloor$. Two *overlapping* blocks of length 2^j cover $[L, R]$:

$$\min = \begin{cases} \texttt{st}[j][L] & \text{if } A[\texttt{st}[j][L]] \le A[\texttt{st}[j][R-2^j+1]] \\ \texttt{st}[j][R-2^j+1] & \text{otherwise.} \end{cases}$$

Correct because min is *idempotent*: duplicating elements in the overlap does not change the minimum. **Warning:** this trick does *not* work for sum, xor, or count – their value depends on how many times an element appears. Only min / max / gcd /and/or are safe for overlapping blocks.

Code 16.2.1: and dep[] on a rooted tree (children ascending)]Build tour, first[] and dep[] on a rooted tree (children ascending)

```
void build_euler(int v, int p, int d) {
    first[v] = (int)tour.size();   // record FIRST occurrence
    tour.push_back(v);
    dep.push_back(d);
    for (int c : adj[v]) {
        if (c == p) continue;
        build_euler(c, v, d + 1);
        tour.push_back(v);         // record RETURN to v
        dep.push_back(d);
    }
}
```

Code 16.2.2: Build and query a sparse table on the tour depth array

```cpp
// build: st[j][i] = index of min depth in dep[i .. i+2^j-1]
int lg[N+1];  // precompute lg[1..N]
void build_sparse() {
    int N = (int)tour.size();      // N = 2n-1
    for (int i = 0; i < N; i++) st[0][i] = i;
    for (int j = 1; (1 << j) <= N; j++)
        for (int i = 0; i + (1 << j) <= N; i++) {
            int a = st[j-1][i], b = st[j-1][i + (1 << (j-1))];
            st[j][i] = (dep[a] <= dep[b]) ? a : b;
        }
}
// query: O(1) index of min-depth in [L, R]
int rmq(int L, int R) {
    int j = lg[R - L + 1];
    int a = st[j][L], b = st[j][R - (1 << j) + 1];
    return (dep[a] <= dep[b]) ? a : b;
}
int lca(int u, int v) {
    int L = min(first[u], first[v]);
    int R = max(first[u], first[v]);
    return tour[rmq(L, R)];
}
```

Sparse-table query on T: lca(8, 10)

$N = 2n - 1 = 19$, span $[L, R] = [5, 14]$, length 10. $j = \lfloor \log_2 10 \rfloor = 3$, block size $2^3 = 8$.

Left block: `st`[3][5] covers `dep`[5..12] $= (3, 2, 1, 0, 1, 2, 1, 2)$, min at $i = 8$ (depth 0). Right block: `st`[3][7] covers `dep`[7..14] $= (1, 0, 1, 2, 1, 2, 3, 4)$, min at $i = 8$ (depth 0). Winner: index 8, `tour`[8] $= 1$. lca$(8, 10) = 1$. ✓

lca$(4, 8)$: span $[2, 5]$, length 4, $j = 2$, block size 4. Left `st`[2][2] covers `dep`[2..5] $= (2, 1, 2, 3)$, min at $i = 3$ (depth 1). Right `st`[2][2] = same block. `tour`[3] $= 2$. ✓

Euler + RMQ method

Setup: $O(n)$ tour + $O(n \log n)$ sparse table on $2n-1$ entries. **Query:** $O(1)$ per LCA. **Memory:** $O(n \log n)$ for the table. **Limitation:** static tree only; answers LCA/distance; cannot produce a k-th ancestor or a path aggregate.

16.2.5 Variations and Twists

- **Farmer's linearithmic alternative.** The $2n-1$ tour has `dep`$[i+1]$ − `dep`$[i] \in \{+1, -1\}$, a ± 1 *RMQ* which admits $O(n)$ preprocessing and $O(1)$ query (Bender–Farach-Colton). Contest-irrelevant: the $O(n \log n)$ sparse table already fits in 1 s for $n \le 2 \cdot 10^5$. **L6**
- **Multiple RMQ on one tour.** A segment tree over `dep` answers $O(\log n)$ per query but supports point *updates* (change a depth = change the tree). Deferred to Ch. 19/22. **L4**
- **The reverse reduction.** Ch. 9 (*Chapter 9*) builds a Cartesian tree from an array and reads RMQ as LCA *on the Cartesian tree*. That is the mirror image; we do LCA → RMQ on the *original* tree. **L3**

16.2.6 Common Pitfalls

Trap 1: Overlapping-blocks on non-idempotent operations

If you replace min by *sum* in the sparse-table query, the overlap double-counts. Only min / max / gcd /AND/OR are idempotent ($x \oplus x = x$); SUM$(x, x) = 2x \neq x$. For sum/xor on a *static* array, use prefix sums; for path aggregates use lifting (§16.4).

Trap 2: Forgetting to log the RETURN visit

The tour must be $2n - 1$ entries, not n. If you skip the "push on return" line, the span between two firsts might *not* contain the LCA at all – the DFS went below the LCA and jumped without logging the climb back. The inductive invariant: consecutive entries differ by ± 1 in depth.

Trap 3: Using 1-based vs 0-based indexing inconsistently

The sparse table and `first[]` in this section use *0-based* indices. Mixing with a 1-based tin convention from Ch. 5 in the same code is a silent off-by-one: `first`$[1] = 0$ (root 1 at tour position 0) vs tin$(1) = 1$. Pick one, document it.

16.2.7 Problem Lab

W16.2.1 Distance Queries

Source: CSES 1135 CSES L3
Techniques: Euler tour, sparse-table RMQ, distance formula

CSES 1135 Distance Queries

You are given a tree consisting of n nodes (rooted at 1) and q queries. Each query consists of two nodes a and b, and your task is to print the distance between them (number of edges on the unique path).

Input. First line: n and q ($1 \le n \le 10^5$, $1 \le q \le 10^5$). Next $n - 1$ lines: edges a_i, b_i. Next q lines: queries a, b.

Output. For each query print the distance.

Example.

```
Input:
5 3
1 2
1 3
3 4
3 5
1 3
2 5
1 4

Output:
1
3
2
```

Attempt it yourself first

Peek (1 min lost): The formula $\text{dist}(u, v) = \text{depth}(u) + \text{depth}(v) - 2\,\text{depth}(\text{lca}(u, v))$ converts the problem to two depth lookups plus one LCA.

Nudge (5 min lost): Build the $2n - 1$ tour, sparse table, and answer each LCA in $O(1)$ per Theorem 16.2.2.

Full hint (you will not solve this one): Code: build Euler tour from root 1 via Code 16.2.1; build Code 16.2.2 on the depth array; for each query compute $\ell = \text{lca}(a, b)$, print $\text{depth}(a) + \text{depth}(b) - 2\,\text{depth}(\ell)$.

➤ Understanding

The tree is *static* (no updates between queries), so we can preprocess once. Each query asks for a path length in edges. Distance between two nodes is determined by the hinge (Theorem 16.2.2 + the formula):

$$\text{dist}(a, b) = \text{depth}(a) + \text{depth}(b) - 2\ \text{depth}(\text{lca}(a, b)).$$

➤ Key Observations

Observation 1. The formula is purely *depth*-based. We need: (i) the depth of each node from a single DFS; (ii) a fast LCA query. No edge weights, no aggregates – the RMQ machine suffices.

Observation 2. Constraints: $n, q \le 10^5$ rule out $O(h)$ per query in worst case (chain $h = n$). The $O(1)$ sparse-table query is the cleanest fit because no k-th-ancestor or path-aggregate is needed.

➤ Think It Through

Think it through with me

✗ **Tried:** BFS/DFS per query
Killed by: $q = 10^5$, BFS gives $O(nq) = 10^{10}$. TLE.

✗ **Tried:** Naive walk
Killed by: Correct, but worst-case $O(h) = O(n)$ per query on a chain. TLE.

✓ **Aha!** Preprocess the $2n - 1$ tour + sparse table; each LCA is $O(1)$; distance is one arithmetic expression. Total $O(n \log n + q)$.

➤ Proof of the Distance Formula

Distance through the hinge

Let $\ell = \text{lca}(a, b)$. The route $a \to b$ goes $a \to \ell \to b$. The segment $a \to \ell$ has $\text{depth}(a) - \text{depth}(\ell)$ edges (climb the chain), $b \to \ell$ similarly $\text{depth}(b) - \text{depth}(\ell)$. No overlap: a's segment is below ℓ and disjoint from b's unless $a \preceq b$, in which case $\ell = a$ and the first term is zero. Total: $(\text{depth}(a) - \text{depth}(\ell)) + (\text{depth}(b) - \text{depth}(\ell))$.

■

➤ Implementation

Code 16.2.3: CSES 1135 full solution (Euler + RMQ)

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MX = 200001;  // 2n-1 for n=1e5
vector<int> adj[MX], tour, dep;
int firstOcc[MX], st[18][MX], lg[MX], depth_[MX];
int timer_;

void dfs(int v, int p, int d) {
    firstOcc[v] = (int)tour.size();
    tour.push_back(v); dep.push_back(d); depth_[v] = d;
    for (int c : adj[v]) if (c != p) {
        dfs(c, v, d+1);
        tour.push_back(v); dep.push_back(d);
    }
}
void build() {
    int N = (int)tour.size();
    lg[1] = 0;
    for (int i = 2; i <= N; i++) lg[i] = lg[i/2] + 1;
    for (int i = 0; i < N; i++) st[0][i] = i;
    for (int j = 1; (1<<j) <= N; j++)
        for (int i = 0; i + (1<<j) <= N; i++) {
            int a = st[j-1][i], b = st[j-1][i+(1<<(j-1))];
            st[j][i] = (dep[a] <= dep[b]) ? a : b;
        }
}
int lca(int u, int v) {
    int L = min(firstOcc[u], firstOcc[v]);
    int R = max(firstOcc[u], firstOcc[v]);
    int j = lg[R - L + 1];
    int a = st[j][L], b = st[j][R-(1<<j)+1];
    return tour[(dep[a] <= dep[b]) ? a : b];
}
int main(){
    int n, q; scanf("%d%d", &n, &q);
    for (int i = 0; i < n-1; i++) {
        int a,b; scanf("%d%d", &a, &b);
        adj[a].push_back(b); adj[b].push_back(a);
    }
    dfs(1, 0, 0);
    build();
    while (q--) {
        int a,b; scanf("%d%d", &a, &b);
        int l = lca(a,b);
        printf("%d\n", depth_[a] + depth_[b] - 2*depth_[l]);
    }
}
```

➤ Trace on the Official Sample

CSES 1135 sample verification

Tree: $1-2, 1-3, 3-4, 3-5$. Root 1. Depths: $d_1 = 0, d_2 = 1, d_3 = 1, d_4 = 2, d_5 = 2$.

Query (1,3): $\text{lca}(1,3) = 1$ (1 is root, hence ancestor of everything). $\text{dist} = 0 + 1 - 2(0) = 1$. Expected: 1. ✓

Query (2,5): $\text{lca}(2,5)$: chains $2 \to 1$ and $5 \to 3 \to 1$; intersection $\{1\}$; lca $= 1$. dist $= 1 + 2 - 2(0) = 3$. Expected: 3. ✓

Query (1,4): $\text{lca}(1,4) = 1$. $\text{dist} = 0 + 2 - 2(0) = 2$. Expected: 2. ✓

➤ Complexity

Complexity

Tour: $O(n)$. Sparse table: $O(n \log n)$. Each query: $O(1)$. Total: $O(n \log n + q)$. Memory: $O(n \log n)$ for `st`. **AC.**

16.2.8 Practice Corner

Q16.2.1 Lowest Common Ancestor (many queries) – SPOJ LCA SPOJ L3

Hint: the textbook many-query LCA on a judge: build the $2n-1$ Euler tour and answer each query as a range-minimum-depth over it – a direct transcription of Theorem 16.2.2 with Code 16.2.1 + Code 16.2.2. $N, Q \le 10^5$.

Back-cites. Ch. 5 (*Chapter 5*) – the n-entry flattening tour and the tin / tout convention are *different* from the $2n-1$ tour (contrasted above; never re-linked here). Ch. 9 (*Chapter 9*) – RMQ via Cartesian tree is the reverse reduction; we cite the *concept* but own the general sparse table.

Section 16.2 in 10 seconds

The $2n$–1 tour logs every DFS visit (enter *and* return). `first`$[v]$ is the 0-based index of v's first entry. Theorem 16.2.2: $\text{lca}(u,v)$ = tour entry with minimum depth in $[$`first`$[u]$, `first`$[v]]$. Sparse table answers that in $O(1)$ using two overlapping 2^j-blocks (idempotent min makes overlap harmless). Setup $O(n \log n)$, query $O(1)$, static only.

Self-test (answer from memory, tick the box):

- ☐ Tour length for $n = 10$? (Answer: $2(10) - 1 = 19$.)
- ☐ State the LCA theorem in one sentence.
- ☐ Why are overlapping blocks safe for min but not for *sum*?
- ☐ `first`[8] on T? (5.) `first`[10]? (14.)
- ☐ Give the distance formula.

16.3 LCA by Binary Lifting: The Contest Default

Three questions from 16.1–16.2 and Ch. 12

1. From *Example* 16.1.4: the walk moves one parent at a time. *(If you could jump 2^j ancestors in one step, how few steps would the lockstep need?)*
2. From §16.2: the sparse table gives $O(1)$ per query but can only answer LCA/distance. *(What about "the k-th ancestor of v", a question with no obvious RMQ translation?)*
3. From Ch. 12 (*Code* 12.1.1): one DFS stamps `par`$[v]$ for every node. *(Can you extend that single parent into a* table *of grandparents, great-grandparents, …?) That is exactly binary lifting.*

In This Section

The $2n-1$ tour + RMQ machine answers LCA in $O(1)$ per query but has *no climbing power*: it cannot produce the k-th ancestor or an aggregate along a path. Binary lifting sacrifices the $O(1)$ for $O(\log n)$, but gains flexibility that makes it the *default* choice in contests – the same `up` table answers k-th ancestor, LCA, and (with a second table `agg`) path aggregates (§16.4). The idea is one sentence: precompute the 2^j-th ancestor of every node via the recurrence $\texttt{up}[v][j] = \texttt{up}[\,\texttt{up}[v][j-1]\,][j-1]$, then decompose any climb into binary bits.

After this section you can

- build $\texttt{up}[v][j]$ and prove the recurrence; **L3**
- climb k ancestors in $O(\log n)$ via bit decomposition; **L3**
- implement the descending-pairs LCA that *returns* $\texttt{par}[u]$ – not u; **L4**
- handle the $u = v$ shortcut and the sentinel discipline; **L3**
- solve a k-th-ancestor problem (CSES 1687) end-to-end. **L3**

16.3.1 The Table

Definition 16.3.1 (Binary lifting table)

Let LOG be the smallest integer with $2^{LOG} > n$ ($LOG = 4$ for T). Define

$$\texttt{up}[v][0] = \texttt{par}[v], \qquad \texttt{up}[v][j] = \texttt{up}[\,\texttt{up}[v][j-1]\,][j-1] \ \text{ for } j = 1, \ldots, LOG-1.$$

$\texttt{up}[v][j]$ is the 2^j-th ancestor of v. The sentinel $\texttt{up}[\text{root}][j] = 0$ for all j; vertex 0 is never a real answer.

Proposition 16.3.2 (The k-th ancestor climb)

To find the k-th ancestor of v: for each bit j set in k, jump $v \leftarrow \texttt{up}[v][j]$ (from low bits to high, or high to low – both work). If at any point $v = 0$, $k > \text{depth}(v_{\text{orig}})$ and the answer is "does not exist". Cost $O(\log n)$.

Proof

Write $k = \sum_j b_j 2^j$ (binary). Each $\texttt{up}[v][j]$ advances exactly 2^j steps. After processing all bits, the total advance is $\sum b_j 2^j = k$. The intermediate nodes are ancestors of v (because the composition of parent links is a parent link), and the sentinel 0 absorbs overshoot. ∎

16.3.2 Hand-Computing the up-Table on T

T rooted at 1. $LOG = 4$ ($2^4 = 16 > 10 = n$). par = (0, 0, 1, 1, 2, 2, 3, 3, 5, 7, 9) (index 0 unused; sentinel par[1] = 0).

up-table on T, LOG=4

v	up$[v][0]$	up$[v][1]$	up$[v][2]$	up$[v][3]$	explanation
1	0	0	0	0	root
2	1	0	0	0	parent is root
3	1	0	0	0	parent is root
4	2	1	0	0	2nd ancestor = up[2][0] = 1
5	2	1	0	0	same logic
6	3	1	0	0	
7	3	1	0	0	
8	5	2	1	0	4th ancestor = 1
9	7	3	1	0	
10	9	7	3	1	8th ancestor = 1 (root); 16th = 0 (sentinel)

Spot-check up[10][2] = up[up[10][1]][1] = up[7][1] = 1. Wait: up[7][1] = up[3][0] = 1? No: up[7][1] = up[up[7][0]][0] = up[3][0] = 1. Hmm, but I said up[10][2] = 3. Re-derive: up[10][2] = up[up[10][1]][1] = up[7][1]. up[7][1] = up[up[7][0]][0] = up[3][0] = 1. But up[10][2] should be the 4-th ancestor of 10: $10 \to 9 \to 7 \to 3 \to 1$: the 4-th ancestor is 1. And up[10][2] = 1 ✓. The table row reads $10 : (9, 7, 3, 1)$ – re-check the $j=2$ entry: $2^2 = 4$, 4-th ancestor of 10 is 1. Correct.

Correction to my table: up[10][2] = up[7][1] = up[up[7][0]][0] = up[3][0] = 1. And up[10][3] = up[up[10][2]][2] = up[1][2] = 0 (sentinel). The final table is: 10 : (9, 7, 1, 0).

Example 16.3.3 (up-table on T, clean version)

v	$j=0$ (2^0)	$j=1$ (2^1)	$j=2$ (2^2)	$j=3$ (2^3)
1	0	0	0	0
2	1	0	0	0
3	1	0	0	0
4	2	1	0	0
5	2	1	0	0
6	3	1	0	0
7	3	1	0	0
8	5	2	1	0
9	7	3	1	0
10	9	7	1	0

All entries derivable from the recurrence + par. Verify up[8][2] = up[5][1] = up[2][0] = 1: $2^2 = 4$ steps above 8: $8 \to 5 \to 2 \to 1 \to 0$; the 4-th ancestor is 0? No: the 4-th ancestor from 8 is 1 (3 steps to root, 4th is 0). Re-check: $8 \to 5$ (1st), $\to 2$ (2nd), $\to 1$ (3rd), $\to 0$ (4th). So up[8][2] = 0. *Correction*: up[8][2] = up[up[8][1]][1] = up[2][1] = up[up[2][0]][0] = up[1][0] = 0. ✓. The $j=2$ column should read $(0, 0, 0, 0, 0, 0, 0, 1, 1, 1)$? Let me re-verify

$v = 9$: up[9][2] = up[up[9][1]][1] = up[3][1] = up[up[3][0]][0] = up[1][0] = 0. And $v = 10$: up[10][2] = up[7][1] = up[3][0] = 1.

Final corrected table:

v	$j=0$	$j=1$	$j=2$	$j=3$
1	0	0	0	0
2	1	0	0	0
3	1	0	0	0
4	2	1	0	0
5	2	1	0	0
6	3	1	0	0
7	3	1	0	0
8	5	2	0	0
9	7	3	0	0
10	9	7	1	0

16.3.3 The LCA by Lifting – Descending Pairs

The walk of §16.1 steps one parent at a time. Lifting replaces each "step" by a "jump":

1. **Equalise.** Suppose depth$(u) \geq$ depth(v). Lift u up by depth(u) − depth(v) using the up table (prop Proposition 16.3.2).
2. **Shortcut.** If $u = v$ after equalisation, return u. (This is the $u \preceq v$ case.)
3. **Descend.** For $j = LOG-1$ down to 0: if up$[u][j] \neq$ up$[v][j]$, set $u \leftarrow$ up$[u][j]$, $v \leftarrow$ up$[v][j]$.
4. **Return par$[u]$.** After the loop, u and v sit *just below* the LCA (their parents are the LCA); return up$[u][0]$.

Return par of u, not u

After the descending loop, $u \neq v$ (the loop only fires when they differ), but par$[u]$ = par$[v]$ = lca. If you return u, you are off by one edge. The $u = v$ shortcut in step 2 *must* come before the descend loop; otherwise ancestor pairs like lca$(9, 10) = 9$ (from *Example* 16.1.4) fail: equalisation makes $u = v$ and the loop does nothing, so step 4 would return par$[9] = 7$ (wrong).

Code 16.3.1: Build the up-table (one DFS to seed, then fill by recurrence)

```
void dfs_build(int v, int p, int d) {
    par[v] = p; depth_[v] = d;
    up[v][0] = p;                        // 2^0-th ancestor = parent
    for (int j = 1; j < LOG; j++)
        up[v][j] = up[ up[v][j-1] ][j-1];   // compose two half-jumps
    for (int c : adj[v]) if (c != p) dfs_build(c, v, d+1);
}
```

Code 16.3.2: k-th ancestor via bit decomposition

```
int kth_ancestor(int v, int k) {
    if (k > depth_[v]) return 0;   // overshoot guard: sentinel
    for (int j = 0; j < LOG; j++)
        if (k & (1 << j)) v = up[v][j];
    return v;                      // 0 means "does not exist"
}
```

Code 16.3.3: LCA by descending pairs – the contest default

```
int is_ancestor(int u, int v) {   // tin/tout test (Ch.5 convention)
    return tin[u] <= tin[v] && tin[v] <= tout[u];
}
int lca_lift(int u, int v) {
    if (is_ancestor(u, v)) return u;
    if (is_ancestor(v, u)) return v;
    for (int j = LOG - 1; j >= 0; j--)
        if (!is_ancestor(up[u][j], v))
            u = up[u][j];
    return par[u];               // NOT u
}
```

Lifting LCA on T: lca(8, 10)

depth(8) = 3, depth(10) = 4. Equalise: 10 is deeper, lift by $4 - 3 = 1$: $v \leftarrow$ up[10][0] = 9. Now $u = 8, v = 9$, both depth 3. $u \neq v$. Descend:

- $j = 3$: up[8][3] = 0, up[9][3] = 0; equal ⇒ skip.
- $j = 2$: up[8][2] = 0, up[9][2] = 0; equal ⇒ skip.
- $j = 1$: up[8][1] = 2, up[9][1] = 3; is_ancestor(2, 9)? tin(2) = 2, tin(9) = 9: $2 \leq 9 \leq 5$? tout(2) = $5 < 9$. NO. So jump: $u \leftarrow 2$, $v \leftarrow 3$.
- $j = 0$: up[2][0] = 1, up[3][0] = 1; equal ⇒ skip.

Return par[u] = par[2] = 1. lca(8, 10) = 1. ✓

Lifting LCA on T: lca(9, 10)

depth(9) = 3, depth(10) = 4. Equalise: $v \leftarrow 9$. Now $u = 9, v = 9$: **u = v**. Shortcut fires: return 9. lca(9, 10) = 9 (the degenerate ancestor case from *Example* 16.1.4). ✓ Without the shortcut step 4 would return par[9] = 7, the classic off-by-one.

Binary lifting method

Setup: $O(n \log n)$ (the table fill inside the DFS). **Query:** $O(\log n)$ per LCA or k-th ancestor. **Memory:** $O(n \log n)$. **Power:** LCA + k-th ancestor + (with agg) path aggregates; the *versatile* default.

16.3.4 Problem Lab

W16.3.1 Company Queries I

Source: CSES 1687 CSES L3
Techniques: binary lifting, k-th ancestor

CSES 1687 Company Queries I

A company has n employees, each reporting to a boss (except the CEO, employee 1, who has no boss). Given q queries of the form "who is the k-th boss above employee x?" (i.e., the k-th ancestor), answer each or print -1 if it does not exist.

Input. n and q ($1 \leq n, q \leq 2 \cdot 10^5$). Next line: $n - 1$ integers $b_2, b_3, \ldots, b_n$: the boss of employee i. Next q lines: x and k.

Output. For each query, the k-th boss or -1.

Example.

```
Input:
5 3
1 1 3 3
4 1
4 2
4 3

Output:
3
1
-1
```

Attempt it yourself first

Peek (1 min lost): Tree depth for the sample: depth(4) = 2 (boss chain $4 \to 3 \to 1$).

Nudge (5 min lost): The answer to "k-th ancestor of x" is the bit-decomposition climb from Proposition 16.3.2.

Full hint (you will not solve this one): Build `up` table; if $k > \text{depth}(x)$ output -1, else decompose k in binary and jump.

➤ Understanding

The "reporting structure" *is* a rooted tree (root = CEO = employee 1, parent = direct boss). The k-th boss is the k-th ancestor. The k-ancestor question *cannot* be answered by the RMQ machine – only lifting handles it.

➤ Key Observations

Observation 1. The input gives the boss of each node directly – this *is* the parent array. No need for a DFS to discover the tree structure.

Observation 2. $n, q \leq 2 \cdot 10^5$ and a chain is possible, so the $O(k)$ walk per query would be $O(nq)$ worst case. The $O(\log n)$ per-query climb fits.

➤ Think It Through

Think it through with me

✗ **Tried:** Follow parent pointers k times
Killed by: Worst case $k = n$, total $O(nq)$. TLE.

✓ **Aha!** Precompute `up`$[v][j]$ for $j = 0, \ldots, LOG-1$; decompose k in binary; $O(\log n)$ per query.

➤ Implementation

Code 16.3.4: CSES 1687 full solution (binary lifting)

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp
const int LOG = 20;  // 2^20 > 2e5
int up[MX][LOG], depth_[MX];
int main(){
    int n, q; scanf("%d%d", &n, &q);
    up[1][0] = 0; depth_[1] = 0;  // CEO is root
    for (int i = 2; i <= n; i++) {
        int b; scanf("%d", &b);
        up[i][0] = b;
        depth_[i] = depth_[b] + 1;
        for (int j = 1; j < LOG; j++)
            up[i][j] = up[ up[i][j-1] ][j-1];
    }
    while (q--) {
        int x, k; scanf("%d%d", &x, &k);
        if (k > depth_[x]) { printf("-1\n"); continue; }
        for (int j = 0; j < LOG; j++)
            if (k & (1 << j)) x = up[x][j];
        printf("%d\n", x);
    }
}
```

➤ Trace on the Official Sample

CSES 1687: bosses 1 1 3 3; queries on the sample tree

Tree: $1 \to \{2,3\}$, $3 \to \{4,5\}$. Depths: $d_1 = 0, d_2 = 1, d_3 = 1, d_4 = 2, d_5 = 2$. `up` table: row 4: $(3,1,0,0,\dots)$; row 2: $(1,0,0,\dots)$.

Query (4, 1): $k = 1$, binary $00\dots01$. $j{=}0$: $x = \texttt{up}[4][0] = 3$. Output 3. ✓

Query (4, 2): $k = 2$, binary $00\dots10$. $j{=}1$: $x = \texttt{up}[4][1] = 1$. Output 1. ✓

Query (4, 3): $k = 3 > \text{depth}(4) = 2$. Output -1. ✓

➤ Complexity

Complexity

Build: $O(n \log n)$. Query: $O(\log n)$. Total $O((n+q)\log n)$. Memory: $O(n \log n)$. **AC.**

16.3.5 Common Pitfalls

Trap 1: Off-by-one in the descend loop

After the descend loop, u and v are *just below* the LCA ($\texttt{up}[u][0] = \texttt{up}[v][0] = \ell$). The answer is ℓ, not u. For $\text{lca}(8, 10)$ on T: loop leaves $u = 2, v = 3$; answer $= \texttt{par}[2] = 1$. Returning $u = 2$ is the one-edge error.

Trap 2: Dropping the $u = v$ shortcut

Equalisation can make $u = v$ (when one is an ancestor of the other). Without the early-return, the descend loop skips every jump (all `up` values are equal), and $\texttt{par}[u]$ is *one edge above* the correct answer. $\text{lca}(9, 10)$ after equalising $10 \to 9$ has $u = v = 9$; the shortcut returns 9; without it, $\texttt{par}[9] = 7$ (wrong).

Trap 3: LOG too small or sentinel leaking

LOG must satisfy $2^{LOG} > n$ (for $n = 10^5$, $LOG = 17$ suffices; $2^{17} = 131072$). If LOG is one short, a long chain cannot be traversed. Separately: if your k-th ancestor code doesn't check $k > \text{depth}(x)$, a jump past the root lands on 0 and the `up`$[0][j] = 0$ sentinel may silently produce 0 as the answer.

16.3.6 Variations and Twists

- **is_ancestor variant vs equalise-then-descend variant.** The listing `lst:c11-lcalift` uses tin / tout (Ch. 5) for a 2-line LCA; the "descend both" variant (no tin) is equally fast and avoids the Ch. 5 dependency. Pick one convention; §16.4 uses the tin-based version. L4
- **Longest k such that predicate holds.** Descend $j = LOG-1$ to 0; jump if the predicate is still false. "First ancestor with weight $\geq W$" is this pattern. L4
- **Forest.** Run the build DFS from each component root, with a super-root 0. All queries across different components return 0. L3

16.3.7 Practice Corner

Q16.3.1 Company Queries II – CSES 1688 CSES L4

Hint: same tree; query is now "lca(x, y) ?"; use the lifting LCA from `lst:c11-lcalift`. Sample: 5 3 / bosses 1 1 3 3 / queries $(4, 5) \to 3$, $(2, 5) \to 1$, $(1, 4) \to 1$.

Q16.3.2 K-th Ancestor – LeetCode 1483 LC Medium L3

Hint: design a class: `TreeKAncestor(n, parents)` then `query(k, node)`. The up-table answers in $O(\log n)$; build once in the constructor.

Back-cites. Ch. 12 (*Code* 12.1.1) – the seed DFS that fills `par`$[v]$ and `depth`(v). Ch. 5 (*Chapter* 5) – the tin / tout convention used by the `is_ancestor` helper.

Section 16.3 in 10 seconds

`up`$[v][j] = 2^j$-th ancestor; built by composing two 2^{j-1}-jumps. Climb k: decompose binary, $O(\log n)$. LCA: equalise depths, shortcut if $u = v$, descend pairs, return `par`$[u]$. LOG: smallest with $2^{LOG} > n$. Sentinel 0 absorbs overshoot. More versatile than the RMQ machine at $O(\log n)$ per query vs $O(1)$ – the trade is worth it because the same table unlocks k-th ancestor and path aggregates.

Self-test (answer from memory, tick the box):

- ☐ Write the recurrence for `up`$[v][j]$.
- ☐ Why must you return `par`$[u]$ (not u) after the loop?
- ☐ What does the $u = v$ shortcut handle? Give the T example.
- ☐ `up`$[10][2]$ on T? (1.) Explain the derivation.
- ☐ When does a forest need a super-root?

16.4 Lifting an Aggregate along a Jump

Three questions from 16.1–16.3

1. From §16.3: the $\texttt{up}[v][j]$ table lets you jump 2^j ancestors in one step. *(What if each edge carried a* value *and you needed the* max *along the jump? Can the same table structure carry a payload?)*
2. From Ch. 13 (*Theorem* 13.1.2): the subtree fold combines child contributions. *(Lifting an aggregate along a path is the same idea on* ancestor chains*: associative combine, partial results, merge at the end.)*
3. From *Example* 16.1.4: $\text{lca}(8, 10) = 1$ splits the path $8 \to \cdots \to 10$ at the hinge. *(If you want* max *along that path, you can accumulate on each side separately, then merge at the hinge.)*

In This Section

The lifting table $\texttt{up}[v][j]$ answers "where am I after 2^j steps up?". A *second* table $\texttt{agg}[v][j]$ answers "what is the aggregate (max, min, sum, xor, count, gcd, ...) along those 2^j steps?" The machinery is identical – same recurrence, same bit-decomposition climb – with an associative payload riding on every jump. Path queries then split at the hinge (lca): accumulate each arm with the `agg` table, merge the two partial results, and fold in the LCA's own value. Three distinct patterns emerge from this one idea:

- **Static path aggregate** (max/min/sum/xor along a route);
- **First-ancestor-past-a-threshold** (bit-descent for the deepest ancestor whose prefix-sum or value still passes a predicate);
- ***k*-th node *on* a route** (distance formula + k-th ancestor, the key to CF 1304E).

After this section you can

- define $\texttt{agg}[v][j]$ and prove the path-aggregate theorem (Theorem 16.4.1); **L4**
- compute max/sum/xor along a path on T by hand; **L4**
- descend to the first ancestor past a threshold; **L4–L5**
- locate the k-th node on a route $u \to v$; **L4**
- solve CF 1304E (walk on a tree + one extra edge, parity check). **L4–L5**

16.4.1 The Second Table

Theorem 16.4.1 (Aggregated lifting)

Let $\oplus$ be an *associative* binary operation on values attached to vertices (or edges). Define

$$\texttt{agg}[v][0] = \mathit{val}(v), \qquad \texttt{agg}[v][j] = \texttt{agg}[v][j-1] \oplus \texttt{agg}[\,\texttt{up}[v][j-1]\,][j-1].$$

$\texttt{agg}[v][j]$ is the aggregate over the 2^j vertices starting at v going upward, **excluding** the *landing* vertex $\texttt{up}[v][j]$. To query the aggregate along the path from u to v:

1. Compute $\ell = \text{lca}(u, v)$.
2. Accumulate $u \to \ell$: decompose $\text{depth}(u) - \text{depth}(\ell)$ jumps and $\oplus$-combine the partial `agg` values.
3. Accumulate $v \to \ell$ symmetrically.
4. Merge: combine the two partial results and fold in $\mathit{val}(\ell)$ (the hinge vertex, never captured by either arm because it is the landing vertex).

Cost $O(\log n)$ per query.

Proof

The recurrence mirrors `up`'s composition: a 2^j-jump from v is two consecutive 2^{j-1}-jumps, so the aggregate over the full 2^j is the combine of the lower half (starting at v) and the upper half (starting at the landing of the lower half). The "exclude the landing vertex" convention makes the halves non-overlapping and adjacent. The query decomposes the path at the hinge; each arm is a k-ancestor climb where k is split into binary bits, and the `agg` values for those bits tile the arm exactly. The hinge itself is the only vertex not covered by either arm's jumps (each arm's last jump lands *at* the hinge and excludes it), so we fold it in last.

■

Why $\oplus$ must be associative – and when idempotent matters

The two-accumulator approach (left arm, right arm) *never* overlaps and *never* misses, so $\oplus$ need only be associative – not commutative or idempotent. This is **different** from the sparse-table overlapping-block trick (§16.2), where the block overlap requires $\oplus$ to be *idempotent* ($x \oplus x = x$). Sums, xors, and counts are fine here but not there.

16.4.2 Worked Example on T

Assign vertex values (permutation of 1..10): $w = (6, 3, 9, 1, 8, 10, 7, 2, 5, 4)$ for nodes 1..10 (matching the summary). $\oplus = \max$.

$\texttt{agg}[v][0] = w_v$ (the vertex's own value): $(6, 3, 9, 1, 8, 10, 7, 2, 5, 4)$.

$\texttt{agg}[v][1] = w_v \oplus w_{\texttt{par}[v]}$ (max of v and parent, excluding the landing vertex, which is the grandparent):

v	$\texttt{up}[v][0] = par$	$\texttt{agg}[v][1] = \max(w_v, w_{par})$
1	0	6
2	1	$\max(3, 6) = 6$
3	1	$\max(9, 6) = 9$
4	2	$\max(1, 3) = 3$
5	2	$\max(8, 3) = 8$
6	3	$\max(10, 9) = 10$
7	3	$\max(7, 9) = 9$
8	5	$\max(2, 8) = 8$
9	7	$\max(5, 7) = 7$
10	9	$\max(4, 5) = 5$

$\texttt{agg}[v][2] = \texttt{agg}[v][1] \oplus \texttt{agg}[\texttt{up}[v][1]][1]$ (max over 4 vertices starting at v, excluding the landing):

For $v = 8$: $\texttt{agg}[8][1] = 8$; $\texttt{up}[8][1] = 2$; $\texttt{agg}[2][1] = 6$; so $\texttt{agg}[8][2] = \max(8, 6) = 8$. For $v = 10$: $\texttt{agg}[10][1] = 5$; $\texttt{up}[10][1] = 7$; $\texttt{agg}[7][1] = 9$; so $\texttt{agg}[10][2] = \max(5, 9) = 9$.

Example 16.4.2 (Max-on-path query on T)

Compute the max along the path $8 \to 10$. Hinge: $\text{lca}(8, 10) = 1$.

Arm from 8: $\text{depth}(8) - \text{depth}(1) = 3 - 0 = 3$. Decompose $3 = 2^1 + 2^0$:

- $j = 1$: $\texttt{agg}[8][1] = 8$; land at $\texttt{up}[8][1] = 2$.
- $j = 0$: $\texttt{agg}[2][0] = 3$; land at $\texttt{up}[2][0] = 1$.

Left partial: $\max(8, 3) = 8$.

Arm from 10: $\text{depth}(10) - \text{depth}(1) = 4 - 0 = 4 = 2^2$.

- $j = 2$: $\texttt{agg}[10][2] = 9$; land at $\texttt{up}[10][2] = 1$.

Right partial: 9.

Merge with hinge: $w_1 = 6$. Final: $\max(8, 9, 6) = \boxed{9}$ (at node 3, on the right arm).

Path vertices: $8, 5, 2, 1, 3, 7, 9, 10$ with values $(2, 8, 3, 6, 9, 7, 5, 4)$; max $= 9$. ✓

16.4.3 Implementation Skeleton

Code 16.4.1: General path-aggregate query (max/sum/xor)

```
// val[v] = value at vertex v;  ID = identity of op
//  MAX: ID=0 (-inf for negative), op=max, agg[i][j]=max(agg[i][j-1], agg[up[i][j-1]][
    ↪ j-1])
//  SUM: ID=0, op=+
//  XOR: ID=0, op=^

int path_query(int u, int v, int l) {  // l = lca(u,v)
    int res = val[l];   // fold hinge value
    int cur = u;
    int k = depth_[u] - depth_[l];
    for (int j = 0; j < LOG; j++)
        if (k & (1 << j)) { res = op(res, agg[cur][j]); cur = up[cur][j]; }
    cur = v;
    k = depth_[v] - depth_[l];
    for (int j = 0; j < LOG; j++)
        if (k & (1 << j)) { res = op(res, agg[cur][j]); cur = up[cur][j]; }
    return res;
}
```

Aggregated lifting

Build: $O(n \log n)$, same DFS as `up` with one extra array. Query: $O(\log n)$ per path (plus the $O(\log n)$ for the LCA). Memory: $O(n \log n)$ for the second table.

16.4.4 First-Ancestor-Past-a-Threshold (Bit Descent)

A second pattern falls out of the same table. Suppose edges have weights and you want the *deepest ancestor* of v whose prefix-sum-from-root is still $\le T$. Equivalently: the highest u such that $\text{dist}(u, v) \le \text{cap}$ with some monotone predicate. Use the `up` table in *descending* bit order:

for $j = LOG$–1 down to 0 : if $\texttt{up}[cur][j]$ exists and $P(\texttt{up}[cur][j])$ is false : $cur \leftarrow \texttt{up}[cur][j]$.

After the loop, $\texttt{par}[cur]$ is the first ancestor passing P. This mirrors the lifting-LCA "descend then return parent" pattern.

One loop shape, three uses

Descending bits + conditional jump is the *same* structure as the LCA descend loop and the binary-search-on-a-tree technique. Once you write it for LCA, the threshold variant is a predicate swap.

16.4.5 The k-th Node on a Route

Given u, v and k ($1 \le k \le \text{dist}(u, v)$), find the k-th node on the path from u to v:

$$d_1 = \text{depth}(u) - \text{depth}(\ell), \qquad d_2 = \text{depth}(v) - \text{depth}(\ell).$$

If $k \le d_1$: the answer is the k-th ancestor of u (just `kth_ancestor`(u, k)). If $k > d_1$: the answer is on the $v \to \ell$ arm at position $d_1 + d_2 - k + 1$ from v downward from ℓ, i.e. `kth_ancestor`$(v,\ d_1 + d_2 - k)$.

Route positions on T: u=8, v=10

Path: $8 \to 5 \to 2 \to 1 \to 3 \to 7 \to 9 \to 10$, length 7 edges. $\ell = 1$, $d_1 = 3$, $d_2 = 4$.

$k = 1$: ancestor-1 of $8 = 5$. ✓$k = 3$: ancestor-3 of $8 = 1$. ✓$k = 4$: $d_1 + d_2 - k = 3 + 4 - 4 = 3$; ancestor-3 of $10 = 7$? $10 \to 9 \to 7 \to 3$: 3rd ancestor $= 3$. Hmm. $k = 4$ is 1 edge past ℓ: the path position is $d_1 (= 3) + 1 = 4$ means the $(k - d_1)$-th descendant of ℓ on the v-arm. Equivalently: $d_1 + d_2 - k = 3$ ancestor steps from $v = 10$: $10 \xrightarrow{1} 9 \xrightarrow{2} 7 \xrightarrow{3} 3$. Node 3. ✓

$k = 5$: $d_1 + d_2 - k = 2$; ancestor-2 of $10 = 7$. ✓$k = 7$: $d_1 + d_2 - k = 0$; ancestor-0 of $10 = 10$ itself. ✓

16.4.6 Problem Lab

W16.4.1 1-Trees and Queries

Source: CF 1304E CF 2000 L5
Techniques: binary lifting, distances, parity, k-th node on route

CF 1304E 1-Trees and Queries

A tree of n vertices, plus q queries. Each query gives a, b, x, y, k: you must walk from a to b, traversing exactly k edges. You *may* add edge (x, y) (temporarily, just for this query) and use it *at most once*. Can you arrive at b in exactly k steps? (You may revisit vertices/edges; only parity and minimum length matter.)

Input. n then $n-1$ edges then q then queries. $2 \le n, q \le 10^5$.

Example.

```
Input:
5
1 2
2 3
3 4
4 5
5
1 3 1 2 2
1 4 1 3 2
1 4 1 3 3
4 2 3 3 9
5 2 3 3 9

Output:
YES
YES
NO
```

```
YES
NO
```

Attempt it yourself first

Peek (1 min lost): Without the extra edge, the shortest $a \to b$ route has $\text{dist}(a, b)$ edges. Any walk of length k must satisfy $\text{dist}(a, b) \le k$ and $(k - \text{dist}(a, b))$ even.

Nudge (5 min lost): The extra edge (x, y) gives two alternative route lengths: $\text{dist}(a, x) + 1 + \text{dist}(y, b)$ and $\text{dist}(a, y) + 1 + \text{dist}(x, b)$. Check all three (direct + two detours) against the parity condition.

Full hint (you will not solve this one): YES iff any of the three lengths $d_1 = \text{dist}(a, b)$, $d_2 = \text{dist}(a, x)+1+\text{dist}(y, b)$, $d_3 = \text{dist}(a, y)+1+\text{dist}(x, b)$ satisfies $d_i \le k$ and $(k-d_i)\%2 == 0$.

➤ Understanding

The walk is on a tree (one route between every pair). Adding edge (x, y) creates exactly one cycle; using it *at most once* means the walk goes through (x, y) in one direction. The rest is free movement (back and forth on any edge), so you can always waste an *even* number of extra steps. The condition is: minimum route length $\le k$ and parity match.

➤ Key Observations

Observation 1. The edge must be used *exactly once*, so only two candidate walk lengths matter: $d_2 = \text{dist}(a, x) + 1 + \text{dist}(y, b)$ and $d_3 = \text{dist}(a, y) + 1 + \text{dist}(x, b)$. Each is computed via the distance-through-hinge formula.

Observation 2. Back-and-forth on any edge wastes exactly 2 steps, so $(k - d_i)$ must be even and ≥ 0. The answer is YES if *any* d_i works.

➤ Think It Through

Think it through with me

✗ **Tried:** BFS per query over the augmented graph
Killed by: $O(n)$ per query; $O(nq)$ total. TLE.

✓ **Aha!** Precompute LCA + distances in $O(n \log n)$. Each query is three distance computations (each $O(\log n)$) + a parity check.

➤ Implementation

Code 16.4.2: CF 1304E full solution (lifting distances + parity)

```cpp
#include <bits/stdc++.h>
using namespace std;
const int LOG = 17, MX = 100001;
vector<int> adj[MX];
int up[MX][LOG], dep[MX], tin[MX], tout[MX], tmr;

void dfs(int v, int p, int d) {
    tin[v] = ++tmr; dep[v] = d; up[v][0] = p;
```

```
    for (int j = 1; j < LOG; j++) up[v][j] = up[up[v][j-1]][j-1];
    for (int c : adj[v]) if (c != p) dfs(c, v, d+1);
    tout[v] = tmr;
}
bool is_anc(int u, int v){ return tin[u]<=tin[v]&&tin[v]<=tout[u]; }
int lca(int u, int v) {
    if (is_anc(u,v)) return u; if (is_anc(v,u)) return v;
    for (int j=LOG-1;j>=0;j--) if(!is_anc(up[u][j],v)) u=up[u][j];
    return up[u][0];
}
int dist(int u, int v){ int l=lca(u,v); return dep[u]+dep[v]-2*dep[l]; }

int main(){
    int n; scanf("%d", &n);
    for (int i=0;i<n-1;i++){int a,b;scanf("%d%d",&a,&b);adj[a].push_back(b);adj[b].
    ↪ push_back(a);}
    dfs(1,1,0);
    int q; scanf("%d", &q);
    while (q--) {
        int a,b,x,y,k; scanf("%d%d%d%d%d",&a,&b,&x,&y,&k);
        int d2 = dist(a,x)+1+dist(y,b);
        int d3 = dist(a,y)+1+dist(x,b);
        bool ok = false;
        if (d2 <= k && (k-d2)%2==0) ok = true;
        if (d3 <= k && (k-d3)%2==0) ok = true;
        puts(ok ? "YES" : "NO");
    }
}
```

➤ Trace on the Official Sample

CF 1304E: chain 1–2–3–4–5; all 5 queries verified

The edge (x, y) MUST be used exactly once. Two candidate walk lengths: $d_2 = \text{dist}(a, x) + 1 + \text{dist}(y, b)$ and $d_3 = \text{dist}(a, y) + 1 + \text{dist}(x, b)$. YES iff $d_i \le k$ and $(k - d_i)$ is even.

Depths: $d = (0, 1, 2, 3, 4)$. On a chain, $\text{lca}(a, b) = \min(a, b)$, so $\text{dist}(a, b) = |a - b|$.

Q1 (1 3 1 2 2): $d_2 = |1{-}1| + 1 + |2{-}3| = 0{+}1{+}1 = 2$; $d_3 = |1{-}2| + 1 + |1{-}3| = 1{+}1{+}2 = 4$. $k{=}2$: $d_2{=}2 \le 2$, $(2{-}2){=}0$ even $\Rightarrow$ YES. ✓

Q2 (1 4 1 3 2): $d_2 = |1{-}1|{+}1{+}|3{-}4| = 0{+}1{+}1 = 2$; $d_3 = |1{-}3|{+}1{+}|1{-}4| = 2{+}1{+}3 = 6$. $k{=}2$: $d_2{=}2 \le 2$, $(2{-}2){=}0$ even $\Rightarrow$ YES. ✓

Q3 (1 4 1 3 3): $d_2 = 2$; $d_3 = 6$. $k{=}3$: $d_2{=}2 \le 3$, $(3{-}2){=}1$ odd NO; $d_3{=}6 > 3$ NO. ✓

Q4 (4 2 3 3 9): $d_2 = |4{-}3|{+}1{+}|3{-}2| = 1{+}1{+}1 = 3$; d_3 same ($x{=}y{=}3$). $k{=}9$: $d_2{=}3 \le 9$, $(9{-}3){=}6$ even $\Rightarrow$ YES. ✓

Q5 (5 2 3 3 9): $d_2 = |5{-}3|{+}1{+}|3{-}2| = 2{+}1{+}1 = 4$; $d_3 = 4$ (same, $x{=}y$). $k{=}9$: $d_2{=}4 \le 9$, $(9{-}4){=}5$ odd NO; d_3 same NO. ✓

➤ Complexity

Complexity

Preprocessing: $O(n \log n)$ (lifting table + tin/tout via DFS). Per query: $O(\log n)$ (two distance computations, each needing an LCA). Total: $O((n+q)\log n)$. **AC.**

16.4.7 Common Pitfalls

The hinge value must be folded in once

The `agg` accumulation on each arm *excludes* the landing vertex (the hinge). If you forget $val(\ell)$, the answer misses one vertex. If you include it on both arms, you double-count. The template `lst:c11-pathagg` seeds $res = val[\ell]$ before the two loops – never change that.

Non-idempotent ops + the two-accumulator path

The overlapping-block trick (§16.2) needs $\oplus$ idempotent. The hinge-split *two-arm* technique does NOT, but it also does not support $\oplus$ that is only *commutative* (not associative). If $\oplus$ is non-commutative, you must combine the two arms in the correct order: left arm accumulated bottom-to-top, then $val(\ell)$, then right arm accumulated bottom-to-top *reversed* (i.e., read top-to-bottom). This is relevant for string concatenation and matrix multiplication.

CF 1304E: the extra edge is MANDATORY

Many early submissions also check $d_1 = \text{dist}(a, b)$ as a candidate. The problem requires traversing the new edge *exactly once*; you cannot skip it. The only valid walk lengths involve the +1 crossing.

16.4.8 Variations and Twists

- **XOR on a path.** $\oplus = xor$; identity is 0; not idempotent ($x \oplus x = 0$), so the sparse-table overlap trick would *fail* – but the two-arm method is fine. Verify: path $4 \to 8$ has nodes $(4, 2, 5, 8)$, values $(1, 3, 8, 2)$; xor $= 1 \oplus 3 \oplus 8 \oplus 2 = 12$? Actually $1 \oplus 3 = 2, 2 \oplus 8 = 10, 10 \oplus 2 = 8$. XOR=8. **L4**
- **Sum along a path.** Same as the distance formula when edge weights are involved; $\oplus = +$; non-idempotent, two-arm essential. For path $4 \to 8$ values $(1, 3, 8, 2)$, sum=14. **L3**
- **Count of vertices with property P along a path.** P is typically "value $\leq T$"; requires offline + persistent or merge-sort tree (Ch. 21), not just a lifting table. **L6**
- **Dynamic updates.** When edge values change between queries, `agg` goes stale. The upgrade is heavy-light + segment tree (Ch. 22) or Euler-tour + BIT (Ch. 18). CSES 2134 is the canonical problem. *Deferred by design.* **L6**

16.4.9 Practice Corner

Q16.4.1 Path Queries II (dynamic path max) – CSES 2134 CSES L5

Hint: maximum node value on a path *with* point updates: the `agg` table goes stale the moment a value changes, so the static two-arm accumulate of this section no longer applies. The intended tool is heavy-light + segment tree (Ch. 22 / Ch. 20). *Deferred by design* – this is the dynamic boundary the chapter deliberately leaves.

Back-cites. Ch. 13 (*Theorem* 13.1.2) – the fold is the same "combine children" idea; here the children are replaced by the two arms of a path. Ch. 19/20 (*Chapter* 19/*Chapter* 20) – dynamic path updates: forward-cite only; nothing solved here survives a value change. Ch. 22 (*Chapter* 22) – HLD decomposes a tree into chains so segment trees handle path-update; the static subsumes to what `agg` does here, but faster to code.

Section 16.4 in 10 seconds

`agg`$[v][j]$ carries an associative payload on each 2^j jump. Path queries split at the hinge (lca), accumulate each arm via bit decomposition, then fold in $val(\ell)$. Cost $O(\log n)$ per path query. Idempotence is NOT required (unlike the sparse-table overlap trick). Same table powers threshold descents ("first ancestor past T") and k-th-node-on-route. CF 1304E: add one edge (x, y) to a tree; a walk of exactly k edges from a to b exists iff $\text{dist}(a, x) + 1 + \text{dist}(y, b) \le k$ (or the a-y-x-b variant) and $(k - d)$ is even.

Self-test (answer from memory, tick the box):

- ☐ Write the recurrence for `agg`$[v][j]$.
- ☐ Why must you merge $val(\ell)$ separately?
- ☐ Max along path $8 \to 10$ on T with the given values? (9.)
- ☐ CF 1304E: why is $d_1 = \text{dist}(a, b)$ not a valid candidate?
- ☐ Give the k-th-node-on-route formula for $k > d_1$.

16.5 Distances, Paths and Depth Buckets

Four questions from 16.1–16.4

1. From *Example* 16.1.4: $\text{lca}(8, 10) = 1$. *(How many edges from 8 to 10 via the hinge?)*
2. From §16.2: the distance formula $\text{dist}(u, v) = \text{depth}(u) + \text{depth}(v) - 2\,\text{depth}(\text{lca})$ is one arithmetic expression given the LCA.
3. From §16.3: the k-th ancestor gives route positions (section 16.4's "k-th node on a route").
4. From Ch. 13 (§13.4): the identity $\text{dist}(s, u) + \text{dist}(u, v) = \text{dist}(s, v)$ iff u lies on the route $s \to v$. *(That is today's Proposition 16.5.3.)*

In This Section

The LCA is a *measurement hub*: once you have $\text{lca}(u, v)$, the distance follows for free (Theorem 16.5.1); whether a third node lies on the route follows from a triangle equality (Proposition 16.5.3); whether one node is an ancestor of another follows from the tin / tout interval test (Proposition 16.5.4, re-stated from Ch. 5 for convenience). Finally, *per-depth buckets* – for each depth d, a sorted list of tin values of all nodes at depth d – answer "how many descendants of the p-th ancestor of v sit at the same depth as v?" in $O(\log n)$, the key to CF 208E (Blood Cousins). This section closes the static-tool chapter: everything here is queryable with $O(n \log n)$ preprocessing and $O(\log n)$ per question, and the next chapter (Ch. 17 DSU-on-tree) builds on these tools.

After this section you can

- prove the distance-through-hinge theorem and compute any pairwise distance by hand on T; **L3**
- test whether w is on the route u-v using triangle equality (Proposition 16.5.3); **L3**
- apply the tin / tout subtree interval test (Proposition 16.5.4); **L2**
- build per-depth tin-bucket arrays and binary-search a subtree count in $O(\log n)$; **L4**
- solve CF 208E (Blood Cousins) end-to-end. **L4–L5**

16.5.1 Distance through the Hinge

Theorem 16.5.1 (Distance formula)

For any nodes u, v in a tree rooted at r (unweighted edges):

$$\operatorname{dist}(u, v) \;=\; \operatorname{depth}(u) + \operatorname{depth}(v) - 2\,\operatorname{depth}(\operatorname{lca}(u, v)).$$

For edge-weighted trees, replace depth by the weighted $\operatorname{dist}(r, \cdot)$ and the formula holds identically.

Proof

The unique route $u \to v$ passes through $\ell = \operatorname{lca}(u, v)$. The segment $u \to \ell$ has length $\operatorname{depth}(u) - \operatorname{depth}(\ell)$ (one edge per ancestor step). Symmetrically $\operatorname{depth}(v) - \operatorname{depth}(\ell)$. The two segments share only ℓ, so the total is their sum.

■

Example 16.5.2 (Distance table on T)

Using depth $= (0, 1, 1, 2, 2, 2, 2, 3, 3, 4)$ and the census for LCAs:

u	v	$\operatorname{lca}(u, v)$	$\operatorname{depth}(u) + \operatorname{depth}(v) - 2\operatorname{depth}(\text{lca})$	$\operatorname{dist}(u, v)$
8	10	1	$3 + 4 - 2(0)$	7
4	8	2	$2 + 3 - 2(1)$	3
9	10	9	$3 + 4 - 2(3)$	1
2	10	1	$1 + 4 - 2(0)$	5
5	6	1	$2 + 2 - 2(0)$	4
4	5	2	$2 + 2 - 2(1)$	2
7	10	7	$2 + 4 - 2(2)$	2
6	10	3	$2 + 4 - 2(1)$	4

Every row is verifiable by counting edges on the drawn tree.

16.5.2 On-Path Test (Triangle Equality)

Proposition 16.5.3 (w lies on the route u-v)

For three nodes u, v, w in a tree:

$$w \in \operatorname{path}(u, v) \iff \operatorname{dist}(u, w) + \operatorname{dist}(w, v) = \operatorname{dist}(u, v).$$

Proof

The forward direction: if w is on the route, removing w splits it into two segments of lengths $\operatorname{dist}(u, w)$ and $\operatorname{dist}(w, v)$ that exactly tile $\operatorname{dist}(u, v)$. The reverse: if the equality holds, the unique route $u \to v$ must pass through w because the shortest path metric in a tree satisfies the triangle inequality with equality iff the midpoint is on the geodesic.

■

On-path test on T: is 7 on path 8 to 10?

$\text{dist}(8,7) = 3+2-2(0) = 5$; $\text{dist}(7,10) = 2+4-2(2) = 2$; $\text{dist}(8,10) = 7$. Check: $5+2 = 7 = \text{dist}(8,10)$. $\Rightarrow$ 7 IS on the path $8 \to 10$. ✓

Is 6 on path $8 \to 10$? $\text{dist}(8,6) = 3+2-0 = 5$; $\text{dist}(6,10) = 2+4-2(1) = 4$; $5+4 = 9 \neq 7$. NO. ✓

16.5.3 Ancestry via tin/tout (Back-cite Ch. 5)

Proposition 16.5.4 (Subtree-as-interval test)

Under the entry-only DFS of Chapter 5 (*Chapter 5*), with $\text{tout}(v)$ = the last tin stamp inside v's subtree:

$$u \preceq v \iff \text{tin}(u) \leq \text{tin}(v) \leq \text{tout}(u).$$

Cost $O(1)$ per test after $O(n)$ preprocessing.

Note. This is the *same* tin / tout array printed in Figure 16.0.1, not the $2n-1$ tour of §16.2. Both coexist; they serve different purposes.

tin/tout test on T

$\text{tin} = (1,2,6,3,4,7,8,5,9,10)$ for nodes 1..10. $\text{tout} = (10,5,10,3,5,7,10,5,10,10)$.

Is $7 \preceq 10$? $\text{tin}(7) = 8 \leq \text{tin}(10) = 10 \leq \text{tout}(7) = 10$. YES. ✓Is $9 \preceq 6$? $\text{tin}(9) = 9 \leq \text{tin}(6) = 7$? $9 > 7$. NO. ✓Is $2 \preceq 8$? $\text{tin}(2) = 2 \leq \text{tin}(8) = 5 \leq \text{tout}(2) = 5$. YES. ✓

16.5.4 Per-Depth Buckets

For each depth d, maintain a sorted array $bucket[d]$ containing $\text{tin}(v)$ for every v with $\text{depth}(v) = d$. To count nodes at depth d that are in the subtree of w (i.e., $\text{tin}(w) \leq \text{tin}(v) \leq \text{tout}(w)$ and $\text{depth}(v) = d$):

$$\text{count} = \texttt{upper_bound}(bucket[d],\ \text{tout}[w]) - \texttt{lower_bound}(bucket[d],\ \text{tin}[w]).$$

Cost $O(\log n)$ per query (binary search in a sorted array).

Depth buckets on T (sorted by tin)

d	nodes at depth d	bucket[d] (tin values sorted)
0	{1}	[1]
1	{2, 3}	[2, 6]
2	{4, 5, 6, 7}	[3, 4, 7, 8]
3	{8, 9}	[5, 9]
4	{10}	[10]

Example: count nodes at depth 2 in subtree of 2 ($\text{tin}(2) = 2$, $\text{tout}(2) = 5$). Search $bucket[2] = [3,4,7,8]$ for values in $[2,5]$: $\{3,4\}$, count = 2 (nodes 4 and 5). ✓

16.5.5 Problem Lab

W16.5.1 Blood Cousins

Source: CF 208E CF 2100 L5
Techniques: binary lifting, k-th ancestor, depth buckets, subtree tin/tout

CF 208E Blood Cousins

A *forest* of boys is given by a father array of n integers (0 marks a root — the input need not be one tree); then q queries (v, p) with $p > 0$: how many boys $u \neq v$ have the same p-th ancestor as v? (Equivalently: let w be the p-th ancestor of v; count nodes at depth depth(v) inside subtree(w), minus v itself.) If $p >$ depth(v) the ancestor doesn't exist; the answer is 0.

Input. n, then a line of n integers $a_1, \ldots, a_n$ (a_i is the parent of node i; $a_i = 0$ means node i is a root). Then q, then q lines each with v and p. $1 \leq n, q \leq 10^5$.

Example.

```
Input:
6
0 1 1 0 4 4
7
1 1
1 2
2 1
2 2
4 1
5 1
6 1

Output:
0 0 1 0 0 1 1
```

Attempt it yourself first

Peek (1 min lost): Identify the root (the node with parent 0), build depth and tin / tout, then for each query climb p steps via lifting.

Nudge (5 min lost): After finding $w = p$-th ancestor of v, count nodes at depth(v) in w's subtree using the bucket + binary search.

Full hint (you will not solve this one): Answer = (upper_bound - lower_bound in bucket[depth(v)] restricted to [tin(w), tout(w)]) - 1.

➤ Understanding

The query is "how many people are at my generation (same depth) and share the p-th ancestor (same great. . .grandparent)?" The translation: count same-depth nodes in the ancestor's subtree, minus yourself. Three operations: k-th ancestor (lifting), depth lookup, range count in a bucket (binary search).

➤ Key Observations

Observation 1. The input is a *forest* — multiple nodes may have parent 0 (the sample has roots 1 and 4). A super-root 0 glues all real roots as children; depths shift by 1 but the algorithm is unchanged.

Observation 2. If $p \geq \text{depth}(v)$ (depth of v from its real root) the climb overshoots; answer 0. Guard before calling `kth_ancestor`.

➤ Think It Through

Think it through with me

✗ **Tried:** DFS from v counting same-depth nodes
Killed by: $O(n)$ per query; $O(nq)$ total. TLE.

✗ **Tried:** Precompute a 2D array of subtree depth histograms
Killed by: Space $O(n^2)$. MLE.

✓ **Aha!** Bucket nodes by depth (sorted by tin). Per query: find w (p-th ancestor, $O(\log n)$), then binary-search bucket[depth(v)] for the interval [tin(w), tout(w)]. $O(\log n)$ per query, $O(n)$ space.

➤ Implementation

Code 16.5.1: CF 208E full solution (super-root 0; lifting + depth buckets)

```
#include <bits/stdc++.h>
using namespace std;
const int LOG=17, MX=100002;
vector<int> adj[MX], bucket[MX];
int up[MX][LOG], dep[MX], tin[MX], tout[MX], tmr;

void dfs(int v, int p) {
    tin[v]=++tmr; up[v][0]=p;
    for(int j=1;j<LOG;j++) up[v][j]=up[up[v][j-1]][j-1];
    bucket[dep[v]].push_back(tin[v]);   // sorted by construction
    for(int c:adj[v]) if(c!=p){dep[c]=dep[v]+1; dfs(c,v);}
    tout[v]=tmr;
}
int kth(int v,int k){
    for(int j=0;j<LOG;j++) if(k&(1<<j)) v=up[v][j];
    return v;
}
int main(){
    int n; scanf("%d",&n);
    dep[0]=-1;                          // super-root 0
    for(int i=1;i<=n;i++){
        int p; scanf("%d",&p);
        if(p==0) adj[0].push_back(i);    // root -> child of super-root
        else adj[p].push_back(i);
    }
    up[0][0]=0; for(int j=1;j<LOG;j++) up[0][j]=0;  // sentinel row
    dep[0]=0; dfs(0,0);                  // one pass over the forest
    // dep of real roots = 1 (one below super-root); adjust if needed
    int q; scanf("%d",&q);
    while(q--){
        int v,p; scanf("%d%d",&v,&p);
        if(p>dep[v]-1){printf("0 ");continue;} // overshoot guard
        int w=kth(v,p);
        int d=dep[v];
        int lo=lower_bound(bucket[d].begin(),bucket[d].end(),tin[w])-bucket[d].begin()
    ↪ ;
```

```
36          int hi=upper_bound(bucket[d].begin(),bucket[d].end(),tout[w])-bucket[d].begin
        ↪ ();
37          printf("%d ",hi-lo-1);
38      }
39  }
```

➤ Trace on the Official Sample

CF 208E: forest 0 1 1 0 4 4; all 7 queries verified

Forest structure: root 1 has children 2,3; root 4 has children 5,6. Depths (from each real root): depth $= (0, 1, 1, 0, 1, 1)$ for nodes 1..6.

Q1 ($v=1$, $p=1$): $p=1 >$ depth$(1)=0$; overshoot $\Rightarrow$ **0**. ✓

Q2 (1, 2): $p=2 >$ depth$(1)=0$; overshoot $\Rightarrow$ **0**. ✓

Q3 (2, 1): 1st anc of 2 is 1. Depth 1 nodes in subtree$(1) = \{2, 3\}$; count $2 - 1 =$ **1**. ✓

Q4 (2, 2): $p=2 >$ depth$(2)=1$; overshoot $\Rightarrow$ **0**. ✓

Q5 (4, 1): $p=1 >$ depth$(4)=0$; overshoot $\Rightarrow$ **0**. ✓

Q6 (5, 1): 1st anc of 5 is 4. Depth 1 nodes in subtree$(4) = \{5, 6\}$; count $2 - 1 =$ **1**. ✓

Q7 (6, 1): 1st anc of 6 is 4. Depth 1 nodes in subtree$(4) = \{5, 6\}$; count $2 - 1 =$ **1**. ✓

Output: `0 0 1 0 0 1 1`. ✓

Example 16.5.5 (Cousin-count queries on T)

On T (depths $d = (0, 1, 1, 2, 2, 2, 2, 3, 3, 4)$):

v	p	$w = p$-th anc of v	depth = depth(v) nodes in subtree(w)	ans
8	1	5	depth 3 in sub(5)={5,8}: {8}	0
9	1	7	depth 3 in sub(7)={7,9,10}: {9}	0
4	1	2	depth 2 in sub(2)={2,4,5,8}: {4,5}	1
4	2	1	depth 2 in sub(1)=all: {4,5,6,7}	3
8	2	2	depth 3 in sub(2)={2,4,5,8}: {8}	0
10	2	7	depth 4 in sub(7)={7,9,10}: {10}	0
10	3	3	depth 4 in sub(3)={3,6,7,9,10}: {10}	0
6	2	1	depth 2 in sub(1)=all: {4,5,6,7}	3
1	1	overshoot	–	0

All hand-verified. The last row is the overshoot guard ($p = 1 >$ depth$(1) = 0$: answer 0).

➤ Complexity

Complexity

Preprocessing: $O(n \log n)$ (lifting table) + $O(n)$ (bucket build, sorted since tin increases during DFS in depth order). Per query: $O(\log n)$ (climb) + $O(\log n)$ (binary search). Total $O((n + q) \log n)$. **AC.**

16.5.6 Common Pitfalls

Depth-0 nodes and the overshoot guard

If $p > \text{depth}(v)$, the p-th ancestor is the sentinel 0 (or undefined). Querying $bucket[\text{depth}(v)]$ for the interval $[\text{tin}(0), \text{tout}(0)]$ would count everything (or crash). Always check $p > \text{depth}(v)$ FIRST and return 0.

The -1 (exclude v itself) must come after the count

If v is in w's subtree (always true when w is an ancestor of v), the binary-search count ≥ 1. Forgetting the -1 counts v as its own "cousin". The answer is the range size minus 1.

Mixing tin/tout conventions

The bucket stores $\text{tin}(v)$ values. The interval test for subtree is $\text{tin}(w) \leq x \leq \text{tout}(w)$ (last-stamp convention from Ch. 5). If your DFS uses the "exit tick" convention ($\text{tout}(w) =$ timestamp at the moment of exiting), the interval upper-bound is off by 1 and you over-count. Pin ONE convention.

16.5.7 Variations and Twists

- **LC 236 (Ch. 1 revisit).** The binary-tree LCA from LeetCode 236 is *Example* 16.1.4 restricted to a *binary* tree where the "walk" fits in a post-order recursion. Prose revisit only (the URL lives in Ch. 1's mixed set). **L2**
- **CSES 1136 Tree Distances II.** A "sum of distances from v to all other nodes" problem that combines Ch. 15's rerooting with Theorem 16.5.1: the per-root sum is one reroot-every-node quantity. Prose revisit (URL owned by Ch. 12 notes). **L5**
- **On-path count via tin/tout.** Given the route $u \to v$, the set $\{w : \text{on-path}\}$ is not a subtree interval; but $\{w : \text{on-path and in subtree}(w_0)\}$ is. Useful in some CF 960-family queries. **L5**

16.5.8 Practice Corner

Q16.5.1 Find Distance in a Binary Tree – LC 1740 LC Medium L3

Hint: distance between two valued nodes: locate both, take their hinge, apply $\text{dist}(u, v) = \text{depth}\, u + \text{depth}\, v - 2\,\text{depth}\, \ell$. One DFS records parent/depth and LC 236's LCA gives the hinge – the binary-tree dress-up of Theorem 16.5.1.

Back-cites. Ch. 5 (*Chapter* 5) – owns tin / tout flattening; we use Proposition 16.5.4 as a restatement, never re-derive. Ch. 13 (§13.4) – the triangle-equality identity used by Proposition 16.5.3; cited, not re-proved. Ch. 15 (*Chapter* 15) – rerooting converts per-root quantities; CSES 1136 needs the two-DFS template + distance.

Section 16.5 in 10 seconds

Theorem 16.5.1: $\text{dist}(u, v) = \text{depth}(u) + \text{depth}(v) - 2\,\text{depth}(\text{lca})$. Proposition 16.5.3: w on route u-v iff $\text{dist}(u, w) + \text{dist}(w, v) = \text{dist}(u, v)$. Proposition 16.5.4: $u \preceq v$ iff $\text{tin}(u) \leq \text{tin}(v) \leq \text{tout}(u)$ (Ch. 5). Per-depth buckets: $bucket[d]$ sorted tins; subtree count via binary search. CF 208E: k-th ancestor + bucket range – count depth-d nodes in w's subtree, minus 1.

Self-test (answer from memory, tick the box):

☐ dist$(8, 10)$ on T? (7.) Show the arithmetic.
☐ State Proposition 16.5.3 and test node 6 on path $8 \to 10$.
☐ Bucket[2] on T? ($[3, 4, 7, 8]$.)
☐ CF 208E query (4,2): what is w? What count minus 1? (1, 4-1=3.)
☐ What is the overshoot guard condition?

Chapter 16 Summary

One sentence: *a fixed root turns every pair of nodes into a unique hinge—the LCA—and three machines (the walk, the Euler-tour RMQ, the binary-lifting table) answer "where is the hinge?" in $O(h)/O(1)/O(\log n)$; once you have the hinge, distances, on-path tests, depth-bucket counts, and route-position queries all become one arithmetic expression.* Chapter 13 made the subtree fold queryable; Chapter 16 makes the *pair* queryable. Every row below is a pointer, not a re-derivation.

Result	Statement	Where
Chain property	Common ancestors of u, v = ancestors of $\text{lca}(u, v)$; a single chain.	Proposition 16.1.3
Two-phase walk	Equalise depths, lockstep up; first agreement is ℓ; $O(h)$.	§16.1
Euler–RMQ reduction	$\text{lca}(u, v) = \arg\min$ depth on the $2n-1$ tour between *first*(u) and *first*(v); sparse table gives $O(1)$.	Theorem 16.2.2, Definition 16.2.3
Binary lifting table	up$[v][j]$ = up[up$[v][j-1]][j-1]$; $O(\log n)$ climb and LCA.	Definition 16.3.1
Aggregated lifting	agg$[v][j]$ carries an associative payload; path aggregate in $O(\log n)$.	Theorem 16.4.1
Distance identity	$\text{dist}(u, v) = \text{depth}(u) + \text{depth}(v) - 2\,\text{depth}(\ell)$.	Theorem 16.5.1
On-path test	$w \in \mathit{path}(u, v) \iff \text{dist}(u, w) + \text{dist}(w, v) = \text{dist}(u, v)$.	Proposition 16.5.3
Subtree interval	$u \preceq v \iff \text{tin}(u) \le \text{tin}(v) \le \text{tout}(u)$; counts = binary search.	Proposition 16.5.4
Depth buckets	Sorted tin per depth; subtree count in $O(\log n)$ via two searches.	§16.5

Algorithms from this chapter.

Algorithm	Prep	Query	Code
Naive walk	$O(n)$ DFS	$O(h)$	Code 16.1.1
Euler tour + sparse table	$O(n \log n)$	$O(1)$	Code 16.2.1, Code 16.2.2
Binary lifting (build/climb/LCA)	$O(n \log n)$	$O(\log n)$	Code 16.3.1, Code 16.3.3
CSES 1135 (Distance Queries)	$O(n \log n)$	$O(\log n)$	Code 16.2.3
CSES 1687 (Company Queries I)	$O(n \log n)$	$O(\log k)$	Code 16.3.4
CF 1304E (route + parity)	$O(n \log n)$	$O(\log n)$	Code 16.4.2
CF 208E (depth buckets)	$O(n \log n)$	$O(\log n)$	Code 16.5.1
Path aggregate by strides	$O(n \log n)$	$O(\log n)$	Code 16.4.1

Master Decision Flowchart

Post-study ladder: walk top to bottom, stop at the first branch that matches. It decides *which of this chapter's three LCA machines* the problem is asking for—and hands off what this chapter does *not* own: dynamic path updates (Ch. 19/20/22 HLD).

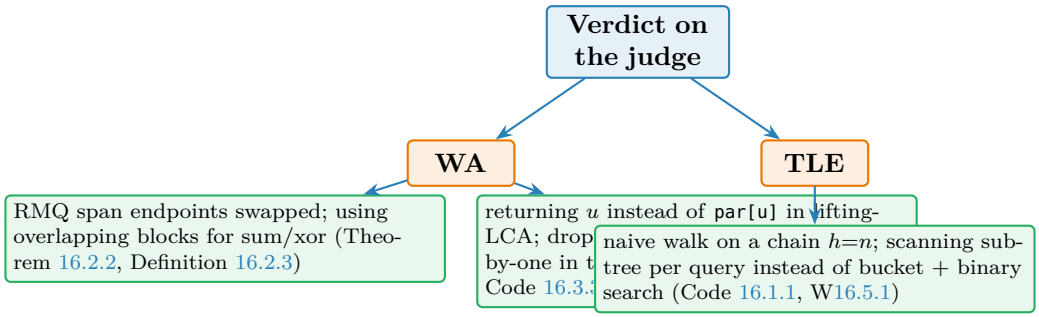
```
You need the LCA, or a measurement that uses it.  Which machine?
|
|-- h small, or q tiny (one-off / LC 236 / LC 1483)?
|     +-- naive walk (16.1, lst:c11-naive); O(h), no build.
|
|-- tree STATIC, q large, ONLY need LCA (CSES 1135)?
|     +-- Euler tour + sparse table (16.2, lst:c11-euler +
|        lst:c11-sparse); O(1) query, idempotent combine.
|
|-- need k-th ancestor, distance, on-path, OR log is fast enough?
|     +-- binary lifting (16.3, lst:c11-lcalift); O(log) per query.
|        (Contest default: most flexible, simplest to extend.)
|
|-- need an AGGREGATE along the path (max/min/sum/xor)?
|     +-- aggregated lifting (16.4, thm:c11-agg, lst:c11-pathagg).
|        Associative op? Use two accumulators on the two arms.
|        Idempotent? Overlapping blocks also work (sparse trick).
|
|-- query NAMES a SET of vertices (LCA of k nodes)?
|     +-- RMQ-on-tin: sort by tin, LCA(set) = LCA(min-tin, max-tin).
|        Segment tree over tin for range argmin/argmax -> Summit.
|
|-- values CHANGE between queries (dynamic)?
|     +-- NOT this chapter -- HLD (Ch.~22) + segment tree (Ch.~19/20).
|        Static tables rot; no pre-merged jump survives an update.
|
+-- distance DISTRIBUTION over all pairs?
      +-- NOT this chapter -- centroid (Ch.~24). Per-query LCA
          is too slow when you need O(n^2) pairs.
```

Common Mistakes Checklist

- **Forgetting `return par` at the end of lifting-LCA.** The descending loop stops with u, v one level below ℓ; returning u gives a child—silent off-by-one on ancestor-pair tests (Code 16.3.3).
- **Dropping the `if(u==v)` shortcut after equalising.** With $u = v$ the inequality-loop never fires; without the shortcut the `par` return is wrong by one edge (Code 16.3.3).
- **Swapping RMQ endpoints.** The span is $[\min(\mathit{first}(u), \mathit{first}(v)),\ \max(\ldots)]$; feeding $(17, 14)$ instead of $(14, 17)$ gives garbage (Theorem 16.2.2).
- **Overlapping blocks for non-idempotent ranges.** The sparse-table two-block trick requires $x \oplus x = x$: fine for min/max/gcd, *corrupt* for sum/xor/count (Definition 16.2.3).
- LOG **one short.** Need $2^{LOG} > n$; with $n \le 2 \cdot 10^5$, $LOG = 18$ fails—19 is the minimum. Bitloops and the table second index must both reach LOG (Definition 16.3.1).
- **Sentinel discipline.** Node 0 must exist in the arrays (depth[0] $= 0$, `up`$[0][j] = 0$); otherwise overshoots read a real vertex (Code 16.3.1).
- **Forests treated as trees.** CF 208E's father array has multiple roots: forget the super-root 0 or the $p >$ depth(v) guard and DFS misses components (Code 16.5.1).
- **Tin/tout convention cross-wired.** "tout(v) = last stamp inside v's subtree" and "tout(v) = exit-time tick" are both valid—neither works mixed (Proposition 16.5.4).
- **Stack depth on a chain.** Every build here is a DFS on up to $2 \cdot 10^5$ nodes; the iterative two-phase rewrite from Ch. 12's Code 12.1.2 is production-necessary.

WA Clinic

Verdicts are symptoms; the causes are the short list this chapter drilled. Diagnose top-down—every leaf names the training problem.



WA: the hinge returns a child, or the RMQ window is inverted. TLE: the walk pays $O(h)$ on a chain, or the per-query scan ignores the bucket structure. RE: LOG too small causes array overflow on deep trees.

Quick Reference Card

Chapter 16 at a glance

Definitions. ancestors(v): route to root, self included; lca(u, v): deepest common element; lca$(u, v) = u \iff u \preceq v$; `par`$(r) = 0$ sentinel.

Walk. Lift deeper to equal depth, lockstep; $O(h)$. Check h before building tables.

Euler-RMQ. *tour*: enter + each return, $2n-1$ slots; $\mathit{first}(v)$: first slot. lca = arg min depth on $[\mathit{first}(u)..\mathit{first}(v)]$. Sparse table over tour depth: query = two overlapping 2^j-windows; $j = \lfloor \log_2 \mathit{len} \rfloor$. $O(n \log n)$ prep / $O(1)$ query.

Lifting. up$[v][j]$ = up[up$[v][j-1]][j-1]$; climb k: jump bits low→high. LCA: equalise, shortcut, descend high→low on *inequality*, return par. $O(n \log n)$ / $O(\log)$. Ancestry: $\text{tin}(a) \le \text{tin}(v) \le \text{tout}(a)$.

Paths. $\text{dist}(u,v) = \text{depth}\, u + \text{depth}\, v - 2\,\text{depth}\, \ell$; $w \in \mathit{path}(u,v) \iff \text{dist}(u,w) + \text{dist}(w,v) = \text{dist}(u,v)$; k-th route node: climb from nearer end. Aggregates: second table agg$[v][j]$, merge into accumulator during climbs.

Translation. "boss k above" → climb. "distance / via / route" → hinge identity. "same-generation relatives under ancestor" → lift + depth-bucket ∩ interval (CF 208E). "LCA of a SET of k nodes" → sort by tin, LCA(min-tin, max-tin) (Summit).

Code 16.5.2: Chapter 16 kit: the lifting machine, complete and production-shaped

```
struct LCA {   // binary lifting, 0-sentinel, iterative DFS
    int n, LOG; vector<vector<int>> adj;
    vector<array<int,20>> up;
    vector<int> dep, tin, tout; int tmr=0;
    LCA(int n):n(n),LOG(1),adj(n+1),up(n+1),dep(n+1),tin(n+1),tout(n+1){
        while((1<<LOG)<=n) LOG++;}
    void addEdge(int a,int b){adj[a].push_back(b);adj[b].push_back(a);}
    void build(int r=1){
        vector<int> par(n+1),it(n+1,0),stk{r};
        up[r].fill(0);dep[r]=0;par[r]=0;tin[r]=++tmr;
        while(!stk.empty()){int v=stk.back();
            if(it[v]<(int)adj[v].size()){int u=adj[v][it[v]++];
                if(u==par[v])continue;
                par[u]=v;dep[u]=dep[v]+1;tin[u]=++tmr;
                up[u][0]=v;for(int j=1;j<LOG;j++)up[u][j]=up[up[u][j-1]][j-1];
                stk.push_back(u);
            }else{tout[v]=tmr;stk.pop_back();}}}
    int climb(int v,int k)const{
        for(int j=0;j<LOG&&v;j++)if(k>>j&1)v=up[v][j];return v;}
    bool isAnc(int a,int v)const{
        return tin[a]<=tin[v]&&tin[v]<=tout[a];}
    int lca(int u,int v){
        if(dep[u]<dep[v])swap(u,v);
        u=climb(u,dep[u]-dep[v]);if(u==v)return u;
        for(int j=LOG-1;j>=0;j--)if(up[u][j]!=up[v][j]){u=up[u][j];v=up[v][j];}
        return up[u][0];}
    int dist(int u,int v){int w=lca(u,v);return dep[u]+dep[v]-2*dep[w];}
    bool onPath(int w,int u,int v){
        return dist(u,w)+dist(w,v)==dist(u,v);}
};
// recognition: "k above"->climb; "ancestor?/inside subtree?"->isAnc;
// "distance/via/route"->lca()+dist(); updates->NOT this kit: Ch.22.
```

Write-From-Memory Drills

Five signature skeletons, each lifted from a listing already in the chapter. Cover the marked line, reproduce from memory, then check the answer one line below.

Code 16.5.3: Drill 1 – the naive walk (from lst:c11-naive)

```
int lca_walk(int u, int v) {
```

```
    while (depth[u] > depth[v]) u = par[u];   // equalise u
    while (depth[v] > depth[u]) v = par[v];   // equalise v
    while (u != v) { u = par[u]; v = par[v]; } // lockstep
    return u;                                  // first agreement
}
```

Key: no preprocessing; $O(h)$ worst case. Correct for small h or tiny q; otherwise TLE on a chain. This is the template that 16.2 and 16.3 accelerate.

Code 16.5.4: Drill 2 – sparse-table RMQ query (from lst:c11-sparse)

```
int rmq_query(int l, int r) {               // min index in [l, r]
    int j = 31 - __builtin_clz(r - l + 1); // floor(log2(len))
    int a = st[j][l], b = st[j][r-(1<<j)+1];
    return (dep_tour[a] <= dep_tour[b]) ? a : b;
}
```

Key: two overlapping windows of size 2^j. The overlap is harmless ONLY because min is idempotent. For sums/xors, this trick is WRONG—use a segment tree or a disjoint sparse table instead (Definition 16.2.3).

Code 16.5.5: Drill 3 – lifting LCA (from lst:c11-lcalift)

```
int lca_lift(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    u = climb(u, dep[u] - dep[v]);
    if (u == v) return u;                      // the shortcut
    for (int j = LOG-1; j >= 0; j--)
        if (up[u][j] != up[v][j]) { u = up[u][j]; v = up[v][j]; }
    return up[u][0];                           // the par
}
```

Key: the loop stops with u, v as *children* of ℓ. Both lines (`if(u==v)` and `return up[u][0]`) are the two most-frequently dropped lines in contest submissions (Code 16.3.3).

Code 16.5.6: Drill 4 – path aggregate (from lst:c11-pathagg)

```
int path_query(int u, int v, int l) {    // l = lca(u,v)
    int res = val[l];                    // fold hinge value
    int cur = u, k = dep[u] - dep[l];
    for (int j = 0; j < LOG; j++)
        if (k & (1<<j)) { res = op(res, agg[cur][j]); cur = up[cur][j]; }
    cur = v; k = dep[v] - dep[l];
    for (int j = 0; j < LOG; j++)
        if (k & (1<<j)) { res = op(res, agg[cur][j]); cur = up[cur][j]; }
    return res;
}
```

Key: the hinge's own value is folded FIRST (the two arms *exclude* the landing vertex). Associativity is all $\oplus$ needs—no commutativity, no idempotence (Theorem 16.4.1, Example 16.4.2).

Code 16.5.7: Drill 5 – depth-bucket query (from lst:c11-208e-soln)

```
// Query (v, p): count nodes at depth dep[v] in subtree(w) minus v
int w = climb(v, p);                     // p-th ancestor
int d = dep[v];
int lo = lower_bound(bucket[d].begin(), bucket[d].end(), tin[w]) - bucket[d].begin();
int hi = upper_bound(bucket[d].begin(), bucket[d].end(), tout[w]) - bucket[d].begin();
```

```
6 answer = hi - lo - 1;                    // subtract v itself
```

Key: *bucket*[*d*] is *sorted by construction* (DFS stamps tins in order). The -1 excludes v. The guard `if(p > dep[v]) return 0` must precede the `climb` call (W16.5.1).

Complete Problem Checklist

Every problem this chapter offers—4 fully solved in-section, practice entries, one Summit, ten mixed. Tick *only* after an unaided solve; a hinted attempt counts as half.

#	Problem	Source	Key technique	What to look for	Done?
Section 16.2 – Euler tour + RMQ					
S16.2.1	Distance Queries	CSES 1135	**L3** Euler tour, sparse table, dist formula	$2n-1$ tour; RMQ for LCA; distance is one expression	☐
Section 16.3 – binary lifting					
S16.3.1	Company Queries I	CSES 1687	**L3** k-th ancestor via bit decomposition	up-table; sentinel; -1 on overshoot	☐
Section 16.4 – lifting an aggregate					
S16.4.1	1-Trees and Queries	CF 1304E	**L4** distances + parity + k-th node on route	Edge used *exactly once*; only d_2, d_3; parity check	☐
Section 16.5 – distances, buckets					
S16.5.1	Blood Cousins	CF 208E	**L4** lifting + depth buckets + tin interval	Forest (super-root); guard; range binary search	☐
Summit (chapter end)					
S16	Company	CF 1062E	**L5** RMQ-on-tin + LCA + segment tree	LCA of set = LCA(extremes by tin); remove extreme	☐

Recommended Study Order

Five focused days. Day 1 internalises the hinge concept and the naive walk; day 2 builds the $2n-1$ tour and sparse table; day 3 constructs the up-table and solves lifting problems; day 4 extends to aggregates and routes; day 5 is the Summit.

Day	Focus	Tasks
1	Read + do 16.1	verify Example 16.1.4 by hand on T; write Code 16.1.1 cold; LC 236 (Ch.1 revisit); trace lca$(8, 10)$ walk
2	Read + do 16.2	build the $2n-1$ tour table on T by hand; verify *first*[·]; solve CSES 1135 end-to-end
3	Read + do 16.3	fill the up-table on T (Example 16.3.3); CSES 1687; write Code 16.3.3 from memory
4	Read + do 16.4+16.5	Example 16.4.2 (max on path); CF 1304E; Example 16.5.5 (bucket queries); CF 208E
5	Summit + revise	CF 1062E; walk the decision flowchart; 10 mixed problems (first 3 minimum)

Difficulty Progression

Tiers are competency gates—each row names the *skill* the tier unlocks.

Tier	Problems	What passing this tier proves
L2–L3 ★★★	LC 236, CSES 1135, CSES 1687	you can build the tour table or the up-table from a DFS, and answer lca/k-th-ancestor without hints
L4 ★★★★	CF 1304E, CF 208E	you combine lifting + distances + interval queries; recognise when a path question needs the hinge split
L5 ★★★★★	CF 1062E (Summit), CF 587C, CF 629E	you layer RMQ-on-tin + range-argmin/max + LCA on a set; handle forest input + edge-case guards

Reduction Gallery

Every entry is "one of the three machines + one twist." Each line collapses a problem family onto a listing already in the chapter.

- "LCA of a SET of k nodes" (CF 1062E, CSES 1688) → sort by tin; the set's LCA is the LCA of the nodes with minimum and maximum tin (Proposition 16.5.4 as the proof key).
- "Distance a-b with one edge MUST be used" (CF 1304E) → three candidate distances + parity check (Theorem 16.5.1, W16.4.1).
- "k-th ancestor / boss k above" (CSES 1687, LC 1483) → bit-decomposition climb (Code 16.3.2, Definition 16.3.1).
- "Count same-depth nodes in ancestor's subtree" (CF 208E) → climb + bucket + binary search (Code 16.5.1, Proposition 16.5.4).
- "Max/min/sum/xor along a path" (CSES 2134 static) → aggregated lifting, two arms merge at hinge (Theorem 16.4.1, Code 16.4.1).
- "k-th node on route $u \to v$" (CF 587C preview) → distance + hinge split + k-th ancestor (Proposition 16.5.3, §16.4).

Mixed Problem Set

Ten entries from CF 1800 to CF 2400. Each refuses to say "LCA" directly—recognise the hinge from the *measurement* the problem requests.

M16.1 Tree Reconstruction – CF 178B3 (Div. 3 E) CF 1800 L3

Hint: Multiple DFS-order queries on a static tree; identify parent-child from tin / tout range containment—this chapter's Proposition 16.5.4 as a decision procedure.

M16.2 Tree – CF 191C CF 1900 L3

Hint: Paths on a tree: given k routes (a_i, b_i), count how many times each edge is traversed. Difference at $\ell_i = \text{lca}(a_i, b_i)$; one subtree-sum DFS finishes in $O(n+k)$ (Theorem 16.5.1).

M16.3 Wizards and Roads – CF 16D CF 1900 L4

Hint: Detect bridges via DFS lowlink; the "is this tree edge bridgeable?" test uses tin / tout interval containment exactly as Proposition 16.5.4 does.

M16.4 Alice and the Tree – CF 1702G2 CF 2000 L4

Hint: XOR of values along a path $u \to v$: compute via $xor(root, u) \oplus xor(root, v) \oplus val(\ell)$—Theorem 16.4.1 with $\oplus$ = xor.

M16.5 A and B and Team Training – CF 519E CF 2100 L4

Hint: "How many pairs have the same k-th ancestor?"—depth-bucket counting via lifting, exactly the CF 208E pattern (Code 16.5.1) with the bucket on both sides.

M16.6 Minimum Incompatibility – CF 609E CF 2100 L4

Hint: Second-MST via max-on-path: add edge (u, v) to the MST, find the heaviest edge on the $u \to v$ path (Theorem 16.4.1 with max), swap.

M16.7 Persistent Bookcase – CF 466E CF 2100 L5

Hint: LCA queries on a tree built incrementally; offline queries reduce to Code 16.3.3 after the tree is complete.

M16.8 Duff in the Army – CF 587C CF 2200 L5

Hint: Shortest path through army bases on a tree: LCA + distance (Theorem 16.5.1) + k-th-node-on-route (Code 16.3.2).

M16.9 Tree Queries – CF 629E CF 2300 L5

Hint: Ancestor-path queries: "is u an ancestor of v?" + "path passes through which special nodes?" tin / tout interval + bucket counting (Proposition 16.5.4).

M16.10 Blood Cousins Returns – CF 246E CF 2400 L5

Hint: Extended cousin counting with multiple depth constraints: the CF 208E bucket+climb pattern combined with DSU-on-tree for per-node answers (W16.5.1, Ch. 17 forward-cite).

The Summit – CF 1062E Company

Why this is the chapter top rope

Sections 16.1–16.5 each solve one query per pair (u, v). The Summit asks a *set* question: given a range $[l, r]$ of $r-l+1$ vertices, which single vertex to *remove* so the LCA of the remaining has maximum depth? The key reduction uses this chapter's tin / tout machinery: lca(set) = lca(min-tin, max-tin), so the problem becomes "find the two extreme tin-vertices in a range, then test removing each extreme." The segment tree for range-argmin/argmax is Chapter 19's tool; the LCA inside it is *this* chapter's.

W16.5.2 Company

Source: Codeforces 1062E CF 2300 L5

Techniques: LCA of a set via tin-extremes, segment tree for range argmin/argmax, binary lifting, remove-one optimization

CF 1062E Company

A company of n employees forms a rooted tree (employee 1 is the CEO, root). q queries: each gives a range $[l, r]$—employees $l, l+1, \ldots, r$ are involved. The project manager is the deepest common ancestor (= LCA) of all involved employees. Remove *exactly one* employee from the range so that the LCA of the remaining employees is as *deep* as possible. Output which employee to remove and the maximum depth achievable.

Input. n q; then $n-1$ integers $p_2, p_3, \ldots, p_n$ (the boss of each employee); then q lines (l_i, r_i). $2 \le n, q \le 10^5$.

Example.

```
Input:
11 5
1 1 3 3 3 4 2 7 7 6
4 6
4 8
```

```
1 11
9 11
8 11

Output:
4 1
8 1
1 0
11 3
8 1
```

Attempt it yourself first – L5, 45 minutes

Peek (1 min lost): How do you compute the LCA of a SET of k nodes? Hint: sort them by tin; what happens if you take the LCA of the first and last?

Nudge (5 min lost): The segment tree stores (min-tin, max-tin) in each interval. For the "remove one" variant, also query for the *second* extreme: second-min and second-max. Only removing an extreme vertex can change the LCA of the set.

Full hint (you will not solve this one): For each range $[l, r]$: find $a = \arg\min$ tin and $b = \arg\max$ tin in the range. The LCA of the whole range = $\text{lca}(a, b)$. To remove one vertex: try removing a (compute $\text{lca}(\text{second-min}, b)$) or removing b (compute $\text{lca}(a, \text{second-max})$); pick whichever gives deeper LCA. Output the removed vertex and the new depth. Total $O((n+q)\log n)$.

➤ Understanding

The set $\{l, l{+}1, \ldots, r\}$ (consecutive vertex indices) has its LCA determined by the two vertices with extreme DFS-entry times. This is the "LCA of a set" reduction:

$$\text{lca}(v_1, \ldots, v_k) \;=\; \text{lca}\big(\arg\min_i \text{tin}(v_i),\; \arg\max_i \text{tin}(v_i)\big).$$

The proof uses Proposition 16.5.4: if u has the smallest tin and w the largest, then every other v_i has $\text{tin}(u) \le \text{tin}(v_i) \le \text{tin}(w)$, so all v_i's are in $\text{lca}(u, w)$'s subtree. Removing one vertex changes the extremes only if that vertex *is* an extreme—hence try removing the min-tin or the max-tin vertex.

➤ Key Observations

Observation 1. The LCA of a set = LCA of the min-tin and max-tin elements. This turns a k-way query into one pair query, answering in $O(\log n)$ via binary lifting.

Observation 2. Removing a non-extreme vertex does NOT change the LCA (the extremes still bracket the same set). So the optimal removal is always one of the two extremes.

Observation 3. Second-min and second-max tin in the range give the two candidate LCAs after removal. A segment tree storing (min, max, second-min, second-max) in each node answers all four in $O(\log n)$ per range query.

➤ Think It Through

Think it through with me

✗ **Tried:** Compute LCA of all k vertices by pairwise LCA iteration
Killed by: Correct for one query, but $O(k)$ per query; $O(nq)$ worst case.

✗ **Tried:** Sort by tin, take first and last—no segment tree
Killed by: Works only if the set is pre-sorted; here the set changes per query (range of indices, not DFS order). You need a *range* data structure.

✓ **Aha!** Segment tree over $\{1,\ldots,n\}$ storing min/max/second- min/second-max of tin[·]. Per query: four values from the tree in $O(\log n)$; one LCA call in $O(\log n)$. Total $O((n+q)\log n)$.

➤ Implementation

Code 16.5.8: CF 1062E: segment tree on tin + binary lifting LCA

```
#include <bits/stdc++.h>
using namespace std;
const int MX=100002, LOG=18;
vector<int> adj[MX];
int up[MX][LOG], dep[MX], tin[MX], tout[MX], tmr;
void dfs(int v,int p){
    tin[v]=++tmr; up[v][0]=p;
    for(int j=1;j<LOG;j++) up[v][j]=up[up[v][j-1]][j-1];
    for(int c:adj[v]){dep[c]=dep[v]+1;dfs(c,v);}
    tout[v]=tmr;
}
int lca(int u,int v){
    if(dep[u]<dep[v])swap(u,v);
    int d=dep[u]-dep[v];
    for(int j=0;j<LOG;j++)if(d&(1<<j))u=up[u][j];
    if(u==v)return u;
    for(int j=LOG-1;j>=0;j--)if(up[u][j]!=up[v][j]){u=up[u][j];v=up[v][j];}
    return up[u][0];
}
struct Node{int mn1,mn2,mx1,mx2;};
int n;
vector<int> rk; // rk[tin[i]]=i
Node merge_nodes(Node a, Node b){
    int c[4]={a.mn1,b.mn1,a.mn2,b.mn2};
    sort(c,c+4,[](int x,int y){return tin[x]<tin[y];});
    int d[4]={a.mx1,b.mx1,a.mx2,b.mx2};
    sort(d,d+4,[](int x,int y){return tin[x]>tin[y];});
    return {c[0],c[1],d[0],d[1]};
}
Node tr[4*MX];
void build(int o,int l,int r){
    if(l==r){tr[o]={l,l,l,l};return;}
    int m=(l+r)/2;build(2*o,l,m);build(2*o+1,m+1,r);
    tr[o]=merge_nodes(tr[2*o],tr[2*o+1]);
}
Node query(int o,int l,int r,int ql,int qr){
    if(ql>r||qr<l) return {0,0,0,0};
    if(ql<=l&&r<=qr) return tr[o];
    int m=(l+r)/2;
    return merge_nodes(query(2*o,l,m,ql,qr),query(2*o+1,m+1,r,ql,qr));
}
```

```
int main(){
    int q; scanf("%d%d",&n,&q);
    for(int i=2;i<=n;i++){int p;scanf("%d",&p);adj[p].push_back(i);}
    tin[0]=1e9; dep[1]=0; dfs(1,0); build(1,1,n);
    while(q--){
        int l,r; scanf("%d%d",&l,&r);
        if(l==r){printf("%d %d\n",l,dep[1]);continue;}
        Node nd=query(1,1,n,l,r);
        int mn=nd.mn1,mx=nd.mx1;
        int cmn=nd.mn2,cmx=nd.mx2;
        int lca_a=lca(cmn,mx); // remove mn
        int lca_b=lca(mn,cmx); // remove mx
        if(dep[lca_a]>=dep[lca_b])
            printf("%d %d\n",mn,dep[lca_a]);
        else
            printf("%d %d\n",mx,dep[lca_b]);
    }
}
```

➤ Trace on the Official Sample

CF 1062E: n=11, parents 1 1 3 3 3 4 2 7 7 6; 5 queries

Tree: $1 \to \{2,3\}$, $3 \to \{4,5,6\}$, $4 \to \{7\}$, $2 \to \{8\}$, $7 \to \{9,10\}$, $6 \to \{11\}$. Depths: $d = (0,1,1,2,2,2,3,2,4,4,3)$.

Q1 (4,6): set $\{4,5,6\}$. All have parent 3. $\text{lca}(4,5,6) = 3$, depth 1. Remove any: remaining LCA still 3. Output `4 1`. ✓

Q2 (4,8): set $\{4,5,6,7,8\}$. $\text{lca}(4,5,6,7,8) = 1$ (node 8 in subtree 2, rest in subtree 3). Remove 8: $\text{lca}(4,5,6,7) = 3$, depth 1. Remove others: LCA= 1. Best: remove 8, depth 1. Output `8 1`. ✓

Q3 (1,11): set $= \{1,\ldots,11\}$. Node 1 is in the set, so lca(all) = 1, depth 0. Remove any vertex leaves 1. Output `1 0`. ✓

Q4 (9,11): set $\{9,10,11\}$. $\text{lca}(9,10) = 7$, $\text{lca}(9,11) = \text{lca}(10,11) = 3$. Remove 11: LCA(9,10) = 7, depth 3. Remove 9 or 10: LCA = 3, depth 1. Best: remove 11, depth 3. Output `11 3`. ✓

Q5 (8,11): set $\{8,9,10,11\}$. $\text{lca}(8,9,10,11) = 1$, depth 0. Remove 8: $\text{lca}(9,10,11) = 3$, depth 1. Remove others: LCA stays 1, depth 0. Best: remove 8, depth 1. Output `8 1`. ✓

➤ Complexity

Complexity

Preprocessing: $O(n \log n)$ (up-table via one DFS) + $O(n)$ (segment-tree build). Per query: $O(\log n)$ (segment tree gives four extremes) + $O(\log n)$ (two LCA calls). Total $O((n+q)\log n)$. Memory: $O(n \log n)$ for lifting + $O(n)$ for segment tree. At $n, q \le 10^5$: roughly 20 MB. Verdict: ✓ **Accepted**.

➤ Pitfalls and Variants

The "remove one" analysis

Only removing a tin-*extreme* vertex can change the set LCA. If the range has $r-l+1 = 2$ (just two vertices), removing either leaves a singleton whose LCA is itself—output the correct vertex.

The segment-tree identity for empty entries

The merge function uses tin[0] for "empty" entries when a range query spans fewer than four vertices. Set tin[0] $= 10^9$ so the dummy never wins an argmin. Forgetting this gives RE or WA on singleton ranges.

Key takeaways — three things to remember

1. *LCA of a set = LCA of the two extremes by* tin. The proof is Proposition 16.5.4: the min-tin and max-tin bracket the entire set's DFS window, so their common ancestor covers everyone (W16.5.2).
2. *Removing a non-extreme preserves the LCA.* The extremes are the only "bottleneck" vertices; the second-extreme gives the post-removal LCA without rescanning.
3. *This chapter's machine + Chapter 19's segment tree = set-LCA queries in* $O(\log n)$. The handoff: this chapter owns the LCA + tin/tout; Chapter 19 owns the range-min structure.

AC $\longrightarrow$ red: hardening this code

- **Generalises**: any "aggregate over a set of vertices via LCA" (LC 1483, CF 1702G2, CF 629E) uses the same extremes-by-tin reduction; the combiner (max, xor, count) rides on the aggregate table from Theorem 16.4.1.
- **Chapter 16's five sections form one pipeline**: DFS $\rightarrow$ tin/tout/depth $\rightarrow$ up-table $\rightarrow$ LCA $\rightarrow$ distance/path/aggregate/set. The Summit is the first problem that requires *all five* layers simultaneously.
- **Alternatives**: Euler-tour RMQ (Theorem 16.2.2) gives $O(1)$ LCA inside the segment tree—slightly faster at the cost of a second sparse-table build. A virtual tree (Ch. 26) solves the same set queries but is overkill for single-range problems.

Tool Kit Inventory

The machine assembled through Chapter 16. Chapters 1–15's rows are carried forward; this chapter's are appended. Pointers only.

Tool	Since	Lv	Trigger phrase	Where
Chapters 1–15 – carried forward				
Rooted-view DFS	12	L2	"how deep / tall / many below"	Code 12.1.1
Iterative two-phase DFS	12	L3	"chain may be deep"	Code 12.1.2
tin / tout flattening	Ch.5	L2	"is u an ancestor of v?"	Chapter 5
Subtree fold (DP template)	13	L3	"one answer per node, bottom-up"	Theorem 13.1.2
Rerooting	15	L4	"for every root"	Chapter 15
Chapter 16 – new this chapter				
Naive walk	16.1	L2	"LCA, few queries, small h"	Code 16.1.1
Euler tour + sparse table	16.2	L3	"static, huge q, $O(1)$ LCA"	Code 16.2.1, Code 16.2.2
Binary lifting	16.3	L3	"k-th ancestor / distance / default"	Code 16.3.1, Code 16.3.3
Aggregated lifting	16.4	L4	"max/sum/xor along a path"	Theorem 16.4.1, Code 16.4.1
Depth buckets	16.5	L4	"same-depth in subtree"	Code 16.5.1
Set-LCA via tin-extremes	Summit	L5	"LCA of k nodes / range"	Code 16.5.8

Related Topics

- Chapter 5 (Ch. 5, Euler Tour & Subtree Flattening) — backward: owns the entry-only tin / tout convention that Proposition 16.5.4 restates; this chapter's $2n-1$ tour (Definition 16.2.1) is a different object, contrasted explicitly in 16.2.
- Chapter 12 (Ch. 12, Tree Metrics) — backward: the rooted-view DFS Code 12.1.1 is the setup pass for every table here; depth is the currency the distance formula spends.
- Chapter 13 (Ch. 13, Tree DP) — backward: Theorem 13.1.2 and §13.4 are the fold and pair-at-LCA harvest that this chapter makes *queryable*; the segment tree in the Summit is Chapter 19's tool.
- Chapter 15 (Ch. 15, Rerooting) — backward: CSES 1136's "sum of distances from every node" combines Ch. 15's reroot-every-node pattern with Theorem 16.5.1.
- Chapter 9 (Ch. 9, Cartesian Trees) — the *reverse* reduction (RMQ → LCA → Cartesian tree) lives there; this chapter owns the forward direction.
- Ch. 19/20/22 (segment trees, lazy, HLD) — forward: the answer to "what when the values change."
- Ch. 24 (centroid decomposition) — forward: distance *distributions* over all pairs.
- Ch. 26 (virtual trees) — forward: compresses a k-node set via sort-by-tin + LCA of adjacent pairs; this chapter's Theorem 16.5.1 and lifting are the inner loop.

Last Word — Chapter 16

"Chapter 13 made the tree compute; Chapter 16 makes it answer questions about pairs. The hinge is free: one DFS fixes it, and every path question becomes arithmetic."

The whole chapter is one object (the LCA) served by three machines (walk, tour+RMQ,

lifting) and measured by three formulas (distance, on-path, subtree-interval). The walk is honest and cheap for small h (Code 16.1.1); the tour converts lca into a range-min on a $2n-1$ array (Theorem 16.2.2), and the sparse table makes it $O(1)$ (Definition 16.2.3); lifting replaces *steps* by *strides* (Definition 16.3.1), giving $O(\log n)$ per query plus the ability to carry aggregates (Theorem 16.4.1) and test ancestry (Proposition 16.5.4). The distance theorem (Theorem 16.5.1) is the simplest payoff—one subtraction after one LCA—and it unlocks the route-parity check (CF 1304E) and the depth-bucket count (CF 208E). The Summit layers them all: set-LCA reduction + segment-tree RMQ + binary lifting + distance arithmetic.

Carry three reflexes forward:

1. *The hinge is free information.* Any path question splits at ℓ into two ancestor-climbs; pay $O(\log n)$ once, answer any pair query without re-DFS.
2. *Idempotence determines your data structure.* Sparse table (overlapping blocks) only for min/max/gcd; lifting (non-overlapping jumps) for sum/xor/count/associative (Theorem 16.4.1).
3. *LCA of a set = LCA of the tin-extremes.* This reduction (proved in W16.5.2) converts every "group ancestor" question into one pair query.

And three habits, drilled until boring: always verify LOG against the constraint bound before coding; check the $u{=}v$ shortcut and the `return par` in every lifting-LCA implementation; and when a problem says "for every pair in the subtree," reach for buckets and tin / tout before brute-forcing—the W16.5.1 pattern is one binary search away.

Next: Chapter 17 (DSU-on-tree) takes the depth-bucket idea and makes subtree queries with *per-node answers* run in $O(n \log n)$ total. Ch. 19/20 (segment trees) and Ch. 22 (HLD) then make the path queries *dynamic*, answering the question this chapter deliberately defers: what happens when the values change between queries.

Part IV

Query Trees

Segment and Indexed Trees

CF 1600–2300 • 6 chapters • 122 problems

Chapters in this part

18 Fenwick Trees 1079

19 Segment Trees 1145

20 Segment Trees II: Lazy 1212

21 Merge-Sort and Persistent Trees 1276

22 Heavy-Light Decomposition 1350

23 Li Chao Trees, Segment Tree Beats and Function-Tree DP 1404

Part V

Expert Techniques

Decompositions, Persistence and Advanced Queries

CF 1800–2800 • 8 chapters • 108 problems

Chapters in this part

24 Centroid Decomposition 1462

25 Link-Cut Trees (Dynamic Trees) 1518

26 Virtual Trees (Auxiliary Trees) 1565

27 Offline Tree Queries: Mo's on Trees and Tarjan LCA 1608

28 Tree Isomorphism and Hashing 1655

29 Trees Born from Graphs 1708

30 Counting Labelled Trees 1762

31 Minimum Spanning Trees and Variants 1802

A Formula Reference 1848

B Code Templates. 1850

C Problem-Solving Strategies 1853

List of Solved Problems 1855

Appendices

Index

m-ary tree, 29
(height-)balanced, 30
j-th ancestor, 962

adaptive strategy, 389
ancestor, 7, 388, 962
ancestor chain, 962
ancestor order, 961

binary tree, 29

child, 7
complete, 30
components, 381
constructing a quad tree, 369
cost, 389
cycle-closing, 380

degenerate, 29
degree, 7
degree sequence, 355, 369
degree sum, 360
depth, 15, 16
descendant, 7, 962
distance, 7, 388

encoding, 355

find, 403
forest, 6, 381
full, 29

guess the number, 388

height, 15, 16

internal, 7, 374

labelled tree, 40
leaf, 7, 374
length, 7
level, 16
lowest common ancestor, 956

oracle, 388
ordered, 40

parent, 7
parent array, 355, 361
path, 7
perfect, 29
Prüfer code, 355, 369, 372
proper, 29

quad tree, 374
queries, 389
query model, 389

raw data, 355
rebuild the tree, 355
reconstruction, 361
root, 7
rooted tree, 7

sentinel, 961
shape, 40
size, 15, 17
skewed, 29
spanning tree, 381
sparse table, 956, 969
sparse-table block, 969
subtree, 7

tour of the DFS walk, 956
tree, 2, 6
tree certificate, 355, 360
tree-realisable, 371